

created: 2026-03-03

updated: 2026-06-08

1 Digital Image: 从连续世界到离散矩阵 p.7

1.1 Digital Camera: sensor array 代替 film p.7

- digital camera 把原来 film 上的连续光照记录，替换成一个二维 sensor array。
- 每个 sensor cell 是 light-sensitive diode: 它把 photons 转换成 electrons, 再由电子数量形成可读的 intensity。
- 这一步已经暗示了计算机视觉的基本对象: 不是连续场景本身，而是 sensor 在有限网格上的 measurement。

Intuition: 为什么图像一开始就是近似。

真实世界的 *irradiance* 是连续变化的: camera 只能在有限数量的 *sensor cells* 上测量它。

所以 *digital image* 同时有两层近似:

- spatial sampling*: 只在离散位置取样;
- value quantization*: 每个位置只存有限精度的数值。

后面所有 *filtering / convolution* 本质上都是在这个 *discrete grid* 上定义局部运算。

1.2 Sampling and Quantization: 位置离散化与数值离散化 p.9

- sampling: 在二维空间上取 regular grid, 把连续 image plane 变成 pixels。
- quantization: 把每个 sample 的连续强度值映射到最近的 integer。
- 如果 sampling rate 不够, 可能出现 aliasing / sampling artifacts: 这就是 slide 中 “Errors due Sampling” 的来源。

Formal View: 从连续函数到 *digital image*。

设连续 *grayscale image* 是函数

$$f:\mathbb{R}^2\rightarrow\mathbb{R}.$$

以水平/竖直 *spacing* $\Delta x, \Delta y$ 采样后, 得到离散网格:

$$I[i,j]\approx Q\left(f(i\Delta x,j\Delta y)\right),$$

其中 $Q(\cdot)$ 是 *quantization function*, 例如把值 round 到 $0,\dots,255$ 。

因此 *digital image* I 不是原函数 f , 而是 f 在有限格点上的有限精度记录。

1.3 Resolution: spatial pixel density p.11

- resolution 是 sampling parameter, 常用 dots per inch (DPI) 或 equivalent spatial pixel density 表示。
- slide 给出的 screen technology 标准值例子是 72 dpi。
- resolution 越高, 单位物理长度上的 samples 越多; 但它不自动保证 image quality, 因为 noise, lens blur, quantization 也会影响结果。

Q&A: resolution 和 image size 是一回事吗。

不是完全一回事。

image size 说的是矩阵有多少 pixels, 例如 $H\times W$ 。

resolution / DPI 说的是这些 *pixels* 映射到物理空间时有多密, 例如每 inch 多少 *samples*。

同样是 1000×1000 的图, 如果打印成很小, *DPI* 高; 打印成很大, *DPI* 低。

1.4 Digital Image Matrix: height $\times$ width $\times$ channel p.12

- color image 可以表示为一个三维矩阵, size 是 $H\times W\times C$ 。
- grayscale image 可以表示为二维矩阵 $H\times W$ 。
- $I(i,j)$ 称为 intensity, 通常用 unsigned 8-bit integer (uint8) 存储, 即取值范围 $0,\dots,255$ 。

Formal Definition: Image Tensor.

对 *color image*:

$$I\in\{0,\dots,255\}^{H\times W\times C}.$$

常见 *RGB image* 中 $C=3$, 分别对应 red, green, blue channels.

对 *grayscale image*:

$$I\in\{0,\dots,255\}^{H\times W}.$$

如果使用 *floating point representation*, 也可以把 *intensity normalization* 到 $[0,1]$:

$$I_{\text{float}}(i,j)=\frac{I_{\text{uint8}}(i,j)}{255}.$$

2 Image as Functions: 把图像看成函数 p.15

2.1 Grayscale Image as Function $f:\mathbb{R}^2\rightarrow\mathbb{R}$ p.15

- grayscale image 可以看作二维位置到 intensity 的函数。
- continuous view 中写作 $f(x,y)$; digital view 中写作 $I[i,j]$ 。
- 这个函数视角很重要, 因为 image transformation / filtering 都可以理解对函数做运算。

Discrete vs Continuous Notation.

slides 中常把 *grayscale image* 写成

$$f:\mathbb{R}^2\rightarrow\mathbb{R}.$$

在真正的 *pixel grid* 上, 更准确的写法是

$$I:\{1,\dots,H\}\times\{1,\dots,W\}\rightarrow\mathbb{R}.$$

两种写法服务不同目的: *continuous notation* 方便讲 *geometric transformation*; *discrete notation* 方便讲 *filtering* 和 *implementation*。

2.2 Point Transformation: 只改 intensity, 不改位置 p.16

- 最简单的 image transformation 是对每个 pixel 独立修改 intensity。
- slide 例子是 brightness shift:

$$g(x,y)=f(x,y)+50.$$

- 这个 transformation 不看 neighborhood, 因此不能 denoise 或 detect structure, 只能改每个位置的 value。

Proof: brightness shift 为什么不改变 geometry.

对任意两个 image locations (x_1,y_1) 和 (x_2,y_2) , brightness shift 输出为

$$g(x_1,y_1)=f(x_1,y_1)+50,\quad g(x_2,y_2)=f(x_2,y_2)+50.$$

位置坐标完全没有变化: 只有每个位置上的 *scalar value* 加了常数。

所以它保持 *image* 的 *spatial arrangement*, 只改变 *intensity scale*。

2.3 Spatial Transformation: 用另一个位置的值生成当前位置 p.18

- spatial transformation 会改变坐标对应关系。
- slide 例子可以理解为 horizontal flip:

$$g(x,y)=f(x,-y).$$

- 这里 output location (x,y) 的值来自 input location $(x,-y)$ 。

Derivation: horizontal flip 的坐标映射。

若 image coordinate 中 y 表示 horizontal axis, 并且原点为中心, 则

$$y\mapsto-y$$

会把左侧点映射到右侧、右侧点映射到左侧。

output 定义为 $g(x,y)=f(x,-y)$, 意思是: 在输出图像的 (x,y) 上, 填入原图对应位置 $(x,-y)$ 的值。

2.4 Thresholded Spatial Transformation: 坐标变换加条件 mask p.20

- slide 中进一步把 spatial transform 和 threshold mask 结合:

$$g(x,y)=f(x,-y)\cdot\mathbf{1}\{f(x,-y)<250\}.$$

- 当 mirrored source pixel 小于 threshold 时保留; 否则乘上 0。
- 这类 operation 说明 image transformation 可以组合: 先重采样位置, 再对 intensity 做条件筛选。

Proof: threshold mask 为什么会删除高亮区域。

indicator function 定义为

$$\mathbf{1}\{A\}=\begin{cases}1,&A\text{ is true,}\\0,&A\text{ is false.}\end{cases}$$

因此

$$g(x,y)=\begin{cases}f(x,-y),&f(x,-y)<250,\\0,&f(x,-y)\geq250.\end{cases}$$

结论: threshold 以上的 *bright pixels* 会被置零, 其余 *pixels* 保留 *mirrored intensity*。

3 Color Space: 颜色的物理与坐标表示 p.23

3.1 What Is Color: light, surface, visual system 的交互结果 p.23

- color 不是单纯的物理量: 它来自 environmental light 和 human visual system 的交互。
- 对同一个 surface, observed color 会受到 illumination spectrum, surface reflectance, camera sensor response / human cones 的共同影响。
- slide 用 monochromatic light 下的 room example 强调: 光源变化可以显著改变观察到的颜色。

Intuition: 为什么颜色不是物体的固定属性。

一个 *surface* 的 *reflectance* 可以比较稳定, 但 *observed color* 取决于入射光。

如果 *illumination* 只包含某个窄波段, 那么即使物体本来能反射多种 *wavelengths*, camera / human eye 也只能看到该 *illumination* 提供的那部分信息。

因此 *vision algorithm* 直接使用 *RGB values* 时, 经常会被 *lighting condition* 影响。

3.2 Linear Color Spaces: 用 primaries 的线性组合表示颜色 p.25

- linear color space 由三个 primaries 定义。
- 一个 color 的 coordinates 是为了 match 该 color 所需的 primaries weights。
- mixing two lights 会在 color space 中产生连接两点的 straight line: mixing three lights 会产生三角形内的颜色。

Formal Definition: linear color coordinates.
设三个 primaries 是 vectors p_1, p_2, p_3 , 某个 color vector c 可以表示为

$$c=\alpha_1p_1+\alpha_2p_2+\alpha_3p_3.$$

则 $(\alpha_1,\alpha_2,\alpha_3)$ 就是该 color 在这个 linear color space 下的 coordinates。

如果只混合两个 colors c_a, c_b , 得到

$$c(t)=(1-t)c_a+tc_b,\quad 0\leq t\leq1,$$

这正是 *color space* 中的一条 line segment。

3.3 Red Green Blue (RGB) Space: monochromatic primaries 的三通道表示 p.26

- Red Green Blue (RGB) space 使用 red, green, blue 三个 primaries。
- 对 monitor 来说, 它们对应三种 phosphors / display primaries。
- RGB image 的每个 pixel 通常存成三元组 (R,G,B) 。

Q&A: RGB 为什么常用。

RGB 与 display hardware 和 camera sensor pipeline 高度匹配: 显示器发光通常由 red/green/blue subpixels 组合, camera 也常通过 color filter array 估计 *RGB channels*。

但 *RGB* 不总是 *perceptually uniform*: 数值距离小不一定代表人眼感知差异小。

3.4 Hue Saturation Value (HSV): 非线性颜色空间 p.27

- Hue Saturation Value (HSV) 是 nonlinear color space。
- Hue 表示色相; Saturation 表示颜色纯度 / 灰度混入程度; Value 表示亮度强度。
- HSV 把 RGB cube 重新组织成更接近 perceptual dimensions 的坐标, 便于做 color-based editing / thresholding。

Comparison: RGB vs HSV.

RGB 更接近设备的 additive color mixing。

HSV 更接近人描述颜色的方式: 是什么颜色、有多鲜艳、有多亮。

但 *HSV* 是 nonlinear transformation, 因此很多 linear filtering / color interpolation 在 *HSV* 里不一定保持 *RGB* 中的线性意义。

4 Histogram: 灰度分布的统计视图 p.29

4.1 Image Histogram: 每个 gray level 出现的频率 p.29

- histogram captures the distribution of gray levels in the image。
- 对 uint8 grayscale image, histogram 通常有 256 个 bins, 对应 gray levels $0,\dots,255$ 。
- 它丢弃 spatial arrangement, 只保留 intensity distribution。

Formal Definition: grayscale histogram.

对 image I , gray level a 的 histogram count 是

$$h(a)=|\{(i,j):I(i,j)=a\}|.$$

如果 *normalize* 成 *probability distribution*:

$$p(a)=\frac{h(a)}{HW}.$$

因为所有 *pixels* 总共是 HW 个, 所以

$$\sum_{a=0}^{255}p(a)=1.$$

Proof: 每个 *pixel* 恰好落入一个 gray-level bin, 因此所有 bin counts 相加等于 HW 。

4.2 Histogram Use Case: 对比度、曝光和 threshold 的线索 p.31

- histogram 可以判断图像是否偏暗、偏亮、contrast 是否过低。
- 如果 histogram 集中在很窄的 gray-level range, 说明 intensity variation 少, 图像可能 low contrast。
- thresholding / segmentation 时, histogram valley 常被用作候选 threshold。

Limitation: histogram 不知道空间结构。

两张图像可以有完全一样的 *histogram*, 但 *spatial layout* 完全不同。

因此 *histogram* 适合描述 global intensity distribution, 不适合直接描述 shape, edge, texture placement.

5 Noise Reduction Motivation: 为什么需要 filtering p.33

5.1 Types of Noise: salt-and-pepper, impulse, Gaussian noise p.34

- salt-and-pepper noise: 随机出现 black and white pixels。
- impulse noise: 随机出现 white pixels。
- Gaussian noise: intensity variations drawn from Gaussian normal distribution。

Formal Noise Models.

additive Gaussian noise 常写成

$$Y(i,j)=X(i,j)+\varepsilon(i,j),\quad \varepsilon(i,j)\sim\mathcal{N}(0,\sigma^2).$$

salt-and-pepper noise 可写成随机替换:

$$Y(i,j)=\begin{cases}0,&\text{with probability }p/2,\\255,&\text{with probability }p/2,\\X(i,j),&\text{with probability }1-p.\end{cases}$$

这两种 *noise* 的统计结构不同, 所以适合的 *denoising filter* 也不同。

5.2 First Attempt: 用 neighborhood average 替代当前 pixel p.35

- basic denoising idea: 把每个 pixel 替换成其 neighborhood values 的 average。
- 两个假设:
 - pixels tend to be like their neighbors;
 - noise processes are independent from pixel to pixel。
- 如果 signal 局部平滑, 而 noise 独立且均值为 0, averaging 可以降低 noise variance。

Proof: averaging 为什么能降低 independent zero-mean noise.

设 neighborhood 中有 n 个观测值:

$$Y_t=X+\varepsilon_t,\quad \mathbb{E}[\varepsilon_t]=0,\quad \text{Var}(\varepsilon_t)=\sigma^2,$$

并假设局部真实 *signal* 近似相同, 都是 X 。平均后:

$$\bar{Y}=\frac{1}{n}\sum_{t=1}^nY_t=X+\frac{1}{n}\sum_{t=1}^n\varepsilon_t.$$

expectation:

$$\mathbb{E}[\bar{Y}]=X.$$

如果 *noises independent*:

$$\mathrm{Var}(\bar{Y}) = \mathrm{Var}\left(\frac{1}{n} \sum_{t=1}^n \varepsilon_t\right) = \frac{1}{n^2} \sum_{t=1}^n \sigma^2 = \frac{\sigma^2}{n}.$$

结论: 平均不会引入 *bias*, 但把 *noise variance* 降低到原来的 $1/n$.

5.3 Weighted Moving Average: 从 uniform weights 到 non-uniform weights p.37

- uniform moving average 使用相同权重, 例如

$$\frac{1}{5} [1, 1, 1, 1, 1].$$

- non-uniform weighted moving average 可以让中心附近的 pixels 更重要, 例如

$$\frac{1}{16} [1, 4, 6, 4, 1].$$

- 这一步是从 simple averaging 过渡到 general filtering 的关键。

Proof: *weights sum to 1* 时 *constant region* 不变。

对一维 *signal*, *weighted average* 写成

$$g[i] = \sum_{u=-k}^k w[u]f[i+u].$$

如果 *local region* 是 *constant*, 即 $f[i+u] = c$, 则

$$g[i] = \sum_{u=-k}^k w[u]c = c \sum_{u=-k}^k w[u].$$

若 $\sum_u w[u] = 1$, 则 $g[i] = c$.

这证明了 *smoothing kernel* 通常需要 *normalize*: *constant brightness* 不应被改变。

5.4 Moving Average in 2D: 二维 neighborhood 上的 smoothing p.39

- 2D moving average 把一维 window 扩展成二维 patch。
- 每个 output pixel 都来自 input image 的一个 local neighborhood。
- box filter 会平滑区域内部, 但也会把 edge 两侧的值混合, 导致 boundary blur。

Intuition: *smoothing* 的代价是 *blur*。

averaging 假设 *neighbor pixels* 和当前 *pixel* 属于同一 *underlying signal*。

在 *flat region* 里这个假设通常成立, 所以 *noise* 被平均掉。

在 *edge* 附近, 这个假设失败: *window* 同时覆盖 *foreground* 和 *background*, 平均值会落在两者之间, 于是 *edge* 被 *blurred*。

6 Correlation Filtering: 用 kernel 做局部线性组合 p.46

6.1 Cross-Correlation Definition: $G = H \otimes F$ p.47

- cross-correlation filtering: replace each pixel with a linear combination of its neighbors。
- filter kernel / mask $H[u, v]$ 指定每个 relative position 的 *weight*。
- 记 input image 为 F , output image 为 G , kernel radius 为 k , 则

$$G[i, j] = (H \otimes F)[i, j] = \sum_{u=-k}^k \sum_{v=-k}^k H[u, v]F[i+u, j+v].$$

Derivation: *from moving average to cross-correlation*。

对 $(2k+1) \times (2k+1)$ *neighborhood*, *uniform average* 是

$$G[i, j] = \frac{1}{(2k+1)^2} \sum_{u=-k}^k \sum_{v=-k}^k F[i+u, j+v].$$

把 *uniform weight* $\frac{1}{(2k+1)^2}$ 替换成 *location-dependent weight* $H[u, v]$, 得到

$$G[i, j] = \sum_{u=-k}^k \sum_{v=-k}^k H[u, v]F[i+u, j+v].$$

这正是 *cross-correlation*。它是 *linear filter*, 因为 *output* 是 *input pixels* 的线性组合。

6.2 Averaging Filter: box kernel p.48

- moving average 的 kernel 是 uniform box filter。

- 对 3×3 average:

$$H = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}.$$

- filter size 从 3×3 增加到 5×5 会增强 *smoothing*, 但也加重 *blur*。

Proof: 3×3 *box filter* 的输出是九个邻居均值。

代入 *cross-correlation definition*:

$$G[i, j] = \sum_{u=-1}^1 \sum_{v=-1}^1 \frac{1}{9} F[i+u, j+v] = \frac{1}{9} \sum_{u=-1}^1 \sum_{v=-1}^1 F[i+u, j+v].$$

这就是 *centered 3×3 neighborhood* 的 *arithmetic mean*。

6.3 Boundary Issues: full, same, valid 与边界外推 p.50

- 当 filter window 靠近 image edge 时, 它会 fall off the edge。
- output size 常见三种选择:
- full**: 包含所有 overlap 情况, output size 变大;
- same**: output size 与 input image 相同;
- valid**: 只保留 kernel 完全落在 image 内的 positions, output size 变小。
- edge extrapolation methods:
- zero fill / black padding;
- replicate border;
- mirror reflect;
- mirror across edge。

Formula: *valid output size*。

若 input 是 $H \times W$, kernel 是 $m \times n$, 且不 *padding*, 则 *valid output size* 是

$$(H-m+1) \times (W-n+1).$$

Proof: *vertical direction* 中 kernel top-left 可以从 row 1 放到 row $H-m+1$, 共 $H-m+1$ 个位置; *horizontal direction* 同理是 $W-n+1$ 。

Comparison: *padding methods*。

zero fill 简单, 但会在边界引入 *artificial dark pixels*。

replicate border 假设边界外继续保持边界值, 适合不想制造黑边的情况。

mirror reflect / *mirror across edge* 假设边界外是图像的镜像延拓, 通常能减少 *edge discontinuity*。

6.4 Identity, Shift, Blur, Sharpening Filters p.52

- identity filter 保留原因:

$$H_{\text{id}} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}.$$

- shift filter 把某个 neighbor 复制到当前位置, 例如

$$H_{\text{shift}} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix}$$

会在 *correlation convention* 下取 $F[i, j+1]$ 。

- sharpening filter* 可写成 *identity* 加上 *high-frequency emphasis*, 例如 *slide* 中的形式:

$$H_{\text{sharp}} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 0 \end{bmatrix} - \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}.$$

Proof: *identity filter* 为什么不改变图像。

代入 *cross-correlation*:

$$G[i, j] = \sum_{u=-1}^1 \sum_{v=-1}^1 H_{\text{id}}[u, v]F[i+u, j+v].$$

除了中心 *weight* $H_{\text{id}}[0, 0] = 1$, 其他 *weights* 全为 0, 所以

$$G[i, j] = F[i, j].$$

Derivation: *sharpening* = *original* + *difference from local average*。

令 *box blur* 为

$$B = \frac{1}{9} 1_{3 \times 3}.$$

slide 的 *sharpening kernel* 是

$$H_{\text{sharp}} = 2\delta - B,$$

其中 δ 是 *identity kernel*。

对 *image* F :

$$H_{\text{sharp}} \otimes F = 2F - (B \otimes F) = F + (F - B \otimes F).$$

$F - B \otimes F$ 是当前 *image* 与 *local average* 的差, 突出 *local differences*; 加回原图后就 *accentuates differences with local average*。

7 Gaussian and Median Filters: 平滑但尽量保留结构 p.62

7.1 Gaussian Filter: nearest neighbors 权重最大 p.62

- Gaussian filter 用二维 Gaussian function 近似生成 kernel。
- 中心附近 pixels 权重大, 远处 pixels 权重小。
- 它 removes high-frequency components from the image, 因此是 low-pass filter。

二维 Gaussian density:

$$G_{\sigma}(x, y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2+y^2}{2\sigma^2}\right).$$

Formal Definition: *discrete Gaussian kernel*。

实际 *filter* 需要 *finite kernel*, 因此在有限 *window* Ω 中 *sample Gaussian*, 再 *normalize*:

$$H[u, v] = \frac{\exp\left(-\frac{u^2+v^2}{2\sigma^2}\right)}{\sum_{(a,b) \in \Omega} \exp\left(-\frac{a^2+b^2}{2\sigma^2}\right)}.$$

denominator 确保 $\sum_{(u,v) \in \Omega} H[u, v] = 1$, 所以 *constant region* 不变。

7.2 Kernel Size and Variance: finite support 与 smoothing extent p.65

- Gaussian function has infinite support, 但 discrete filters use finite kernels。

- kernel size 太小会 truncate Gaussian tail, 近似质量变差。
- variance* / σ determines extent of smoothing: σ 越大, spread 越宽, smoothing 越强。

Proof: σ 控制 *spread*。

Gaussian exponent 是

$$-\frac{x^2+y^2}{2\sigma^2}.$$

固定距离 $r^2 = x^2 + y^2$ 时, σ 越大, $-\frac{r^2}{2\sigma^2}$ 的绝对值越小, 远离中心的位置仍有较大权重。

因此 larger σ 会把更远的 *neighbors* 纳入有效平均, *smoothing* 更强。

7.3 Properties of Smoothing Filters: positive, sum-to-one, low-pass p.68

- values positive;
- weights sum to 1, 所以 constant regions same as input;
- amount of smoothing proportional to mask size / effective support;
- remove high-frequency components, 是 low-pass filter。

Explanation: *high-frequency* 为什么被 *smoothing* 抑制。

high-frequency components 表示 *intensity* 在短距离内快速变化。

smoothing filter 把 *neighborhood* 中多个 values 做 *weighted average*。快速正负变化会在平均中互相抵消, 而 *slowly varying* / *constant component* 保留得更多。

所以 *smoothing* 在 *spatial domain* 是 *local averaging*, 在 *frequency intuition* 中就是 *attenuation of high frequencies*。

7.4 Mean vs Gaussian: uniform average 与 distance-aware average p.64

- mean / box filter 给 window 内所有 pixels 相同权重。
- Gaussian filter 根据 distance 衰减权重, 中心附近更重要。
- Gaussian 通常比 box filter 更自然, 因为它减少了 hard window boundary 带来的 artifacts。

Comparison: *box kernel* 的 *sharp cutoff*。

box filter 在 *window* 内权重完全相同, *window* 外权重突然变成 0。

Gaussian filter 权重随距离平滑下降, 因此在许多图像处理中更稳定, 也与 *scale-space theory* 更契合。

7.5 Median Filter: nonlinear, spike removal, edge preserving p.70

- median filter 不引入新的 pixel values: 输出来自 *neighborhood* 中已有的中位数。
- 它对 impulse / salt-and-pepper noise 很有效, 因为 isolated spikes 不容易成为 median。
- 它是 nonlinear filter。
- median filter 常比 mean filter 更 edge preserving。

Proof: *median filter* 是 *nonlinear filter*。

若 operator M 是 *linear*, 应满足

$$M(a+b) = M(a) + M(b).$$

考虑一维 3-pixel window:

$$a = [0, 0, 100], \quad b = [0, 100, 0].$$

则

$$M(a) = 0, \quad M(b) = 0.$$

但

$$a+b = [0, 100, 100], \quad M(a+b) = 100.$$

因此

$$M(a+b) \neq M(a) + M(b),$$

median filter 不是 *linear filter*。

Intuition: *median* 为什么适合 salt-and-pepper noise。

salt-and-pepper noise 是少量 *pixels* 被替换成极端值 0 或 255。

在一个 *small neighborhood* 中, 只要 *corrupted pixels* 不超过一半, *median* 仍会落在未污染 *pixels* 的 *typical range*。

mean filter 会把极端值也平均进去, 因此更容易被 *spikes* 拉偏。

8 Convolution: 先翻转 kernel, 再做 correlation p.74

8.1 Convolution Definition: flip both dimensions p.74

- convolution 的操作:
- flip the filter in both dimensions;
- then apply cross-correlation。
- 对二维 discrete image:

$$(H * F)[i, j] = \sum_{u=-k}^k \sum_{v=-k}^k H[u, v]F[i-u, j-v].$$

- 与 cross-correlation 的差别是 kernel index 的 sign。

Derivation: *convolution* = *correlation with flipped kernel*。

定义 *flipped kernel*:

$$\tilde{H}[u, v] = H[-u, -v].$$

对 $\tilde{H}$ 做 *cross-correlation*:

	<i>Fourier transform</i> 把 <i>signal</i> 分解成频率成分。 <i>convolution</i> 在 <i>spatial domain</i> 中是“邻域混合”，在 <i>frequency domain</i> 中变成每个 <i>frequency coefficient</i> 乘上 <i>filter response</i>： $\mathcal{F}(f * g) = \mathcal{F}(f)\mathcal{F}(g).$ 这解释了为什么 <i>Gaussian smoothing</i> 是 <i>low-pass</i>: <i>Gaussian kernel</i> 的 <i>frequency response</i> 会削弱 <i>high-frequency coefficients</i>。 8.4 Optical Convolution: camera shake as blur kernel p.77 - camera shake 可以被建模成 sharp image 与 blur kernel 的 convolution。 - blur kernel 描述 exposure time 内 camera / scene 的 motion path。 - deblurring 的目标通常是估计 blur kernel 或从 blurred observation 中恢复 sharp image。 <i>Model: image blur by convolution.</i> 一个简化 <i>observation model</i> 是 $B = K * I + \eta,$ 其中 <i>I</i> 是 <i>sharp image</i>, <i>K</i> 是 <i>blur kernel</i>, <i>B</i> 是 <i>observed blurred image</i>, η 是 <i>noise</i>。 如果 <i>K</i> 已知, <i>deconvolution</i> 尝试反解 <i>I</i>: 如果 <i>K</i> 未知, 就是 <i>blind deconvolution</i>。 9 Separable Filters: 把 2D filter 拆成两个 1D filters p.79 9.1 Separability Definition: row convolution + column convolution p.79 - 有些 filter 是 separable, 可以 factor into two steps: - convolve all rows with a 1D filter; - convolve all columns with a 1D filter。 - 如果 2D kernel 可以写成 outer product: $H[u, v] = a[u]b[v],$ 则它是 separable。 <i>Proof: separable 2D convolution</i> 等价于两次 <i>1D convolution</i>。 若 $H[u, v] = a[u]b[v]$, 则 $(H * F)[i, j] = \sum_u \sum_v a[u]b[v]F[i - u, j - v].$ 先对 <i>rows</i> / <i>horizontal direction</i> 用 <i>b</i>: $R[i, j] = \sum_v b[v]F[i, j - v].$ 再对 <i>columns</i> / <i>vertical direction</i> 用 <i>a</i>: $\sum_u a[u]R[i - u, j] = \sum_u a[u] \sum_v b[v]F[i - u, j - v].$ 这与原始 <i>2D convolution</i> 相同。 9.2 Computational Speedup: 从 $O(k^2)$ 到 $O(2k)$ p.79 - 对每个 pixel, 直接用 $k \times k$ kernel 需要 k^2 次乘法。 - separable filter 需要一次 horizontal <i>k</i>, 一次 vertical <i>k</i>, 共约 2<i>k</i> 次乘法。 - 对大 kernel, speedup 很明显。 <i>Complexity Derivation.</i> 对 $H \times W$ image: $O(HWk^2).$ <i>separable filtering cost:</i> $O(HWk) + O(HWk) = O(2HWk).$ <i>ratio approximately:</i> $O(HWk) + O(HWk) = O(2HWk).$
$(\tilde{H} \otimes F)[i, j] = \sum_{u,v} \tilde{H}[u, v]F[i+u, j+v] = \sum_{u,v} H[-u, -v]F[i+u, j+v]$. 令 $a = -u, b = -v$, 得到 $\sum_{a,b} H[a, b]F[i - a, j - b],$ 这就是 <i>convolution</i>。 8.2 Shift Invariance: 同样 pattern 在任何位置行为相同 p.75 - shift invariant means operator behaves the same everywhere。 - output value depends on local pattern in the image neighborhood, not on the absolute position of that neighborhood。 <i>Formal Statement: translation equivariance.</i> 令 <i>shift operator</i> T_Δ 定义为 $(T_\Delta F)[i, j] = F[i - \Delta_i, j - \Delta_j].$ <i>convolution</i> 满足 $H * (T_\Delta F) = T_\Delta (H * F).$ 这说明先 <i>shift image</i> 再 <i>filter</i>, 等价于先 <i>filter</i> 再 <i>shift output</i>。 8.3 Algebraic Properties: commutative, associative, distributive p.76 convolution 的常用性质: $f * g = g * f$ $(f * g) * h = f * (g * h)$ $f * (g + h) = (f * g) + (f * h)$ $a f * g = f * a g = a (f * g)$ identity / unit impulse: $\delta = [\dots, 0, 0, 1, 0, 0, \dots], \quad f * \delta = f.$ Fourier transform: $\mathcal{F}(f * g) = \mathcal{F}(f) \cdot \mathcal{F}(g).$ <i>Proof Sketch: commutativity.</i> 对一维 <i>discrete convolution</i>: $(f * g)[n] = \sum_m f[m]g[n - m].$ 令 $r = n - m$, 即 $m = n - r$: $(f * g)[n] = \sum_r f[n - r]g[r].$ <i>scalar multiplication</i> 可交换, 所以 $\sum_r g[r]f[n - r] = (g * f)[n].$ 因此 $f * g = g * f$。 <i>Proof Sketch: identity impulse.</i> unit impulse $\delta[m]$ 只有 $m = 0$ 时为 1, 其他位置为 0。 $(f * \delta)[n] = \sum_m f[m]\delta[n - m].$ 只有 $n - m = 0$, 即 $m = n$ 的项不为 0, 所以 $(f * \delta)[n] = f[n].$ <i>(Advance) Convolution Theorem Intuition.</i>	$\frac{HWk^2}{2HWk} = \frac{k}{2}.$ <i>kernel</i> 越大, <i>separability</i> 带来的加速越大。 9.3 Gaussian Filter Is Separable p.80 二维 Gaussian: $G_\sigma(x, y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right).$ 可以分解为两个一维 Gaussian: $G_\sigma(x, y) = \left(\frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{x^2}{2\sigma^2}\right)\right) \cdot \left(\frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{y^2}{2\sigma^2}\right)\right).$ <i>Proof: Gaussian separability.</i> 从二维 Gaussian 开始: $G_\sigma(x, y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right).$ 利用 <i>exponent</i> 的加法性质: $\exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right) = \exp\left(-\frac{x^2}{2\sigma^2}\right) \exp\left(-\frac{y^2}{2\sigma^2}\right).$ 同时 $\frac{1}{2\pi\sigma^2} = \frac{1}{\sqrt{2\pi}\sigma} \frac{1}{\sqrt{2\pi}\sigma}.$ 所以 $G_\sigma(x, y) = g_\sigma(x)g_\sigma(y),$ where $g_\sigma(t) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{t^2}{2\sigma^2}\right).$ 9.4 How to Tell Separability: analytic manipulation, rank, SVD p.81 - 方法 1: try to manipulate the analytic form, 看能否写成 $a[u]b[v]$。 - 方法 2: 把 discrete kernel 看成 matrix, 检查 rank 是否为 1。 - 方法 3: 用 singular value decomposition (SVD): 如果只有一个 nonzero singular value, 则 rank 1, filter separable。 <i>Formal Criterion: rank-1 matrix.</i> 若 <i>kernel matrix</i> $H \in \mathbb{R}^{m \times n}$ 可写为 $H = ab^T,$ 其中 $a \in \mathbb{R}^m, b \in \mathbb{R}^n$, 则 <i>H</i> 的每一列都是 <i>a</i> 的 <i>scalar multiple</i>, 所以 <i>rank</i> 是 1。 反过来, 若 $\text{rank}(H) = 1$, 则 <i>SVD</i> 给出 $H = \sigma_1 u_1 v_1^T,$ 取 $a = \sqrt{\sigma_1} u_1, b = \sqrt{\sigma_1} v_1$, 就有 $H = ab^T$。 <i>(Advance) Approximate Separability.</i> 若 <i>kernel rank</i> 大于 1, 也可以用 <i>top singular components</i> 做 <i>approximation</i>: $H \approx \sum_{r=1}^R \sigma_r u_r v_r^T.$ 每个 <i>rank-1 term</i> 都对应一对 <i>1D filters</i>, 因此可以用少量 <i>separable filters</i> 近似复杂 <i>2D filter</i>。
	10 Filter Applications:从 feature extraction 到 template matching p.83 10.1 Hybrid Images: low frequency 与 high frequency 的感知组合 p.83 - hybrid images 使用 filtering 组合不同 spatial frequency 的信息。 - close viewing distance 时 high-frequency details 更明显; far viewing distance 时 low-frequency structure 更明显。 - 这说明 filters 不只是 denoising tool, 也可以操控 perceptual representation。 <i>Construction: hybrid image.</i> 给两张 <i>aligned images</i> <i>A, B</i>: $A_{\text{low}} = G_\sigma * A,$ $B_{\text{high}} = B - G_\sigma * B.$ <i>hybrid image:</i> $H = A_{\text{low}} + \lambda B_{\text{high}}.$ 其中 G_σ 是 <i>Gaussian low-pass filter</i>, $B - G_\sigma * B$ 是 <i>high-pass component</i>。 10.2 Filters for Features: 从 raw pixels 到 intermediate representation p.84 - feature filter 把 raw pixels 映射到后续 processing 使用的 intermediate representation。 - goal: reduce amount of data, discard redundancy, preserve what is useful。 - edge filters, texture filters, gradient filters 都是这种思路。 <i>Intuition: feature filter</i> 在保留什么。 <i>raw image</i> 中有大量 <i>redundant information</i>: 相邻 <i>pixels</i> 高度相关, 很多 <i>smooth regions</i> 对识别任务贡献有限。 <i>feature filters</i> 试图保留对 <i>downstream task</i> 有用的 <i>structures</i>, 例如 <i>boundaries, corners, oriented textures</i>。 10.3 Filters in Modern Convolutional Neural Networks (CNN) p.85 - Convolutional Neural Networks (CNN) 使用 convolution filters layer by layer 提取 features。 - 与 handcrafted filters 不同, CNN filters 通常通过 training data 自动学习。 - 但核心 computation 仍是 local weighted combination + nonlinear activation。 <i>Connection: classical filters 和 CNN filters.</i> <i>classical vision</i> 里我们手工设计 <i>Gaussian, sharpening, edge filters</i>。 <i>CNN</i> 中, <i>kernel weights</i> 是 <i>learnable parameters</i>: $Y_C[i, j] = \sum_d \sum_u \sum_v W_{c,d} u[v] X_d[i - u, j - v] + b_C.$ 其中 <i>d</i> 是 <i>input channel</i>, <i>c</i> 是 <i>output channel</i>。 这仍然是 <i>convolution</i> / <i>correlation</i> 形式, 只是 <i>weights</i> 由 <i>optimization</i> 学出来。 10.4 Template Matching: filter as matched template p.86 - filters can be templates: filter looks like the pattern it is intended to find。 - matched filters 使用 filter response 来检测某个 pattern 是否出现在 image 中。 - slide 提到用 normalized cross-correlation score 找 template, 并用 normalization control for relative brightness。 <i>Formal Definition: normalized cross-correlation.</i> 设 <i>template</i> 是 <i>T</i>, <i>image patch</i> 是 $P_{i,j}$。normalized cross-correlation (<i>NCC</i>) 可写成 $\text{NCC}(i, j) = \frac{\sum_u, v (T[u, v] - \bar{T})(P_{i,j}[u, v] - \bar{P}_{i,j})}{\sqrt{\sum_u, v (T[u, v] - \bar{T})^2} \sqrt{\sum_u, v (P_{i,j}[u, v] - \bar{P}_{i,j})^2}}.$ <i>score</i> 越接近 1, <i>template</i> 与 <i>patch</i> 越相似。

Proof: *NCC* 对 *additive brightness shift* 不敏感。

若 *image patch* 变成

$$P'_{i,j}[u,v] = P_{i,j}[u,v] + c,$$

则 *patch mean* 也变成

$$\bar{P}'_{i,j} = \bar{P}_{i,j} + c.$$

center 后:

$$P'_{i,j}[u,v] - \bar{P}'_{i,j} = P_{i,j}[u,v] + c - (\bar{P}_{i,j} + c) = P_{i,j}[u,v] - \bar{P}_{i,j}.$$

因此 *NCC numerator* 和 *denominator* 都不受 *additive brightness shift* 影响。

10.5 Correlation Map: response peak 表示 candidate location p.88

- template 在 image 上滑动后, 每个位置得到一个 correlation score.
- 所有 scores 组成 correlation map.
- response peak 对应最可能的 template location.

Practical Caveat: template matching 的限制.

template matching 对 scale、rotation、deformation、occlusion 都比较敏感。

如果 *object* 在 *scene* 中大小或方向变化明显, 单一 *fixed template* 很难稳定检测。

这也是后续 *feature descriptors*、*learned representations*、*object detectors* 会发展的原因。

11 Summary: L2 的主线 p.91

- digital image 是 sampled and quantized 的矩阵表示。
- image-as-function 让 transformations 和 filtering 都能用数学形式描述。
- color space 决定如何解释 multi-channel values: RGB 设备友好, HSV 感知维度更直接。
- histogram 描述 gray-level distribution, 但丢弃 spatial structure.
- denoising 的 basic assumption 是 neighboring pixels similar 且 noise independent.
- cross-correlation / convolution 是局部线性 filtering 的核心形式。
- Gaussian smoothing 是 normalized low-pass filter; median filter 是 nonlinear edge-preserving spike remover.
- separable filters 用 rank-1 factorization 把 2D filtering 加速为两次 1D filtering.

filter applications 从 classical feature extraction 延伸到 CNN 和 template matching.

Concept Map.

L2 的逻辑可以压缩成一条链:

digital image $\rightarrow$ *image as function* $\rightarrow$ *local neighborhood operation* $\rightarrow$ *correlation / convolution* $\rightarrow$ *smoothing / sharpening / feature extraction*.

后续课程里的 *edges*、*keypoints*、*CNN*、*dense prediction* 都会反复使用这条链上的概念。

created: 2026-03-10

updated: 2026-05-04

1 Perspective Projection Geometry

1.1 Focal Length

- focal length 越大, view 越窄; 远处物体看起来被拉近。

1.2 Camera Coordinate System

- optical center / camera center *C* 放在 origin (0,0,0)。
- Z* axis 叫 optical axis / principal axis.
- X*, *Y* axes 与 image axes 平行。
- 使用 right-handed coordinate system.

1.2.1 Principal Point and Image Coordinates

- optical axis 与 image plane 的交点是 principal point *p*。
- camera center 到 principal point 的距离是 focal length *f*。
- image axes 记作 *x*, *y*, 这是 image coordinate system.

1.3 Projection Ray and Ambiguity

- 3D point *Q* 的坐标先假设在 camera coordinate system 中。
- 从 *Q* 到 camera center 的直线叫 projection line.
- projection line 与 image plane 的交点是 image point *q*。
- 同一条 projection line 上所有 3D points 都投影到同一个 *q*。

1.3.1 Depth Ambiguity

- 只看单张 image, 无法知道 point 沿 projection ray 距 camera 多远。
- 因此 single image 本身不能直接给出真实 3D size。
- 需要 scale reference 或额外几何约束。

1.4 Projection Equations

对 camera coordinate 中的 3D point $Q = (X, Y, Z)^T$, similar triangles 给出:

$$Q = (X, Y, Z)^T \mapsto \left(\frac{fX}{Z}, \frac{fY}{Z}, f \right)$$

如果 image coordinate origin 不在 principal point, 而在 image 的 (0,0), 则:

$$Q = (X, Y, Z)^T \mapsto \left(\frac{fX}{Z} + p_x, \frac{fY}{Z} + p_y \right)$$

- $p = (p_x, p_y)$ 是 principal point.
- 最后的 image point 要丢掉 depth coordinate.
- 因为有 division by *Z*, perspective projection 不是普通 linear transformation.

Math Derivation: Similar Triangles.

slides 中最关键的一步是用 *similar triangles* 得到 *perspective division*。

设 camera center 在 *origin*, image plane 到 camera center 的距离为 *f*, 3D point 为 $Q = (X, Y, Z)^T$ 。

$$\frac{x}{f} = \frac{X}{Z}, \quad \frac{y}{f} = \frac{Y}{Z}$$

所以相对 *principal point* 的 *image coordinate* 是:

$$x = \frac{fX}{Z}, \quad y = \frac{fY}{Z}$$

若 *image origin* 在左上角, 需要加上 *principal point* $p = (p_x, p_y)$:

$$q = \begin{bmatrix} fX/Z + p_x \\ fY/Z + p_y \end{bmatrix}$$

最后去掉 *depth coordinate*. 这里的 *division by Z* 是 *perspective projection* 的核心, 也不是 *linear* 的地方。

2 Homogeneous Coordinates and Intrinsics

2.1 Homogeneous Coordinates

- homogeneous coordinates 给 vector 末尾 append 一个 1。
- 在 projective geometry 中, scale 不改变 point:

$$(x, y, 1)^T \sim (sx, sy, s)^T$$

- 从 homogeneous coordinate 回到 ordinary coordinate 时, 要除以最后一个 coordinate.

2.2 Lines in Homogeneous Coordinates

- 2D line $ax + by + c = 0$ 可以写成 homogeneous vector $l = (a, b, c)^T$ 。
- point 写成 $x = (x, y, 1)^T$ 。
- point on line 的条件是:

$$l^T x = 0$$

- 给两个 points $x_1 = (x_1, y_1, 1)^T$ 和 $x_2 = (x_2, y_2, 1)^T$, 穿过它们的 line 是:

$$l = x_1 \times x_2$$

2.3 Perspective Projection as Matrix Multiplication

原来的 non-linear division 可以通过 homogeneous coordinates 推迟到最后一步:

$$\begin{bmatrix} fX + p_x Z \\ fY + p_y Z \\ Z \end{bmatrix} = \begin{bmatrix} f & 0 & p_x \\ 0 & f & p_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \end{bmatrix}$$

- matrix multiplication 先产生 homogeneous image coordinate.
- 最后再 divide by third coordinate, 得到 image point.

Math Derivation: Homogeneous Form.

从 *perspective projection* 开始:

$$q = \begin{bmatrix} fX/Z + p_x \\ fY/Z + p_y \\ 1 \end{bmatrix}$$

在 *homogeneous coordinates* 中, 可以把 *denominator Z* 移到整体 *scale* 里:

$$q \sim \begin{bmatrix} fX + p_x Z \\ fY + p_y Z \\ Z \end{bmatrix}$$

于是 *projection* 可以写成一次 *matrix multiplication*:

$$\begin{bmatrix} fX + p_x Z \\ fY + p_y Z \\ Z \end{bmatrix} = \begin{bmatrix} f & 0 & p_x \\ 0 & f & p_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \end{bmatrix}$$

读出 *actual image coordinate* 时, 再除以第三个 *coordinate*。

2.4 Camera Intrinsics

camera calibration matrix / intrinsic parameter matrix:

$$K = \begin{bmatrix} f & 0 & p_x \\ 0 & f & p_y \\ 0 & 0 & 1 \end{bmatrix}$$

投影可以写成:

$$x = KX$$

其中 *x* 是 homogeneous image coordinate, *X* 是 camera coordinate 中的 3D point.

2.4.1 Non-Square Pixels and Skew

更一般地, pixel 可能不是 square:

$$K = \begin{bmatrix} f_x & 0 & p_x \\ 0 & f_y & p_y \\ 0 & 0 & 1 \end{bmatrix}$$

如果 image axes 之间有 skew angle θ , *K* 还会包含 skew term.

Full Intrinsic Matrix from Slides.

square pixels 且无 *skew* 时:

$$K = \begin{bmatrix} f & 0 & p_x \\ 0 & f & p_y \\ 0 & 0 & 1 \end{bmatrix}$$

non-square pixels 时:

$$K = \begin{bmatrix} f_x & 0 & p_x \\ 0 & f_y & p_y \\ 0 & 0 & 1 \end{bmatrix}$$

若 *x*, *y* *image axes* 之间有 skew angle θ :

$$K = \begin{bmatrix} f_x & -f_y \cot \theta & p_x \\ 0 & f_y / \sin \theta & p_y \\ 0 & 0 & 1 \end{bmatrix}$$

Intrinsics.

internal camera parameters 描述 camera 自己的成像几何: *focal length*、*principal point*、*pixel aspect ratio*、*skew*, 它们不描述 camera 在 *world* 中的位置和朝向。

3 Projection Properties and Vanishing Points

3.1 Basic Projection Properties

- points map to points.
- lines map to lines.
- planes map to planes.
- plane through principal point 且 orthogonal to image plane 时, projects to a line.

3.2 Vanishing Point

- parallel lines 在 image 中会 converge at a vanishing point.

- world* 中每个不同 *direction* 都有自己的 *vanishing point*.
- all lines with the same 3D direction intersect at the same vanishing point.

3.2.1 Why Same Direction Gives One Point

设一条 3D line 为:

$$X = V + tD$$

将 $t \rightarrow \infty$ 后, image point 只依赖 *direction D*, 不依赖 line 上的起点 *V*。
Why Same Direction Gives Same Vanishing Point.
对 *direction* $D = (D_x, D_y, D_z)^T$ 的所有 *parallel lines*, 投影中当 $t \rightarrow \infty$ 时, 起点 *V* 的影响消失, 所以 *vanishing point* 只由 *D* 和 *K* 决定。

设 line 为 $X(t) = V + tD$, 其中 $V = (V_x, V_y, V_z)^T$, $D = (D_x, D_y, D_z)^T$ 。用

$$K = \begin{bmatrix} f & 0 & p_x \\ 0 & f & p_y \\ 0 & 0 & 1 \end{bmatrix}$$

投影得到 *homogeneous image point*:

$$\tilde{x}(t) = K(V + tD) = \begin{bmatrix} f(V_x + tD_x) + p_x(V_z + tD_z) \\ f(V_y + tD_y) + p_y(V_z + tD_z) \\ V_z + tD_z \end{bmatrix}$$

因此 *actual image coordinate* 是:

$$x(t) = \frac{f(V_x + tD_x) + p_x(V_z + tD_z)}{V_z + tD_z}$$

$$y(t) = \frac{f(V_y + tD_y) + p_y(V_z + tD_z)}{V_z + tD_z}$$

令 $t \rightarrow \infty$:

$$\lim_{t \rightarrow \infty} x(t) = \frac{fD_x + p_x D_z}{D_z} = \frac{fD_x}{D_z} + p_x$$

$$\lim_{t \rightarrow \infty} y(t) = \frac{fD_y + p_y D_z}{D_z} = \frac{fD_y}{D_z} + p_y$$

limit 中没有 *V*, 所以同一个 3D *direction* 的所有 *parallel lines* 会去同一个 *vanishing point*。

3.3 Finding 3D Direction from Vanishing Point

把具有 *direction D* 的 line 平移到 camera center, 这条 line 与 image plane 的交点, 就是该 *direction* 的 *vanishing point*。

3.3.1 Recovering Direction with *K*⁻¹

如果已知 camera intrinsic matrix *K*, vanishing point *x* 可以反推出 3D direction:

$$x = KD$$

$$D = K^{-1}x$$

- 只能确定方向向量!

Math Detail: Direction Is Up to Scale.

vanishing point 应写成 *homogeneous image point* $\tilde{x}_v = (u, v, 1)^T$ 。

$$\tilde{x}_v \sim KD$$

因此:

$$D \sim K^{-1} \tilde{x}_v$$

这里的 *D* 只确定到 *scale*; 如果需要 *physical direction*, 通常再 *normalize* 成 *unit vector*。

3.4 Lines Parallel to Image Plane

- 3D 中 parallel to image plane 的 lines, 在 image 中仍然 parallel.
- 考虑 *vanishing point*, 可理解为它们 intersect at infinity.

3.5 Vanishing Line and Horizon Line

- 对同一个 3D plane 上的 lines, 它们的 vanishing points lie on a line, 叫 vanishing line.
- ground plane 的 vanishing line 叫 horizon line.
- parallel planes in 3D have the same horizon line in the image.
- ** 考虑将平面平移到 camera center**

3.6 Height from Horizon

从 image 判断 camera 离 ground 多高。

- 当 camera image plane orthogonal to ground plane, 且 ground plane flat 时:
- camera height 等于 horizon intersects a building 的高度。
- 考虑地平面和当前的高度平面的 vanishing line 是一样的

4 Orthographic Projection

4.1 Definition

- orthographic projection 不做 division.
- 它直接 drop Z coordinate.

$$\begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

Math Detail: No Perspective Division.

orthographic projection 的重点是没有 Z -division.

$$(X, Y, Z, 1)^T \mapsto (X, Y, 1)^T$$

或写作:

$$\begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

所以 depth Z 不会影响 image coordinate; parallel lines 也不会因为 depth 改变而 converge.

4.2 Orthographic vs Perspective

- perspective projection 中, 3D parallel lines 在 image 中不一定 parallel.
- orthographic projection 中, 3D parallel lines 在 image 中仍然 parallel.
- orthographic model 适合 depth variation 相对小、object 离 camera 很远的场景近似。

5 Camera Extrinsics and Projection Matrix

5.1 Intrinsics vs Extrinsics

完整 projection 需要两类参数:

- internal parameters: camera intrinsics K .
- pose in the world: camera position and orientation.

两个 coordinate systems:

- world coordinate system.
- camera coordinate system.

5.2 Why World and Camera Coordinates

- scene 中的 3D points 通常按 world coordinate system 记录。
- projection 公式要求 points 在 camera coordinate system 中。
- 因此投影前要把 world coordinates transform 到 camera coordinates.

5.3 Camera Extrinsics

- extrinsics 描述 camera 在 world 中的 pose.
- rotation R 描述 camera orientation.
- translation t 描述 camera position / coordinate origin offset.
- final transformation 把 world coordinate 中的 point 转到 camera coordinate.

Coordinate Transform.

常见写法是 $X_C = RX_w + t$. 如果 camera center 在 world coordinate 中为 C , 也常写作 $X_C = R(X_w - C)$, 其中 $t = -RC$.

5.4 Projection Matrix P

projection matrix P 累积了 intrinsic 和 extrinsic parameters:

$$x = PX$$

其中 P 是 3×4 matrix, $X = (X, Y, Z, 1)^T$ 是 homogeneous 3D point, 计算 image point:

- compute PX , 得到 3×1 homogeneous vector.
- divide all coordinates by third coordinate.
- drop the last coordinate.

Math Derivation: Full Projection Matrix.

slides 把 projection matrix 分成 intrinsics, canonical projection, rotation, translation:

$$P = \underbrace{\begin{bmatrix} f & 0 & p_x \\ 0 & f & p_y \\ 0 & 0 & 1 \end{bmatrix}}_{\text{intrinsics } K} \underbrace{\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}}_{\text{projection } [I|0]} \underbrace{\begin{bmatrix} R_{3 \times 3} & t_{3 \times 1} \\ 0_{1 \times 3} & 1 \end{bmatrix}}_{\text{extrinsics}}$$

更常用地写成:

$$P = K[R \mid t]$$

对 homogeneous world point $X_w = (X, Y, Z, 1)^T$:

$$\tilde{x} = PX_w = \begin{bmatrix} uZ_c \\ vZ_c \\ Z_c \end{bmatrix}$$

再做 homogeneous normalization:

$$(u, v) = \left(\frac{\tilde{x}_1}{\tilde{x}_3}, \frac{\tilde{x}_2}{\tilde{x}_3} \right)$$

5.4.1 Compact Notation

compact notation:

$$P = K[R \mid t]$$

- 有时也写作 $P = K[R \ t]$.
- 实际 calibrated camera 通常会直接给出 P .
- 有了 P , projection 就只是 matrix multiplication 加 homogeneous normalization.

6 Calibration and Single-View Applications

6.1 Camera Calibration

general procedure:

- 放一个已知 3D geometry 的 pattern 在 camera 前。
 - 拍照。
 - detect image corners.
 - 建立 2D image points 与 3D pattern points 的 correspondences.
 - 由这些 2D-3D correspondences 计算 K .
- #### 6.2 Calibration from Vanishing Geometry
- 如果 image 中能看到 3 个 orthogonal directions 的 lines, 例如 cube 或 room.
 - 即使不知道 camera, 也可以估计 camera matrix K 、rotation R 和 translation t .
 - 这依赖 vanishing points 和 orthogonality constraints.

6.3 Single View Metrology

- 对 man-made scenes, image 中有大量 straight lines.
 - 利用 vanishing points, horizon line, orthogonality 等约束, 可以从 single image reconstruct 3D scene.
 - 相关方向: Single View Metrology.
- #### 6.4 Inserting Objects into Images
- 目标: 把 3D CAD model realistic 地 project 到 image 中。
 - 需要:
 - CAD model 的 vertices and faces.
 - camera intrinsics K .
 - object pose, 即 R 和 t .
 - ground plane / placement constraints.

6.4.1 Rendering Equation

rendering 的核心仍是 projection equation:

$$\begin{bmatrix} uZ \\ vZ \\ Z \end{bmatrix} = K[R \mid t] \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

然后 divide by third coordinate, drop homogeneous coordinate.

Math Detail: CAD Vertex Rendering.

对 CAD model 中每个 vertex $X_{cad} = (X, Y, Z, 1)^T$, 先用 object pose 给出该 vertex 相对 camera / world 的 R, t , 再投影:

$$\tilde{x} = K[R \mid t]X_{cad} = \begin{bmatrix} uZ \\ vZ \\ Z \end{bmatrix}$$

image coordinate 是:

$$(u, v) = \left(\frac{\tilde{x}_1}{\tilde{x}_3}, \frac{\tilde{x}_2}{\tilde{x}_3} \right)$$

这就是 slides 中 “divide with third coord and drop it” 的具体数学步骤。

6.5 From Fun to Useful

- 实际应用: generate 3D boxes and score them with image features.
- 例子: 3D Object Proposals for Accurate Object Class Detection.
- 这类方法把 camera geometry 与 object detection 结合起来。

created: 2025-10-02

updated: 2026-05-23

1 Stereo: 两张图用 correspondence 消除 depth ambiguity 2

1.1 Single-View Ambiguity: 一个 pixel 只确定一条 ray 2

- 单个 camera 里, 一个 image point p 对应一整条 projective ray.
- ray 上所有 3D points 都会投影到同一个 p .
- 所以 single image 不能直接恢复 depth: 这和 L3 的 depth ambiguity 是同一个问题。

1.2 Two Views: 第二张图把 ray 约束成 epipolar line 3

- 在 left camera 中经过 P 的 projective ray, 投到 right image 上会变成一条 line.
- 如果能在 this line 上找到 matching point, 就能用 two rays 做 triangulation.
- stereo 的核心流程:
- find correspondence;
- triangulate;
- recover 3D point.

Core Intuition.

单张图: 一个 pixel 对应一条 ray, depth 不确定。

两张图: 同一个 3D point 在两个 camera 里各给一条 ray; 两条 rays 的交点就是 3D point. 实际算法的难点不是 triangulation, 而是找到 reliable correspondence.

2 Parallel Calibrated Cameras: 已知 baseline 时 stereo 最简单 8

2.1 Baseline B : right camera 只是向右平移 8

- 假设两个 cameras 已经 calibrated; intrinsics 和 extrinsics 已知。
- right camera 只是相对 left camera 向右平移一段距离。
- 这段距离叫 baseline, 记作 B .
- 两个 optical axes parallel, 两个 image planes coplanar, 并且 focal length 为 f .

2.2 Horizontal Epipolar Lines: parallel stereo 中 $y_l = y_r$ 10

- 对 3D point P , 它在 left/right image 上的投影分别是 $p_l = (x_l, y_l)$ 和 $p_r = (x_r, y_r)$.
- parallel calibrated cameras 中, 对应点满足:

$$y_l = y_r$$

- 所以找 match 时不需要搜索整张 right image.
- 只需要沿同一条 scanline / horizontal line 搜索。

Why $y_l = y_r$.

C_l, C_r, P 构成一个 epipolar plane.

parallel stereo 中两个 image planes 在同一个平面姿态下, epipolar plane 与两个 image planes 的交线都是 horizontal line.

因此 left point p_l 在 right image 的候选 match 只能落在 $y = y_l$ 上。

2.3 Disparity: 越近 d 越大 35

- 对应点的 horizontal displacement 叫 disparity:

$$d = x_l - x_r$$

- closer objects have larger disparity.
- farther objects have smaller disparity.
- disparity map 给每个 pixel 一个 disparity value.

2.4 $Z = fB/d$: disparity 反推出 depth $\frac{f}{d} = \frac{B}{d}$ 16

从 birdseye view 看 parallel stereo, 可用 similar triangles 得到 depth:

$$Z = \frac{fB}{d}$$

其中:

- Z 是 point 到 camera 的 depth.
- f 是 focal length.
- B 是 baseline.
- $d = x_l - x_r$ 是 disparity.

因此 depth 和 disparity 成反比:

$$Z \propto \frac{1}{d}$$

Math Derivation: $Z = fB/d$.

设 left camera center 为 C_l , right camera center 为 C_r , baseline 为 B .

在 rectified stereo 中:

$$x_l = \frac{fX}{Z}$$

right camera 的坐标系相对 left camera 平移 B , 所以同一点的 right image coordinate 可写成:

$$x_r = \frac{f(X - B)}{Z}$$

两式相减:

$$d = x_l - x_r = \frac{fX}{Z} - \frac{f(X - B)}{Z} = \frac{fB}{Z}$$

所以:

$$Z = \frac{fB}{d}$$

这也是 slides summary 中最重要的公式。

2.5 Full 3D: 有了 Z 就能反推 X, Y 73

一旦有了 Z , 可以从 camera projection equation 反推 X, Y .

若 principal point 为 (p_x, p_y) :

$$X = \frac{(x_l - p_x)Z}{f}, \quad Y = \frac{(y_l - p_y)Z}{f}$$

- disparity 给 depth.
- depth 加上 image coordinate 给 full 3D point.
- 这就是 stereo reconstruction 的基本几何闭环。

3 Correspondence: stereo 的难点是可靠匹配 19

3.1 Scanline Search: parallel stereo 只沿同一行找 match 20

- 对每个 left point $p_l = (x_l, y_l)$, 目标是找 right point $p_r = (x_r, y_r)$.
- parallel stereo 中只在 $y_r = y_l$ 的 horizontal line 上搜索。
- patch around (x_l, y_l) 应该和 patch around (x_r, y_r) 相似。

3.2 Patch Matching: 对应点的局部窗口应相似 22

- 基本做法: 比较 local image patch.
- 常见 matching cost:
- SSD / SAD:
- normalized cross-correlation:
- learned matching cost.
- 2015 年以后可以训练 neural network classifier 来估计 stereo matching cost.

Matching Cost: SSD and NC.

设 left patch 以 $p = (x, y)$ 为中心, right patch 用 disparity d 对齐到 $p_d = (x - d, y)$.

window 记作 W , 其中一个 offset 为 $q = (u, v)$.

SSD / sum of squared differences:

$$\text{SSD}(p, d) = \sum_{q \in W} (I_l(p + q) - I_r(p_d + q))^2$$

SSD 越小, 两个 patches 越像。

NC / normalized correlation:

$\text{NC}(p, d) = \frac{\sum_{q \in W} (I_l(p + q) - \bar{I}_l) (I_r(p_d + q) - \bar{I}_r)}{\sqrt{\sum_{q \in W} (I_l(p + q) - \bar{I}_l)^2} \sqrt{\sum_{q \in W} (I_r(p_d + q) - \bar{I}_r)^2}}$ NC 越大, 两个 <i>patches</i> 越像; 实际作为 <i>cost</i> 时常用 $-\text{NC}$ 或 $1 - \text{NC}$。 I 只是 <i>image intensity function</i> / 图像灰度函数 <i>Learned Matching Cost</i>. <i>Slides</i> 提到 Zbontar and LeCun, “Computing the Stereo Matching Cost with a Convolutional Neural Network”, <i>CVPR 2015</i>。	Source: http://www.cvlibs.net/datasets/kitti/ 4 General Epipolar Geometry: 任意 camera pose 仍能把搜索限制到线 44 4.1 General Case: web photos 通常不是 parallel cameras 44 - 现实中不一定能把两个 cameras 固定成 parallel。 - web images、tourism photos、multi-view reconstruction 都是 arbitrary viewpoints。 - 因此需要 general epipolar geometry。 <i>Project: Photosynth / Photo Tourism</i>. 问题: 从网络照片或旅游照片中恢复 <i>3D scene</i>, 而这些照片通常来自不同 camera pose。
这里的思想是: 不是手写 <i>patch similarity</i>, 而是用 <i>CNN</i> 学习 “两个 <i>patches</i> 是否对应同一个 <i>3D point</i>”。 <i>Paper: MC-CNN Stereo Matching</i>. 问题: 传统 <i>stereo matching cost</i> 依赖手写 <i>patch similarity</i>, 遇到光照、纹理弱、遮挡时不稳。	方法: <i>Photo Tourism / Photosynth</i> 使用 <i>image matching</i>、<i>camera pose estimation</i>、<i>structure-from-motion</i>, 把 <i>unordered photo collections</i> 组织成可浏览的 <i>3D reconstruction</i>。
意义: 这是 <i>slides</i> 解释 <i>general epipolar geometry</i> 的动机: 现实 <i>multi-view images</i> 往往不是 <i>parallel stereo</i>。	Source: https://photosynth.net/ 4.2 Epipoles: 另一个 camera center 在当前 image 中的位置 51 - left camera center C_l 投影到 right image, 得到 right epipole e_r。 - right camera center C_r 投影到 left image, 得到 left epipole e_l。 - epipole 可以理解为 “另一个 camera center 在当前 image 里的位置”。 4.3 Epipolar Plane: C_l, C_r, P 决定两条 epipolar lines 55 C_l, C_r, P 三点确定一个 plane, 叫 epipolar plane。 - epipolar plane 与 left image plane 的交线是 left epipolar line。 - epipolar plane 与 right image plane 的交线是 right epipolar line。 - 所有 epipolar lines 都经过对应 image 的 epipole。 <i>Geometry Summary</i>. 对任意 <i>3D point P</i>:
意义: 这是 <i>slides</i> 中 “Version 2015: Train a Neural Network classifier” 的例子, 说明 <i>correspondence</i> 可以从 <i>handcrafted matching</i> 转向 <i>learned matching cost</i>。 3.3 Disparity Map, Patch Size Tradeoff: 小 patch 细但 noisy, 大 patch 稳但糊 37 - smaller patches: - 保留 more detail: - 但更 noisy, 容易 mismatch。 - bigger patches: - disparity map 更 smooth: - 但边界会被抹平, thin structures 容易丢。 3.4 Energy Minimization: 让 disparity map 更平滑 38 - 可以在 matching cost 上加 global optimization。 - 典型方法: Markov Random Field (MRF) / energy minimization。 - 直觉: 相邻 pixels 的 disparity 往往应该平滑, 但 depth discontinuity 处允许跳变。 <i>Stereo Energy</i>. <i>stereo</i> 常见 <i>energy</i> 形式:	C_l, C_r, P 在同一个 <i>epipolar plane</i> 上。
$E(d) = \sum_p D_p(d_p) + \lambda \sum_{(p,q) \in N} V(d_p, d_q)$ $D_p(d_p)$ 是 <i>data term</i>: 当前 <i>disparity</i> 是否让两个 <i>patches</i> match。	这个 plane 切 left/right image planes, 得到一对 <i>epipolar lines</i>。
$V(d_p, d_q)$ 是 <i>smoothness term</i>: 相邻 <i>pixels</i> 的 <i>disparity</i> 是否过度变化。	所以 left image 中的一个 <i>point</i>, 不会在 right image 中对应任意位置, 而只会对应到一条 <i>epipolar line</i> 上。 4.4 Epipolar Constraint: 2D search 降成 1D search 58 - parallel stereo 中, matching search 被限制在 horizontal line。 - general stereo 中, matching search 被限制在 epipolar line。 - 这把 2D search 降成 1D search。 - all matches lie on lines that intersect in epipoles, 这又给了额外几何约束。 5 Fundamental Matrix: 一个 3×3 matrix 描述 point-to-line 映射 62 5.1 General-Camera Pipeline: 先估 F, 再 rectify 后做 stereo 62 general cameras 的 stereo pipeline: - 先找一些 reliable image matches。 - 用 matches 估计 fundamental matrix F。 - 用 F 得到每个 point 的 epipolar line。 - 用 F 计算 rectification homographies, 把两张图变成 parallel / rectified。 - 之后沿 horizontal lines 做 stereo matching。 5.2 Definition of F: left point 映射到 right epipolar line 63 fundamental matrix F 把 left image point 映射到 right epipolar line:
Source: http://www.cs.toronto.edu/~urtasun/publications/yamaguchi_et_al_cvpr14.pdf	$l_r = F p_l$ p_l 是 left image 中的 homogeneous point。 l_r 是 right image 中的 homogeneous line。 F 是 3×3 matrix。 - 同一对 cameras 的所有 points 都共享同一个 F。 5.3 $l_r = F p_l, F = [e_r]_{\times} H \pi$: epipole 和 homography 构造 point-to-line map 64 <i>slides</i> 给出的构造直觉: - 把 left ray 延伸到任意 plane π, 得到 plane 上的 point X。 X 在 right image 中的投影是 p_r。 - right epipolar line 经过 e_r 和 p_r:

$l_r = e_r \times p_r$ plane-induced homography 给出:	$p_r = H \pi p_l$
所以:	$l_r = e_r \times H \pi p_l$
交叉乘可以写成 skew-symmetric matrix:	$l_r = [e_r]_{\times} H \pi p_l$
因此:	$F = [e_r]_{\times} H \pi$
<i>Cross Product Matrix</i>. 对 $e = (e_1, e_2, e_3)^T$:	$[e]_{\times} = \begin{bmatrix} 0 & -e_3 & e_2 \\ e_3 & 0 & -e_1 \\ -e_2 & e_1 & 0 \end{bmatrix}$
于是 $e \times x = [e]_{\times} x$。 5.4 $p_r^T F p_l = 0$ by true match 必须落在 epipolar line 上 67 如果 p_r 是 p_l 的 true match, 则 p_r 必须落在 right epipolar line l_r 上:	$p_r^T l_r = 0$
代入 $l_r = F p_l$:	$p_r^T F p_l = 0$
这是 fundamental matrix 最重要的约束。 <i>Main Thing to Remember</i>. 对任意 <i>matching pair</i> (p_l, p_r):	$p_r^T F p_l = 0$
这里 p_l, p_r 都是 <i>homogeneous image coordinates</i>, 例如 $p_l = (x_l, y_l, 1)^T$。	这条式子表达的是: right point p_r lies on the epipolar line generated by left point p_l。 5.5 Estimating F: 至少 7 个 reliable correspondences 68 设 matching points 为:
$(x_l, y_l) \leftrightarrow (x_r, y_r)$	令:
$F = \begin{bmatrix} f_{11} & f_{12} & f_{13} \\ f_{21} & f_{22} & f_{23} \\ f_{31} & f_{32} & f_{33} \end{bmatrix}$	把 $p_r^T F p_l = 0$ 展开, 得到一条关于 F entries 的 linear equation:
$x_r x_l f_{11} + x_r y_l f_{12} + x_r f_{13} + y_r x_l f_{21} + y_r y_l f_{22} + y_r f_{23} + x_l f_{31} + y_l f_{32} + f_{33} = 0$	F 有 9 个 entries, 但 scale 不重要, 所以自由度少 1。 <i>slides</i> 说: 本质上只需要至少 7 correspondences。 - 实际中 matches 越多越好, 然后用 least squares / robust fitting 估计。 <i>why only 7</i>. e_r the right epipole, we have that $e_r^t l_t = 0$. Therefore, $e_r^t (F p_l) = 0$ For all p_l. So we have that $e_r^t F = 0$. <i>Why More Matches Help</i>. 少量 matches 容易被 noise 和 outliers 影响。
更多可靠 matches 会形成 over-constrained system, 可以用 least squares 稳定估计 F。	如果有 outliers, 通常需要配合 RANSAC。

5.6 Rectification: 把 epipolar lines 拉成水平线 69

- 有了 F , 可以计算两个 homographies。
 - 这些 homographies 会把两张 image plane warp 成 parallel / rectified 的形式。
 - rectification 之后, epipolar lines 变成 horizontal lines。
 - 然后就回到 parallel stereo 的 matching problem。
- 6 Camera Matrices from F : relative pose 可从 correspondences 恢复 71**
- 6.1 Projective Ambiguity: 可选 $P_l = [I \mid 0]$ 和 $P_r = [[e_r]_{\times} F \mid e_r]$ 71**
- 有了 F , 可以在 projective ambiguity 下选择一组 camera projection matrices:

$P_l = [I_{3 \times 3} \mid 0]$ $P_r = \begin{bmatrix} [e_r]_{\times} F & e_r \end{bmatrix}$ - 这说明即使不知道两个 cameras 的 relative pose, 也能从 image correspondences 中恢复一组 projective cameras。 - 这对 web images / unordered photo collections 很重要。 <i>Why $P_r = [[e_r]_{\times} F \mid e_r]$.</i> 目标: 已知 fundamental matrix F, 构造一组 cameras P_l, P_r, 让它们产生同一个 epipolar geometry。	
---	--

先选择 left camera 为 canonical camera:	$P_l = [I \mid 0] = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}$
它把 <i>3D homogeneous point</i> $X = (X, Y, Z, W)^T$ 投影成:	$p_l \sim P_l X = (X, Y, Z)^T$
所以投影到同一个 left image point $p_l = (u, v, w)^T$ 的整条 <i>3D ray</i> 可以写成:	$X = \begin{bmatrix} p_l \\ \lambda \end{bmatrix} = (u, v, w, \lambda)^T$
因为:	$P_l \begin{bmatrix} p_l \\ \lambda \end{bmatrix} = p_l$

无论 λ 怎么变, left image 上仍然是同一个 p_l。	
令 right camera 先写成一般形式:	$P_r = [M \mid m]$
left camera center 是 P_l 的 null vector:	$C_l = (0, 0, 0, 1)^T, \quad P_l C_l = 0$
right epipole e_r 的定义是 left camera center 投影到 right image:	$e_r \sim P_r C_l$

若 $P_r = [M \mid m]$, 则:	$P_r C_l = [M \mid m] \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} = m$
因此可以取 $m = e_r$, 也就是:	$P_r = [M \mid e_r]$
现在看 left image 中同一个 p_l 对应的 <i>3D ray</i>. 把	

$X = \begin{bmatrix} p_l \\ \lambda \end{bmatrix}$	7 Summary: parallel stereo 用 disparity, general stereo 先估 F 73
投到 <i>right image</i> :	7.1 Parallel Cameras: $d = x_l - x_r$ 且 $Z = fB/d$ 73 - 已知 intrinsics / extrinsics, 且 optical axes parallel. - correspondences 沿 horizontal lines 搜索。 - disparity:
$p_r \sim P_r X = [M \mid e_r] \begin{bmatrix} p_l \\ \lambda \end{bmatrix} = Mp_l + \lambda e_r$	$d = x_l - x_r$
当 λ 改变时, 所有 <i>possible</i> p_r 都落在经过 Mp_l 和 e_r 的同一条 line 上。	depth:
所以 <i>right epipolar line</i> 是:	$Z = \frac{fB}{d}$
$l_r = e_r \times Mp_l = [e_r]_{\times} Mp_l$	- disparity and depth are inversely proportional. - 有了 depth, 就可以恢复 full 3D。
但 <i>fundamental matrix</i> 定义的是:	7.2 General Cameras: F 给出 epipolar constraint 并支持 rectification 74 - 先找 unconstrained image matches。 - 至少需要 7 reliable correspondences 来估计 F, 更多更稳。 - fundamental matrix 给出 epipolar constraint:
$l_r = Fp_l$	$p_r^T Fp_l = 0$
因此希望两者表示同一条 line:	- 给定 F, 可以: - compute epipolar lines; - rectify both images; - estimate camera projection matrices up to ambiguity; - 再进入 stereo matching / triangulation。
$[e_r]_{\times} Mp_l \sim Fp_l$	8 Beyond Two-View Stereo: 多视角和主动光也能恢复 depth 75
<i>slides</i> 选择一个方便的 <i>representative</i> :	8.1 Structure from Motion: 多于两张图时同时恢复 pose 和 structure 75 - 如果同一 scene 有 more than two views, 就是 structure-from-motion (SfM)。 - SfM 同时估计: - camera poses; 3D structure。 - 典型目标: minimize re-projection error。
$[e_r]_{\times} M = [e_r]_{\times} [e_r]_{\times} F$	8.2 Bundle Adjustment: 最小化 re-projection error 76 bundle adjustment 解一个 nonlinear optimization problem:
此时:	$\min_{P_i, X} \sum_i \sum_j \text{dist}(x_{ij}, P_i X_j)$
$[e_r]_{\times} M = [e_r]_{\times} [e_r]_{\times} F$	P_i 是第 i 个 camera projection matrix。 X_j 是第 j 个 3D point。 x_{ij} 是第 j 个 point 在第 i 张 image 中的 observation。 - objective 是让 projected point 尽量接近 observed image point。
用 <i>identity</i> :	8.3 Structured Light: 用 projector 简化 correspondence 77 - structured light 把已知 light pattern 投到物体上。 - 它本质上简化了 correspondence problem。 - 可以只用 one camera, 因为 projector 提供了另一种 “view” 或 coding。
$[e_r]_{\times} [e_r]_{\times} F = e_r(e_r^T F) - (e_r^T e_r)F = -(e_r^T e_r)F \sim F$	8.4 Kinect: structured infrared light 提供 depth cues 78 - Kinect 使用 structured infrared light。 - infrared pattern 给 surface points 编码。 - camera 观察 pattern deformation, 从而推断 depth。
又因为 $e_r^T F = 0$, 所以:	created: 2026-03-24 updated: 2026-05-23
$[e_r]_{\times} [e_r]_{\times} F = e_r(e_r^T F) - (e_r^T e_r)F = -(e_r^T e_r)F \sim F$	1 Local Feature
<i>epipolar line</i> 是 <i>homogeneous line</i> , <i>scale</i> 不重要, 所以这个 <i>construction</i> 与原来的 F 等价。	1.1 Raw patches as local descriptors - 无法使用 (encoding the pixels around) very sensitive to even small shifts, rotations
因此可以取:	1.2 SIFT (Scale Invariant Feature Transform) - Gaussian Smoothing “Pyramid” - Rotated Grid For Histogram - 首先找到主方向 (梯度), 其他点的方向转正之后, 每小格子进行统计, 获得 8d 向量。
$l_r = Fp_l$	16 格, 128d descriptor
且:	2 Keypoint Selection
$e_r^T l_r = 0$	2.1 Corner - Harris Corner Detector - pixel changes in all directions are large = corner - a possible tool: Orthogonal Diagonalization
所以对所有 p_l :	
$e_r^T Fp_l = 0$	
因此:	
$e_r^T F = 0$	
e_r 是 F^T 的 null vector。 - 类似地, e_l 是 F 的 null vector。	

$\Delta^2 = (D_x \Delta x + D_y \Delta y)^2$ $= \begin{pmatrix} \Delta x & \Delta y \end{pmatrix} \begin{pmatrix} D_x^2 & D_x D_y \\ D_x D_y & D_y^2 \end{pmatrix} \begin{pmatrix} \Delta x \\ \Delta y \end{pmatrix}$	2.2 Scale-Invariant Interest Points Laplacian of Gaussian => Approximated by Difference of Gaussian
3 Keypoint Matching	
4 Homography	
4.1 Basic Transformation	
4.2 Affine Transformations Parrallel Lines remain Parallel	
4.3 Projective Transformations Parallel lines do not necessarily remain parallel	
4.4 Image Mosaic 4.4.1 获得全景图片 - Image Reprojection - Mosaic is a synthetic wide-angle camera, we need to map two 2-D picture from a same center of projection!	
4.4.2 Homography lecture5 Keypoints, p.63 Matrix is equivalent under scale. Therefore, we need only 8 unknowns. - solve for $Ah = b$ where $h = [a, b, c, d, e, f, g, h]^T$ - If over-constrained, solve using least-squares: $\min \ Ah - h\ ^2$	
5 Robust Fitting with RANSAC	
5.1 Outliers may be an erroneous paor of matching points from two images. affect least squares fit	
5.2 RANSAC 2 fit lines 只选择 s 个点随机. 考虑他们的 line, 在考虑这条线能 fit 的点是否足够多, 如果是, 就用这些点 refit	<i>Notes.</i>
5.3 Trials	
5.4 RANSAC 2 find transformation lecture5 Keypoints, p.95 如果 transformation fit 的 match 足够多, 就 accept	
5.5 Pros and Cons pro: simple, applicaitons, works well in practice cons: doesn't work well for low inlier ratios (too many iterations, or can fail completely), Can' t always get a good initialization of the model based on the minimum number of samples. created: 2026-03-31 updated: 2026-05-23	
1 CNN: 从视觉皮层到卷积网络 3	
1.1 History: CNN 的核心直觉来自 local receptive field 3 - Hubel & Wiesel 研究 visual cortex: 神经元只对局部 visual field 有反应。 - topographical mapping / retinotopy: 皮层中相邻 cells 对应视觉场中相邻区域。 - 这对应 CNN 的两个关键 bias: - local connectivity: 每个 neuron 只看局部区域。 - spatial structure: 保留 image 的 $W \times H \times C$ 结构。	<i>Historical Path.</i> <i>Perceptron: Rosenblatt, 1957</i> , 第一次把 camera input 和 learning rule 接起来。
<i>Adaline/Madaline: Widrow and Hoff, 1960.</i>	
<i>Back-propagation: Rumelhart et al., 1986</i> , 让多层网络训练重新变得可行。	
<i>Deep learning revival: Hinton and Salakhutdinov, 2006.</i>	
1.2 Neocognitron 到 LeNet/AlexNet: CNN 的结构逐渐成型 10 - Neocognitron: Fukushima 1980。 ”sandwich” architecture: $SCSCSC \dots$。 - simple cells: modifiable parameters。 - complex cells: pooling。 - LeNet-5: LeCun et al. 1998, 用 gradient-based learning 做 document recognition。 - AlexNet: Krizhevsky, Sutskever, Hinton 2012, 让 CNN 成为 ImageNet classification 的关键转折点。	

1.3 Fully Connected Layer: 会打平 spatial structure 14 $32 \times 32 \times 3$ image 被 stretch 成 3072×1 vector。 - FC layer 的每个 output 是一行 weights 和 input 的 dot product。 - 问题: image 的 2D spatial layout 被打散; 局部邻接关系没有被显式利用。	
2 Convolution Layer: 保留 spatial structure 的局部线性算子 15	
2.1 Filter: 在空间上 slide , 沿 depth 全连接 16 - input volume 形状为 $32 \times 32 \times 3$。 - 一个 $5 \times 5 \times 3$ filter 在 image 上滑动。 - 每个位置计算一个 dot product:	$5 \cdot 5 \cdot 3 = 75$
- 加上 bias 后得到一个 activation value。 - filter 通常覆盖 input 的完整 depth。 - 例外: group conv / depthwise conv 会改变这个规则。	
2.2 Activation Map: 一个 filter 产生一张 map 19 - 一个 filter 扫过所有 spatial locations, 得到一张 activation map。 $32 \times 32 \times 3$ input, $5 \times 5 \times 3$ filter, 不加 padding 时 spatial size 变成 28×28。 - 多个 filters 会 stack 多张 activation maps。 - 因此 output depth 等于 filters 数量。	
2.3 ConvNet: CONV 和 activation function 交替堆叠 21 - ConvNet 可以看成一串 convolution layers。 - 每层通常跟一个 nonlinearity, 如 ReLU。 - 前几层学 low-level patterns, 后面逐渐组合成 higher-level features。	
2.4 Output Volume: 卷积层尺寸公式 28 假设 input 是 $W_1 \times H_1 \times C$, conv layer 有: - number of filters: K。 - filter size: F。 - stride: S。 - zero padding: P。 输出为 $W_2 \times H_2 \times K$:	$W_2 = \frac{W_1 - F + 2P}{S} + 1, \quad H_2 = \frac{H_1 - F + 2P}{S} + 1$
参数量:	$F^2 CK + K$
其中 $F^2 CK$ 是 filter weights, K 是 biases。	
2.5 Example: $32 \times 32 \times 3$ 输入和 10 个 5×5 filters 24 - input volume: $32 \times 32 \times 3$。 - filters: 10 个 5×5 filters。 - stride $S = 1$, padding $P = 2$。 输出 spatial size:	$\frac{32 + 2 \cdot 2 - 5}{1} + 1 = 32$
所以 output volume:	$32 \times 32 \times 10$
每个 filter 参数量:	$5 \cdot 5 \cdot 3 + 1 = 76$
整层参数量:	$76 \cdot 10 = 760$
2.6 Common Settings: padding 常用来保持 spatial size 29 $F = 3, S = 1, P = 1$: spatial size 不变。 $F = 5, S = 1, P = 2$: spatial size 不变。 $F = 5, S = 2, P = ?$: 用于 downsampling, padding 取能整除的值。 $F = 1, S = 1, P = 0$: 1×1 convolution。	
2.7 1×1 Convolution: 只混合 channel , 不混合 spatial neighborhood 30 1×1 conv 在每个 pixel 位置做 channel-wise dot product。 - 例: $56 \times 56 \times 64$ input, 用 32 个 $1 \times 1 \times 64$ filters。 - output 变成 $56 \times 56 \times 32$。 - 作用: 调节 channel dimension, 常用于 bottleneck / projection。	
2.8 Neuron View: conv layer 是局部连接 + 参数共享 32 一个 activation value 可以看成是一个 neuron 的输出。	

- 这个 neuron 只连接 input 的 5×5 local receptive field.
- 一张 28×28 activation map 是 28×28 个 neuron outputs.
- 同一张 map 内所有 positions 共享同一个 filter 参数。
- 若有 5 个 filters, 则同一 input region 会被 5 个不同 neurons 观察, 输出 volume 是 $28 \times 28 \times 5$ 。

3 Pooling, FC, Activation Functions 37

3.1 Pooling Layer: 降低 spatial size, 不改变 channel 数 37

- pooling 让 representation 更小、更 manageable。
- 每个 activation map 独立做 pooling。
- max pooling 例子: 2×2 filter, stride 2, 每个窗口取 max。

假设 input 是 $W_1 \times H_1 \times C$, pooling 有 spatial extent F 和 stride S :

$$W_2 = \frac{W_1 - F}{S} + 1, \quad H_2 = \frac{H_1 - F}{S} + 1$$

输出为:

$$W_2 \times H_2 \times C$$

参数量为 0。

3.2 Fully Connected Layer: 末端做全局决策 40

- FC layer 中每个 neuron 连接整个 input volume。
- 常出现在 classifier head。
- 与 conv 相比, FC 不再利用 local connectivity 和 weight sharing。

3.3 Activation Functions: nonlinearity 让网络能组合复杂函数 42

- 常见 activation functions:
- Sigmoid。
- tanh。
- ReLU。
- Leaky ReLU。
- Maxout。
- ELU。

3.4 Sigmoid: 可解释但训练不友好 44

- squashes numbers to range $[0, 1]$ 。
- 早期常被解释为 neuron firing rate。
- 三个问题:
- saturated neurons kill gradients。
- output not zero-centered。
- exp() 计算更贵。

3.5 ReLU: 现代 CNN 的默认选择之一 45

- 定义:

$$f(x) = \max(0, x)$$

- 正半轴不饱和, gradient 更稳定。
- computationally efficient。
- 实践中比 sigmoid/tanh 收敛更快。
- 缺点:
- output not zero-centered。
- 当 $x < 0$ 时 gradient 为 0, 可能出现 dead ReLU。

4 Training: Initialization, Normalization, Regularization 46

4.1 Constant Initialization: 所有 neuron 会学成一样 47

- 如果 W 初始化成 constant, 同层 neurons 对称。
- 它们收到相同 gradient, 更新后仍然相同。
- 结果: 多个 neurons 没有学习不同 features。

4.2 Small Random Initialization: 深层 activation 可能趋近 0 48

- 第一想法: 从 zero-mean Gaussian 采样 small random numbers。
- 例: standard deviation 为 10^{-2} 。
- 问题: 深层网络中 activations 会越来越小。
- 结果: gradients 也趋近 0, no learning。

4.3 Xavier Initialization: 让各层 activation scale 更稳定 50

- Xavier initialization 使用:

$$\text{std} = \frac{1}{\sqrt{D_{in}}}$$

- 目标: 让 forward pass 中各层 activation 的 scale 不至于爆炸或消失。
- 来源: Glorot and Bengio, AISTAT 2010。

4.4 Normalization: 训练中仍要控制 feature scale 51

- initialization 只保证初始状态。
- training 过程中 scale 会继续变化, 所以需要 normalization。
- 对单个 sample 的 feature dimension 做 normalize。

设 feature 数为 d :

$$\mu_L = \frac{1}{d} \sum_{j=1}^d x_j, \quad \sigma_L^2 = \frac{1}{d} \sum_{j=1}^d (x_j - \mu_L)^2$$

$$\hat{x}_j = \frac{x_j - \mu_L}{\sqrt{\sigma_L^2 + \epsilon}}, \quad y_j = \gamma \hat{x}_j + \beta$$

- γ, β 是 learnable affine parameters。
- 直觉: 先标准化, 再让模型学回需要的 scale/shift。

4.5 Beyond Training Error: 真正关心 generalization gap 52

- optimizer 能降低 training loss。
- 但目标是 new data 上的 error。
- 需要减少 train/val gap, 而不是只盯 training error。

4.6 Early Stopping: validation accuracy 下降时停止 53

- always track validation performance。
- 当 validation accuracy 下降时停止训练。
- 或者训练很久, 但保存 validation 上最好的 model snapshot。

4.7 Data Augmentation: 训练时主动制造变化 54

- 对 input image 做随机 transform, 但 label 不变。
- 常见组合:
- translation。
- rotation。
- stretching。
- shearing。
- lens distortions。
- 直觉: 让模型学到 task-relevant invariance。

4.8 Regularization in Practice: training 加噪, testing marginal-ize 56

- training: add random noise。
- testing: marginalize over the noise。
- examples:
- Dropout。
- Data Augmentation。
- Stochastic Depth。
- Random Crop。
- Mixup。

5 CNN Architectures: AlexNet, ResNet, DenseNet, Depthwise Separable Conv 57

5.1 ILSVRC: CNN 之后进入 depth revolution 58

- ImageNet Large Scale Visual Recognition Challenge 展示了 CNN 架构的快速演化。
- AlexNet 是第一个 CNN-based winner。
- 后续 VGG, GoogLeNet, ResNet 等把网络越堆越深。

5.2 AlexNet: ImageNet 2012 的转折点 59

- architecture 由 CONV, MAX POOL, NORM, FC 等模块组成。
- details:
- first use of ReLU。
- 使用 Local Response Normalization。
- heavy data augmentation。
- trained on GTX 580 GPU with only 3GB memory。
- network split across 2 GPUs。

5.3 ResNet: 用 residual mapping 解决深层优化问题 63

- plain CNN 堆得更深时, 56-layer model 在 training/test error 上都更差。
- 这不是 overfitting, 因为 training error 也更差。
- 假设: 问题是 optimization, deeper models harder to optimize。

Residual Mapping.

Deep model 理论上 representation power 更强。

若想至少不比 shallow model 差, 可以让新增 layers 学 identity mapping。

ResNet 不直接学 $H(x)$, 而是学 residual:

$$H(x) = F(x) + x$$

如果 $F(x) = 0$, 则 $H(x) = x$, block 就退化为 identity mapping。

5.4 ResNet Architecture: stack residual blocks 71

- 每个 residual block 通常有两个 3×3 conv layers。
- 周期性:
- double number of filters。
- spatially downsample by stride 2。
- activation volume 的 spatial dimension 减半。

- 开头有一个 conv stem。
- 末端没有大量 FC layers:
- last conv 后 global average pooling。
- 只保留 FC 1000 to output classes。
- 因此理论上可以处理 variable input image sizes。

5.5 Bottleneck Block: ResNet-50+ 的效率设计 76

- deeper networks 使用 bottleneck layer 提高效率。
- 典型结构:
- 1×1 conv: project to fewer channels。
- 3×3 conv: 在较少 feature maps 上计算。
- 1×1 conv: project back。
- 例: 先投影到 $28 \times 28 \times 64$, 再投回 $28 \times 28 \times 256$ 。

5.6 ResNet Results: 非常深的网络可以正常训练 78

- 能训练 152-layer ImageNet model 和 1202-layer Cifar model。
- deeper networks 重新获得更低 training error。
- ILSVRC 2015 classification winner: top-5 error 3.6%。
- 同时拿下 ILSVRC 和 COCO 2015 多项第一。

5.7 DenseNet: 每层连接到之后所有层 80

- dense block 中, 每层都 feedforward 连接到后续所有层。
 - 作用:
 - alleviates vanishing gradient。
 - strengthens feature propagation。
 - encourages feature reuse。
 - 课件 punchline:50-layer DenseNet 可以 outperform 152-layer ResNet。
- 5.8 Depthwise Separable Conv: 把 standard conv 分解成 depth-wise + pointwise 81
- standard conv 同时做 spatial mixing 和 channel mixing。
 - depthwise separable conv 分两步:
 - depthwise conv: 每个 channel 独立做 spatial conv。
 - pointwise conv: 用 1×1 conv 混合 channels。
 - 可以看成 tensor factorization / low-rank decomposition。

参数量从:

$$K^2 C^2$$

降低为:

$$K^2 C + C^2$$

- PyTorch 中 depthwise conv 可用 group conv 表达。
- MobileNet / Xception 使其流行, ConvNeXt 等现代实践继续采用。

6 Attention and Self-Attention 84

6.1 CNN Limitations: local receptive field 对 long-range interaction 不够直接 85

- CNN 的 receptive field 随 depth 增长。
- 但 long-range dependency 需要堆很多层。
- large kernels 可以缓解, 例如 RepLkNet 中 31×31 kernel。
- CNN 还隐含 translation equivariance。
- Transformer 的动机: global interaction, dynamic connections, flexible modeling。

6.2 Attention Layer: output 是 inputs 的 weighted sum 87

给定:

- query vector: $q \in \mathbb{R}^{D_Q}$ 。
- data vectors: $X \in \mathbb{R}^{N \times D_X}$ 。

计算:

$$e_i = f_{att}(q, X_i)$$

$$a = \text{softmax}(e)$$

$$y = \sum_i a_i X_i$$

- attention weights 表示 query 和每个 input 的 relevance。
- output 是所有 values 的 linear combination。

6.3 General Attention: 用 key/value projection 分离匹配和输出 88

给定 query matrix $Q \in \mathbb{R}^{N_Q \times D_Q}$ 和 data vectors $X \in \mathbb{R}^{N_X \times D_X}$:

$$K = XW_K, \quad V = XW_V$$

$$E = \frac{QK^T}{\sqrt{D_Q}}$$

$$A = \text{softmax}(E, \text{dim} = 1)$$

$$Y = AV$$

- K 用来算 similarity。
- V 用来被 weighted sum。

6.4 Self-Attention: 每个 input 同时产生 query, key, value 89

输入 $X \in \mathbb{R}^{N \times D_{in}}$:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

其中:

$$Q, K, V \in \mathbb{R}^{N \times D_{out}}$$

similarity:

$$E = \frac{QK^T}{\sqrt{D_Q}} \in \mathbb{R}^{N \times N}$$

attention:

$$A = \text{softmax}(E, \text{dim} = 1)$$

output:

$$Y = AV \in \mathbb{R}^{N \times D_{out}}$$

- 每个 output vector 都是所有 value vectors 的混合。
- 这是 set of vectors 内部的 global interaction。

6.5 Masked Self-Attention: 不允许看未来 token 90

- 用于 language modeling。
- 对不允许看的 positions, 把 similarity 改成 $-\infty$ 。
- softmax 后这些位置的 attention weight 变成 0。
- 直觉: 预测 next word 时, 当前位置不能偷看后面的词。

6.6 Multi-Head Self-Attention: 并行运行多个 attention heads 91

- H 个 self-attention heads 并行运行。
- 每个 head 有自己的 weights。
- head outputs stack 后, 再用 output projection 融合。
- 通常 head dimension:

$$D_H = \frac{D}{H}$$

给定 $X \in \mathbb{R}^{N \times D}$:

$$Q, K, V \in \mathbb{R}^{H \times N \times D_H}$$

$$E = \frac{QK^T}{\sqrt{D_Q}} \in \mathbb{R}^{H \times N \times N}$$

$$A = \text{softmax}(E, \text{dim} = 2)$$

$$Y = AV \in \mathbb{R}^{H \times N \times D_H}$$

reshape 后:

$$Y \in \mathbb{R}^{N \times H D_H}$$

最后:

$$O = YW_O \in \mathbb{R}^{N \times D}$$

6.7 Self-Attention 是 4 个 matrix multiplies 93

- QKV projection:

$$[N \times D][D \times 3H D_H] \rightarrow [N \times 3H D_H]$$

split and reshape:

$$Q, K, V : [H \times N \times D_H]$$

- QK similarity:

$$[H \times N \times D_H][H \times D_H \times N] \rightarrow [H \times N \times N]$$

- V-weighting:

$$[H \times N \times N][H \times N \times D_H] \rightarrow [H \times N \times D_H]$$

reshape:

$$[N \times H D_H]$$

- Output projection:

$[N \times HD_H][HD_H \times D] \rightarrow [N \times D]$	
- compute 随 sequence length 增长为 $O(N^2)$。 - full attention matrix 的 memory 也是主要瓶颈。	
6.8 FlashAttention: 不显式存完整 attention matrix 96	
- 若 $N = 100K, H = 64, H \times N \times N$ attention weights 会非常大。 - slide 给出的估计: 约 1.192 TB, 普通 GPU 放不下。 - FlashAttention 把 QK similarity 和 V-weighting 合并计算。 - 关键: 不存 full attention matrix。 - 结果: 大 N 更可行, memory 可降到 $O(N)$。	
7 Transformer and Vision Transformer 98	
7.1 Convolution vs Self-Attention: grid bias vs global vector interaction 98	
- convolution works on N-dimensional grids。 - self-attention works on sets of vectors。 - convolution: - 对长序列不友好, 需要很多层扩大 receptive field。 - outputs 可并行计算。 - self-attention: - 每个 output 直接依赖所有 inputs。 - highly parallel, 主要是 matrix multiplies。 - compute 主要来自 $O(N^2)$, memory 压力大。	
7.2 Transformer Block: interaction 只靠 self-attention 99	
- input: set of vectors x。 - output: set of vectors y。 - self-attention 是 vectors 之间唯一的 interaction。 - LayerNorm 和 MLP 都对每个 vector 独立作用。 - compute 主要来自 6 个 matmuls: 4 from self-attention。 2 from MLP。	
7.3 Transformer Scale: 结构没大变, 规模变得大 100	
- Transformer 是 identical Transformer blocks 的 stack。 - Original Transformer: 12 blocks, $D = 1024, H = 16, N = 512, 213M$ params。 - GPT-2: 48 blocks, $D = 1600, H = 25, N = 1024, 1.5B$ params。 - GPT-3: 96 blocks, $D = 12288, H = 96, N = 2048, 175B$ params。	
7.4 Vision Transformer: 把 image 切成 patches 当 token 101	
- input image 例: $224 \times 224 \times 3$。 - 切成 patches, 例如 $16 \times 16 \times 3$。 - 每个 patch flatten 后维度为: $16 \cdot 16 \cdot 3 = 768$	
- 通过 linear transform 投到 D 维。 - 这些 D-dim vectors 输入 Transformer。	
7.5 ViT Classification Head: pool patch outputs 再分类 102	
- Transformer 对每个 patch 输出一个 vector。 - 不使用 masking: 每个 image patch 可以看所有其他 patches。 - positional encoding 告诉 Transformer patch 的 2D position。 - 对 $N \times D$ patch vectors 做 average pooling 得到 $1 \times D$。 - 线性层 $D \rightarrow C$ 输出 class scores。	
7.6 Pre-Norm Transformer: LayerNorm 放进 residual branch 前 103	
- slide 先指出: LayerNorm outside residual connections 会让 model 难以学 identity function。 - solution: 把 LayerNorm 移到 Self-Attention 和 MLP 之前, 放在 residual connection 内。 - 这样 training 更 stable。	
7.7 RMSNorm: 用 root mean square 替代 LayerNorm 105	
- RMSNorm 用 Root-Mean-Square Normalization。 - input: x, shape D。 - output: y, shape D。 - weight: γ, shape D。	
$y_i = \frac{x_i}{\text{RMS}(x)} \cdot \gamma_i$	
$\text{RMS}(x) = \sqrt{\epsilon + \frac{1}{N} \sum_{i=1}^N x_i^2}$	
训练通常更 stable。	

7.8 Patch Embedding Limitation: ViT 会丢掉 translation equivariance 106

- patch embedding 把 image 变成 tokens。
- 这种做法弱化了 CNN 自带的 translation equivariance。
- 这也是后续 ConvNeXt 等工作重新思考 CNN design 的动机。

8 ConvNeXt: 把 Transformer design ideas 放回 CNN 108

8.1 Main Idea: Transformer ideas + CNN efficiency 108

- ConvNeXt: Liu et al., 2022。
- 目标:
 - bring Transformer design ideas into CNNs。
- keep a pure convolutional architecture。
- improve performance without attention。
- punchline:

ConvNeXt = Transformer ideas + CNN efficiency

8.2 Patchify Stem: 用 non-overlapping conv patch 替代传统 stem 109

- ResNet stem:
 - 7×7 conv, stride 2。
 - Max Pool。
- ConvNeXt stem:
 - 4×4 conv, stride 4。
- LayerNorm。
- key point: kernel size = stride, 因此得到 non-overlapping patches。

8.3 Inverted Bottleneck: 先 expand 再 project back 110

- structure:
 - Depthwise Conv。
 - 1×1 Conv, expand $4\times$ 。
 - 1×1 Conv, project back。
- changes:
 - expand channels then reduce。
 - use LayerNorm and GeLU。

8.4 Other Improvements:GeLU,LayerNorm,独立 downsampling 111

- activation function:
 - ReLU $\rightarrow$ GeLU。
- fewer activations, 只保留 one per block。
- normalization:
 - BatchNorm $\rightarrow$ LayerNorm。
- fewer normalization layers。
- separate downsampling layer:
 - ResNet: downsampling inside blocks。
 - ConvNeXt: independent layer between blocks。
- structure: LayerNorm $\rightarrow 2 \times 2$ Conv with stride 2。

8.5 ConvNeXt V2: architecture + training strategy co-design 112

- ConvNeXt V1:
 - improve CNN using Transformer design ideas。
- focus on architecture design。
- ConvNeXt V2:
 - combine architecture + training strategy。
- co-design with Masked Autoencoders (MAE)。

9 U-Net Family: dense prediction 中的 long skip architecture 114

9.1 U-Net Origin: 少量训练图像也要精细分割 114

- U-Net: Biomedical Image Segmentation, MICCAI 2015。
- source: https://arxiv.org/abs/1505.04597。
- authors 的 main idea:
 - "very few training images"。
 - "more precise segmentations"。
- high-resolution features combined with upsampled output。
- 没有 fancy stories, 核心就是 localization 需要高分辨率信息。

9.2 U-Net Intuition: images are multi-scale 115

- ResNet skip connections 主要服务 single-scale 或 down-sampling。
- U-Net 处理的是 down-process-up。
- dense prediction 需要 high-resolution output。
- long skip connection 把 encoder 的高分辨率 features 接到 decoder。
- U-Net 早于 ResNet, 和 Highway Networks 同期。

9.3 U-Net Descendants: dense prediction everywhere 116

- generative models:

- Stable Diffusion 使用 U-Net family 思想。
- U-ViT 虽然不是 CNN, 但 still U-Net。
- long skip connection 对 diffusion-based image modeling 很关键。
- language models:
 - language modeling 也可看作 dense prediction。
- H-Net: Dynamic Chunking for End-to-End Hierarchical Sequence Modeling。

- byte-in-byte-out autoregressive LM。
- point cloud:
 - PointNet++。

U-ViT Slide Quote.

课件强调: *for diffusion-based image modeling, long skip connection 是 crucial.*

CNN-based U-Net 的 downsampling / upsampling operators 并不总是必要; 真正持续发光的是 long skip.

created: 2026-05-23

updated: 2026-05-23

1 Recognition Tasks: 从图像里识别什么 p.2

1.1 一张 tourist photo 可以对应很多任务 p.2

- recognition 不只是给整张图一个 label。
- 同一张图可以问很多层次的问题:
 - Identification: 这是什么 landmark? p.3
- Scene classification: 这是哪类 scene? p.4
- Object classification: window 里是 person / car / ...? p.5
- Image annotation: 场景里有哪些 object types? p.6
- Detection: 某类 objects 在哪里? p.7
- Segmentation: 哪些 pixels 属于每个 object class? p.8
- Pose estimation: 每个 object 的 pose 是什么? p.9
- Attribute recognition: 颜色、大小等 attributes。p.10
- Action recognition: 图像里正在发生什么? p.11
- Surveillance / reasoning: 为什么会发生? p.12

2 Image Classification: 固定 label set 下的核心任务 p.14

2.1 Semantic Gap: 像素 tensor 到语义 label 的距离 p.15

- computer 看到的是整数 tensor, 例如 $800 \times 600 \times 3$, RGB 值在 $[0, 255]$ 。
- human 看到的是 cat / dog / truck / landmark 等 semantic category。
- image classification 的核心难点: 从 low-level pixels 学到 high-level semantics。

2.2 Classification 的视觉挑战 p.16

- viewpoint variation: camera 一动, 所有 pixels 都会变。p.16
- background clutter: 目标和背景混在一起。p.17
- illumination: 同一物体在不同光照下像素差异很大。p.18
- occlusion: 目标部分被遮挡。p.19
- deformation: 非刚体形变会改变局部结构。p.20
- intraclass variation: 同一 class 内部差异很大。p.21

2.3 Supervised Learning Pipeline p.23

- 很难 hard-code 一个 algorithm 来识别 cat。
- supervised classification 的基本流程:
 - collect images and labels;
 - train a classifier;
 - evaluate on new images。
- 问题: large-scale training 需要大量人工 labeled data。p.25

3 Self-Supervised Learning: 不用人工 label 学 representation p.25

3.1 Pretext Task and Downstream Task p.26

- pretext task: 由 data itself 自动生成监督信号。
- 不需要 manual annotation, 但训练目标仍然可以是 supervised objective, 例如 classification / regression。
- downstream task: 真正关心的应用任务, 通常数据更少, 但有 labels。
- 直觉: 先从大量无标注数据学 feature, 再把 feature 迁移到具体任务。

Formal Distinction.

Pretext task 的 labels/outputs 自动从数据生成。p.27

<i>Downstream task 使用你关心的 labeled dataset. p.28</i>

3.2 Inpainting: 预测缺失像素 p.30

- 一个早期 pretext task: mask 掉图像中间区域, 让模型 reconstruct missing pixels。
 - 如果模型能补全缺失区域, 说明它必须学到 object, texture, scene context。
- 代表工作: Context Encoders, Pathak et al., 2016。
- 局限: learned representation 可能被某个特定 pretext task 绑住。

3.3 MAE: 大比例 mask 的重建式 self-supervised learning p.35

- MAE = Masked Auto Encoder。

- idea: 像 ViT 一样把 image 切成 non-overlapping patches。
- uniform sample 很大比例 patches 做 mask, 典型比例是 75%。p.37
- 高 mask ratio 让任务更难, 也更有意义。

3.4 MAE Encoder: 只看 visible patches p.38

- encoder 只处理 unmasked patches, 典型是 25%。
- 每个 visible patch 经过 linear projection, 加 positional embedding。
- 用 Transformer blocks 编码。
- 因为 token 少数, encoder 可以做得很大。

3.5 MAE Decoder: 只为 reconstruction 服务 p.39

- decoder 把 encoder outputs 和 shared mask tokens 合并。
- mask tokens 放回原来被 mask 的位置, 并加 positional encoding。
- decoder 使用 Transformer blocks, 再接 linear projection 重建 pixels
- **post-training 后 decoder 不再使用; 真正保留的是 encoder**。**
- 这是 asymmetrical autoencoder design: **** 大 encoder, 小 decoder**。**

3.6 MAE Reconstruction Loss p.40

- reconstruction target 是 pixel space。

- loss 用 MSE:

$$\mathcal{L}_{\text{MAE}} = \frac{1}{|\mathcal{M}|} \sum_{i \in \mathcal{M}} \|x_i - \hat{x}_i\|_2^2$$

- 只在 masked patches 上算 loss。
- ablation 关注 masking ratio, decoder depth/width, mask token, reconstruction target, data augmentation, mask sampling method, training schedule。p.41

4 Contrastive Representation Learning: 拉近正样本, 推远负样本 p.46

4.1 General Formulation p.47

- reference sample: x 。
- positive sample: x^+ , 应该和 x 表示同一语义。
- negative sample: x_j^- , 应该和 x 语义不同。
- 给定 score function $s(\cdot, \cdot)$, 学习 encoder f :
- positive pair (x, x^+) score 高;
- negative pair (x, x^-) score 低。

4.2 InfoNCE-style Loss p.48

给定 1 个 positive 和 $N - 1$ 个 negative samples:

$$L = -\mathbb{E}_X \left[\log \frac{\exp(s(f(x), f(x^+)))}{\exp(s(f(x), f(x^+))) + \sum_{j=1}^{N-1} \exp(s(f(x), f(x_j^-)))} \right]$$

- 分子: positive pair 的 similarity。
- 分母: positive 加所有 negatives 的 similarity。
- loss 最小化会把 positive 拉近, 把 negatives 推远。

4.3 SimCLR: augmentation 生成 positive pairs p.49

- score function 用 cosine similarity:

$$s(u, v) = \frac{u^T v}{\|u\| \|v\|}$$

- 用 projection network $g(\cdot)$ 把 feature 投到 contrastive space。
- 从同一张 image 生成两个 random augmented views:
 - random cropping;
 - random color distortion;
 - random blur。
- $f(\cdot)$ 产生 representation h_i, h_j 。
- $g(\cdot)$ 产生 contrastive embeddings z_i, z_j 。
- 训练目标: maximize agreement between augmented views。

4.3.1 SimCLR Training Pipeline p.49

给定一张 image x :

$\tilde{x}_i = t(x), \quad \tilde{x}_j = t'(x)$

- t, t' 是两次随机 data augmentation。
 - $\tilde{x}_i, \tilde{x}_j$ 来自同一张原图, 所以是 positive pair。
 - batch 里其它 images 的 views 都作为 negatives。
- encoder 和 projection head:

$h_i = f(\tilde{x}_i), \quad h_j = f(\tilde{x}_j)$
$z_i = g(h_i), \quad z_j = g(h_j)$

- f : 真正想学的 visual encoder。

- h : 通用 image feature, 留给 downstream tasks.
 - g : projection head, 通常是小 MLP, 只服务 contrastive objective.
 - z : 训练时拿来算 contrastive loss 的 embedding.
- 4.3.2 NT-Xent Loss:** 每个 **view** 都要找回自己的 **positive p.48**
- 对 positive pair (i, j) :

$$\ell_{i,j} = -\log \frac{\exp(s(z_i, z_j)/\tau)}{\sum_{k \neq i} \exp(s(z_i, z_k)/\tau)}$$

- τ 是 temperature.
 - 分子: 当前 view i 和它的 positive view j .
 - 分母: 当前 view i 和 batch 内所有其它 views.
 - loss 让 positive pair 更近, 让 negatives 更远.
 - batch 越大, negative samples 越多, contrastive signal 越强.
- 4.3.3 为什么 downstream 用 h , 不是用 z p.49**
- $z = g(h)$ 是为了让 contrastive loss 更容易优化.
 - projection head 可能会丢掉一部分 downstream task 需要的信息.
 - 所以训练完通常丢掉 g , 保留 encoder f .
 - downstream feature 取:

$$h = f(x)$$

- 直觉:
 - h : 更通用的视觉表示.
 - z : 为拉近/推远这个训练目标定制过的空间.
- Why Projection Head Helps.*
- SimCLR* 的经验结论是: 在 h 后面加一个 *non-linear projection head g* , *contrastive pretraining* 会更好.

- 但 *downstream evaluation* 使用 h 而不是 z . 这说明 *contrastive objective* 需要一个“训练用空间”, 而通用 *feature* 最好留在 *encoder output*.
- 4.4 Linear Evaluation: 冻结 encoder 后训练线性分类器 p.51**
- 先在 ImageNet 上用 SimCLR 训练 feature encoder.
 - 冻结 encoder.
 - 用 labeled data 在 feature 上训练 linear classifier.
 - 这用来评估 representation 是否可线性分离.
- 4.4.1 h 作为 downstream feature 的三种用法 p.51**
- Linear evaluation:**

$$h = f(x), \quad \tilde{y} = Wh + b$$

- 冻结 f .
- 只训练 W, b .
- 如果 linear classifier 表现好, 说明 h 已经编码了 class-level semantics.

Fine-tuning:

$$\tilde{y} = \text{Head}(f(x))$$

- 保留 f , 接 task-specific head.
- f 和 head 一起更新.
- 可用于 classification、detection、segmentation 等.

Feature extraction:

$$h_i = f(x_i)$$

- 把 f 当固定 feature extractor.
 - downstream 可以做 image retrieval、kNN classification、clustering、feature visualization.
 - retrieval 时常直接比较 h_i, h_j 的 cosine similarity.
- 4.5 MoCo: 用 queue 扩大 negative samples p.52**
- SimCLR 需要大 batch 才有很多 negatives.
 - MoCo 用 running queue of keys 保存 negative samples.
 - query encoder 正常 backprop.
 - key encoder 不直接 backprop, 标成 no grad.
 - 这样 decouple mini-batch size 和 number of keys.
- 4.6 MoCo Momentum Encoder p.53**
- key encoder 通过 momentum update 缓慢跟随 query encoder:

$$\theta_k \leftarrow m\theta_k + (1 - m)\theta_q$$

- θ_q 是 query encoder 参数.
 - θ_k 是 key encoder 参数.
 - m 越接近 1, key encoder 变化越慢, queue 中 keys 更稳定.
- Why Queue + Momentum Encoder.*

为什么不直接从 *dataset* 随机取几张图当 *negatives*?

contrastive loss 需要的是 *feature-level negative keys*, 不是 *raw images*:

$$k_i^- = f(x_i^-)$$

如果每个 *step* 都额外随机取很多 *images* 并重新 *forward*, 计算会很贵。

如果提前把整个 *dataset* 的 *features* 算好, 这些 *features* 又会随着 *encoder* 更新而 *quickly stale*.

MoCo queue 的折中: 复用最近 *mini-batches* 已经算好的 *keys*.

为什么 *momentum encoder* 会变化慢?

key encoder 不通过 *loss* 直接 *backprop*. 而是用 *EMA* 跟随 *query encoder*:

$$\theta_k^{(t)} = m\theta_k^{(t-1)} + (1 - m)\theta_q^{(t)}$$

若 $m = 0.999$, 每一步只吸收 0.1% 的新 *query encoder* 参数.

展开后:

$$\theta_k^{(t)} = (1 - m)\theta_q^{(t)} + m(1 - m)\theta_q^{(t-1)} + m^2(1 - m)\theta_q^{(t-2)} + \dots$$

所以 *key encoder* 是很多历史 *query encoders* 的加权平均, 像 *low-pass filter*.

直觉: *query encoder* 学得快, 负责被 *gradient* 推进; *key encoder* 跟得慢, 负责给 *queue* 提供稳定的 *keys*.

4.7 MoCo v2: SimCLR + MoCo 的混合 p.54

- 来自 SimCLR:
- non-linear projection head;
- strong data augmentation.
- 来自 MoCo:
- momentum-updated queue;
- 支持大量 negative samples;
- 不依赖超大 TPU batch.

4.8 DINO: Self-Distillation with No Labels p.55

- DINO 使用 student / teacher networks.
- teacher 通过 EMA 从 student 更新, 不直接 backprop. p.56
- loss 让 student prediction 匹配 teacher prediction.
- teacher output 会做 centering + sharpening, 避免 collapse.

$$\theta_{\text{teacher}} \leftarrow \tau\theta_{\text{teacher}} + (1 - \tau)\theta_{\text{student}}$$

- DINO 的一个现象: ViT attention map 可以自动聚焦到 object regions, 即使没有 label.
- DINO Pseudocode Sketch.*
- 输入 *image x* , 生成两个 *random views x_1, x_2* .

student 输出 s_1, s_2 , *teacher* 输出 t_1, t_2 .

loss 交叉匹配:

$$\mathcal{L} = \frac{1}{2}H(t_1, s_2) + \frac{1}{2}H(t_2, s_1)$$

teacher output stop-gradient, *teacher params* 用 *EMA* 更新. p.57

4.9 DINOv2: 更大数据、更强局部表示 p.58

- DINOv2 的 PCA visualization 显示 patch features 可以捕捉 object parts.
- 对 pose、style、甚至 object instance 变化, 相关 parts 仍然能匹配. p.58
- 和 DINO 相比:
- data pipeline: 从 curated ImageNet 扩展到 142M images;
- objective: global DINO + local masked image modeling / iBOT;
- scaling: ViT-g 级别, 配合 FlashAttention 和 FSDP. p.59

5 JEPA: 在 representation space 做预测 p.61

5.1 Contrastive vs Regularized Methods: 不用 negatives 也要防止 collapse p.61

- contrastive methods 的基本思路:
- training samples 应该落在 low-energy regions;
- suitably-generated contrastive samples 应该被推到 high-energy regions;

- negatives 越多, 区分信号越强, 但随 dimension 增大 scaling 很差.
- regularized methods 的基本思路:
- 不显式构造大量 negatives;
- 用 regularizer 限制 low-energy space 的体积;
- 让 representation 保留信息, 同时避免所有输入 collapse 到同一个表示.

Contrastive vs Regularized Intuition.

contrastive learning 像是在说: “这个 *positive* 要近, 那些 *negatives* 要远.”

regularized learning 更像是在说: “我不一定列出所有 *negatives*, 但我要限制模型不能把所有东西都映射到同一个低能量区域.”

两者都在解决同一个底层问题: 学 *representation* 时, 不能让模型走捷径. *contrastive methods* 用 *negatives* 防捷径; *regularized methods* 用信息量约束、防 *collapse regularizer* 防捷径.

5.2 JEA: Joint Embedding Architecture p.62

- JEA 对两个相关输入 x, y 分别计算 abstract representations:

$$s_x = \text{Enc}_x(x), \quad s_y = \text{Enc}_y(y)$$

- 目标是在 representation space 对齐 s_x 和 s_y .
- 它不在 pixel space 比较 x, y , 而是在 embedding space 比较抽象表示.
- 直觉: 如果 x, y 是同一对象、同一场景、相邻时刻或相关区域, 它们的 embeddings 应该有可预测关系.

5.3 JEPA: Joint Embedding Predictive Architecture p.63

- JEPA 在 JEA 基础上加入 predictor.
- 它不是直接让 $\text{Enc}_x(x)$ 和 $\text{Enc}_y(y)$ 靠近, 而是从 x 的表示预测 y 的表示:

$$s_x = \text{Enc}_x(x), \quad s_y = \text{Enc}_y(y)$$

$$\hat{s}_y = \text{Pred}(s_x, z)$$

$$\mathcal{L}_{\text{pred}} = \|\hat{s}_y - s_y\|^2$$

- z 是可选 latent variable, 用来表达预测中的不确定性.
 - 关键点: prediction 发生在 representation space, 不发生在 raw input space.
 - $\text{Enc}_y(y)$ 可以通过 invariances 去掉 irrelevant details, 因此 predictor 不需要预测 y 的所有像素细节.
- 5.3.1 JEPA 不做 pixel reconstruction p.63**
- MAE-style reconstruction (Masked Autoencoder):

$$\text{visible patches} \rightarrow \text{missing pixels}$$

- JEPA-style prediction:

$$\text{context representation} \rightarrow \text{target representation}$$

- JEPA 关心 target 的 abstract semantics, 而不是每个 low-level detail.
 - 这让模型更像在学 world structure, 而不是在补纹理.
- 为什么不直接预测 *pixels*.
- pixel-level prediction* 会逼模型猜很多对语义不重要的东西:

- 精确纹理;
- 背景细节;
- 光照和颜色;
- *sensor noise*;
- 多种可能但都合理的局部细节.

这些细节很难预测, 也不一定服务 *downstream recognition*.

JEPA 把 *target* 先编码成 $\text{Enc}_y(y)$, 让 *encoder* 自己通过 *invariance* 去掉无关细节. 于是 *predictor* 学的是“缺失部分在语义表示上应该是什么”, 而不是“每个 *pixel* 应该是什么值”.

5.3.2 JEA 和 JEPA 的区别 p.62

- JEA: 两个 embeddings 对齐.

$$\text{Enc}_x(x) \approx \text{Enc}_y(y)$$

- JEPA: 从一个 embedding 预测另一个 embedding.

$$\text{Pred}(\text{Enc}_x(x), z) \approx \text{Enc}_y(y)$$

- JEA 更像 matching / alignment.
- JEPA 更像 predictive world modeling.

JEPA 的“预测”为什么重要.

如果只是让两个 *views* 的 *embeddings* 靠近, 模型主要学习 *invariance*.

如果要求从 *context* 预测 *target representation*, 模型还要学习 *relation*: 哪些上下文会导致哪些目标表示.

对 *image / video / robot learning* 来说, 这种 *relation* 更接近“world model”: 看到一部分世界后, 预测另一部分或未来状态的抽象表示.

5.4 Training a JEPA: prediction + information constraints p.64

training cost 中有四类目标:

- maximize information content in representation of x ;
- maximize information content in representation of y ;
- minimize prediction error;
- minimize information content of latent variable z .

5.4.1 四个目标分别在防什么 p.64

- $\text{Enc}_x(x)$ 要有信息: 否则 context representation 没法支持预测.
- $\text{Enc}_y(y)$ 要有信息: 否则 target representation 会 collapse 成常量.
- prediction error 要小; 否则 x 和 y 的结构关系没有被学到.
- z 的信息量要小; 否则 predictor 可以把所有答案藏进 z , 绕过从 x 学结构.

为什么 latent variable z 不能太强.

z 的作用是表达 *ambiguity*.

例如只看到一个图像局部时, 缺失区域可能有多种合理解释. z 可以帮助表示这种不确定性.

但如果 z 容量太大, 模型可能把 *target* 的答案全塞进 z , predictor 就不需要从 x 学任何规律.

所以 *JEPA* 的训练目标会限制 z 的 *information content*: 允许它表达不确定性, 但不能让它成为作弊通道.

5.5 JEPA 和 H-JEPA 的边界 p.65

- JEPA 不是 generative model;
- prediction 发生在 representation space;
- 它不直接生成 y .
- JEPA 不是 probabilistic model;
- y encoder 通常不可逆;
- 没有简单方式得到 normalized distribution $p(y|x)$.
- H-JEPA 是 hierarchical;
- latent variables 会送到下一层;
- 低层可以预测局部/短期结构;
- 高层可以预测更抽象/长期结构.

为什么 *JEPA* 不是 *generative model*.

generative model 通常要能产生 *data* 本身, 或者定义 $p(y|x)$.

JEPA 只预测 $\text{Enc}_y(y)$. 如果 Enc_y 丢掉了颜色、纹理、噪声等细节, 那么从 $\hat{s}_y$ 无法唯一还原 y .

这正是 *JEPA* 的设计目的: 不能把力浪费在恢复所有细节, 而是学 *abstract, predictable representations*.

5.6 Representation Collapse 和 SIGReg p.66

- JEPA 只做 representation prediction 会有一个 trivial solution: 所有输出都输出同一个表示.
- SIGReg 的作用是强制 latent space 保持足够的分布结构和方差, 防止这种 collapse.
- collapse: 所有 inputs 被映射到 constant 或 low-dimensional representation.
- 这样 prediction loss 可以很低, 但 representation 没有信息.
- LeWorldModel / LeWM 的思路: prediction loss + SIGReg regularizer. p.66
- SIGReg 强制 latents 接近 Gaussian distribution. p.67
- 用 random projections 和 Epps-Pulley test 检查 normality.
- 目标: 保留 variance, 防止 collapse.

Collapse Intuition.

如果所有 y 都被编码成同一个常量 c :

$$\text{Enc}_y(y) = c$$

predictor 只要永远输出 c :

$$\hat{s}_y = c$$

prediction loss 就可以很低。

但这样的 representation 完全没有区分能力。它没有学到 object、scene、motion 或 world structure，只是学会了一个 trivial solution。

SIGReg 的直觉。

SIGReg 的目标不是让模型记住 labels，而是强制 latent features 保持统计上的“展开”。

如果 features collapse，variance 会消失，分布不可能像 Gaussian。

SIGReg 通过 random projections 检查特征分布，并用 Epps-Puley test 约束 normality。直觉上，它要求 representation 在 latent space 中保留足够方差和形状，从而阻止所有样本挤到一个点。

6 Vision-Language Representation Learners p.69

6.1 从 fixed label classification 到 zero-shot generalization p.70

- supervised classifiers 预测固定 object categories。
- 这些 categories 通常是 one-hot labels，例如 ImageNet-1k。p.70
- 问题：fixed label set 限制了模型能说什么。
- self-supervised models 如 MoCo、SimCLR、MAE 不用人工 label 学 feature。p.71
- 常规流程：
- pre-training on large-scale data；
- fine-tuning on task-specific data；
- deployment。p.72

vision-language representation learner 的目标：把 fine-tuning 前移到大规模 image-text pretraining 中，让模型能 zero-shot 迁移。为什么 language 能带来 zero-shot。

传统 classifier 的输出维度是固定的，例如 1000 个 ImageNet classes。

language encoder 让类别变成 text prompt，例如 “a photo of a cat”、“a photo of a medical X-ray”。

这样新类别不需要重新训练分类头，只要写成 text，就能和 image embedding 比较。

6.2 CLIP: Contrastive Language-Image Pre-training p.76

- CLIP 把 image 和 language 放进同一个 embedding space。
- data: 400M image-text pairs from Internet。
- model: ResNets / ViTs，最大约 300M parameters。
- loss: contrastive loss。
- compute: 250-600 GPUs，最多 18 days。
- Internet 上天然有很多 image-caption pairs。p.77

6.2.1 CLIP 的训练对象是 pair，不是 class label p.76

- supervised ImageNet:

image $\rightarrow$ one-hot class

- CLIP:

(image, text) $\rightarrow$ matched pair

- 训练信号不再是人工固定类别，而是 image-text 是否匹配。
- 因为 text 可以描述 open vocabulary，所以迁移能力更强。

6.3 CLIP Contrastive Objective: batch 内做 image-text matching p.78

- image encoder 产生 image embedding。
- text encoder 产生 text embedding。
- 正样本：同一 pair 的 image/text。
- 负样本：wrong text 或 wrong image。p.78
- objective:

- image 要靠近正确 caption；
- text 要靠近正确 image；
- batch 内其他 image/text pair 被推远。p.79

6.3.1 相似度矩阵视角 p.78

给定 batch 中 N 对 image-text pairs:

$$I_i = \text{fimg}(x_i), \quad T_j = \text{ftext}(t_j)$$

构造相似度矩阵:

$$S_{ij} = I_i^T T_j$$

- diagonal S_{ii} 是 matched pair。
- off-diagonal $S_{ij}, i \neq j$ 是 mismatched pairs。
- image-to-text loss: 给 image 找正确 text。
- text-to-image loss: 给 text 找正确 image。

- CLIP 同时训练两个方向。

CLIP 和 SimCLR 的关系。

SimCLR 的 positive pair 来自同一张 image 的两种 augmentation。

CLIP 的 positive pair 来自同一个 Internet image-text pair。

两者都是 contrastive: positive 拉近，batch 中其它样本作为 negatives 推远。

差别在于 SimCLR 是 image-image representation learning，CLIP 是 image-language representation learning。

6.4 CLIP Zero-Shot Classification p.80

- 先用 contrastive loss 训练 vision-language network。
- 对每个 class 写 prompt，例如 “a photo of a {class}”。
- 比较 image embedding 和每个 text prompt embedding。
- 选 similarity 最大的 class。
- 与传统 supervised learning 相比，CLIP 对 distribution shift 更 robust。p.81

6.4.1 Zero-shot classification pipeline p.80

对测试图像 x :

$$I = \text{fimg}(x)$$

对候选类别 C 写 prompts:

$$p_c = \text{“a photo of a c”}$$

$$T_c = \text{ftext}(p_c)$$

预测:

$$\hat{c} = \arg \max_{c \in C} I^T T_c$$

Prompt 是新的 classifier weights。

在普通 classifier 里，最后一层 weight W_c 表示 class c 。

在 CLIP zero-shot 里，text prompt embedding T_c 扮演了类似 classifier weight 的角色。

所以 changing label set 变成了 changing prompts，而不是重新训练最后一层。

6.5 为什么 CLIP 成功: transferability + scale + simplicity p.84

- vision-language supervision 不是新想法；
- ConVIRT 用 paired medical images and text。p.82
- ICMLM 做 image-conditioned masked language modeling。p.83
- CLIP 的成功关键：
- Transferability: bridging vision and language；
- Scalability: 400M data；
- Simplicity: contrastive learning 适合大规模 pre-training。
- scale 对比：

- CLIP Transformer 约 307M params；
- ImageNet ResNet101 约 44.5M params。p.85
- CLIP training data 约 400M images；
- ImageNet training data 约 1.28M images。p.86

为什么是 scale，而不只是 idea。

vision-language paired supervision 早就存在。

CLIP 真正把它做强，靠的是：

- Internet-scale image-text data；
- 简单可扩展的 contrastive objective；
- 大模型容量；
- open-vocabulary text interface。

所以 CLIP 的意义不是“第一次想到 image-text”，而是把 image-text supervision 扩展到了足够大的规模。

6.6 Modality Gap: image cone 和 text cone 没有完全重合 p.87

- paired text embeddings 和 visual embeddings 不一定完全 matched。
- 两个 encoder 可能产生两个 cones。p.88
- contrastive learning 可以保持 pairwise ordering，但不一定消除 modality gap。p.89

Modality Gap 的直觉。

CLIP 只要求 matched image-text pair 比 unmatched pair 更相似。

它不要求 image embedding 和 text embedding 在几何上完全重合。

因此两个 modality 可能各自形成一个 cone: 配对样本相对更近，但整体分布仍然有 gap。

这解释了为什么 CLIP 很擅长 ranking / retrieval / zero-shot classification，但 embedding space 仍然不是完美的跨模态统一空间。

6.7 SigLIP: 用 sigmoid loss 解耦 batch size p.90

- SigLIP 修改 CLIP loss。
- 目标: decouple batch size from comparison。
- CLIP 的 softmax contrastive loss 会把一个 batch 内所有 pairs 放在同一个 normalized competition 里。
- SigLIP 把 image-text matching 改成 pairwise binary classification。slide 中的 pairwise sigmoid loss:

$$\mathcal{L}_{ij} = \log \frac{1}{1 + e^{z_{ij}(-t x_i \cdot y_j + b)}}$$

整批 loss:

$$\mathcal{L} = -\frac{1}{|B|} \sum_{i=1}^{|B|} \sum_{j=1}^{|B|} \mathcal{L}_{ij}$$

- x_i 是 image embedding， y_j 是 text embedding。
- t 是 learnable temperature， b 是 learnable bias。
- $z_{ij} = +1$ 表示 diagonal positive pair。
- $z_{ij} = -1$ 表示 off-diagonal negative pair。
- larger data: WebLI 10B/12B。

SigLIP 为什么能解耦 batch size。

CLIP softmax loss 中，一个 image 要在 batch 里的所有 texts 中选正确 text。batch 越大，comparison set 越大。

SigLIP 把每个 (image, text) pair 当作独立的 binary decision:

- matched pair: label +1；
- mismatched pair: label -1。

这样 batch size 不再以同样方式绑定到 softmax competition 的类别数。训练仍然可以利用 batch 内 pairs，但 objective 更像 pairwise logistic regression。

6.8 CoCa: CLIP-style contrastive + captioning decoder p.92

- CoCa 给 CLIP-like model 加 decoder。
- 除 contrastive objective 外，再加 captioning loss。
- 直觉: 不仅要 image-text 对齐，还要能生成描述性 text。

Contrastive Loss 和 Captioning Loss 分别补什么。

contrastive loss 擅长学全局对齐：

image embedding $\leftrightarrow$ text embedding

但它不要求模型逐词解释图像内容。

captioning loss 要求 decoder 生成 text sequence:

$$p(w_1, w_2, \dots, w_T \mid \text{image})$$

这会给模型更细粒度的 language supervision。CoCa 的思路是：同时保留 CLIP 的 retrieval / zero-shot 能力和 captioning 的生成监督。

6.9 CLIP-style Models the limitation p.94

- CLIP 对关系和组合语义不够敏感。
- 例子: there is a mug in some grass vs there is some grass in a mug。
- hard negative fine-tuning:
- horse eating grass vs grass eating horse。p.95
- hard positives / compositionality:
- positive: A black cat and a brown dog;
- hard negatives: A brown cat and a black dog, A brown dog and a black cat。p.96
- image-level captions supervision 太粗。p.97
- image-level label 可以只给 living room。
- 但图中局部对象如 house plants、couch 也存在，却不一定被 image caption 监督到。

- 可能需要 region captions + bounding box coordinates。
- 为什么 CLIP 不擅长关系。
- CLIP 的训练信号通常是 image-level caption。

如果 caption 只要求整体匹配，模型可以靠 object co-occurrence 和关键词完成任务，不一定精确理解关系：

- mug in grass 和 grass in mug 有相同关键词；
- horse eating grass 和 grass eating horse 有相同 object/action words；
- black cat and brown dog 与 brown cat and black dog 只交换属性绑定。

这些例子要求模型理解 binding: 哪个属性属于哪个 object, 哪个 object 是 action subject, 哪个是 object。

hard negatives 的作用就是故意构造“词差不多但关系错”的文本，逼模型学习 composition。

7 Vision-Language Models: 图像作为 LLM 的输入 p.98

7.1 LLaVA: Large Language and Vision Assistant p.99

- motivation: LLM 做 next token prediction，却能泛化到 math、sentiment、symbolic reasoning 等任务。

- 问题：能不能让 model 接收 image + text，再输出 text？
- 这就是 Vision-Language Models (VLMs)。
- VLM 不只是做 image classification，而是让图像进入 language interface。从 CLIP 到 VLM。

CLIP 输出的是 embedding similarity，适合 retrieval / zero-shot classification。

VLM 要输出 text sequence，适合问答、描述、推理和 instruction following。

所以 VLM 通常要把 visual features 接入 LLM，让 LLM 在生成 token 时能 condition on image。

7.2 LLaVA 的关键: 把 visual information 接到 LLM p.100

- vision encoder 先把 image 转成 visual tokens / image features。
- projector 把 visual features 映射到 LLM token embedding space。
- LLM 把 image tokens 当作上下文的一部分。
- 训练 recipe 通常分两步：
- image-text alignment；
- instruction tuning。p.101

7.2.1 LLaVA 的模块关系 p.101

典型结构:

image $\xrightarrow{\text{vision encoder}}$ visual features

visual features $\xrightarrow{\text{projector}}$ visual tokens

[visual tokens, text tokens] $\xrightarrow{\text{LLM}}$ output text

- vision encoder 提供视觉语义。
- projector 负责 modality adapter。
- LLM 负责语言生成和 instruction following。

为什么需要 projector。

vision encoder 的 feature space 和 LLM 的 token embedding space 不一样。

projector 的作用是把 visual features 变成 LLM 能读的“伪 token embeddings”。

如果没有这个 adapter，LLM 无法直接把图像特征当成上下文。

7.2.2 LLaVA 两阶段训练 p.102

- stage 1: image-text alignment。
 - 通常冻结部分 backbone。
 - 训练 projector，让 visual tokens 能被 LLM 接受。
 - stage 2: visual instruction tuning。
 - 用 instruction-style image-text data。
 - 让模型学会按用户问题输出自然语言答案。
- Alignment 和 Instruction Tuning 的区别。
- alignment 解决“图像特征能不能接到语言模型里”。

instruction tuning 解决“接进去之后，会不会按人类问题回答”。

前者偏 representation alignment，后者偏交互能力和回答格式。

7.3 Flamingo: 用 gated cross-attention 融合视觉特征 p.103

- Flamingo 是另一种 fuse visual features 的方式。
- 视觉侧先经过 encoder 和 down-sampling。p.104
- 在语言模型中插入 gated cross-attention。p.105
- masked attention 控制不同 text token 能看到哪些 visual/text context。p.106

• 这种设计支持 in-context learning: 给 few-shot image-text examples, 再回答新 query。p.107

Flamingo 和 LLaVA 的融合方式。

LLaVA 常见做法是把 projected visual features 当作 tokens 放进 LLM context。

Flamingo 则在 language model 层中加入 gated cross-attention, 让 language hidden states 在生成过程中 cross-attend 到 visual features。

gated 的意义是: 模型可以学习什么时候依赖 vision, 什么时候主要依赖 language prior, 为什么 Flamingo 能做 in-context learning。

Flamingo 的输入可以包含多组 image-text examples。

masked attention 控制 token 只能看合适的上下文, 避免未来信息泄露。

这样模型可以像 LLM 读 few-shot text examples 一样, 读 few-shot multimodal examples, 再回答新的 image/text query。

8 Look at Your Data: 数据本身也会出问题 p.108

8.1 Validation Data Leakage p.109

- val data 可能被泄露到训练或模型选择流程里。
- 结果: reported validation performance 过于乐观。
- slide 引用例子:
- https://arxiv.org/abs/1912.11370。p.109
- https://arxiv.org/abs/2508.17416。p.110

Leakage 为什么危险。

validation set 的作用是模拟 unseen data。

如果 validation data 泄露进 training, pretraining, hyperparameter search 或 benchmark construction, 模型表现就不再代表真实泛化。

对大模型尤其麻烦: 训练数据巨大, benchmark 很可能在网络语料中出现过。于是高分可能来自 memorization / contamination, 而不是能力提升。

8.2 Labels can be wrong p.111

- ImageNet-1k 这类经典数据集也可能有错误 label。
- 训练和评估时, 模型不一定是在学真实世界类别, 而是在拟合 dataset annotation。
- slide 给出外部 source:https://www.fws.gov/sites/default/files/documents/2024-04/6-Pr2,-0.1,1.5,0.5,... p.13

Sliding Window 的问题。

sliding window 的主要问题是搜索空间太大:

- location 很多;
- scale 很多;
- aspect ratio 很多;
- 每个 class 都可能要判断一次。

label 错误会带来两个问题:

- training 时, 模型被错误监督信号拉偏;
- evaluation 时, 模型可能给出正确语义答案, 却被 benchmark 判错。

当数据集很大时, 单个错误看似不重要, 但系统性 label noise 会影响类别边界、长尾类别表现和 benchmark 可信度。

8.3 Classes themselves can be wrong p.112

- 类别体系本身也可能不合理。
- 问题不只是 individual label noise, 而是 ontology / taxonomy 可能错。
- slide 引用: https://arxiv.org/abs/2603.0657。p.112
- 做 vision model 时要看数据:
- label 是否可信;
- split 是否干净;
- class definition 是否和真实任务一致;
- benchmark 是否真的测到了想测的能力。

Class Ontology 也会出错。

label noise 是“这张图标错了”。

ontology problem 是“这个类别体系本身就不适合这个任务”。

例如类别之间可能重叠、粒度不一致、文化或生物学分类不准确, 或者 label name 不对应视觉上可区分的概念。

这会让模型看起来在分类, 其实是在学习一个有问题的 taxonomy。

created: 2026-05-23

updated: 2026-05-23

1 Object Detection: 分类之外还要定位 p.3

object detection 的目标是同时回答两个问题:

- where: 图像里 object 在哪里, 用 tight bounding box localize。
- what: 每个 box 属于哪个 class。

和 image classification 相比, detection 的输出不再是一个全图 label, 而是一组 structured predictions:

$$\{(b_i, c_i, s_i)\}_{i=1}^N$$

- b_i : bounding box coordinates。
- c_i : object class。
- s_i : confidence score。
- N : 图像中预测出的 object 数量, 不是固定常数。

Detection 为什么比 Classification 难。

classification 只需要判断整张图最像哪个 class。

detection 还要处理:

- object count 不固定;
- object scale 不固定;
- object position 不固定;
- 多个 objects 可能重叠或遮挡;
- 背景区域远多于 object 区域。

所以 detection 通常要把 classification problem 和 localization problem 绑在一起做。

2 Detection Approaches: 三类基本思路 p.4

课件把 object detection 的历史路径分成几类:

- sliding window: 不先选区域, 直接在很多窗口上跑 classifier, You Only Look Once (YOLO) 是神经网络时代的一阶段版本。p.5
- category-independent region proposal: 先找可能包含 object 的区域, 再分类, 例如 Region-based Convolutional Neural Network (R-CNN) / Fast Region-based Convolutional Neural Network (Fast R-CNN) / Faster Region-based Convolutional Neural Network (Faster R-CNN)。p.14
- matching patches against a codebook/vocabulary: 用局部 patch 或 query 去匹配 object structure, 例如 Implicit Shape Model, 后面 Detection Transformer (DETR) 用 learned queries 替代 data patches。p.19

2.1 Sliding Window: 把 detection 变成很多次 binary classification p.5

sliding window 的直觉很直接:

- 用一个固定大小的 window 在图像上滑动。
- 每个位置问 classifier: 这个 window 里有没有目标, 比如 sheep?
- 得到很多 confidence scores。
- 保留高分窗口, 合并重复检测。

课件示例中, 不同窗口得到的 sheep confidence 可以是 0.4, 0.6, 0.2, -0.1, 1.5, 0.5, ... p.13

Sliding Window 的问题。

sliding window 的主要问题是搜索空间太大:

- location 很多;
- scale 很多;
- aspect ratio 很多;
- 每个 class 都可能要判断一次。

早期方法需要手工设计 feature 和高效 cascade。深度学习以后, YOLO / Single Shot MultiBox Detector (SSD) / RetinaNet 这类 single-stage detectors 把 dense prediction 放进一个网络 forward pass 中, 才让这个方向重新变得实用。

2.2 Region Proposal: 先找 object-like regions, 再分类 p.15

region proposal 的思想是: 不要在所有窗口上穷举分类, 先生成一批可能含 object 的 candidate regions。

proposal 可以来自:

- object segmentation / grouping super-pixels;
- filtering sliding windows;
- binary classification: object vs. non-object。p.16

Region Proposal Network (RPN) 的核心可以看成这件事的 learnable Convolutional Neural Network (CNN) 版本:

$$\text{RPN} = \text{objectness classification} + \text{box refinement}$$

也就是先判断一个 anchor 是否像 object, 再预测 box corrections。p.17

2.3 Implicit Shape Model: 局部 patch 为 object center 投票 p.19

Implicit Shape Model 代表了一条更早的 detection 思路:

- 从训练图像中学习 local patches / visual words。
- 每个 patch 不只表示局部 appearance, 还携带它相对 object center 的 offset。
- 测试时, 检测到的 patches 对潜在 object center 投票。
- 多个局部证据在同一个位置聚集, 就支持那里存在 object。

从 Implicit Shape Model 到 DETR 的对比。

Implicit Shape Model 使用从 data 中抽取的 patches / visual vocabulary 去匹配 object。

DETR 不再显式从图像中抽 patch 当 codebook, 而是学习一组 object queries。每个 query 像一个“槽位”, 负责从 image features 中吸收一个 object 的证据。

这就是课件的 spoiler: DETR uses learned queries instead of patches extracted from data。p.22

3 Evaluation: mean Average Precision (mAP) at Intersection over Union (IoU) 是检测核心指标 p.24

3.1 mean Average Precision (mAP) at Intersection over Union (IoU) 的三个部分 p.24

检测常用指标写作 mAP@IoU:

- m: mean over classes。
- AP: Average Precision (AP), 基本上是 precision-recall curve 的 area under curve。
- IoU: 判断 predicted box 是否 match ground-truth box 的 overlap threshold。

AP 的具体 interpolation method 会影响数值, 不同 benchmark 的实现细节要对齐。课件引用了 Common Objects in Context (COCO) Application Programming Interface (API) 的 evaluation implementation。p.25

3.2 Precision-Recall: 按 confidence 从高到低扫描 p.25

对某个 class, 检测结果通常按 confidence score 排序:

- 从高到低依次考虑 predictions。
 - 对每个 prediction, 找 unmatched ground-truth (GT) boxes 中 IoU 最高的那个。
 - 如果 $\text{best IoU} \geq \text{threshold}$, 则记作 true positive (TP), 并把对应 GT 标记为 matched。
 - 否则记作 false positive (FP)。
 - 随着纳入的 predictions 数量增加, 得到一条 precision-recall curve。
- 没有被任何 prediction match 到的 ground-truth object 记作 false negative (FN)。

precision 和 recall 定义为:

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall} = \frac{TP}{TP + FN}$$

AP 是这条 curve 下的面积, mAP 再对 classes 取平均。

为什么同一个 GT 只能 match 一次。

如果同一个 object 周围预测出很多高度重叠的 boxes, 不能让它们都算 TP。

evaluation 中一个 ground-truth box 只能被一个 prediction match。第一个高分预测如果已经 matched, 后面重复预测即使 IoU 很高, 也会被算作 FP。

这也是为什么 detectors 通常需要 non-maximum suppression (NMS), 或者像 DETR 那样在训练目标里直接鼓励一对一 matching。

3.3 IoU: Intersection over Union p.27

两个 boxes A, B 的 IoU 定义为:

$$\text{IoU}(A, B) = \frac{\text{area}(A \cap B)}{\text{area}(A \cup B)}$$

- IoU = 1: 两个 boxes 完全重合。
- IoU = 0: 两个 boxes 没有交集。
- threshold 越高, 要求 localization 越准。

常见 benchmark 会报告不同 thresholds 下的 AP, 例如 AP@0.5 或 COCO 风格的多 threshold average。

4 Learn to Detect: classification + localization p.29

神经网络检测器通常把 detection 拆成两个 head:

- classification head: 预测 class scores, 使用 softmax / cross-entropy loss。
- localization head: 预测 bounding box coordinates 或 box offsets, 使用 regression loss。

课件中的示意是:

feature vector $\rightarrow$ class scores

feature vector $\rightarrow (x, y, w, h)$

整体训练用 multitask loss:

$$\mathcal{L} = \mathcal{L}_{cls} + \lambda \mathcal{L}_{box}$$

其中 $\mathcal{L}_{box}$ 可以是 L2 loss、Smooth L1 loss、Generalized Intersection over Union (GIoU) loss 等, 具体取决于 detector。

为什么常预测 box corrections。

直接预测 absolute coordinates 比较困难, 因为 objects 的位置和大小变化很大。

很多 detectors 会先给一个 reference box, 例如 proposal 或 anchor, 再预测 correction:

$$(d_x, d_y, d_w, d_h)$$

直觉上, 网络回答的是: “从当前 proposal 出发, box center 应该往哪里移, 宽高应该怎么缩放。”

这样 regression target 更稳定, 也更容易学习。

5 R-CNN Family: 从 region proposals 到 learnable proposals p.31

5.1 R-CNN: 每个 proposal 独立跑 CNN p.31

R-CNN 的基本 pipeline:

- 用 Selective Search 等方法生成约 2000 个 region proposals。
- 对每个 proposal crop / warp 成 CNN 输入大小。
- 每个 proposal 独立通过 CNN 提取 feature。
- 对 feature 做 object classification。
- 再预测 box corrections (d_x, d_y, d_w, d_h)。p.36

R-CNN 的关键意义是把 CNN feature hierarchy 用到了 object detection, 但它的瓶颈也很明显:

$$\text{one image} \rightarrow \sim 2000 \text{ CNN forward passes}$$

所以课件说它 very slow。p.37

R-CNN 的计算浪费在哪里。

相邻 region proposals 通常高度重叠。

如果每个 proposal 都单独 crop raw image 再跑 CNN, 那么重叠区域的 convolution 会被重复计算很多次。

更自然的做法是: 整张图先过 CNN, 得到 shared conv feature map; 然后在 feature map 上为每个 proposal crop features。

这就是 Fast R-CNN 的核心改动。p.38

5.2 Fast R-CNN: 先卷整图, 再 crop feature p.39

Fast R-CNN 把 R-CNN 的顺序反过来:

- 整张 image 只通过 CNN backbone 一次。
- 在 shared feature map 上, 根据每个 proposal crop 对应 feature。
- 用 Region of Interest Pooling (RoI Pool) / Region of Interest Align (RoI Align) 得到固定大小 feature。
- 后面接 classification head 和 box regression head。

它避免了 2000 次独立 CNN forward passes, 把主要计算共享在整图 feature extraction 上。

5.3 RoI Pool: 任意大小 proposal 到固定大小 feature p.45

RoI Pool 的目标是把每个 proposal 对应的 feature region 变成 fixed spatial size, 例如 7×7 :

- 把 image coordinate 中的 proposal 映射到 feature map coordinate。
- 将这个 rectangular feature region 划分成 $H \times W$ 个 bins。
- 每个 bin 内做 max pooling。
- 输出固定尺寸 feature, 例如 $C \times 7 \times 7$ 。

这样后面的 fully connected layers 可以处理所有 proposals。

RoI Pool 和 RoI Align。

RoI Pool 会把 coordinates 量化到 feature map grid 上, 因此存在 rounding / quantization error。

RoI Align 后来用 bilinear interpolation 避免粗糙量化, 对 segmentation 和精细 localization 更友好。

本讲重点是 Fast/Faster R-CNN 的共享 feature 思路, 所以课件主要用 RoI Pool 解释 crop features。p.52

5.4 Fast R-CNN 的剩余瓶颈: proposal 仍来自 CPU heuristic p.54

Fast R-CNN 共享了 CNN computation, 但 region proposals 仍由 Selective Search 等 heuristic 算法生成。

问题:

- proposal generation 在 Central Processing Unit (CPU) 上运行;
- runtime 仍可能被 proposals dominated;
- proposals 不是 end-to-end learned。

课件的下一步是: learn them with a CNN instead。p.55

5.5 Region Proposal Network: 在 feature map 上预测 anchors p.57

RPN 从 image features 直接生成 proposals。

课件给的 feature map 例子:

image : $3 \times 640 \times 480$ features : $512 \times 20 \times 15$	
先想象每个 feature map location 上有一个固定大小 anchor box. p.58 对每个 anchor, RPN 预测: - objectness: anchor 是否包含 object. p.59 - box corrections: 修正 anchor 到更好的 proposal. p.60 实际中, 每个 location 会放 K 个不同 scale / aspect ratio 的 anchors. p.61 如果 feature map 是 $H \times W$, 每个位置有 K 个 anchors, 则候选 proposal 数是:	
KHW	
RPN 会按 objectness score 排序, 取 top ~ 300 proposals 送到第二阶段. p.62 RPN 输出张量. 如果 feature map 是 $H \times W$, 每个位置有 K 个 anchors:	
- objectness logits: $K \times H \times W$, 或二分类写成 $2K \times H \times W$. - box offsets: $4K \times H \times W$.	
课件为了直观先用 $K = 1$ 展示:	
$1 \times 20 \times 15$	
表示每个位置一个 objectness score.	
$4 \times 20 \times 15$	
表示每个位置四个 box correction numbers. 5.6 Faster R-CNN: two-stage detector p.63 Faster R-CNN 把 RPN 插入 Fast R-CNN, 用 learnable proposals 替代 Selective Search: - first stage: backbone network + region proposal network. - second stage: RoI Pool / Align crop features, 再预测 object class 和 bbox offset. p.64 因此 Faster R-CNN 是典型 two-stage object detector. <i>Two-Stage Detector</i> 的取舍. <i>two-stage detectors</i> 通常更重, 但 <i>localization</i> 和分类更细:	
- stage 1 专找 <i>object-like regions</i>; - stage 2 对较少 <i>proposals</i> 做更精细的 <i>classification</i> / <i>box regression</i>.	
<i>single-stage detectors</i> 则倾向于一次性 <i>dense predict</i>, 速度更快, 但要处理大量 <i>anchors</i> / <i>grid cells</i> 的 <i>class imbalance</i>. 6 Single-Stage Detectors: YOLO / SSD / RetinaNet p.67 single-stage detector 不再显式生成一批 category-independent proposals 后再分类, 而是在 dense grid 上直接输出 box 和 class. 课件给的统一描述: - 把 image 划分成 grid, 例如 7×7. - 每个 grid cell 有 B 个 base boxes. - 每个 base box 回归到 final box, 输出 $(d_x, d_y, d_h, d_w, \text{confidence})$. - 每个 cell 或 box 预测 C 个 class scores, 包括 background. 输出张量形状:	- 只做一次 network forward. - 图像被分成 $S \times S$ grid. p.69 - 每个 grid cell 输出若干 boxes 和 class probabilities. p.70 对每个 box, YOLO 输出: $P(\text{object})$: box 中是否有 object. - bounding box: (x, y, h, w). $P(\text{class})$: 属于各类别的概率. 课件示例中 $B = 2$, 每个 grid cell 负责两个 candidate boxes. p.71 6.2 YOLO 的 post-processing: 多 box 需要去重 p.73 YOLO 会产生 many bounding boxes with different object probabilities. p.73 常见处理流程: - 对每个 predicted box 计算 class-specific score. - 丢掉低 confidence boxes. - 对每个 class 做 NMS, 保留高分且不重复的 boxes. NMS 的基本规则: - 先选最高分 box. - 删除与它 IoU 太高的低分 boxes. - 重复直到没有 boxes. YOLO 的优势和代价. YOLO 的优势是快: 一个 forward pass 直接给出 dense predictions.
	代价是它必须在固定 grid / anchors 上处理不同 scale、密集小物体、遮挡和 class imbalance.
	后续 YOLO 版本、SSD、RetinaNet 等都在 anchor design、feature pyramid、loss design 和 post-processing 上做了很多改进. 7 DETR: 用 Transformer 直接预测一组 boxes p.76 DETR = Detection Transformer. 课件强调三点: - Simple object detection pipeline: 直接从 Transformer 输出一组 boxes. - No anchors, no regression of box transforms. - 用 bipartite matching 把 predicted boxes 和 ground-truth boxes 配对, 再训练 box coordinates. p.76 7.1 DETR Pipeline: fixed set prediction p.77 DETR 通常包含: - CNN backbone 提取 image feature map. - 加 positional encoding. - Transformer encoder 处理 image tokens. - Transformer decoder 接收一组 learned object queries. - 每个 query 输出一个 class distribution 和一个 bounding box. 输出是固定数量 N 的 predictions:
	$\{(\hat{c}_i, \hat{b}_i)\}_{i=1}^N$
	其中 class distribution 中包含一个 special no-object class, 表示该 query 没有负责真实 object. <i>Object Query</i> 的直觉. <i>object query</i> 可以理解成一个可学习的 <i>detection slot</i>.
	decoder 中每个 <i>query</i> 通过 cross-attention 从 <i>image features</i> 里取信息, 最后决定自己是否对应某个 <i>object</i>.
	和 <i>anchor</i> 不同, <i>query</i> 初始不是某个固定 <i>spatial box</i>; 它更像一个 <i>learnable slot</i>, 通过训练学会分工. 7.2 Bipartite Matching: 让 prediction 和 ground-truth (GT) 一对一 p.76 DETR 的训练目标需要解决 set prediction 的 permutation problem: - GT objects 没有顺序. - 模型输出的 N 个 queries 也没有固定顺序. - 所以不能直接说第 i 个 prediction 对应第 i 个 GT. DETR 使用 Hungarian matching 找最小代价的一对一匹配:
	$\hat{\sigma} = \arg \min_{\sigma} \sum_{j=1}^M \mathcal{C}(y_j, \hat{y}_{\sigma(j)})$
	其中 M 是 ground-truth object 数量, C 通常结合: - classification cost: - L1 box distance: - IoU / GIoU cost. 匹配完成后: - matched predictions 学对应 class 和 box.
	unmatched predictions 学 no-object. <i>DETR</i> 为什么不太需要 <i>NMS</i>. 传统 <i>dense detectors</i> 会对同一个 <i>object</i> 产生很多 overlapping boxes, 所以需要 <i>NMS</i> 去重.
	<i>DETR</i> 的 <i>matching loss</i> 直接鼓励一个 <i>GT object</i> 只由一个 <i>query</i> 负责. 重复预测同一个 <i>GT</i> 不会都得到正样本监督.
	因此 <i>DETR</i> 的输出天然更像 <i>set prediction</i>, 减少了 <i>hand-designed post-processing</i>. 7.3 DETR 的取舍 p.81 DETR 把 object detection 重写成 end-to-end set prediction, 设计上很干净: - 不需要 anchors. - 不需要手写 proposal generation. - 不需要传统 NMS. 结构和 Transformer 的 global reasoning 对齐. 但原始 DETR 也有已知代价: - 训练收敛慢; - 小物体检测困难; - high-resolution dense features 和 attention cost 之间有张力. 后续 Deformable DETR、DETR with Improved deNoising anchor boxes (DINO) detector、Grounding DINO 等工作都在改进这些问题. 8 Open-Vocabulary Detection: 从固定类别到自然语言查询 p.86 8.1 Closed-Vocabulary Detection 的限制 p.86 传统 detectors 通常在固定类别集上训练: - COCO: 常见是 80 classes. - Large Vocabulary Instance Segmentation (LVIS): 约 1200 classes. 问题是: - out-of-vocabulary objects 会被忽略或误分类. - 手工标注 "every object in the world" 不现实. - 真实应用中, 用户想找的类别可能不在训练 label set 里. open-vocabulary detector 需要: - 使用 natural language labels, 而不是固定 class list. - 对 unseen categories zero-shot generalize. - 对 known categories 仍保持竞争力. 8.2 Open-World Localization Vision Transformer (OWL-ViT): 用 text query 做 open-world localization p.87 OWL-ViT 的贡献: - open-vocabulary detection: 检测 text 描述的 objects, 不限于训练 labels. - zero-shot generalization: 不 retrain 也能找 novel categories, 例如 "espresso machine". - simplicity + scaling: large-scale pretraining + Vision Transformer (ViT) + end-to-end fine-tuning. OWL-ViT 可以理解成把 Contrastive Language-Image Pre-training (CLIP)-style image-text representation 接到 detection head 上. 8.3 OWL-ViT Stage 1: image-text contrastive pre-training p.88 第一阶段做 large-scale contrastive pre-training: - vision encoder: ViT-B / ViT-L / ViT-H, patch size 16 或 32; 也可以是 ResNet-50 (R50) + ViT-H. - text encoder: 12-layer, 8-head Transformer. - data: 3.6B image-text pairs. - batch size: 256. - text 和 image encoders 都从 scratch 训练. 目标是让 image embedding 和 text embedding 进入可比较的 semantic space. 8.4 OWL-ViT Stage 2: 加 detection heads 做 fine-tuning p.89 第二阶段加入 detection heads, 在 medium-sized detection datasets 上 fine-tune: - text encoder 保留 CLIP-style 功能. - inference 时, 用户输入任意 text labels, 例如 "espresso machine", 得到 query embedding. - vision side 移除 token pooling + projection layer. - 对每个 output token representation 做 linear projection, 得到 per-object image embeddings. - 最大预测 object 数约等于 tokens 数, 例如 576+. - box coordinates 来自 separate Multilayer Perceptron (MLP) head. - detection datasets 包括 LVIS、COCO、Objects365. - text encoder frozen, 只重新训练 ViT / detection side. <i>OWL-ViT</i> 怎么把分类头变成文本查询. 普通 <i>detector</i> 的分类头是固定矩阵:
$W \in \mathbb{R}^{C \times d}$	
其中 C 是固定类别数.	
OWL-ViT 用 text encoder 把 label phrase 编成 query embedding:	
$q_t = f_{\text{text}}(\text{"espresso machine"})$	
image token / region embedding 为:	
$z_i = f_{\text{vision}}(x)_i$	
然后用 similarity 判断第 i 个位置是否对应该 text query:	
$s_i = z_i^T q_t$	
	这样 <i>changing classes</i> 变成 <i>changing text prompts</i>, 而不是重新训练分类头. 8.5 LVIS 和 rare categories p.90 LVIS 常被用来测试 rare / unseen categories: - training 可能只用 base / common categories. - testing 看 all categories 和 rare categories. $A_{\text{P}}^{\text{rare}}$ 基本反映 zero-shot / few-shot open-vocabulary 能力. p.91 课件强调 OWL-ViT 在 unseen classes 上有 competitive zero-shot performance. 8.6 Image-Conditioned Detection: 用图像 embedding 做 query p.92 除了 text query, 也可以用 image embedding query 输入图像: - 给一个 reference image / crop. - 编码成 query embedding. - 在目标图像中找最相关 objects. 课件指出 OWL-ViT 在 image-conditioned detection 上相比 task-specific models 有明显优势, 核心 idea 是 use image embeddings instead of text to query the input image. 8.7 OWLv2: 用 self-training 扩大 detection supervision p.93 OWL-ViT v1 的限制: - pre-training data 非常大; - detection fine-tuning data 相比很少. OWLv2 的改进: - 用 OWL-ViT 在大量 web-scraped image-text data 上自动生成 pseudo-labels. - pseudo-labels 包括 bounding boxes + class labels. - 用这些 noisy supervision 扩大 detection examples. - 从几十万 detection examples 扩展到 billions. 结果: - rare-category detection 明显提升. - 课件给的数字: $A_{\text{P}}^{\text{rare}}$ 从 31.2% 到约 44.6%. <i>Self-Training</i> 的关键风险. <i>self-training</i> 的好处是把 <i>detector</i> 自己的预测变成训练数据, 极大扩大 <i>detection supervision</i>.
	风险是 <i>pseudo-label</i> 有噪声: - box 不准; - label 可能错; - long-tail categories 容易被常见类别吞掉.
	所以 OWLv2 的意义不只是 "生成更多数据", 还在于用 <i>scaling</i> 抵消 <i>noisy labels</i>, 让 open-vocabulary detector 从 web-scale data 中受益. 9 Grounding DINO: open-set detector with language-aware regions p.94 Grounding DINO 是 open-set object detection 方法: - 用户给 natural language prompts. - 模型无需额外训练即可检测 prompt 描述的 objects. 课件特别提醒: 这里的 DINO 是 detector 方向的 DINO, 不是前面 self-supervised representation learning 里的 DINO. p.95 9.1 从 closed-set DINO detector 到 Grounding DINO p.96 Grounding DINO 基于 closed-set detector DINO: - DINO detector = DETR with improved denoising anchor boxes for end-to-end object detection.

- 它属于 DETR 系列 object detector，而不是 self-distillation DINO。Grounding DINO 的核心变化：
- 学 language-aware region embeddings。
- 每个 region 可以在 language-aware semantic space 中被分类到 novel categories. p.97

9.2 Open-Set Detection 的应用 p.98

Grounding DINO 这类 open-set detector 可以用于：

- zero-shot image detection on new datasets。
- 和 Stable Diffusion 结合做 image editing。
- embodied AI 中按语言指令 grounding objects。

Open-Vocabulary vs. Open-Set.

open-vocabulary detection 强调类别由 language vocabulary 指定，不固定在训练 label set。

open-set detection 更强调测试时可能出现训练中没有见过的 objects / categories，模型要能识别或定位这些 novel concepts。

两者经常一起出现：language prompt 给了开放词表接口，region-language alignment 提供了 zero-shot grounding 能力。

10 Lecture Map: 从传统 detection 到 language-grounded detection

这讲可以压缩成一条发展线：

- detection = localization + classification. p.3
- 早期思路包括 sliding window、region proposal、implicit shape matching. p.4
- evaluation 用 mAP@IoU，先排序 predictions，再按 IoU 匹配 GT. p.24
- R-CNN 把 proposals 分类，但每个 proposal 独立 CNN 很慢。p.37
- Fast R-CNN 共享 image features,用 RoI Pool crop proposal features. p.45
- Faster R-CNN 用 RPN 学 region proposals，形成 two-stage detector. p.64
- YOLO / SSD / RetinaNet 直接 dense predict boxes + classes，是 single-stage detectors. p.67
- DETR 把 detection 变成 set prediction,用 object queries + bipartite matching. p.76
- OWL-ViT / Grounding DINO 把 detection 接到 language space，支持 open-vocabulary / open-set detection. p.86

Exam / Review Focus.

复习时优先抓这些对比：

- R-CNN vs. Fast R-CNN: proposal 是否单独跑 CNN，还是共享整图 feature。
- Fast R-CNN vs. Faster R-CNN: proposal 是 heuristic Selective Search，还是 learned RPN。
- Faster R-CNN vs. YOLO: two-stage proposals + refinement，还是 one-stage dense prediction。
- YOLO vs. DETR: anchor/grid dense predictions + NMS,还是 learned queries + bipartite matching。
- closed-vocabulary vs. open-vocabulary: 固定 classifier weights,还是 text/image query embeddings。

created: 2026-05-25

updated: 2026-05-26

1 Dense Prediction: 从一个 label 到每个 pixel 一个输出 p.2

dense prediction 指一类 computer vision tasks:模型不是给整张图一个 sparse label，而是对输入图像中的每个 spatial location / pixel 产生预测。典型输出通常和输入有相同或相关的空间分辨率：

$$I \in \mathbb{R}^{H \times W \times 3} \rightarrow Y \in \mathbb{R}^{H \times W \times C}$$

其中 Y_{ij} 是 pixel (i, j) 的 prediction。不同任务中 C 的含义不同：

- semantic segmentation: C 是类别数，每个 pixel 预测 class distribution。
- depth estimation: $C = 1$ ，每个 pixel 预测距离 camera 的 depth。
- optical flow: $C = 2$ ，每个 pixel 预测 motion vector (u, v) 。
- image generation / diffusion: 每个 pixel 或 latent location 预测 clean image / noise。

Dense Prediction vs. Image Classification.

image classification 的输出是一个全局 label，例如 cat。

dense prediction 的输出是一张 map: 每个位置都有 label、depth、flow、mask probability、noise vector 或其他 structured value。

所以 dense prediction 的核心矛盾是：high-level semantics 需要大 receptive field，但 pizel-level output 又需要 precise localization。

1.1 Image Segmentation: semantic / instance / panoptic p.3
image segmentation 是 dense prediction 的核心任务：给图像里的每个 pixel 分配语义或实例相关的 annotation。

课件区分三类 segmentation：

- semantic segmentation: 每个 pixel 预测 object / stuff class，例如 person、car、road；不区分同类不同实例。
- instance segmentation: 在 semantic label 之外，还要区分同类中的不同 object instance，例如不同人用不同 mask。
- panoptic segmentation:统一 semantic segmentation 和 instance segmentation，同时处理 Stuff 和 Things。

这里的关键词是：

- semantic 回答“what”：这个 pixel 是什么类别。
- instance 回答“which”：这个 pixel 属于哪个 object instance。
- panoptic 同时回答“what”和“which”。

Stuff 和 Things。

Things 是可数的 object instances，例如 person、car、dog。

Stuff 是没有明确实例边界的背景或材质区域，例如 sky、road、grass、wall。

panoptic segmentation 的目标是让一整张图中的 every pizel 都有解释：Things 要分 instance，Stuff 只需要 class region。

1.2 Depth Estimation: 每个 pixel 到 camera 的距离 p.4

depth estimation 的目标是从一张或多张图像预测 depth map：

$$D \in \mathbb{R}^{H \times W}$$

其中 D_{ij} 表示 pixel (i, j) 对应 scene point 到 camera 的距离或相对深度。monocular depth estimation 是 ill-posed problem，因为单张图像会丢失真实 3D scale：

- 同一个 2D projection 可能来自不同真实 3D scenes。
- size、perspective、occlusion、texture gradient 等 cues 只能提供间接信息。
- 没有 multi-view geometry 时，absolute scale 很难唯一确定。

1.3 Image Generation: 预测 clean image / noise 也是 dense prediction p.5

diffusion models 可以从 dense prediction 的角度理解：

$$\text{noisy image } x_t \rightarrow \mathbb{E}[\text{clean image} \mid x_t]$$

或者更常见地预测每个位置的 noise：

$$\epsilon_{\theta}(x_t, t)$$

这仍然是 dense prediction，因为模型在每个 pixel 或 latent location 上输出一个 value，而不是只输出一个 global class label。

2 Fully Convolutional Network (FCN): 把 classifier 变成 pixel-wise predictor p.7

Fully Convolutional Network (FCN) 的出发点：

- 已有一个 pre-trained image classification network。
- 想要 arbitrary-sized image 上的 pixel-wise predictions。
- fully connected layer 可以等价改写成 convolutional layer。

如果把 classification network 最后的 fully connected layers 换成 convolutional layers，网络就可以在整张 feature map 上滑动式地产生 dense outputs。从 Cat Classifier 到 Segmentation Model。

先想一个普通 cat classifier：

$$\text{image} \rightarrow \text{CNN feature map} \rightarrow \text{one label: cat}$$

它最后只输出一个 label，是因为最后几层把 spatial information 压掉了。

FCN 的想法是：不要把最后的 feature map 展平后接 fully connected layer，而是保留 feature map 的二维结构。

假设 backbone 最后得到的是：

$$20 \times 15 \times 512$$

也就是每个 spatial location 都有一个 512 维 feature vector。现在用一个 1×1 convolution 做分类，相当于对每个 location 各自问一次：

这个位置像 road / person / car / sky 吗？

于是输出不是一个 class vector，而是一张小的 score map：

$$20 \times 15 \times C$$

这张 map 还太小，所以再 upsample 回原图大小，例如 $640 \times 480 \times C$ 。每个 pixel 取分数最高的 class，就是 semantic segmentation。

Fully Connected Layer 为什么能变成 Convolution。

对固定大小 feature map，fully connected layer 看起来是把所有 spatial positions 展平后做矩阵乘法。

如果把权重 reshape 成覆盖整个 spatial support 的 convolution kernel，它就等价于一次 convolution。

更重要的是：改成 convolution 以后，同一个权重可以应用到更大的输入图像上，输出变成 spatial map，而不是单个 vector。

2.1 FCN 面临的两个问题: sparse, low-resolution output; top-layer features abstract but not localized p.8

FCN 的挑战来自两个方向：

- sparse, low-resolution output:classification backbone 经过多次 stride / pooling，最后 feature map 分辨率很低。
- top-layer features abstract but not localized: 高层 feature 语义强，但空间定位粗。

这和 dense prediction 的需求冲突：

$$\text{semantic abstraction} \quad \text{vs.} \quad \text{spatial precision}$$

所以 dense prediction architecture 通常要同时做两件事：

- downsample: 扩大 receptive field，学习 high-level semantics。
- upsample: 恢复 spatial resolution，让输出对齐原图。

2.2 FCN 的 practical solution: 先 downsample, 再 upsample p.9

直接在 full image resolution 上运行所有 convolution 很贵，而且低层 local features 不够语义化。

实际方案是 encoder-decoder style:

- encoder / backbone: 通过 convolution、pooling、stride 逐步降低 resolution。
 - predictor: 用 1×1 convolution 在 feature map 上产生 class scores。
 - decoder / upsampler: 把 low-resolution scores 放大回 input resolution。
- FCN 中常见两种 upsampling：
- bilinear upsampling: 无学习参数，直接插值到原图大小。
 - learned upsampling: 用 transposed convolution 学习上采样 kernel. p.12

2.3 Skip Fusion: 高层语义和低层定位相加 p.12

FCN 不只把最终低分辨率 score map 直接放大，还可以融合不同层级的 feature maps。

直觉：

- high-level feature map: semantic strong, spatial coarse。
- lower-level feature map: semantic weaker, spatial finer。
- fusion: 让预测同时知道“what”和“where”。

FCN 的 skip fusion 常用 summing: 把上采样后的高层 prediction 和较低层 prediction 对齐后相加。

为什么 Dense Prediction 常需要 Skip Connection。

如果只用最后一层 feature map，输出虽然语义稳定，但边界会很粗。

如果只用很浅层 feature map，边界细，但 class semantics 不可靠。

skip connection 把 coarse semantic map 和 fine spatial map 连起来，是 segmentation、depth、generation architecture 中反复出现的设计。

2.4 Semantic Segmentation Metrics: pixel accuracy 和 Intersection over Union (IoU) p.14

semantic segmentation 常见 metrics：

- mean pixel classification accuracy: 所有 pixels 上的平均分类准确率。
- mean per-category pixel classification accuracy: 先按类别算 pixel accuracy，再对类别平均。
- Intersection over Union (IoU):衡量 predicted region 和 ground-truth region 的重叠程度。

对某个 class c：

$$IoU_c = \frac{|P_c \cap G_c|}{|P_c \cup G_c|} = \frac{TP_c}{TP_c + FP_c + FN_c}$$

mean Intersection over Union (mIoU) 再对 classes 取平均：

$$mIoU = \frac{1}{C} \sum_{c=1}^C IoU_c$$

为什么 mIoU 比 pixel accuracy 更可靠。

pixel accuracy 容易被大面积背景主导。

如果一张 road scene 里 road / sky 占比很高，模型即使完全不识别小 object，也可能有不错的 pixel accuracy。

IoU 会同时惩罚 false positive 和 false negative，对 region overlap 更敏感；mIoU 又避免大类完全压过小类。

3 Operations for Dense Prediction: upsampling 如何恢复 resolution p.16

dense prediction 中最核心的 operation 是 upsampling: 把低分辨率 representation 变回高分辨率 output。

常见选择：

- interpolation: nearest / bilinear。
- transposed convolution: learnable upsampling。
- unpooling: 利用 pooling indices 恢复 sparse high-resolution map。
- resize + convolution: 先插值再普通 convolution，常用于避免 checkerboard artifacts。

3.1 Transposed Convolution: 用 filter 在 output 上 paint p.17

transposed convolution 的直觉是：

- input 中每个 value 都会乘上一个 filter。
- 这个 scaled filter 被放到 output 的对应位置。
- 多个 filter copies 重叠的地方相加。

课件说它像“paint” in the output。1D 例子中，它等价于用 flipped filter 做某种 convolution。

为什么叫 Transposed Convolution。

普通 convolution 可以写成一个 matrix multiplication：

$$y = Cx$$

其中 C 是由 convolution kernel 生成的稀疏矩阵。

transposed convolution 对应：

$$x' = C^T y$$

它不是严格的 inverse convolution，而是普通 convolution 线性算子的 transpose。这个名字比“deconvolution”更准确。

3.2 Stride-2 Transposed Convolution: 插零再卷积 p.18

transposed convolution 可以看成 backwards-strided convolution。为了增加 resolution，使用 output stride > 1 。

对 stride 2 的情形，可以理解为：

- 在 input 相邻 rows / columns 中间插入 zeros。
- 用 flipped filter 做 convolution。
- 得到更高分辨率 output。

这有时也叫 fractional input stride 1/2。p.23

Checkerboard Artifact.

transposed convolution 如果 kernel size 和 stride 搭配不好，不同 output positions 会被不均匀数量的 filter copies 覆盖。

结果可能出现 checkerboard artifact: 生成图或 segmentation map 中有网格状纹理。

常见缓解方式是 resize + convolution，或者小心选择 kernel / stride，让 overlap 更均匀。课件引用的 Distill animation 专门解释这个现象。

3.3 Max Unpooling: 用 pooling switches 放回位置 p.24

unpooling 是 transposed convolution 的替代方案，尤其和 max pooling 成对出现。

max pooling 时，除了保存 pooled value，还可以保存 argmax index：

$$\text{switch}_{ij} = \arg \max_{(u,v) \in \text{pool window}} x_{uv}$$

max unpooling 时：

- 把 pooled value 放回原来 argmax 的位置。
- pool window 里其他位置填 0。
- 后续 convolution 再把 sparse map densify。

Unpooling 的优点和局限。

unpooling 的优点是保留了 *encoder* 中 *max response* 的位置，所以比纯插值多一点 *spatial memory*。

局限是 *output* 一开始很 *sparse*，很多位置是 0；模型还需要后续 *convolution* 去补充细节。

4 Architectures for Dense Prediction: DeconvNet / U-Net / Feature Pyramid Network p.25

这一部分从 operation 进入 architecture: dense prediction networks 通常都有 encoder-decoder 或 multi-scale fusion 结构。

核心问题是：如何同时保留 high-level semantic context 和 high-resolution spatial detail。

4.1 DeconvNet: encoder-decoder + unpooling / deconvolution p.26

DeconvNet 的结构可以理解为：

- encoder: 类似 classification network: convolution + pooling 得到 high-level features。
 - decoder: 用 unpooling 和 deconvolution / transposed convolution 逐步恢复 resolution。
 - output: 为每个 pixel 预测 segmentation class。
- DeconvNet 的关键点是 decoder 不是简单插值，而是学习如何从 low-resolution feature map 恢复 detailed segmentation mask。

4.2 U-Net: concatenate skip features, 最后统一预测 p.28

U-Net 也是 FCN style architecture，但融合方式更强：

- encoder downsampling path 提取多尺度 features。
- decoder upsampling path 逐步恢复 resolution。
- 每个 decoder stage 和对应 encoder stage 做 skip connection。
- 和 FCN 的 summing 不同，U-Net 常用 concatenation。
- 最后在 high-resolution feature map 上统一预测。

U-Net 为什么适合 *dense prediction*。

U-Net 的 *skip concatenation* 保留了更多低层 *spatial detail*，而不是只把 *score maps* 相加。

这对 *biomedical image segmentation* 特别重要，因为边界、小结构、细长结构经常决定结果质量。

后来的 *segmentation*、*diffusion U-Net*、*image-to-image translation* 都大量复用了这个 *encoder-decoder + skip* 的思想。

4.3 Feature Pyramid Network (FPN): 给低层 feature 加高层 context p.29

Feature Pyramid Network (FPN) 的目标是改善 lower-level feature maps 的 predictive power：

- 高层 feature 语义强但分辨率低。
- 低层 feature 分辨率高但语义弱。
- FPN 通过 top-down pathway 和 lateral connections, 把 higher-level context 加到 lower-level maps 上。

在 object detection 中, FPN 通常让不同 pyramid levels 负责不同尺度的 boxes：

- high-resolution levels: 小 objects。
- low-resolution levels: 大 objects。
- predictor parameters 可以在不同 levels 之间共享或保持结构一致。

FPN 和 *U-Net* 的关系。

两者都在做 *multi-scale fusion*。

U-Net 更像完整的 *encoder-decoder*：逐步 *upsample*，最后输出 *dense maps*。

FPN 更强调构造一组 *semantic-rich pyramid feature maps*，让 *detection / segmentation heads* 在不同尺度上使用它们。

U-Net 是“把 feature 解码回一张细图”；FPN 是“造一组语义增强的多尺度特征图”

5 Instance Segmentation: Mask Region-based Convolutional Neural Network (Mask R-CNN) p.31

instance segmentation 要输出：

$$\{(b_i, c_i, m_i)\}_{i=1}^N$$

其中：

- b_i : bounding box。
- c_i : class label。
- m_i : 该 instance 的 binary mask。

Mask Region-based Convolutional Neural Network (Mask R-CNN) 可以概括为：

$$\text{Mask R-CNN} = \text{Faster R-CNN} + \text{FCN on RoIs}$$

也就是在 Faster R-CNN 的 detection pipeline 上, 为每个 Region of Interest (RoI) 增加 mask head。p.32

5.1 Mask R-CNN Pipeline: class / box / mask 三个输出 p.33

从 RoIAlign features 出发, Mask R-CNN 同时预测：

- class label: 这个 RoI 属于哪个 object class。
- bounding box: 对 RoI 位置做 refinement。
- segmentation mask: 该 RoI 内每个 pixel 是否属于 object。

mask head 的输出通常是 class-specific binary masks：

$$M \in [0, 1]^{K \times m \times m}$$

其中 K 是 object classes, $m \times m$ 是 RoI 内 mask resolution。

训练时对 ground-truth class 对应的 mask 使用 per-pixel sigmoid binary cross-entropy：

$$\mathcal{L}_{mask} = -\frac{1}{m^2} \sum_{u,v} [y_{uv} \log \hat{y}_{uv} + (1 - y_{uv}) \log(1 - \hat{y}_{uv})]$$

为什么 *Mask R-CNN* 用 *Independent Binary Masks*。

semantic segmentation 常用 *multinomial setting*: 每个 *pixel* 必须在所有 *classes* 中选一个。

Mask R-CNN 的 *mask* 是 *RoI-level instance mask*。给定一个 *detected RoI*，它只需要回答：这个 *pixel* 是否属于当前 *object instance*。

所以它用 *per-class independent sigmoid masks*，而不是在所有 *classes* 上做 *pixel-wise softmax*。classification head 已经负责 *object class*, *mask head* 专注 *shape*。

5.2 RoIAlign vs. RoIPool: 避免 quantization error p.34

先说共同目标: RoIPool 和 RoIAlign 都是在做 Region of Interest (RoI) feature extraction。

Faster R-CNN / Mask R-CNN 前面会产生很多 candidate boxes，也就是 RoIs：

$$\text{RoI} = (x_1, y_1, x_2, y_2)$$

这些 RoIs 的大小不一样：

- 一个 person RoI 可能很高很窄。
- 一个 car RoI 可能比较宽。
- 一个 small object RoI 可能只有几十个 pixels。

但后面的 classification / box regression / mask head 需要固定大小输入，例如：

$$7 \times 7 \times C$$

所以 RoIPool / RoIAlign 要解决的问题是：

$$\text{variable-size RoI} \rightarrow \text{fixed-size feature}$$

5.2.1 RoIPool: 把 RoI 切成 bins, 然后 max pooling p.34

RoIPool 的流程：

- backbone 先把整张 image 变成 feature map。
- 把 image coordinates 里的 RoI 映射到 feature map coordinates。
- 把这个 RoI 区域划分成固定数量的 bins，例如 7×7 。
- 每个 bin 里面做 max pooling。
- 得到固定大小 feature，例如 $7 \times 7 \times C$ 。

直觉上，RoIPool 是在问：

这个 *proposal* 区域里的每个小格子，最强的 *feature response* 是什么？

问题是 RoIPool 需要取整：

- proposal coordinates 从 image 映射到 feature map 时要 quantize。
- bin boundaries 也要 quantize。
- bin 内部选 feature 时相当于落到离散 grid 上。

这种 nearest-neighbor / rounding 会造成 feature 和原图 object boundary misalignment。

RoIPool 的小例子。

假设原图中的 *RoI* 映射到 *feature map* 后坐标是：

$$(3.4, 5.7, 12.8, 18.2)$$

RoIPool 不能直接处理小数边界，于是可能把它 *round* 成：

$$(3, 6, 13, 18)$$

然后再切成 7×7 bins，每个 *bin* 的边界也要取整。

对 *classification* 来说，这种误差可能还能接受；但对 *mask* 来说，边界偏一点就会直接影响 *pixel-level segmentation*。

5.2.2 RoIAlign: 不取整, 用 bilinear interpolation 采样 p.35

RoIAlign 用 bilinear interpolation 替代粗糙取整。p.35

它的流程和 RoIPool 很像，但关键差别是：RoIAlign 保留 floating-point coordinates。

RoIAlign 的流程：

- 把 RoI 映射到 feature map coordinates，但不 round。
 - 把 RoI 分成固定数量的 bins，例如 7×7 。
 - 在每个 bin 中选若干采样点。
 - 每个采样点的 feature 用 bilinear interpolation 算出来。
 - 对采样点做 average pooling 或 max pooling，得到该 bin 的 feature。
- bilinear interpolation 的意思是：如果采样点落在四个 feature grid points 中间，就用周围四个点按距离加权求值。

如果采样点是 (x, y) ，周围四个 grid points 的值是 $Q_{11}, Q_{12}, Q_{21}, Q_{22}$ ，那么输出不是强行取最近点，而是类似：

$$f(x, y) = \sum_{i,j} w_{ij} Q_{ij}$$

其中 w_{ij} 由采样点到四个邻居的距离决定。

RoIAlign 的关键不是 *pooling*，而是 *align*。

RoIAlign 不是说完全不用 *pooling*。

它仍然会把 *RoI* 切成固定数量的 bins，也仍然会把每个 bin 变成一个 *feature value*。

真正的改动是：坐标不再粗暴取整，而是在连续坐标上用 *interpolation* 采样。

所以名字叫 *Align*：让 *RoI feature* 和原图中的 *object* 尽量对齐。

5.2.3 两者的核心区别

为什么 *RoIAlign* 对 *Mask* 更重要。

bounding box regression 对几个 *pixels* 的 *offset* 可能还能容忍。

mask prediction 是 *pixel-level output*，边界位置很敏感。

如果 *RoI feature* 已经因为 *quantization* 对不齐，*mask head* 再强也会继承这个误差。*RoIAlign* 的核心价值就是 *preserve spatial alignment*。

但要注意：*Mask R-CNN* 的 *mask head* 基本只看 *RoIAlign* 截出来的 *fixed-size RoI feature*，而不是同时看整张原图。

也就是说，*mask head* 的任务是：

$$\text{given this RoI feature, predict the object mask inside this RoI}$$

如果前面的 *proposal / refined box* 没有完整包住 *object*，*RoI* 外面的部分通常很难被 *mask head* 找回来。输出 *mask* 最后也会被放回这个 *RoI* 范围内。

所以 *Mask R-CNN* 很依赖 *box quality*: *RPN* 和 *box regression* 负责先把 *object* 大致框完整，*mask head* 再在框内细化边界。

总结：

方法；做什么；坐标处理；主要问题 / 优势

RoIPool; variable-size RoI $\rightarrow$ fixed-size feature; round / quantize; 简单，但会 misalign

RoIAlign; variable-size RoI $\rightarrow$ fixed-size feature; no quantization + bilinear interpolation; 更精确，适合 mask / keypoint

一句话记：

RoIPool 是“框出来，取整，切格子，池化”；**RoIAlign** 是“框出来，不取整，用插值精确采样，再池化”。

5.3 Keypoint Prediction 也是 dense map prediction p.40

Mask R-CNN 框架还可以扩展到 keypoint prediction。

如果有 K 个 keypoints，可以训练模型输出 K 张 one-hot heatmaps：

$$\hat{H} \in \mathbb{R}^{K \times H \times W}$$

每个 heatmap 对应一个 keypoint 的 location distribution。训练时可以对 spatial positions 做 cross-entropy。

这说明 dense prediction 不只用于 segmentation mask，也可以用于 pose / keypoint localization。

6 Interactive Segmentation: 从 scribbles / points 到 promptable segmentation p.42

interactive segmentation 的目标是：用户提供少量 interaction，模型输出目标 object mask。

常见 prompts：

- scribbles: foreground / background 涂鸦。p.42
- points: 绿色 foreground point、红色 background point。p.44
- boxes: 框出目标区域。
- masks: 已有 mask 或 coarse mask refinement。
- text: 用语言描述目标。

这种任务强调 human-in-the-loop: 用户不用完整标注 mask，只需要给 sparse guidance。

6.1 Segment Anything Model (SAM): promptable segmentation foundation model p.45

Segment Anything Model (SAM) 被课件称为第一个 promptable segmentation foundation model。

它的目标不是只为某个固定 dataset 或 class set 做 segmentation，而是：

- 输入 image。
- 输入 prompt，例如 points、boxes、text、mask。
- 输出 valid segmentation mask。
- 在 zero-shot setting 中泛化到新数据。

SAM 的系统由三个部分构成: task、model、data engine。p.46

6.2 SAM Task: Promptable Segmentation p.47

SAM 把 prompt 分成两类：

- sparse prompts: clicks、boxes、text。
- dense prompts: masks。

promptable design 的意义是：SAM 可以作为更大系统中的组件，而不是只跑单一 segmentation benchmark。

例如：

- detection model 先给 box，SAM 根据 box 出 mask。
- user click 给 foreground / background points，SAM 输出候选 masks。
- language grounding model 找到 object，SAM 做 precise segmentation。

6.3 SAM Model: image encoder / prompt encoder / mask decoder p.48

SAM 的模型结构可以拆成：

- image encoder: heavy Vision Transformer (ViT)，先把 image 编成 dense image embeddings。
- prompt encoder: 把 point / box / text / mask prompts 编码成 prompt embeddings。
- mask decoder: 用 prompts 和 image embeddings 预测 masks。

课件还指出 SAM 的 image encoder 和 Masked Autoencoder (MAE) pre-training 有关联: encoder 只处理一部分 patches, 强调 scalable feature learning。p.49

SAM 的 *prompt encoding*。

point prompt: 位置 *positional encoding* + *foreground / background learnable embedding*。

box prompt: 两个 *corner* 的 *positional encoding* + *corner type embedding*。

text prompt: *Contrastive Language-Image Pre-training (CLIP)* *test encoder*。

mask prompt: *convolutions* 后和 *image embeddings* 做 *element-wise sum*。

6.4 SAM Mask Decoder: prompt-image 双向 attention p.51

SAM 的 mask decoder 使用 transformer decoder layers：

- prompts 内部做 self-attention。
- query prompts 对 image embeddings 做 cross-attention。
- pointwise Multilayer Perceptron (MLP) 提供非线性。
- image tokens 也可以对 prompts 做 cross-attention。
- 最后用 transposed convolution upsample 到 mask resolution。

SAM 会预测三个 mask outputs: p.52

- 用来覆盖 whole / part / sub-part 等 ambiguity。
 - 每个 mask 有一个 output token。
 - training 时只对多个 masks 中 minimum loss 的那个 backpropagate。
 - 另有 Intersection over Union (IoU) prediction token 用于 rank masks。
- 为什么 *SAM* 一次输出多个 *masks*。
- 一个 *prompt* 可能本身有歧义。

例如点在人身上，用户可能想要整个人、上半身、衣服区域，甚至一个局部部件。

SAM 输出多个 *candidate masks*，再用 *predicted IoU* 排序，可以把 *ambiguity* 显式

保留下来，而不是强行只给一个答案。

6.5 SAM Efficiency: heavy encoder offline, light decoder interactive p.53

SAM 的交互效率来自分工：

- heavy image encoder 先提取 image embeddings。
- embeddings 可以存在 browser。
- lightweight prompt encoder 和 mask decoder 在用户交互时快速运行。这让用户每次点击时不需要重新跑整个 ViT image encoder。

6.6 Model-in-the-Loop Data Engine: SA-1B 如何被构建 p.54

SAM 的 data engine 分三个阶段：

- assisted manual stage: 从已有 datasets 开始，模型预测后由人类修正，经过多轮 retraining。
 - semi-automatic stage: 用当前模型输出提高 mask diversity，人类 refine，周期性 retrain。p.55
 - fully automatic stage: 已有 ambiguity-aware model 后，用 32×32 grid points 自动 prompt，保留 high-confidence masks，并做 Non-Maximum Suppression (NMS)。p.56
- Data Engine 的关键点。

SAM 的能力并不只是 architecture 带来的。

它把 model improvement 和 dataset growth 绑在一起：模型越好，越能帮助标注更多 masks；数据越多，模型又进一步变强。

这和后面 Depth Anything / OWLv2 的 self-training 思路类似：foundation model 很大程度上来自 scalable data loop。

6.7 SAM Evaluation and Limitations p.57

SAM 的 zero-shot single point valid mask evaluation：

- training dataset: whole Segment Anything 1 Billion Mask dataset (SA-1B)。
 - test datasets: 23 diverse segmentation datasets。p.57
 - 结果：1 point prompt 下显著超过 baselines，多点时也有竞争力。p.58
- 局限：p.59
- 可能漏掉 fine structures。
 - 有时 hallucinate small disconnected components。
 - boundaries 可能错误。
 - 多点交互下，dedicated interactive segmentation methods 可能更强。
 - domain-specific tools 在专门领域可能更好。
 - text-to-mask 不完全 robust。
 - 虽然初始化用 self-supervised technique，但主要能力来自 large-scale supervised training。

6.8 Grounded-Segment-Anything: Grounding DINO + SAM p.60

Grounded-Segment-Anything 把 open-vocabulary detection 和 segmentation 组合起来：

text prompt → Grounding DINO → boxes / regions → SAM → masks

它的能力包括：p.61

- automatic image annotation: raw image → labeled masks。
- controllable image editing: 用 diffusion models + masks 编辑特定 objects。
- 3D human motion understanding。

Grounding DINO 和 SAM 的分工。

Grounding DINO 负责“which object should we segment?”, 也就是把 language prompt grounding 到 image regions。

SAM 负责“what is the precise mask?”, 也就是根据 region / point / box prompt 产生 pixel-level segmentation。

两者组合后，language 指令就能驱动 dense mask output。

6.9 SAM 2:从 image segmentation 到 video object segmentation p.62

SAM 2 (Segment Anything Model 2) 是 promptable segmentation foundation model：

- 支持 images 和 videos。
- 用 points / boxes 等简单 prompts segment any object。
- zero-shot setting 下不需要 task-specific training。

SAM 2 的 video extension 关键是 memory mechanism：p.63

- 存储 previous frames 的信息。
- 支持跨 video 的 consistent tracking。
- motion / occlusion 下保持 object identity。

- 支持 streaming processing 和 interactive correction。
- 能力：p.64
- track objects across video frames。
 - segment multiple objects consistently。
 - higher accuracy + fewer interactions，课件写约 3× less。
 - faster and more robust than SAM。

7 Depth Estimation: 从 supervised / stereo 到 Depth Anything p.65

depth estimation 是 dense prediction 的另一条主线：每个 pixel 输出一个 depth value。

传统 learning-based methods 可以分成：

- supervised learning: 用 labeled single image data，模型输出 depth，与 target depth 监督训练。p.66
- unsupervised learning: 用 stereo data，通过 view consistency / photometric reconstruction 等信号训练。p.67

Supervised Depth 和 Stereo Self-Supervision 的差别。

supervised monocular depth 依赖 ground-truth depth label，通常来自 LiDAR、RGB-D camera 或 synthetic data。

stereo self-supervision 不直接需要 depth label，而是利用左右相机视角的几何关系：如果 depth 预测正确，就应该能把一个视角 warp 到另一个视角并重建图像。

前者 label expensive，后者依赖 camera geometry 和 photometric assumptions。

7.1 Depth Anything: labeled + unlabeled data 的 teacher/student p.68

Depth Anything: Unleashing the Power of Large-Scale Unlabeled Data (CVPR 2024) 的核心：

- mix labeled and unlabeled datasets。
- teacher/student network。

teacher network: p.69

- frozen state-of-the-art (SOTA) encoder。
 - LDiW-pretrained ZoeDepth。
 - 对 65M unlabeled images 生成 pseudo ground truth。
- student network:
- nearly-frozen encoder，带 strong semantic prior。
 - lightweight from-scratch head。

- 使用 DINOv2 + transformer head。
- training data: 1.5M labeled images + 65M unlabeled images 的 teacher pseudo labels。

7.2 Depth Anything 为什么可能超过 teacher p.70

课件提出问题：student 如何 learn beyond teacher?

关键是 perturbation trick: p.71

- 对 unlabeled images 加 color jitter。
- 加 Gaussian blurring。
- 使用 CutMix。
- 让 student 从 perturbed / mixed input 学 teacher 给出的 pseudo label。

Perturbation Trick 的直觉。

如果 student 只是复制 teacher 在原图上的输出，它最多学到 teacher 的函数。

perturbation 让 student 看到更宽、更丰富的输入分布，同时仍然被要求输出稳定 depth。

这像 consistency regularization：模型不应因为 color jitter、blur、CutMix 这类扰动就改变几何理解。大量 unlabeled data + strong semantic backbone 让 student 有机会泛化得比 teacher 更好。

7.3 Depth Anything 的应用: image / video editing p.72

depth map 可以作为 image / video generation 的控制 signal。

例如 ControlNet-style generation 中：

- depth map 提供 scene geometry / layout。
 - text prompt 提供 semantic style / object description。
 - diffusion model 根据 depth condition 生成保持结构一致的 image / video。
- 这把 depth estimation 和 generation 串起来：前者是 dense geometry predictor，后者使用 dense condition 做 controlled synthesis。

7.4 Depth Anything V2: 用 synthetic data 解决 real depth label noise p.73

depth ground truth 也有 label noise：

- real sensor depth 可能稀疏、缺失、反光失真。
 - datasets 之间 scale / calibration / noise pattern 不一致。
- Depth Anything V2 的核心思想：p.74
- 用 synthetic data 获得更精确的 depth labels。
 - 再 bridge sim-to-real gaps。

训练结构：p.76

- teacher 在 synthetic images 上训练，减少真实数据 label noise。
- student 在 real images 上用 pseudo labels 训练，处理 sim-to-real gap。Synthetic Data 的优势和风险。

synthetic depth 的优势是几何 label 可以非常精确，因为渲染引擎知道真实 scene depth。

风险是 domain gap: synthetic image 的 texture、lighting、object distribution 和真实世界不同。

V2 的思路是先利用 synthetic label 的准确性训练 teacher，再用真实图像上的 pseudo labels 把能力迁移回 real domain。

7.5 Beyond Depth Anything: 从 single-image depth 到 view-consistent geometry p.78

Depth Anything series 的限制：

- single-image depth 在 multi-view images 中可能 inconsistent。
- 不能恢复 true 3D structure。
- 没有 explicit camera geometry notion。

新的目标是学习 view-consistent representation：模型不仅预测 depth，还要学习 geometry 本身。

7.6 Depth Anything 3: depth + ray + camera pose p.79

Depth Anything 3 被课件描述为 toward 3D foundation models 的一步：

- plain transformer。
- 使用 depth-and-ray targets。
- teacher-student supervision。
- training objectives 包括 depth estimation、cross-view geometry alignment、camera pose estimation。

它在一些 benchmarks 表现不错，但某些 depth estimation benchmarks 上仍 underperform Depth Anything V2，并且 physical consistency 仍不完美。

Depth vs. 3D Geometry。

单张 depth map 是 2.5D 表示：每个 pixel 一个距离，但没有完整 scene topology，也不保证跨视角一致。

真正的 3D representation 需要考虑 camera rays、pose、multi-view correspondence 和 geometry consistency。

所以 Depth Anything 3 的意义是把 dense prediction 从 per-image depth 推向 multi-view geometric foundation model。

8 Generation as Dense Prediction: GAN / Pix2Pix / Diffusion p.80

课件最后把 image generation 放回 dense prediction 视角：生成模型不只是 sample 一个 image，而是在 pixel / patch / latent grid 上持续预测 dense values。

粗略时间线：p.81

- 2014: Generative Adversarial Network (GAN), adversarial training paradigm。
- 2014: Conditional Generative Adversarial Network (cGAN), 用 label / condition 控制生成。
- 2017: Pix2Pix, 用 conditional GAN + U-Net 做 image-to-image translation。
- 2019: StyleGAN, style-based generator + progressive growing。
- 2020: Denoising Diffusion Probabilistic Model (DDPM), iterative denoising paradigm。
- 2022: Latent Diffusion Model (LDM) / Stable Diffusion, 在 latent space 中 diffusion。
- 2023: Diffusion Transformer (DiT), 用 Vision Transformer 替代 U-Net backbone。

8.1 GAN / cGAN: Generator and Discriminator the two-player game p.82

GAN 有两个网络：

- Generator (G): 把 latent noise $z \sim p_z(z)$ 映射到 data space，产生 synthetic sample $G(z)$ 。
- Discriminator (D): 二分类器，判断输入来自真实数据 $p_{data}(x)$ 还是 generator。

标准 minimax objective:

discriminator 的目标：

- real samples 输出 high $D(x)$ 。
- generated samples 输出 low $D(G(z))$ 。

generator 的目标：

- 让 $D(G(z)) \rightarrow 1$ 。
- fool discriminator，使 generated samples 看起来像 real。

8.2 Training Dynamics and Nash Equilibrium p.85

GAN 训练交替进行：

- 固定 G ，更新 D ，提升 real/fake discrimination。
 - 固定 D ，更新 G ，最小化 $\log(1 - D(G(z)))$ ，或为了更好梯度最大化 $\log D(G(z))$ 。
- 如果 D 对每个 x 都接近 optimal，则 generator 的目标和 Jensen-Shannon divergence 有关：

$$JS(p_{data} \parallel p_g)$$

Nash equilibrium 下：

- G 产生的 samples 与 real data indistinguishable。
- $D(x) = 0.5$ 对所有 inputs。
- generated distribution 接近 real distribution。

为什么 GAN 难训练。

GAN 不是单模型 supervised loss，而是两个模型互相追逐。

如果 D 太强， G 梯度可能消失；如果 G 找到少量能骗过 D 的 patterns，就可能 mode collapse。

所以后来的 image generation 主流逐步转向 diffusion：训练目标更稳定，虽然 sampling 更慢。

8.3 Pix2Pix: paired image-to-image translation p.86

Pix2Pix 使用 conditional GAN 学习 paired training data 上的 mapping:

$$x \rightarrow y$$

例如：

- edge map → photo。
- segmentation map → street scene。
- sketch → realistic object。

它不是从 random noise 直接生成，而是用 input image 作为 condition。输出 image 中每个 pixel 都由 input image 的对应区域和 context 决定，所以这是 dense pixel-to-pixel prediction。

8.4 Pix2Pix Generator: U-Net 再次出现 p.88

Pix2Pix 的 generator 使用 U-Net architecture：

- encoder 把 input image 压到 high-level representation。
- decoder 逐步 upsample 到 output image。
- skip connections 保留低层 spatial details。

这再次说明 U-Net 是 dense prediction 的通用 backbone：segmentation、depth、image translation、diffusion 中都会反复出现。

8.5 PatchGAN: Discriminator 也可以是 dense predictor p.89

PatchGAN discriminator 不输出整张图一个 scalar，而是输出 spatial probability map:

$$D(x, y) \in [0, 1]^{H' \times W'}$$

每个 output scalar 判断原图中对应 70×70 patch 是否 realistic。

课件给的 architecture：

- 5 convolutional layers。
- batch normalization。
- LeakyReLU activations。
- 每个 output scalar 的 receptive field 是 70×70。

PatchGAN 的作用：

- 关注 local patches，而不是整张图一个全局判断。
- 更强地约束 high-frequency details 和 textures。
- 让 generated image 的局部结构更 sharp。

PatchGAN 为什么也是 Dense Prediction。

虽然 PatchGAN 是 discriminator，不是 generator，但它输出的是 spatial map。

每个 map location 都在判断对应 patch 的 realism。

这和 semantic segmentation 的形式很像，只是 label 从“class”变成“real/fake patch”。

8.6 Denoising Diffusion Probabilistic Model (DDPM): 逐步预测 noise p.90

DDPM 学习 reverse gradual noising process。forward process 逐步加 Gaussian noise:

$$q(x_t \mid x_{t-1}) = \mathcal{N}(x_t; \sqrt{1 - \beta_t} x_{t-1}, \beta_t I)$$

reverse process 学习预测 noise $\epsilon_\theta(x_t, t)$ ，并从 x_t 得到 x_{t-1} ：

$$x_{t-1} = \frac{x_t - \frac{\beta_t}{\sqrt{1-\alpha_t}}\epsilon_\theta(x_t, t)}{\sqrt{1-\beta_t}} + \sigma_t z$$

dense prediction view:

- 输入 noisy image x_t 。
- 输出每个 pixel / channel 上的 noise ϵ 。
- 或等价地预测 clean image x_0 。

8.7 Latent Diffusion Model (LDM): 在 latent space 做 dense prediction p.92

Latent Diffusion Model (LDM) 不在 pixel space 中直接 diffusion, 而是在 Variational Autoencoder (VAE) 学到的 compressed latent space 中 diffusion。

组件:

- VAE encoder E : 把 image x 压缩成 latent z 。
 - VAE decoder D : 把 latent z 解码回 reconstructed image $\hat{x}$ 。
 - U-Net denoiser: 在 latent space 中预测 noise。
 - text encoder, 例如 Contrastive Language-Image Pre-training (CLIP): 编码 text prompts 做 conditioning。
- dense prediction view:

$$z_t \rightarrow \epsilon_\theta(z_t, t, c)$$

模型在 latent grid 上预测 noise, 而不是在 pixel grid 上预测 noise。

LDM 的优势: p.94

- 在低维 latent space 中做最贵的 denoising step。
- 比 vanilla pixel-space diffusion 更 efficient。
- 在较少参数和计算量下得到高质量 FID / IS scores。

Pixel Space vs. Latent Space.

pixel space 分辨率高, 直接 denoise 成本大。

latent space 把图像压缩成较小 feature grid, 让 diffusion 模型处理更低维的 dense representation。

最终 image detail 由 VAE decoder 恢复, 所以 LDM 的 dense prediction 发生在 latent map 上。

8.8 Diffusion Transformer (DiT): 用 Vision Transformer 做 denoiser p.95

Diffusion Transformer (DiT) 用 Vision Transformer (ViT) 替代 DDPM / LDM 中常见的 U-Net backbone。

结构:

- patchify: 把 image 或 latent split 成 patches, 例如 16×16 。
- transformer blocks: 用 self-attention 在 patches 之间建模型 global dependency。
- Adaptive Layer Normalization (adaLN): 注入 timestep 和 class conditioning。
- output: 对每个 patch 预测 noise。

DiT 的意义:

- transformer scaling laws 更清晰。
- self-attention 更容易捕捉 long-range relation。
- generation 仍然是 dense prediction, 只是 dense unit 从 pixel / latent location 变成 patch token。

9 Lecture Map: Dense Prediction 的统一主线 p.6

这讲可以整理成一条统一线索:

- dense prediction = every pixel / location has a prediction. p.2
- segmentation, depth, generation 都可以放进这个框架。 p.3
- FCN 把 classifier 改成全卷积结构, 用 1×1 conv 和 upsampling 做 pixel-wise prediction. p.7
- dense prediction 的基本矛盾是 semantic abstraction vs. spatial localization. p.8
- transposed convolution, unpooling, skip connection 是恢复 resolution 的基本工具。 p.17
- DeconvNet, U-Net, FPN 都在解决 multi-scale feature fusion. p.26
- Mask R-CNN 把 Faster R-CNN 扩展到 instance-level mask prediction. p.32
- SAM 把 segmentation 变成 promptable foundation model, 并通过 data engine 扩大 mask supervision. p.45
- SAM 2 把 promptable segmentation 推到 video object segmentation. p.62
- Depth Anything 用 teacher/student 和 unlabeled data scaling 做 monocular depth. p.68

• diffusion / Pix2Pix / PatchGAN 说明 generation 也可以被看作 dense prediction. p.89

Exam / Review Focus.

复习时优先抓这些对比:

- classification vs. dense prediction: global label vs. spatial map。
- semantic / instance / panoptic segmentation: what / which / both。
- FCN vs. U-Net: score fusion by summing vs. feature fusion by concatenation。
- transposed convolution vs. unpooling: learned painting vs. pooling switches。
- RoIPool vs. RoIAlign: quantized bins vs. bilinear interpolation。
- Mask R-CNN vs. semantic segmentation: RoI-level instance masks vs. full-image class map。
- SAM vs. traditional segmentation: fixed class prediction vs. promptable segmentation。
- Depth Anything V1 / V2 / V3: unlabeled scaling, synthetic labels, multi-view geometry。
- GAN / Pix2Pix / DDPM / LDM / DiT: generation models can also be read as dense predictors over pixels, patches, or latents。

created: 2026-05-27

updated: 2026-05-27

1 Synthetic Data for Visual Perception: 用可控世界生成难以人工标注的监督 p.3

Synthetic data 指人工生成的数据, 而不是直接从真实传感器采集的数据。它可以来自 simulation, 3D engine, procedural generation 或 generative model。

这讲的核心不是“synthetic data 能不能替代 real data”, 而是:

- synthetic data 提供真实世界很难或几乎不可能人工标注的 structured supervision。
- real data 负责把模型拉回真实 distribution。
- 现代视觉系统常用的范式是 synthetic pre-training + real-world adaptation。

典型 synthetic labels 包括:

- depth: 每个 pixel 的 metric depth 或 relative depth。
- optical flow / scene flow: 每个 pixel 或 3D point 的 motion。
- surface normal: 每个 pixel 的 surface orientation。
- segmentation: semantic / instance masks。
- point trajectory: query point 在 video 中跨时间的位置和 visibility。
- six degrees of freedom (6-DoF) pose: object 的 3D rotation + translation。

Real Data vs. Synthetic Data.

real data 的优势是真实 distribution, 但 label expensive, privacy risk 高, 而且 rare scenarios 很难覆盖。

synthetic data 的优势是 perfect Ground Truth (GT), scalable, controllable。渲染器知道 scene geometry, material, camera, lighting, 所以 dense annotation 可以“免费”得到。

代价是 domain gap: synthetic image 可能 lighting 太干净, texture 太简单, sensor noise 不真实, 导致模型 synthetic accuracy 高但 real accuracy 差。

1.1 Historical Evolution: 从 toy examples 到 standard pre-training p.4

课件给出的时间线:

- 2010s: FlyingThings、Grand Theft Auto V (GTA5) 等合成数据开始用于 flow, segmentation 等任务。
 - 2020s: Kubric 这类 procedural generator 成为 tracking, video understanding, dense prediction 的标准预训练来源。
 - future: Generative Artificial Intelligence (GenAI) / world models 可能进一步生成更真实、更多样的训练世界。
- paradigm shift:
- before 2015: synthetic data 常被看成 toy examples, real data 是 gold standard。
 - after 2020: synthetic pre-training 在 FlowNet, Persistent Independent Particles (PIPs)、CoTracker 等任务中成为常规操作。
 - trend: hybrid training, 先在 synthetic data 上学 geometry / correspondence, 再用 real data fine-tune 或 self-train。

1.2 How Synthetic Data is Generated: 三条路线 p.5

synthetic data 的来源可以分为三类:

- 3D engines:
 - 例子: Unreal, Unity, GTA5, SYNTHIA, AI2-THOR。
 - 优势: 场景真实感较强, 可以直接输出 RGB + GT。
 - 限制: asset-dependent, diversity 受限。
- procedural generation:

• 例子: Blender + randomization, physics simulation, FlyingThings, Kubric。

- 优势: 无限 variation, 完美 GT, 适合系统性覆盖 motion, occlusion, geometry。
- 限制: 外观 realism 可能不如真实数据或精心制作的 3D world。
- neural / generative:
 - 例子: diffusion models, Generative Adversarial Network (GAN) refinement, Neural Radiance Field (NeRF), 3D Gaussian Splatting (3DGS), Zero-1-to-3。
 - 优势: 图像 realism 强, 容易生成多样外观。
 - 限制: GT 可能 noisy / inconsistent, 尤其是 geometry, pose, normal 这类需要严格 3D consistency 的 label。

三种路线的核心 trade-off.

3D engine 重在 realistic scenes, 但受限于 asset 和场景设计。

procedural generation 重在 controllability 和 perfect annotation, 常适合 tracking, flow, pose, depth, normal 这类 geometry-heavy tasks。

generative route 重在 visual realism, 但如果只生成 RGB 而没有一致的 3D world state, GT 会变得不可靠。

1.3 Domain Gap: synthetic accuracy 不等于 real accuracy p.6

domain gap 是 synthetic data 的中心问题:

$$P_{\text{syn}}(x, y) \neq P_{\text{real}}(x, y)$$

synthetic domain 常见特点:

- simplified lighting。
- clean textures。
- no sensor noise。
- complex lighting。
- motion blur。
- sensor noise。
- missing depth / holes / edge artifacts。

如果模型只在 synthetic domain 上训练, 它可能学到过于干净的 appearance cue, 到了真实图像就失效。

1.4 Standard Paradigm: Synthetic → Real p.7

现代方法常用两阶段:

- synthetic pre-training:
 - 输入: Kubric, FlyingThings 等带 perfect GT 的 synthetic data。
 - 作用: 学底层 correspondence, geometry, motion, shape prior。
- real-world adaptation:
 - 输入: real videos / real images。
 - 监督: teacher model 生成 pseudo labels, 或者少量真实标注。
 - 作用: 做 distribution alignment。

课件强调 quality > quantity: CoTracker3 使用 15K videos, 可以超过 BootS-TAPIR 使用 15M videos 的路线, 说明 pseudo-label quality 和 data design 有时比 blind scaling 更重要。

为什么 synthetic pre-training 仍然有用。

synthetic world 虽然不完全真实, 但它能提供真实世界标不出来的信号: 长期 occlusion 后的 point identity, 每个 pixel 的 exact depth, 每个 surface 的 normal, object 的 exact 6-DoF pose。

这些 supervision 会让模型先学到 task structure。真实数据再负责校准 appearance distribution, sensor noise 和 edge cases。

1.5 Representative Synthetic Datasets and Generators p.8

1.5.1 FlyingThings / MPI Sintel / Monkaa: optical flow and scene flow p.8

FlyingThings3D 是 large-scale synthetic stereo dataset, 提供 3D scene flow 和 optical flow GT. p.9

MPI Sintel 来自动画电影 frames, 提供 dense optical flow GT, 并有 albedo, clean, final render 等。 p.10

Monkaa 是 photorealistic animated scenes, 提供 dense optical flow. p.12

这些 dataset 的共同价值是: motion field 可以从渲染器和 3D scene state 精确计算出来, 不需要人工逐 pixel 标注。

1.5.2 Kubric: object-centric procedural video generator p.13

Kubric 是 DeepMind 的 procedural data generation framework:

- Blender + PyBullet pipeline。
- asset sources 包括 ShapeNet, Google Scanned Objects (GSO), KuBasic。
- 生成 RGB, depth, normal, segmentation, optical flow, object coordinates, point trajectories 和 occlusion labels. p.14

• MOVi dataset family 由 Kubric 生成。 p.15

worker pipeline 可以理解为:

Asset Source → Scene → PyBullet + Blender → layers + metadata

Kubric 的关键是 task-agnostic: 一次 render pass 中拿到多任务 GT, 因此可以服务 tracking, flow, segmentation, 6-DoF pose, video understanding。

1.5.3 Hypersim / Replica: photorealistic indoor geometry p.16

Hypersim 提供 photorealistic indoor scenes, 并带 metric depth 和 dense surface normals. p.17

Replica 提供 high-quality 3D reconstructions, 也能渲染 RGB, depth, surface normal 等 dense GT. p.18

Replica 的 Turing Test 展示了 real captures 和 mesh renderings 的接近程度。 p.20

1.5.4 GTA5 / AI2-THOR: game engine and interactive simulation p.22

GTA5 代表 open-world game engine synthetic data:

- RGB frames。
- pixel-level semantic GT。
- 可用于 segmentation 和 pose。

AI2-THOR 代表 interactive indoor simulation:

- RGB / depth / normals / semantic / instance annotations。
- 支持 iTHOR interactive scenes 和 physics。

1.6 Key Messages of Synthetic Data p.25

这讲的总论可以压缩成四句话:

- synthetic data is a complement, not a substitute。
- domain gap is the central challenge。
- data quality often beats quantity。
- hybrid training is the dominant paradigm。

2 Synthetic Data for Point Tracking: 长期轨迹和 occlusion 几乎只能合成监督 p.27

Point tracking 的输入输出:

- input: video sequence + query points。
 - output: 每个 query point 的 trajectory over time, 以及 visibility flags。
- 和普通 optical flow 不同, point tracking 要跟踪任意 query point, 而且要 long-term through occlusions。

形式上, 对于 query point $p_i = (x_i, y_i)$, tracker 需要输出:

$$\{(x_{i,t}, y_{i,t}, v_{i,t})\}_{t=1}^T$$

其中 $v_{i,t}$ 表示该点在第 t 帧是否 visible。

2.1 Why Tracking Needs Synthetic Data p.28

真实人工标注 point tracking 非常困难:

- 一个 point 可能需要跟踪 200+ frames。
- occlusion 后再次出现, 需要判断 identity 是否仍然是同一个 surface point。
- 每一帧都要给 visibility flag。
- sub-pixel position 很难人工稳定标注。

synthetic data 可以提供:

- exact 3D point positions across frames。
- ray-casting 得到 automatic occlusion detection。
- dense, sub-pixel accurate trajectories。

Occlusion 为什么是 tracking 的核心难点。

如果点一直可见, tracking 更像 local matching。

一旦 point 被遮挡, 模型必须记住它属于哪个物体 surface, 并在重新出现时恢复 identity。 synthetic data 能用 3D scene state 判断“这个点是否真实可见”, 这是普通视频人工标注很难做到的。

2.2 Synthetic-to-Real Pipeline in Tracking p.29

tracking 中主流路线:

- pre-train on Kubric / TAP-Vid-Kubric synthetic GT。
- 得到 Persistent Independent Particles (PIPs), Tracking Any Point with per-frame Initialization and temporal Refinement (TAPIR), CoTracker 等 initial model。
- 用 teacher tracker 在 unlabeled real videos 上生成 pseudo-labels。
- self-train student, 得到 real-world adapted tracker。

关键问题是:

- synthetic data enough 吗?
- real videos 需要多少?
- pseudo-label quality 如何控制?

2.3 Persistent Independent Particles (PIPs): Tracking Any Point with Synthetic Pre-training p.30

PIPs 是早期 tracking any point 方法:

- 使用 RAFT-style correlation volume.
- 用 12-block Multilayer Perceptron Mixer (MLP-Mixer) iteratively refine trajectories.
- 处理 8-frame chunks.
- 使用 zero-velocity initialization.
- visibility-aware linking 把 short tracks 链接成长轨迹.
- 训练数据是 FlyingThings++ synthetic data.

PIPs 的限制是 chunking 和 sequential chaining 较慢, 而且 long-term occlusion 处理仍受限.

2.4 Tracking Any Point Video (TAP-Vid)-Kubric: 专门为 long-term tracking 设计的数据 p.31

TAP-Vid-Kubric 的特点:

- long trajectories, 至少 24+ frames.
- ray-casting 得到 perfect visibility.
- 1, 141 training sequences.

它的重要性在于:这是早期带 long-term occlusion labels 的 synthetic tracking dataset, 让模型可以直接学习 occlusion-aware point identity.

2.5 TAP-Net and TAPIR: 从 per-frame matching 到 unified refinement p.32

Tracking Any Point Network (TAP-Net):

- per-frame cost-volume search.
- fully-convolutional and parallel.
- 速度快, 但没有 temporal smoothing, 所以容易 jitter.

TAPIR 结合 TAP-Net 和 PIPs. p.33

TAPIR 的三个 upgrade: p.34

- TAP-Net initialization 替代 PIPs 的 slow chaining.
- fully-convolutional network 替代 MLP-Mixer, 去掉 chunking.
- position uncertainty, 保护下游 3D reconstruction 等任务.

TAPIR 的结果: p.35

- DAVIS 比 PIPs 高约 19%.
- Kinetics 比 TAP-Net 高约 10%.
- 速度约 150 frames per second (FPS), 而 PIPs 是 seconds per clip 级别。

PIPs, TAP-Net, TAPIR 的关系.

PIPs 强在 *iterative refinement*, 但慢。

TAP-Net 强在 *per-frame global matching*, 但轨迹容易 jitter.

TAPIR 把 TAP-Net 的 *fast initialization* 和 *temporal refinement* 结合起来, 并显式预测 *uncertainty*, 因此是 *point tracking pipeline* 的一次统一。

2.6 CoTracker: joint tracking and cross-track attention p.38

classical independent tracking 是:

$$p_i \rightarrow t_i$$

每个 point 单独得到 trajectory, 不共享信息。

CoTracker 的 joint tracking 是:

$$\{p_i\}_{i=1}^N \rightarrow \{t_i\}_{i=1}^N$$

模型让 tracks 之间通过 attention 共享信息。

cross-track attention 的意义:

- neighboring points 可能属于同一个 rigid object 或同一 surface.
- 一个点 occluded 时, 附近仍可见点可以帮助恢复.
- synthetic occlusion labels 提供了“遮挡不等于 identity 消失”的训练信号。

2.7 TAPTR: 把 point tracking 看成 detection p.39

Tracking Any Point with Transformers (TAPTR) 采用 Detection Transformer (DETR)-style architecture, 并在 TAP-Vid-Kubric 上训练.

核心想法:

- tracking as detection: 在每一帧中, query point 的 tracking 就是在检测它最匹配的位置。
- structured point query: 每个 query 有 position part 和 content part.
- cross-attention 采样 local image features, 保留 geometric detail.
- self-attention 让 neighboring points share context.
- temporal attention 在 frames 之间传播信息。

Tracking 和 Detection 的类比.

detection 中 *object query* 在图像中寻找一个 *object*.

tracking 中 *point query* 在每一帧中寻找同一个 *surface point*. 差别是 *point tracking* 还需要 *temporal consistency* 和 *visibility reasoning*.

2.8 BootsTAPIR: 用 15M real videos 做 teacher-student self-training p.40

BootsTAPIR 的路线:

- 在 Kubric synthetic GT 上 pre-train.
- 用 TAPIR 作为 teacher 处理 unlabeled real videos.
- 生成 trajectories + visibility pseudo labels.
- 用 15M real videos 训练 student.

结论:

- real data scaling 显著提升 generalization.
- 但 compute cost 巨大.
- 它验证了 synthetic pre-training + real pseudo-labeling 的有效性。

2.9 CoTracker3: better pseudo-labels beat more data p.41

CoTracker3 的问题意识是: BootsTAPIR 用 15M videos 才达到 state-of-the-art (SOTA), 是否真的需要这么多?

CoTracker3 的答案:

- simpler architecture.
- multi-teacher pseudo-labeling.
- 只用 15K videos.
- 达到或超过 SOTA.

架构简化: p.42

- no global matching.
- simple MLP for 4D correlation.
- unified updates for tracks / confidence / visibility.
- online / offline 使用相同 architecture.
- 参数量约比 CoTracker 少 2x, 速度比 LoCoTrack 快 27%.

multi-teacher pseudo-labeling: p.43

使用 diverse frozen teachers: CoTracker3 online/offline, CoTracker, TAPIR.

- 每个 batch 随机采样一个 teacher.
- 用 Scale-Invariant Feature Transform (SIFT) sampling 找 good-to-track keypoints, 过滤 ambiguous regions.
- 用 Huber loss 训练 student.

training details: p.44

- freeze confidence / visibility head, 避免 real pseudo-label training 时遗忘.
- self-training with own predictions 带来约 +1.2 points.
- every teacher matters.
- random sampling 优于 averaging.
- SIFT 优于 LightGlue, SuperPoint, DISK.

结果: p.45

- 用约 1000x less data 超过 BootsTAPIR.
- offline model 在 occlusion 场景中更强, 因为它可以使用 all frames 做 interpolation.

2.10 Track-On-R: verifier-guided pseudo-labeling p.46

Track-On-R 的思想是: 不要信任单个 teacher.

pipeline:

- 多个 teacher trackers 给候选 trajectories.
- verifier network 评估每条 candidate 的质量.
- 动态选择最好的 pseudo labels.
- 使用 multi-teacher voting.

verifier training:

- 在 synthetic data K-EPIC 上训练.
- perturb GT trajectories 生成 6-12 个 candidates.
- 用 soft contrastive learning 学 frame-level reliability score.
- 在真实 videos 上逐 frame 选择最佳 teacher prediction.

2.11 Key Takeaways for Point Tracking p.48

point tracking 这一节的主线:

- manual annotation of long trajectories is infeasible.
- TAP-Vid-Kubric 第一次系统提供 long-term occlusion labels.
- 主流路线是 pre-train on Kubric $\rightarrow$ pseudo-label real videos.
- CoTracker3 说明 quality > quantity.
- frontier 是更强 pseudo-label selection, 例如 Track-On-R verifier.

3 Synthetic Data for 6-DoF Pose Estimation: 从 CAD rendering 到 few-shot RGB-D matching p.50

6-DoF pose estimation 的目标是估计 object 相对 camera 的 3D pose:

$$T_{CO} = (R, t), \quad R \in SO(3), \quad t \in \mathbb{R}^3$$

其中 R 是 rotation, t 是 translation.

3.1 Model-Based 6-DoF Pose Estimation p.50

model-based setting 假设 inference 时有 object 的 Computer-Aided Design (CAD) model.

pipeline:

- training: 从 CAD models render synthetic images.
- inference: 给定 RGB / optional depth, Region of Interest (RoI) 和 CAD model.
- output: 6D pose.

常见策略是 render & compare:

- 对候选 pose 渲染 object.
- 把 rendered view 和 observed crop 比较.
- 迭代 refine pose.

优势:

- 给一个新的 3D model 就能快速部署到 novel object.
- synthetic views 可以覆盖多视角、多光照、多背景。

3.2 Model-Free 6-DoF Pose Estimation p.51

model-free setting 不要求高质量 CAD model, 只需要 novel object 的少量 labeled RGB-D views.

输入:

- a few support views.
- query scene patch.

输出:

- query scene 中 object 的 6D pose.

核心是 dense RGB-D prototypes matching: 在 support views 和 query scene patch 之间提取 dense prototypes 并做 matching.

Model-Based vs. Model-Free.

model-based 方法依赖 CAD model, 适合工业物体、机器人操控中可提前建模的物体。

model-free 方法更接近人类视觉: 看过几个视角, 就能在新场景里找一个物体并估计 pose.

synthetic data 在两者中都重要: model-based 可以直接 render CAD views; model-free 可以用大量 synthetic objects 学到 dense correspondence 和 RGB-D fusion.

3.3 Few-Shot 6D Pose Estimation (FS6D): few-shot 6D pose with ShapeNet6D, A specific realization of MF6Dp.52

FS6D 的目标:

- 从 few RGB-D support views 估计 novel object 的 6D pose.
- 不需要 CAD model.

方法:

- RGB branch 和 point cloud branch 编码 support / query.
- 通过 dense RGB-D prototype matching 建 correspondence matrix.
- 用 Umeyama algorithm 从 correspondences 解 pose.

ShapeNet6D dataset: p.53

- 800K images.
- 12K objects.
- 当时最大 synthetic 6D pose dataset.

3.4 FS6D Online Data Augmentation: texture blending + deformation p.55

FS6D 用 online data augmentation 缓解 sim-to-real gap:

- 把 ImageNet / Microsoft Common Objects in Context (MS-COCO) 的 real-world images 通过 UV mapping blend 到 object meshes 上.
- 不做复杂人工 simulation lighting / material, 而是保留 real-world texture distribution.

- online shape deformation 增加 shape diversity.
- online generation 避免大量 offline storage.

RGB-D complementary cues: p.56

RGB texture 对 smooth / texture-rich objects 很关键, 例如 decorated bowl.

- depth geometry 对 texture-less objects 很关键, 例如 plain egg.
- joint RGB-D reasoning 比单独 RGB 或 depth 更稳.

3.5 MegaPose: render & compare with synthetic data p.57

MegaPose 使用 render & compare:

- network 判断 pose error.
- iterative refinement.
- 数据来自 thousands of objects 的 Blender renders.

coarse-to-fine architecture: p.58

- coarse estimator: 从 random pose hypotheses 开始, render and score, 选择较好的 initial pose.
- refiner: 围绕当前 pose 生成 multiview renderings, 迭代 refine.

Render & Compare 的直觉.

pose estimation 可以看成 inverse graphics: 如果 pose 正确, 把 CAD model 从该 pose 渲染出来, 应该和真实 crop 对齐.

network 不必直接从图像回归全部 pose, 而是学习“当前 pose hypothesis 错在哪里”, 然后逐步修正.

3.6 FoundationPose: pure synthetic training 的 unified framework p.59

FoundationPose 是 CVPR 2024 方法, 统一 model-based 和 model-free tracking.

innovations:

- unified model-based / model-free framework.
- neural implicit novel views.
- pure synthetic training.
- training data: 600K scenes, 使用 Large Language Model (LLM) textures.

LLM-aided synthetic data generation: p.60

- LLM 生成 texture descriptions.
- diffusion model 从 text 生成 textures 并映射到 3D mesh.
- physics + rendering 组成 scenes, 并用 path tracing 渲染.

texture augmentation result: p.61

- TexFusion 用 diffusion 生成 seamless textures.
- 相比 random texture blending, 减少 random patch seams.

training at scale: p.62

- performance 随 training scale 提升.
- around 400K scenes 后趋于饱和.
- final model 使用 600K scenes.

- insight 仍然是 quality > pure scale.

Benchmark for 6D Object Pose Estimation (BOP) result: p.63

- FoundationPose 在 BOP leaderboard overall rank 1.
- 同时超过 model-based 和 model-free tracks.
- 全部来自 pure synthetic training.

3.6.1 Average Distance of Model Points (ADD) and ADD-S Metrics p.62

ADD 衡量 estimated pose 和 GT pose 作用到 object model points 后的平均距离:

$$ADD = \frac{1}{|\mathcal{M}|} \sum_{x \in \mathcal{M}} \|(Rx + t) - (\hat{R}x + \hat{t})\|_2$$

对于 symmetric objects, Average Distance of Model Points for Symmetric Objects (ADD-S) 使用 nearest GT point, 避免对称物体的等价 pose 被错误惩罚:

$$ADD-S = \frac{1}{|\mathcal{M}|} \sum_{x_1 \in \mathcal{M}} \min_{x_2 \in \mathcal{M}} \|(Rx_1 + t) - (\hat{R}x_2 + \hat{t})\|_2$$

3.7 Key Takeaways for Pose Estimation p.64

pose estimation 中 synthetic data 的趋势:

- FS6D: 用 ShapeNet6D 的 800K images / 12K objects 学 few-shot dense matching.
- MegaPose: render & compare, 把 CAD rendering 用作 pose refinement signal.
- FoundationPose: LLM + diffusion 生成高质量 textures, pure synthetic training 就能在 BOP 上达到 overall rank 1.
- texture augmentation 从 random domain randomization 走向 LLM + diffusion guided TexFusion.

4 Synthetic Data for Depth Estimation: 从 synthetic depth 到 any-camera 和真实传感器 p.66

Monocular depth estimation 可以分成:

- metric depth:
 - 输出 absolute meters.
 - 例子: SceneFlow, Hypersim, Virtual KITTI.
 - relative depth:
 - 输出 up-to-scale ordering.
 - 例子: MiDaS, Dense Prediction Transformer (DPT), Marigold.
- metric depth 更适应 robotics 和 3D reconstruction, relative depth 更容易跨 dataset 泛化.

4.1 Marigold: Stable Diffusion fine-tuned for depth p.67

Marigold 的 idea:

- fine-tune Stable Diffusion for depth prediction.
- 输出 affine-invariant depth.
- training data: Hypersim + Virtual KITTI synthetic depth.
- result: single GPU, few days training, zero-shot depth 有 20%+ gain.

training / inference: p.68

- training on synthetic depth.
- inference 使用 ensemble denoising.

为什么 *diffusion prior* 能用于 *depth*.

Stable Diffusion 本来学到的是强 *image prior*, Marigold 把它 *fine-tune* 到 *depth* 输出上, 相当于把 *generative model* 的 *representation* 转成 *geometry predictor*.

这里 *synthetic depth* 很重要, 因为真实 *metric depth label* 可能 *sparse*、有噪声, 而 *synthetic render* 可以给 *dense clean supervision*.

4.2 Depth Any Camera (DAC): generalizing depth model to any camera p.69

DAC 处理的问题:

- 普通 *depth models* 多在 *perspective cameras* 上训练.
- fisheye* / 360° *images* 会产生 *distortion*.
- 直接应用 *perspective depth model* 会失效.

solution:

- Equi-Rectangular Projection (ERP) representation.
 - gnomonic Field of View (FoV) alignment.
 - 不需要 *fisheye training data*, 也能 *zero-shot* 到 *fisheye* / 360° *cameras*.
- DAC training / inference: p.70
- training: *perspective samples* 做 online ERP augmentation, 加入 FoV-align 和 *multi-resolution augmentation*.
 - inference: any camera input $\rightarrow$ ERP image $\rightarrow$ ERP metric depth $\rightarrow$ original camera depth.

4.3 What is ERP? p.71

Equi-Rectangular Projection (ERP) 是把 *spherical camera model* 展开成矩形图像:

- 每个 pixel 对应 *latitude* λ 和 *longitude* φ .
 - full space 是 180° *latitude* $\times$ 360° *longitude*, 像展开的 *globe*.
 - 只需要 *image height* 就能定义 ERP space.
 - 不同 FoV *cameras* 可以投到 unified ERP space.
- gnomonic projection 建立 *spherical coordinates* 和 *tangent plane co-ordinates* 的对应关系:

$$(\lambda, \varphi) \leftrightarrow (x_t, y_t)$$

这个设计让一个 *depth model* 能处理不同 *camera models*.

4.4 Camera Depth Models: sim-to-real gap in depth sensors p.72

Camera Depth Models (CDMs) 不是 *monocular depth estimator*. 它的输入输出是:

$$(\text{RGB}, \text{raw noisy depth}) \rightarrow \text{denoised metric depth}$$

真实 *depth sensor* 的问题:

- noisy readings.
- incomplete depth.
- edge artifacts.
- holes.

CDM 的目标是作为 *depth cameras* 的 *plug-in*, 把 raw sensor depth 清理成更准确的 *metric depth*.

4.5 CDM Architecture and Training p.73

CDM 的 *synthetic training idea*:

- 从 *synthetic render* 得到 clean GT depth.
- 学 *camera-specific value noise* 和 *hole noise*.
- 把 *learned sensor noise* 加到 *synthetic depth* 上, 模拟真实 raw depth.
- 模型输入 RGB + noisy depth, 监督输出 clean GT depth.

architecture:

- Sensor Depth Generation: *value noise model* + *hole noise model*.
 - Camera Depth Model: RGB image 和 raw depth 分别经过 *Vision Transformer (ViT)* encoder.
 - token fusion 后使用 *DPT decoder* 输出 *predicted depth*.
- ByteCameraDepth dataset: p.74
- 7 cameras.
 - 10 modes.

用于学习 *camera-specific holes* / *value noise models*.

real robot deployment: p.75

- policy 在 *clean simulated depth* 上训练, 不额外加入 *noise*.
- deployment 时 raw depth + RGB $\rightarrow$ CDM $\rightarrow$ clean depth $\rightarrow$ policy.
- 结果: real robots 上接近 *simulation-level accuracy*, *zero fine-tuning*.

CDM 和 *Monocular Depth* 的区别.

monocular depth 从 RGB 猜 *depth*, 本质上是 *ill-posed*: 同一张图可能对应多个 *3D scenes*.

CDM 已经有 raw depth, 只是 raw depth 有 *sensor artifacts*. 它学习的是 *sensor noise removal* 和 *sim-to-real correction*, 因此更像 *camera-specific denoising / calibration module*.

5 Synthetic Data for Surface Normal Estimation: 从 dense normal GT 到 geometry-aware priors p.76

Surface normal estimation 的目标是: 对图像中的每个 pixel 预测一个 3D unit vector, 描述该 surface patch 在 camera coordinate 下的朝向. p.77

形式上, normal map 可以写成:

$$N(u, v) = \mathbf{n}_{u, v} \in \mathbb{S}^2, \quad \|\mathbf{n}_{u, v}\|_2 = 1$$

其中 $\mathbb{S}^2$ 是 3D unit sphere.

surface normal 的难点在于 *inverse problem* 本身是 *ill-posed*: 同一张 2D image 可以来自无限多个 3D shapes. p.78

典型 *ambiguity* 是 *shading ambiguity*:

- 同样的 *intensity pattern* 可能来自 *convex surface*.
- 也可能来自 *concave surface*.
- 如果没有 *lighting*, *material*, *camera intrinsics* 或 *geometry prior*, 模型只能从 *learned prior* 中猜测.

Surface Normal 和 *Depth* 的区别.

depth 预测每个 *pixel* 到 *camera* 的距离; *surface normal* 预测局部 *surface orientation*.

两者都属于 *dense geometry prediction*, 但 *normal* 更强调 *local shape* 和边界处的几何折线. 一个平面即使 *depth* 值不断变化, 它的 *normal* 也可以保持近似常量; 反过来, *normal map* 的 *discontinuity* 往往对应 *object boundary* 或 *surface crease*.

5.1 Omnidata: 从 3D scans 自动生成 mid-level cues p.79

Omnidata 是 ICCV 2021 的 *synthetic* / *rendered data pipeline*, 目标是从 3D scans 中自动生成大量 *mid-level visual cues*.

它的基本流程是:

$$3\text{D mesh} \rightarrow \text{rendering} \rightarrow \text{RGB} + \text{perfect normal map}$$

关键数字和设定:

- 约 14.5M images. p.80
 - 来自多个 3D scan sources / datasets.
 - 输出 21 种 *mid-level cues*, 包括 *surface normal*, *reshading*, *depth*, *edge*, *segmentation* 等. p.81
 - normal label* 是 *dense per-pixel normal*, 并要求 *rotation consistency*.
- Omnidata 的重要性在于: *normal* 这种 *dense label* 很难人工标注, 但如果有 3D mesh, *rendering pipeline* 可以直接计算每个 *visible surface point* 的 GT *normal*.

为什么 3D mesh 能给 *perfect normal GT*.

对每个 *rendered pixel*, *renderer* 知道这个 *ray hit* 到 *mesh* 上的哪个 *triangle* 或 *interpolated surface point*.

mesh 本身有 *face normal* / *vertex normal*, 因此可以把该点的 *local surface orientation* 直接投到 *camera coordinate* 下, 得到 *per-pixel normal label*.

这类 *label* 的精度不受人工点击误差影响, 但受限于 *mesh quality*、*rendering realism* 和 *scanned scene coverage*.

5.2 Omnidata 的限制: scale 大, 但 inductive bias 不一定够 p.82

Omnidata 的问题不是“没有数据”, 而是数据和真实世界之间仍然有 *gap*:

- scale 很大: 需要 14.5M images.
- domain gap: *synthetic* / *rendered data* 比真实图像更 *clean*, 真实世界有复杂 *lighting*, *sensor noise*, *motion blur*, *material effects*.
- limited scene information: 如果没有 *Bidirectional Reflectance Distribution Function (BRDF)*, *lighting*, *material* 等信息, *reshading* 和 *normal* 仍可能有 *blind guess*.

所以 *surface normal* 这一节的主线不是单纯扩大数据量, 而是: 怎样把 *camera geometry* 和 *local surface geometry* 作为 *inductive bias* 放进模型.

5.3 DSINE: 用 ray direction 和 local normal rotation 替代盲目堆数据 p.83

DSINE 是 CVPR 2024 Oral 的 *surface normal estimation* 方法. 课件强调它的核心结论:

$$\text{better priors} > \text{more data}$$

DSINE 只用了约 160K images, 就能匹配 Omnidata 使用 14.5M images 的效果.

它的两个关键 *geometry prior*:

- per-pixel ray direction*: 显式告诉网络每个 pixel 对应的 *camera ray*.
- local normal rotation*: 不直接回归邻域 *normal*, 而是预测相邻 *normal* 之间的 *rotation relation*.

DSINE 的核心直觉.

normal 不是纯 *image-to-image translation*. 一个 *pixel* 的 *normal* 必须和 *camera ray*, *visibility constraint*, 邻域 *surface geometry* 共同一致.

如果模型不知道 *camera intrinsics*, 它需要从数据里隐式学 *FoV* / *resolution* / *principal point* 的影响; *DSINE* 直接把 *ray direction* 输入网络, 相当于把 *camera geometry* 从“需要猜”的变量变成显式条件.

5.3.1 Ray Direction Encoding: 把 camera intrinsics 显式喂给网络 p.84

对 pixel (u, v) , 给定 focal lengths f_u, f_v 和 principal point (c_u, c_v) , DSINE 计算 focal-length normalized *ray direction*:

$$\mathbf{r}(u, v) = \left[\frac{u - c_u}{f_u}, \frac{v - c_v}{f_v}, 1 \right]^\top$$

这个 *ray direction* 会被 fed into intermediate layers of the network, 而不是只在输入端拼接一次.

带来的好处:

- 模型不需要从 RGB 数据中隐式学习 *camera intrinsics*.
- 对任意 *resolution*, *aspect ratio*, *Field of View (FoV)* 都更稳.
- camera geometry* 从 *data-dependent cue* 变成 *explicit coordinate condition*.

为什么 *FoV* 会影响 *normal prediction*.

同一个 *image pattern* 在 *wide-angle camera* 和 *narrow-angle camera* 下, 对应的 *3D ray directions* 不同.

如果模型只看 *RGB patch*, 它可能把 *perspective distortion* 当成 *object shape*; *ray direction encoding* 让模型知道“这个 *pixel* 是从哪个方向看出去的”, 因此 *normal prediction* 更容易跨 *camera generalize*.

5.3.2 Ray ReLU: 把 normal 限制在 visible hemisphere p.85

DSINE 使用 Ray ReLU activation 来编码 *visibility constraint*. 直觉是: 一个 *surface* 如果 *facing away from the camera*, 就不可见. 因此 *visible surface normal* 和 *camera ray* 的夹角必须满足几何可见性约束.

Ray ReLU σ_{ray} 的操作:

- 如果 *predicted normal n* 在 *ray direction r* 上有不合法的 *facing component*, 就把它沿 *r* 的分量投影到边界.
- 再把结果 *re-normalize* 成 unit vector.
- 得到 *guaranteed visible* 的 *output normal*.

可以把它理解为 *normal space* 上的 ReLU: 普通 ReLU 把 *scalar activation* 限制到非负区间, Ray ReLU 把 *normal* 限制到 *camera* 可见的半球内.

Visibility Constraint 的符号直觉.

不同论文可能用 *camera-to-surface ray* 或 *surface-to-camera ray*, 因此 *dot product* 的正负号会随 *convention* 改变.

重要的不是记某个固定符号, 而是几何条件: *predicted normal* 不能指向 *camera* 看不到的一侧. *Ray ReLU* 的作用就是把 *normal* 投回 *visible hemisphere*.

5.3.3 Rotation Between Normals: 把 local geometry 写成旋转关系 p.86

DSINE 不只预测每个 *pixel* 的 *normal*, 还建模邻域 *normal* 之间的 *local rotation*.

对中心 pixel i 和邻居 j , 设当前邻居 *normal* 为 $\mathbf{n}_j^t$, DSINE 预测 *rotation* R_{ij} , 并用它把 *neighbor normal* 旋转到中心 *pixel* 的 *normal estimate*:

$$\mathbf{n}_i^{t+1} \approx R_{ij} \mathbf{n}_j^t$$

rotation 用 Lie group / Lie algebra 表示:

$$R_{ij} = \exp(\theta_{ij}[\mathbf{e}_{ij}]_{\times}) \in SO(3)$$

其中:

- θ_{ij} 是 *rotation angle*.
- $\mathbf{e}_{ij}$ 是 *rotation axis*.
- $[\mathbf{e}_{ij}]_{\times}$ 是 *skew-symmetric matrix*.
- $\exp: \mathfrak{so}(3) \rightarrow SO(3)$ 把 *axis-angle* 表示映射到 3D *rotation matrix*.

为什么预测 *relative rotation* 比直接预测 *normal* 更自然 p.87.

直接回归 *absolute normal* 容易受 *camera pose*, *global alignment*, *lighting* 和 *texture* 影响.

相邻 *surface* 的 *relative angle* 更稳定: 平面内部通常接近 0° , 两个正交 *surface* 之间通常接近 90° , 这和相机整体姿态关系较小.

rotation axis 还常常对应 *image* 中两块 *surface* 的 *intersection line*, 也就是 2D *edge*. CNN 擅长检测 *edge*, 因此这个 *inductive bias* 更贴近视觉网络的能力.

5.3.4 DSINE 具体预测什么 p.88

对每个 pixel i , 以及它 5×5 window 内的每个 *neighbor j*, DSINE 预测三类量:

- rotation angle*:

$$\theta_{ij} \in [0, \pi]$$

实际实现中可以用 *sigmoid* 再乘 π . flat *surface* 内部的 θ 接近 0° , *orthogonal surfaces* 之间的 θ 接近 90° .

- rotation axis* 的 2D offset:

$$(\delta u_{ij}, \delta v_{ij})$$

这个 2D offset 定义 *projected axis*, 再结合几何约束恢复 3D axis $\mathbf{e}_{ij}$.

- fusion weight*:

$$w_{ij}$$

它用于 *down-weight disconnected* / *non-smooth regions*, 例如 *object boundary* 或 *strong crease*.

Fusion Weight 的作用.

如果 *pixel i* 和 *neighbor j* 属于同一个 *smooth surface*, 那么 *neighbor normal* 经过 *rotation* 后应该能帮助估计中心 *normal*.

如果二者跨过 *object boundary*, 直接融合会把两个不同物体的 *normal* 混在一起. *w_{ij}* 就是在学习“这个邻居不可信”.

5.3.5 Iterative Refinement: 旋转、可见性裁剪、加权融合 p.89

从 initial normal $\mathbf{n}^0$ 开始, DSINE 做 $N_{\text{iter}} = 5$ 次 *refinement*. 每次更新:

$$\mathbf{n}_i^{t+1} = \frac{\sum_j w_{ij} \cdot \sigma_{\text{ray}}(R_{ij} \mathbf{n}_j^t, \mathbf{r}_i)}{\left\| \sum_j w_{ij} \cdot \sigma_{\text{ray}}(R_{ij} \mathbf{n}_j^t, \mathbf{r}_i) \right\|}$$

四个步骤:

- 用 *R_{ij}* rotate *neighbor normals*.
- 用 Ray ReLU clip 到 *visible hemisphere*.
- 用 *learned weights w_{ij}* 做 *weighted fusion*.
- normalize* 并重复迭代.

这个过程把 *surface normal estimation* 从一次性的 *dense regression*, 改成了 *geometry-constrained message passing*.

5.4 Marigold for Normal Estimation: diffusion prior 也可以预测 normal p.90

Marigold 原本是 CVPR 2024 的 *diffusion-based depth estimation* 方法, 但同样的 *diffusion paradigm* 可以用于 *surface normal estimation*.

基本想法:

- 使用 *Stable Diffusion V2.1* 的强 *image prior*.
- 在 *synthetic normal maps* 上 *fine-tune*.
- RGB image 作为 *latent condition*, 类似 ControlNet 的 *conditioning* 思路.

- 在 *latent space* 中预测 *noise* ϵ 或 *clean latent* x_0 .

用于 *normal estimation* 时的特点:

- normal map* 看起来 *sharp*.
- 需要 *ensemble* 来降低 *variance*.
- inference* 慢, 例如 5 samples 约 10s.

它的核心限制是 *stochasticity*: *diffusion sampling* 本来就是随机过程, 但 *surface normal estimation* 理想上应该是 *deterministic geometry prediction*.

Diffusion Prior 的优点和矛盾.

diffusion model 的优点是 *image prior* 强, 可以生成 *sharp-looking geometry outputs*.

但 *normal prediction* 不是开放式生成任务. 给定同一张 *RGB image*, 我们通常希望 *normal map* 稳定、可复现, 并且和真实 *geometry* 对齐.

所以 *diffusion normal estimator* 的问题不是“不够漂亮”, 而是 *repeated sampling* 的 *variance*, *video temporal inconsistency*, 以及 *ensemble cost*.

5.5 StableNormal: 让 diffusion normal estimation 更稳定 p.91

StableNormal 处理的问题是: GeoWizard, Marigold 这类 *diffusion-based normal estimators* 可以生成 *sharp-looking normals*, 但多次采样之间 *variance* 高.

核心冲突:

diffusion is stochastic but normal estimation should be deterministic

StableNormal 的 key idea 是:

- reliable initialization: You-Only-Sample-Once (YOSO)。
- stable refinement: Semantic-Guided Refinement / SG-DRN。

在 stability 和 sharpness 之间做平衡。

5.6 StableNormal Method: YOSO initialization + SG-DRN refinement p.92

StableNormal 是 two-stage pipeline。

Stage 1: YOSO initialization。

- 从 Gaussian noise 做 one-step estimation。
- 采用 x_t -parameterization。
- 使用 DecoupleLoss, 把 diffusion loss 分成 generative term 和 reconstruction term。

结果已经可以达到与 DSINE 这类 SOTA regression method 接近的水平。

Stage 2: SG-DRN refinement。

- 使用 DINOv2 semantic features 作为 auxiliary condition。
- 用 lightweight Semantic-Guided Network (SGN) g_ψ 注入 global context。

从 t_+ = 401 开始做 10-step Denoising Diffusion Implicit Models (DDIM) refinement。

- 目标是减少 local ambiguity, 同时不牺牲 sharp boundaries。

为什么 *DINOv2 semantic features* 有帮助。

surface normal 是几何任务, 但 *RGB* 中的局部纹理常常不足以判断 *shape*。

DINOv2 features 提供 *object-level / semantic-level context*。例如看到“桌面”“墙角”“椅背”时, 模型可以借助物体类别和结构 *prior* 约束 *local normal*。

这不是用 *semantic label* 替代 *geometry*, 而是用 *high-level context* 减少局部 ambiguity。

5.7 StableNormal Results: 更低 variance, 更快 inference p.93

StableNormal 的结果用 Mean Angular Error (MAE) 衡量, 越低越好。

课件给出的表格中:

- DSINE 在 NYUv2 和 ScanNet 上很强。
- StableNormal 在 ScanNet、iBims-1、DIODE 上达到 best 或 second-best。
- DIODE 上从 Marigold 的 16.67°、DSINE 的 18.45° 提升到 13.70°。ablation:
- YOSO only; DIODE-indoor 17.12°, 已经是 strong baseline。
- without DINO: 15.61°。
- full StableNormal: 13.70°。
- SG-DRN + DSINE initialization: 14.75°, 不如 full pipeline。efficiency:
- GeoWizard: 5 samples 约 10s, variance 1.37。
- StableNormal: single pass 约 3s, variance 0.410。
- 不需要 ensemble。

StableNormal 和 DSINE 的关系。

DSINE 是 *geometry-aware regression route*: 把 *camera ray*、*visibility*、*local rotation* 作为 *hard inductive bias*。

StableNormal 是 *diffusion route* 的改造: 保留 *diffusion prior* 的 *sharpness*, 但通过 *YOSO* 和 *semantic-guided refinement* 降低随机性。

两者代表两条不同路线: 一种是显式几何约束, 一种是生成式 *prior* 的稳定化。

5.8 Key Insight: right inductive bias can replace orders of magnitude of data p.94

surface normal 这一节的总结是:

better inductive bias > more data

三条路线的对比:

- Omndata: 用 14.5M synthetic images 和 3D scans 自动生成 dense normal labels。
 - DSINE: 用约 160K synthetic images, 加上 ray direction / Ray ReLU / local normal rotation, 匹配大规模数据路线。
 - StableNormal: 把 diffusion prior 用在 normal estimation 上, 再用 deterministic initialization 和 semantic-guided refinement 解决 stochasticity。
- 最终 takeaway:
- synthetic data 让 dense normal supervision 可规模化获得。

- 只堆数据不够, camera geometry 和 local surface relation 是关键 inductive bias。

diffusion prior 能提高 visual sharpness, 但必须控制 variance 和 inference cost。

- 这一节和前面的 depth / pose / tracking 一样, 都指向同一个原则: synthetic data 的价值不只是 quantity, 而是提供真实世界难以标注的 structured supervision。

created: 2026-05-29

updated: 2026-05-29

1 Vision-Language Data: 从网页 alt-text 到 multimodal pre-training p.3

alt 在网页里通常是 alternative text 的缩写, 意思是“替代文本”。

Vision-Language (V+L) data 最常见的形式是 image-text pair:

(x_i, t_i) = (image, natural language caption)

其中 x_i 是图像, t_i 是自然语言描述。web-scale V+L data 最自然的来源是网页中的 HTML `<img>` tag 和 alt attribute:

- `<img>` tag 给出 image URL。
- alt attribute 原本用于 accessibility。
- 对 Contrastive Language-Image Pre-training (CLIP)、text-to-image generation、Vision-Language Model (VLM) 和 Vision-Language-Action (VLA) 来说, alt-text 变成了最重要的弱监督信号。

这讲的核心问题不是“能不能从网页抓到很多图片”, 而是:

web-scale image-text pairs 很多, 但质量、对齐、语言和文化分布都很乱。

所以 data curation 的目标是在 scale、quality、diversity、fairness 之间做选择。

Alt-text 为什么能成为监督信号。

好的 alt-text 把视觉内容和语言概念绑定在一起。例如“A golden retriever playing fetch in a sunny park” 同时告诉模型 *object*、*attribute*、*action* 和 *scene*。

CLIP 这类模型不需要人工分类 labels。它只需要大量 (image, text) pair, 然后学习让匹配的 image embedding 和 text embedding 接近, 不匹配的 pair 远离。

1.1 Alt-Text Spectrum: good, bad, mismatched p.4

good alt-text 的特点:

- descriptive and specific。
 - visually grounded。
 - precisely matches the image。
 - rich in visual detail。
- bad alt-text 常见几类:
- raw filename: IMG_20210101_001.jpg、DSC_0456.png。
 - ad copy: Click here to buy now!。
 - layout metadata: banner_ad_728x90。
 - generic placeholder: Image, Photo, Screenshot。

最危险的是 mismatched alt-text: 文字本身语法正确、概念丰富, 但描述的是另一张图。例如 text 是“A cat sleeping on a red sofa”, image 却是一只 dog running on grass。p.7

text-only filter 会保留这个 pair, 因为“cat”和“sofa”都是看起来很好的视觉概念, 但模型会学到错误 association:

dog on grass $\longleftrightarrow$ cat on sofa

为什么 mismatched pair 比 filename 更危险。

filename 类型的 bad alt-text 通常只是不提供信息, 模型最多学不到东西。

mismatched pair 会提供错误监督。它看起来像高质量 caption, 因此更容易通过 text-side filtering, 并且在 contrastive learning 中把错误 image-text relation 拉近。

1.2 Why Scale Matters: dataset 越来越大, performance 也上升 p.8

典型 web-scale V+L datasets:

Dataset; Pairs

CLIP WebImageText (WIT); 400M

ALIGN; 1.8B

LAION-5B; 5.85B

WebLI-100B; 100B

经验规律是: data scale 增大时, zero-shot performance 通常上升; filtering、deduplication、metadata balancing 等 curation 还会进一步影响结果。

但 scale 不是免费午餐。dataset 越大, 越会放下面三组 tension:

- quality vs. scale。
- precision vs. diversity。

- bias vs. fairness。

1.3 Use Cases: generation, Vision-Language Model (VLM), Vision-Language-Action (VLA) p.9

Text-to-image generation:

- Stable Diffusion、DALL-E、Midjourney、Imagen 等都依赖 massive image-text data。
- 模型学习 text description $\leftrightarrow$ image content mapping。
- data quality 决定 generation quality, data diversity 决定 style / concept coverage。

Vision-Language Model (VLM): p.10

- CLIP 可以做 zero-shot visual recognition。
- Modern VLMs 如 GPT-4V、PaLI、PaLI-Gemma 可以做 captioning、Visual Question Answering (VQA)、Optical Character Recognition (OCR)、chart QA 等。

共同点是先在 web-scale image-text pairs 上做 multimodal pre-training。

Vision-Language-Action (VLA): p.11

VLA = visual perception + language instruction + physical action。

- 代表模型: RT-2、OpenVLA。
- pipeline 是 web-scale V+L pre-training $\rightarrow$ robot data fine-tuning。

1.4 Web Data Pipeline and Core Challenge p.12

典型 pipeline:

- 从 Common Crawl 读取网页快照。
- parse HTML `<img>` tags。
- extract image URL + alt-text。
- download images。
- 得到 raw image-text pairs。
- 做 safety filtering、deduplication、quality filtering、packaging。Common Crawl 每月约 300 TB, 因此 extraction 本身就是 distributed engineering problem。
- 这讲的主线可以压缩成三条 curation philosophy: p.13
- text-side filtering: 只看文字, 用 language knowledge 过滤 alt-text。
- image-text alignment: 看图像和文字是否真的匹配。
- embracing noise: 减少过滤, 让 scale 和 diversity 承担更多角色。

2 Text-Side Filtering: 用语言概念筛选 alt-text p.15

Text-side filtering 的核心假设是:

alt-text quality can be judged by language analysis alone。

也就是说, 不看 image, 只看 text。好的 alt-text 通常包含 concrete nouns、action verbs、descriptive adjectives; 坏的 alt-text 通常包含 filenames、ads、size labels、generic terms。

最大优势是 cheap: 不需要 pre-trained vision model, 也不需要 GPU inference, 适合 billion-scale 数据预处理。

2.1 General Method: vocabulary, matching, balancing p.16

text-side filtering 通常有三步:

- build concept vocabulary:
- Wikipedia high-frequency words。
- WordNet synsets。
- Wikipedia article titles。
- high Pointwise Mutual Information (PMI) bi-grams。
- sub-string match:
- 检查 alt-text 是否包含 vocabulary 中的 entry。
- 如果没有任何 meaningful concept, 通常丢弃。
- balance frequencies:
- 对每个 concept 做 hard cap 或 probabilistic sampling。
- 防止 head concepts 例如“photo”淹没 long-tail concepts。

PMI 衡量两个词的 co-occurrence 强度:

$$\text{PMI}(x, y) = \log \frac{p(x, y)}{p(x)p(y)}$$

如果“ice”和“cream”一起出现的概率远高于独立出现, ice cream 就会成为 high-PMI bi-gram。

Text-side filtering 的优缺点。

优点是 transparent、interpretable、cheap, 并且 concept distribution 可控。

限制是 English-centric, 并且完全无法验证 image-text semantic consistency, 只要文字看起来好, 它就会通过, 即使图片完全不匹配。

2.2 WebImageText (WIT): Contrastive Language-Image Pre-training (CLIP) 背后的 mystery dataset p.18

WebImageText (WIT) 是 CLIP 使用的 private training data:

- 400M image-text pairs。
 - 来源描述为“various publicly available sources on the internet”。
 - 使用 500k query vocabulary。
 - 每个 query 最多 20,000 pairs。
 - dataset 没有公开, 因此 community 无法 reproduce 或 audit。
- WIT 的 query vocabulary 由四部分构成: p.19
- English Wikipedia 中出现 ≥ 100 次的 high-frequency words。
 - high-PMI bi-grams。
 - Wikipedia article titles。
 - WordNet synsets。

active search 使用这 500k queries 在 internet 上找 image-text pairs。hard cap 20,000 pairs per query 的作用是保留 long-tail representation p.20

如果没有 cap, 常见词会支配 dataset。例如“photo”可能出现 tens of millions of times, 导致 dataset 过度集中在 head concepts。

2.3 WebImageText (WIT) vs. Yahoo Flickr Creative Commons 100 Million (YFCC100M): WebImageText 的优势主要来自 scale p.22

YFCC100M 是公开 Flickr dataset, 包含约 100M images with metadata。但 metadata quality 很差:

- filenames 被当成 titles。
 - camera EXIF 被当成 descriptions。
 - 过滤后只有约 15M English natural language descriptions。
- CLIP appendix 的 same-size comparison 显示, YFCC 和 WIT performance 很接近。课件强调的结论是:

WIT 的优势不是神秘 curation magic, 而主要是 400M scale。

这也解释了为什么后续 MetaCLIP 的目标是公开复现 WIT curation, 而不是重新发明一个新架构。

2.4 Metadata-Curated Contrastive Language-Image Pre-training (MetaCLIP): Demystifying Contrastive Language-Image Pre-training (CLIP) Data p.23

MetaCLIP 的问题意识:

- WIT 是 black box。
 - CLIP 的成功可能很大程度上来自 data, 而不只是 architecture。
 - data curation method 应该 public and auditable。
- MetaCLIP 从 Common Crawl 透明重建 WIT 风格的数据, 并证明公开 curation 可以 match 或 beat WIT performance。

重建的 500k vocabulary; p.24

Component; Count; Details

WordNet synsets; 86,654; all noun synsets

Wiki uni-grams; 251,465; words appearing ≥ 100 times

Wiki bi-grams; 100,646; PMI threshold ≈ 30

Wiki titles; 61,235; pageviews threshold ≈ 70

Total; 500k; OpenAI exact threshold unknown

MetaCLIP data source 是 Common Crawl 15 snapshots, 从 Jan 2021 到 Jan 2023。raw pool 有 1.6B image-text pairs 和 5.6B total matches, 平均每个 text 有 3.5 matches。p.25

web metadata 呈现极端 long-tail:

- 114k metadata entries zero matches。
- 16k entries, 即 3.2% entries, 占据 > 94.5% matches。

2.5 Metadata-Curated Contrastive Language-Image Pre-training (MetaCLIP) Balancing: 为什么 cap 是 20,000 p.26

MetaCLIP 发现 20,000 正好是 cumulative distribution 的 knee:

- above 20,000: head concepts 进入 exponential growth。
- below 20,000: long tail 接近 linear growth。

因此设定 max count $t = 20,000$ 可以把 distribution 从 exponential 拉得更接近 linear。

对每个 metadata entry, MetaCLIP 使用 sampling probability:

$$p = \min \left(1, \frac{t}{\text{count}} \right)$$

如果某个 concept count 小于 t , 全部保留; 如果 count 大于 t , 按比例抽样, 使 expected count 接近 t 。

Balancing 的直觉。

这不是让所有 concept 完全均匀, 而是防止最热的概念占据大部分训练预算。

对 contrastive pre-training 来说, long-tail concept 很重要。模型需要见到“mangos-teen”、“sumo”、“zanzibar”这类 rare concept, 才能在 downstream zero-shot 中识别它们。

2.6 Metadata-Curated Contrastive Language-Image Pre-training (MetaCLIP) Results p.27

MetaCLIP 在 ImageNet zero-shot top-1 上超过 CLIP: Scale; Model; MetaCLIP; CLIP 400M; ViT-B/32; 65.5%; 63.4% 400M; ViT-B/16; 70.8%; 68.3% 400M; ViT-L/14; 76.2%; 75.5% 1B; ViT-L/14; 79.0%; NA 2.5B; ViT-H/14; 80.5%; NA 2.5B; ViT-bigG/14; 82.1%; NA 结论:

- WIT-style text-side curation 可以被公开复现。
- concept balancing 对 performance 很重要。
- 但 text-side filtering 的 blind spot 仍然存在: 它不看 image, 因此无法判断 image-text 是否真的对齐。 p.28

3 Image-Text Alignment Verification: 从 tags 到 learned scores p.30

text-side filtering 的 fatal blind spot 是: 它完全不看图像。 alignment verification 的目标是估计:

$$s(x, t) = \text{how well image } x \text{ matches text } t$$

然后保留 $s(x, t)$ 高的 pairs。 alignment verification 有三代方法: p.31

- tag overlap: external vision model 生成 tags, 再和 alt-text 做 overlap。
- embedding similarity: CLIP 把 image 和 text 编码到 shared embedding space, 再计算 cosine similarity。
- learned filter + benchmark: 专门训练 filtering model, 并用 DataComp 这类 benchmark 做公平比较。

3.1 Filtering Paradox: 过滤掉的不只是 noise, 还有 diversity p.33 filter 的定义来自它的 training distribution. 如果 filter 在 Western internet data 上训练, 那么它会把 “good” 近似定义成 Western internet content. 可能被系统性丢弃的内容:

- non-Western cultural scenes。
- low-income community imagery。
- low-resource languages。

因此 filtering 同时移除 noise 和 diversity. 这个问题会反复出现在 LAION、DataComp、No Filter 和 WebLI-100B 中。 Filtering Paradox.

更强的 filter 不一定意味着更好的 dataset。

如果 benchmark 是 ImageNet / COCO 这类 Western-centric benchmark, 那么 optimized filter 可能确实提高 benchmark score, 但会牺牲文化多样性、低资源语言和低收入地区内容。

3.2 Conceptual Captions 3M (CC3M): 早期 image-text verification p.34

Conceptual Captions 3M (CC3M) 的目标是从 web 自动抽取高质量 image-text pairs, 用于 image captioning。

- 四步 pipeline: p.37
- Image Filtering:
 - exclude small images, non-JPEG、Not Safe For Work (NSFW) 等。
 - Text Filtering:
 - exclude non-English、ALL-CAPS、no-noun、rare-word、negative-sentiment text。
 - Image-Text Filtering:
 - Google Cloud Vision API 预测 image labels。
 - 检查 labels 和 alt-text 的 overlap。
 - overlap 超过 threshold 才保留。 p.35
 - Text Transformation:
 - hypernymize proper nouns。
 - remove temporal expressions。
 - replace numbers with #。

CC3M 的 human evaluation precision 是 90.3%, 但 scale 只有 3.3M pairs。 p.36 它适合 captioning, 因为 captioning 更重视 precision; 但对 V+L pre-training 来说, recall 和 diversity 可能更重要。

3.3 Conceptual Captions 12M (CC12M): 放松 filter, 扩大 long-tail p.38

Conceptual 12M (CC12M) 的核心动机:

For V+L pre-training, recall may matter more than precision.

策略是保留 image-text filtering, 但放松 unimodal image / text filters。 和 CC3M 相比, CC12M 的变化包括: p.39 Rule; CC3M; CC12M max aspect ratio; 2.0; 2.5 alt-text length; stricter; 3-256 words POS checking; required prepositions; no preposition check duplicate-word ratio; stricter; ≤ 0.2 rare-word threshold; 5; 20 hypernymization; yes; no name replacement; none; <PERSON> 对 25% alt-text 最重要的变化是 stop hypernymizing. 对 pre-training 来说, 原始 entity names 比过度泛化后的 hypernym 更有价值。 long-tail gains 很明显: p.40

- luffy: $0 \rightarrow 152$ 。
- mangosteen: $0 \rightarrow 212$ 。
- zanzibar: $0 \rightarrow 1, 138$ 。
- pokemon: $1 \rightarrow 8, 615$ 。
- mehndi: $3 \rightarrow 9, 218$ 。
- cyberpunk: $5 \rightarrow 5, 247$ 。

CC12M precision 是 76.6%, 低于 CC3M 的 90.3%; 但 scale 是 12.4M. 约为 CC3M 的 4 $\times$ 。 NoCaps out-of-domain coverage 比 CC3M 高 5.8 $\times$ 。 p.41 CC3M vs. CC12M 的核心 lesson. CC3M 是 high precision, low recall, 更像为 captioning 定制。

CC12M 是 lower precision, higher recall, 更适合 representation learning. 它说明 web-scale pre-training 不一定需要每条 caption 都特别干净, 覆盖 long-tail concept 可能更重要。

3.4 Large-scale Artificial Intelligence Open Network (LAION): 用 Contrastive Language-Image Pre-training (CLIP) 自己过滤 web-scale data p.42

Large-scale Artificial Intelligence Open Network (LAION) 的 breakthrough 是用 CLIP similarity 做 filtering:

$$s(x, t) = \cos(f_{\text{image}}(x), f_{\text{text}}(t))$$

相比 Cloud Vision API:

- 不需要 predefined label table。
- 不需要 word overlap。
- 可以直接用 image-text embedding similarity。

LAION-400M 到 LAION-5B 的意义是 democratization: release URLs + metadata + CLIP similarity scores, 让 community 可以下载并调整 thresholds。 LAION pipeline: p.43

- Distributed web filtering:
 - parse Common Crawl WAT files。
 - CLD3 language detection。
 - Distributed downloading:
 - async requests。
 - 300 parallel workers。
 - 10k links per batch。
 - Content filtering:
 - text length ≥ 5 chars。
 - image size ≥ 5 KB。
 - CLIP ViT-B/32 cosine similarity threshold:
 - English: ≥ 0.28 。
 - other languages: ≥ 0.26 。
- 结果是约 90% raw data 被过滤, 保留 5.85B pairs。 LAION-5B language breakdown: p.44
- English: 2.32B, 约 40%。
 - multilingual: 2.26B, 100+ languages。
 - undetected: 1.27B。

multilingual subset 中 top languages 主要是 European languages, 例如 Russian、French、German、Spanish: Chinese 约 6.3%。 African、South Asian、Southeast Asian languages 很小。

3.5 Large-scale Artificial Intelligence Open Network (LAION) 的问题: democratized dataset, not filtering model p.46

- LAION-400M 在 ImageNet zero-shot 上低于 CLIP:
- ViT-B/32: LAION-400M 62.9% vs. CLIP 63.3%。
 - ViT-L/14: LAION-400M 72.8% vs. CLIP 75.6%, gap -2.8 pp。 LAION-2B-en ViT-L/14 可以接近 CLIP:
 - LAION-2B-en: 75.2%。

- CLIP WIT: 75.6%。
- 但 LAION-2B-en 需要约 34B samples seen, 而 CLIP WIT 约 12.8B samples seen. 也就是说, 需要更多 compute 来补偿更低 data quality。 LAION 的四个问题: p.47
- no concept balancing, head concepts dominate。
 - Western-centric bias, English 和 European languages 过多。
 - performance gap, CLIP filtering 可能成为 bottleneck。
 - black-box dependency, pipeline 依赖 proprietary OpenAI CLIP。
- 3.6 DataComp: 把 filtering 变成 fair benchmark p.48 DataComp 是 benchmark, 不是单一 method. 它的设计原则是:

fix model architecture, training hyperparameters, compute budget; open dataset

- CommonPool construction funnel: p.48
- 88B possible URL-text pairs。
 - attempt to download $\approx 40\text{B}$ 。
 - successfully downloaded $\approx 16.8\text{B}$ 。
 - NSFW filtering 后 $\approx 13.6\text{B}$ 。
 - eval-set dedup 后 $\approx 13.1\text{B}$ 。
 - random sample 得到 12.8B xlarge pool。

DataComp 四个 scales: p.49

- Small: 12.8M samples。
- Medium: 128M samples。
- Large: 1.28B samples。
- XLarge: 12.8B samples。

evaluation 包含 38 downstream tasks。

3.7 DataComp Findings: filtering beats no filtering, but threshold matters p.50

Finding 1: 在固定 training compute 下, 每种 filtering strategy 都优于 no filtering。

- XLarge scale:
- no filtering: ImageNet 72.3%。
 - best filtering: ImageNet 79.2%。
 - gain: +6.9 percentage points。
- Finding 2: optimal CLIP-score threshold 不是越高越好。 p.51 top 30% CLIP similarity scores 是 sweet spot:
- top 10% precision 更高, 但 diversity 下降太多。
 - top 30% 在 quality 和 diversity 之间更平衡。

Finding 3: deduplication consistently helps, 虽然单项 gain 有时很小。 p.52 Finding 4: ImageNet 和 38-task average correlation 很高, $r \approx 0.99$, 但 ImageNet 可能和某些 specific tasks 负相关。 因此不能只优化 ImageNet。 DataComp 的隐含限制。 DataComp 把 filtering comparison 科学化了, 但 benchmark 本身仍然有倾向。

如果 38 tasks 主要代表 Western-centric performance, 那么 top 30% sweet spot 也只是 “Western benchmark sweet spot”, 不一定是文化多样性或 low-resource language 的 sweet spot。

3.8 Data Filtering Networks (DFN): 专门训练 filtering model p.53

Data Filtering Networks (DFN) 的核心想法是: 不要直接用 generic CLIP similarity, 而是训练一个专门用于 high-quality alignment filtering 的 model。 training recipe:

- initialization: OpenAI CLIP ViT-B/32。
- train on HQITP-350M: 357M human-verified image-text pairs。
- fine-tune on MS COCO、Flickr30k、ImageNet with OpenAI prompt templates。
- weight ensembling + data augmentation。

workflow: p.54

HQ Data $\rightarrow$ train DFN $\rightarrow$ filter uncurated data $\rightarrow$ DFN-induced dataset

3.9 Data Filtering Networks (DFN) Core Discovery: filtering quality $\neq$ ImageNet accuracy p.55

- DFN 的反直觉结论:
- A better ImageNet model is not necessarily a better filter。
- 原因是二者任务不同:
- ImageNet accuracy 衡量的是 image category recognition。

- filtering quality 衡量的是 image-text alignment quality。
- 一个 model 即使 ImageNet accuracy 高, 也可能不擅长判断 caption 是否具体、准确、和 image 对齐。
- DFN ablation: p.56
- augmentation: +0.006。
 - fine-tuning: +0.010。
 - OpenAI initialization: +0.012。
- public DFN 只用 CC12M + CC3M + Shutterstock 15M 训练, 也能 match OpenAI CLIP filtering performance on DataComp medium / large。 p.57

3.10 Data Filtering Networks (DFN) Poisoning Insight: filter training data quality 极其敏感 p.58

- DFN dataset poisoning experiment:
- 从 10M high-quality CC12M samples 开始。
 - 逐渐用 unfiltered Common Crawl 替换。
 - train a series of DFNs。
- 观察:
- DFN 自己的 ImageNet accuracy 只是逐渐下降。
 - DFN 作为 filter 的 performance 在加入少量 poisoned data 后迅速崩塌。
- 结论:

filter training data quality > filter model accuracy 这说明 learned filter 的瓶颈不是模型大小, 而是 clean alignment supervision 的质量。

3.11 Data Filtering Networks (DFN) Results: DFN-2B and DFN-5B p.59

- DFN-2B:
- 从 DataComp 12.8B CommonPool 取 top 15%, 约 1.9B pairs。
 - ViT-L/14, 12.8B samples seen。
 - ImageNet: 81.4%。
 - DataComp 38-task average: 0.669。
- DFN-5B:
- same DFN applied to 42B pool。
 - ViT-H/14。
 - ImageNet: 84.4%。
 - DataComp average: 0.710。

课件给出的 comparison:

- LAION-2B + ViT-G/14 with 34B samples seen < DFN-2B + ViT-L/14。
- DFN-2B uses 16 $\times$ less compute。

alignment verification 的总结: p.60

- CC3M / CC12M: tag overlap, 简单但 coarse。
- LAION: CLIP similarity, democratizes scale, 但 black-box 和 biased。
- DataComp: fair comparison, 发现 top 30% sweet spot。
- DFN: dedicated learned filter, quality 更高, 但需要 clean training data。

4 Rethink: Embracing Noise at Scale p.62

前两条路线都在过滤:

- text-side filtering 过滤 bad text。
- alignment verification 过滤 mismatched image-text pairs。

但 filtering 有 hidden cost: filter 可能把 cultural diversity 当成 noise。 p.62 如果 filter 在 Western data 上训练, 它可能丢弃 African village scenes、Indian street festivals、low-income household photos 等。 这一节的核心问题:

What if we do not filter?

4.1 Scale Dilutes Random Noise, but Amplifies Systematic Bias p.64

scale hypothesis 的直觉:

- signal 是 structured and recurrent。
- random noise 是 structured and inconsistent。

例如 web 上有 1M 个 “dog” + dog photo 的正确 pair, 只有 100k 个 “dog” + cat photo 的错误 pair。 随着数据变大, 正确 pattern 会反复出现, 随机错误会被 diluted。 但 caveat 很关键:

random noise is diluted by scale; systematic bias is learned by scale。

如果 bias 是 stereotype 或 collection bias, scale 不会自动修正, 反而可能把它学成稳定统计规律。

4.2 A Large-scale ImaGe and Noisy-text Embedding (ALIGN): Noisy Web Text is Good Enough at Scale p.65

ALIGN 的论文标题已经说明路线:

Scaling Up Visual and Vision-Language Representation Learning With Noisy Text Supervision

核心 recipe:

- > 1B image-text pairs.
- simple dual-encoder.
- contrastive loss.
- minimal filtering.

ALIGN 的 filtering 几乎只做 basic cleaning: p.66

image side:

- remove NSFW.
- short side > 200 px.
- aspect ratio < 3.
- discard alt-text shared by > 1000 images.

text side:

- discard alt-text shared by > 10 images.
- discard rare words not in top 100M uni-/bi-grams.
- length 3-20 words.

ALIGN 不做 semantic filtering, 不做 CLIP filtering, 不做 image-text alignment check, 不用 Cloud Vision API. 最后得到 1.8B pairs.

4.3 A Large-scale ImaGe and Noisy-text Embedding (ALIGN) Results: scale beats clean small data p.67

controlled ablation 比较 ALIGN noisy data 和 CC3M clean data:

Scale; ALIGN noisy; CC3M clean

3M; worse; better

6M; equal; equal

12M; better; worse

结论: 当 scale 足够大时, noisy data can beat clean data.

ALIGN results: p.68

- ImageNet zero-shot: ALIGN 76.4% vs. CLIP 76.2%.
- ImageNet fine-tuned: 88.64%.
- Flickr30k text-to-image zero-shot: ALIGN 75.7% vs. CLIP 68.7%.
- MSCOCO text-to-image zero-shot: 45.6%.

ALIGN embeddings 还表现出 compositionality: 可以做 image + text 或 image - text retrieval, 例如 panda photo + “Australia” 检索 koala, living room with cat - “cat” 检索 cat-free living room. p.69

4.4 No Filter: Filtering is Bias p.70

No Filter 的核心 argument:

English-only or CLIP-filtered data harms low-income and non-Western

ImageNet / COCO 这类 benchmark 捕捉不到这种损失。

motivating example: Google Landmarks Dataset v2 (GLDv2) 中, English-only SigLIP 把 Iran 的 Milad Tower 预测成 Canada 的 CN Tower, 把 Brazil 的 São Paulo Cathedral 预测成 Notre-Dame of Montreal. global-trained models 能正确分类。p.71

原因是 English-filtered pre-training data 中很少出现 non-Western landmarks.

4.5 Dollar Street: income-stratified visual benchmark p.72

Dollar Street 包含:

- 38,479 in-home photos.
- 289 household items.
- 63 countries.
- 每张图片有 monthly income、region、country metadata.

evaluation subset:

- 96 topics mapped to ImageNet classes.
- 21,536 images.
- 17,228 train / 4,307 test.

Dollar Street income-level zero-shot classification: p.73

- English-only model:
- high income > 1998/month: 62.4%.
- low income 0-200/month: 29.9%.
- gap: 32.5 pp.
- global data model:
- gap shrinks to 27.4 pp.

结论:English filtering 对 low Socioeconomic Status (SES) communities 伤害更大。

4.6 Geographically Diverse Evaluation (GeoDE), Multicultural Reasoning over Vision and Language (MaRVL), and

Global Fine-Tuning Recipe p.74

GeoDE:

- 61,940 images.
- 6 continents.
- 40 classes.
- zero-shot: English-only 91.82%, global 92.84%, gain +1.02%.
- geo-localization 10-shot: English-only 16.41%, global 26.23%, gain +9.82%.

Multicultural Reasoning over Vision and Language (MaRVL):

- languages: Indonesian, Chinese, Swahili, Tamil, Turkish.
- English-only 68.30%, global 69.96%, gain +1.66%.
- largest gains on Indonesian, Turkish, Chinese.

fine-tuning bridge experiment: p.75

三种模型:

- en: English-only captions.
- globe-tl: global images, captions machine-translated to English.
- globe: raw multilingual global data, original captions retained.

关键发现:

- global pre-train → English fine-tune: ImageNet 很快追上 English-only, 同时保留 GLDv2 diversity advantage.
- English pre-train → global fine-tune: 无法恢复 missing cultural knowledge.

practical recipe:

global pre-training + brief English fine-tuning

课件给出的 fine-tuning 长度是约 50k steps.

4.7 Quality Filtering Makes Diversity Worse p.76

No Filter 还测试了: 如果对 global data 做 quality filtering, 会发生什么?

GeoDE:

- tl: 92.84%.
- en: 91.82%.
- filtered tl: 90.57%.
- filtered en: 90.53%.

过滤后, global data 相对 English data 的 advantage 明显缩小。

结论:

“quality” itself is Western-centric. Filtering is the problem.

No Filter 的 takeaway: p.77

- English filtering removes non-Western / low-SES content.
- Western benchmarks do not capture this loss.
- learned filter amplifies bias.
- current practical recipe 是 global pre-training + English fine-tuning.

4.8 Summary of Rethink p.78

ALIGN 和 No Filter 都反对过度 filtering, 但角度不同:

Work; Claim; Main Lesson

ALIGN; scale compensates noise; noisy data is fine if you have enough

No Filter; filtering itself is bias; pre-train globally, fine-tune locally 共同点:

- 都需要 billion-scale data.
- 都说明 filtering 的 hidden cost 很大.
- 都把问题从 “how to filter better” 推向 “whether and why to filter”.

5 Web Language Image 100B (WebLI-100B): 三条路线在 100B scale 汇合 p.80

WebLI-100B 的问题意识:

- LAION-5B 约 5.85B pairs.
- DataComp pool 约 12.8B pairs.
- WebLI-100B 达到 100B image-text pairs. 是 prior art 的约 10×.

核心 research questions:

- scale gain 在 100B 会不会 saturate?
- text filtering、alignment filtering、embracing noise 三种 philosophy 在极限 scale 下表现如何?

5.1 Three Regimes at Scale p.81

WebLI-100B 在同一 experimental setup 下比较三种 regimes:

Regime; Description; Philosophy

Raw 100B; minimal filtering only, safety + Personally Identifiable Information (PII); scale dilutes noise

CLIP-filtered 5B; 从 100B pool 中取 CLIP-L/14 score top pairs; model-based filtering

Language-rebalanced 100B; 7 low-resource languages upsampled to 1% each; metadata curation

这相当于把前面三条路线放到 100B scale 做正面比较。

5.2 Engineering and Training Setup p.82

100B scale 的 engineering challenges:

- storage: 仅 URL + caption metadata 就约 10 TB, image storage 更大.
 - compute: one epoch 需要 days to weeks.
 - fair compare: 1 epoch ×100B 和 10 epochs ×10B 到底哪个更公平?
- training setup: p.83
- data source: Common Crawl + basic safety filtering.
 - 100B unique pairs.
 - model scales:Vision Transformer (ViT)-B/16、ViT-L/16、ViT-H/16.
 - loss: Sigmoid Loss for Language Image Pre-training (SigLIP).
 - ablations: 1B / 10B / 100B.
 - CLIP-filtered subset: top 5B by CLIP-L/14.
 - language rebalancing: 7 low-resource languages upsampled to 1%.

5.3 Western Benchmarks Saturate p.84

ViT-L/16 compute-matched results show Western benchmarks nearly saturate from 10B to 100B:

Benchmark; 1B; 10B; 100B

ImageNet ZS error; 31.2; 29.7; 28.5

CIFAR-100 ZS error; 25.0; 23.8; 23.4

COCO I2T@1 error; 49.7; 47.2; 45.3

Flickr T2I@1 error; 39.9; 32.3; 32.5

Pet ZS error; 14.4; 12.5; 9.5

ImageNet zero-shot from 10B 到 100B 几乎 flat. Wilcoxon signed-rank test 的 $p = 0.9$, 说明差异没有统计显著性。

结论:

Traditional Western accuracy metrics alone cannot justify 100B scale

5.4 Cultural Diversity Gains at 100B p.85

Dollar Street 10-shot classification:

- ViT-L/16: 10B → 100B gain +5.8%.
- ViT-H/16: 10B → 100B gain +5.4%.

这远大于 Western benchmark 的 < 1% gains.

Wilcoxon test $p = 0.002$, 在 99% confidence 下 significant.

原因是 cultural long-tail concepts 需要极大 scale 才能在模型中变得 salient. 10B 对 English / Western common concepts 可能已经足够, 但对 low-income household、regional objects、non-Western landmarks 仍然不够。

5.5 Multilinguality: low-resource languages benefit more p.86

Crossmodal-3600 (XM3600) 包含 36 languages, 用于 zero-shot image-text retrieval.

核心发现:

- low-resource languages 从 100B 中获益比 high-resource languages 更大.
 - model size 越大, 这个差异越明显.
 - high-resource languages 在 10B 已接近 saturate; low-resource languages 在 10B 仍然几乎 invisible.
- language rebalancing 把 7 个 low-resource languages 从 < 0.5% up-sample 到 1%; p.87
- Bengali.
 - Filipino.
 - Hindi.
 - Hebrew.
 - Maori.
 - Swahili.
 - Telugu.

结果:

- low-resource languages 在 XM3600 上显著提升.
- high-resource languages 略降, 但总体仍可接受.
- overall multilingual benchmark improves.

这相当于把 MetaCLIP 的 concept balancing 思想应用到 language meta-data 上。

5.6 Fairness: stereotypes 不会随 scale 自动消失 p.88

WebLI-100B 的 fairness 结论有一个重要区分:

performance parity improves, but stereotype bias does not auto-correct

gender-occupation bias:

- random ImageNet images 有约 85% probability 被关联到 “Male” 而不是 “Female”.
- 1B → 10B → 100B, bias 并没有下降.
- association bias 如 secretary → female、manager → male 也没有随 scale 消失.

原因:

- random noise 可以被 scale dilute.
- systematic bias 会被模型学成 stable statistical regularity.

performance parity 方面, scale 有帮助: p.89

- Dollar Street income-group accuracy gap 从 1B 到 100B 变小.
- GeoDE geographic region gap 也有变小趋势.

因此:

- scale helps coverage parity and performance parity.
- scale does not automatically fix stereotypes.

5.7 Web Language Image 100B (WebLI-100B) as Convergence p.90

WebLI-100B 的总结是: 三种 philosophies 在 100B scale 不再是互斥的, 而是 complementary tools.

Raw 100B:

- dilutes random noise.
- improves cultural diversity.
- improves multilinguality.

CLIP-filtered 5B:

- filtering returns diminish at 100B.
- still hurts diversity.
- not worth it at full scale for broad pre-training.

Language-rebalanced 100B:

- balancing works for languages too.
- complements raw scale.
- metadata curation reappears.

最终 recipe:

raw scale for long-tail+rebalancing for fairness+targeted filtering for s

6 Synthesis: 三种 curation paradigm 和不可避免的三角权衡 p.92

这讲的三种 paradigm:

Paradigm; Representative Works; Core Mechanism; Main Limitation
Filter Text Only; WIT, MetaCLIP; 不看 image, 只用 vocabulary 和 balancing; 无法验证 image-text consistency

Verify Alignment; CC3M, CC12M, LAION, DataComp, DFN; 看 image-text 是否匹配; filter bias persists

Embrace Noise; ALIGN, No Filter; scale dilutes noise; raw data 也有 collection bias

WebLI-100B 验证了: 在 100B scale, 三者不是简单谁替代谁, 而是根据目标组合使用。

6.1 Inevitable Triangle p.93

No free lunch: 任何 curation strategy 都在三组 desiderata 之间做选择。

6.1.1 Quality vs. Scale

filtering 提高 per-sample quality, 但减少 diversity 和 scale.

no filtering 保留 scale, 但引入 noise.

DataComp 的 top 30% 是 Western benchmark sweet spot, 不一定是 cultural diversity sweet spot.

6.1.2 Precision vs. Diversity

strict filtering:

- high precision.
- low diversity.
- good for captioning.

relaxed filtering:

- lower precision.
- higher diversity.
- good for pre-training.

CC12M 和 ALIGN 都选择了更 relaxed 的路线。

6.1.3 Filter Bias vs. Raw Data Bias

filters carry training bias.例如 CLIP-based filter 继承 CLIP 的 Western-centric representation.

raw data carries collection bias.例如 English websites 本来就更多, 某些地区和语言本来就少。

因此没有 bias-free choice, 只有 conscious trade-off.

6.2 Five Key Takeaways p.94

• **text-side filtering** 是最低成本、最透明的起点。MetaCLIP 证明 WIT-style curation 可以公开发现：当 concept balancing 和 interpretability 重要时，它很有价值。

• **image-text alignment verification** 让 filtering 更科学。从 LAION 到 DataComp 到 DFN, filtering 变得可比较、可优化。但 learned filter 会继承 training data bias, 所谓 sweet spot 本身也带 bias。

• **embracing noise** 在 10B+ scale 后变得可行，并且对 cultural diversity 很关键。No Filter 给出的当前 practical recipe 是 global pre-training + brief English fine-tuning。

• **100B-scale reality**: Western benchmarks 会 saturate, 但 cultural diversity 和 multilinguality 仍然显著增长。stereotype bias 不会被 scale 自动修正。

• **future direction** 不是寻找唯一最好的 filter, 而是针对不同 goals 选择和组合 curation strategies。

这讲和前几天讲的连接。

Lecture 10 讨论 *synthetic data* 时反复出现 *quality > quantity, structured supervision, domain gap*。

Lecture 11 的 web V+L data 也有类似主题，但矛盾换成了 *quality vs. diversity, web data* 的问题不是缺 *scale*, 而是 *scale* 中混入了 *random noise, systematic bias, language imbalance* 和 *cultural long-tail*。

因此现代 *multimodal pre-training* 的数据策略更像系统设计：不是“清洗得越干净越好”，而是要明确 *downstream goal* 是 *Western benchmark, global diversity, low-resource language, safety*, 还是 *fairness*。

created: 2025-10-02
updated: 2026-05-31

```
““dataviewjs
let content = await dv.io.load(dv.current().file.path);
// 匹配所有 [!info] 或 [!info]+ 开头的 callout 块 (含多行)
let regex = /> \[!info\]\+?(?:\s*(.*))?(?:\n(?:> .*)*)?/gm;
[...content.matchAll(regex)].forEach((m, i) => {
let text = m[0]
.split(/\r?\n/)
.map(l => l.replace(/> ?/, ").trim()) // 去掉开头的">"
.join(' ')
.replace(/\^[!info\]\+?\s*/, ""); // 去掉 "[!info]+"
dv.paragraph('i + 1.{text}');
});
““
```

1 Image Generation as Density Modeling: 从 $p(x)$ 到可采样图像分布 p.4

Image generation 的目标是学习真实图像的 distribution:

$x \sim p(x) \implies$ realistic images.

如果我们能显式得到 $p(x)$, 那么 generation 就是从这个 density 里 sample. Normalizing Flow 和 Autoregressive Model 属于 explicit density model:

- Normalizing Flow 可以 exact likelihood, 也可以通过 invertible transformation sample。
- Autoregressive Model 把 joint density 分解成 ordered conditional distribution, 逐 token / pixel 生成。

但 tractable density 的代价很高:

- Normalizing Flow 要求 transformation 可逆, architecture 被 constrained。
- Autoregressive Model 要固定 ordering, 并且生成是 sequential 的, 对 image 这种高维连续结构会比较慢, 也不自然。

因此 diffusion models 的出发点不是直接 parameterize 一个容易算的 $p_{\theta}(x)$, 而是学习一个从 noise 到 data 的生成过程。

(Advance) 为什么 density modeling 是统一语言。

很多生成模型表面上训练目标不同,但本质都在解决同一个问题:如何表示和采样 $p_{data}(x)$ 。

- Autoregressive Model**: 直接写出 $p(x) = \prod_i p(x_i \mid x_{<i})$ 。
 - Normalizing Flow**: 通过 *change of variables* 写出 exact density,
 - Variational Autoencoder (VAE)**: 引入 *latent variable*, 用 Evidence Lower Bound (ELBO) 训练。
 - Energy-Based Model (EBM)**: 只学习 *unnormalized density*。
 - Diffusion Model**: 把 *sampling* 写成逐步 *denoising / reverse-time dynamics*。
- 1.1 Energy-Based Model (EBM) and Metropolis-Hastings Sampling p.5**

Energy-Based Model (EBM) 把 density 写成:

$$p(x) = \frac{e^{-E(x)}}{Z}, \quad Z = \int e^{-E(x)} dx.$$

$E(x)$ 是 energy function。低 energy 对应高 probability, 高 energy 对应低 probability。困难在于 partition function Z 通常无法计算, 因为它要对整个 image space 积分。

Metropolis-Hastings (MH) Markov Chain Monte Carlo (MCMC) 的好处是可以不显式计算 Z 。从当前样本 x 提议一个 x' , 用接受概率

$$\min\left(1, \frac{p(x')}{p(x)}\right) = \min\left(1, e^{-E(x') + E(x)}\right)$$

来决定是否移动。这里 Z 被 ratio 抵消掉。

但是在高维 image space 里, random-walk proposal 会非常慢:

- 局部随机走动很难跨过 low-probability barrier。
- high-dimensional space 里有效区域很稀疏, proposal 大多无效。
- mixing time 很长, 很难在有限时间内得到好样本。

这推动我们使用 **gradient information** 来指导采样。

1.2 Langevin Dynamics and Score Function p.6

Langevin dynamics 使用 score function:

$$s(x) = \nabla_x \log p(x).$$

对 EBM 来说:

$$\nabla_x \log p(x) = \nabla_x (-E(x) - \log Z) = -\nabla_x E(x),$$

因为 $\log Z$ 对 x 是常数。
Langevin update 是:

$$x_{k+1} = x_k + \eta \nabla_x \log p(x_k) + \sqrt{2\eta} z_k, \quad z_k \sim \mathcal{N}(0, I).$$

其中:

- deterministic term $\eta \nabla_x \log p(x_k)$** 把样本推向 high-density region。
- noise term $\sqrt{2\eta} z_k$** 保持 exploration, 避免只做 gradient ascent。

这给出一个关键思路: 如果我们不学习 density 本身, 而是学习 $\nabla_x \log p(x)$, 就可以做 **gradient-guided sampling**, 同时绕开 Z 。

(Advance) 为什么 *Langevin sampling* 有效。

Langevin dynamics 的连续时间形式是:

$$dx = \nabla_x \log p(x) dt + \sqrt{2d} dW_t.$$

它的作用不是单纯做 *gradient ascent*, 而是让当前 *sample distribution $q_t(x)$* 随时间演化。对应的 *Fokker-Planck equation* 是:

[!Notes]- (Advance) *Fokker-Planck equation* 是什么

Fokker-Planck equation 是一个描述 *probability density* 如何随时间变化的 *partial differential equation (PDE)*。如果 *stochastic process* 写成:

$$dx = b(x, t) dt + \sigma(x, t) dW_t,$$

那么它不仅描述单个 *sample path* 怎么随机移动, 也会诱导整体 *density $q_t(x)$* 的演化。*Fokker-Planck equation* 描述的就是这个 $q_t(x)$ 。

在最常见的 *isotropic constant noise* 情况:

$$dx = b(x) dt + \sqrt{2d} dW_t,$$

density evolution 是:

$$\frac{\partial q_t(x)}{\partial t} = -\nabla \cdot (q_t(x) b(x)) + \Delta q_t(x).$$

第一项 $-\nabla \cdot (q_t b)$ 是 *drift* 造成的 *probability mass transport*: 样本被 *vector field $b(x)$* 推着走。第二项 Δq_t 是 *Brownian noise* 造成的 *diffusion*: 概率质量向周围扩散。

对 *Langevin dynamics*, $b(x) = \nabla_x \log p(x)$, 所以就得到下面这个方程。

$$\frac{\partial q_t(x)}{\partial t} = -\nabla \cdot (q_t(x) \nabla_x \log p(x)) + \Delta q_t(x).$$

其中第一项来自 *score drift*, 第二项来自 *Gaussian noise diffusion*。检查目标分布 $p(x)$ 是否是 *stationary distribution*: 令 $q_t(x) = p(x)$, 则右边变成:

$$-\nabla \cdot (p(x) \nabla_x \log p(x)) + \Delta p(x).$$

因为:

$$p(x) \nabla_x \log p(x) = \nabla_x p(x),$$

所以:

$$-\nabla \cdot (\nabla_x p(x)) + \Delta p(x) = -\Delta p(x) + \Delta p(x) = 0.$$

因此 $p(x)$ 是 *Langevin dynamics* 的 *stationary distribution*。直觉上, *score drift* 把样本拉向 *high-density region*, *noise* 负责持续探索; 两者平衡时, 样本停留的整体分布就是 $p(x)$ 。

在 *image generation* 里, x_k 虽然数学上是高维空间中的一个点, 但视觉上就是一张图像。若图像大小是 $H \times W \times C$, 则 $x_k \in \mathbb{R}^{HWC}$, *score $\nabla_x \log p(x_k)$* 是同维度的修改方向, 告诉每个 *pixel channel* 该往哪里动。

2 Score Matching: 直接学习 density 的梯度度 p.8

Score Matching 的目标是学习:

$$s_{\theta}(x) \approx \nabla_x \log p_{data}(x).$$

最直接的 objective 是 Fisher divergence:

$$\frac{1}{2} \mathbb{E}_{p_{data}} \left[\|s_{\theta}(x) - \nabla_x \log p_{data}(x)\|^2 \right].$$

问题是 $\nabla_x \log p_{data}(x)$ 不知道。Hyvarinen 的 *score matching* 通过 *integration by parts*, 把这个目标改写成不需要 true score 的形式:

$$\min_{\theta} \mathbb{E}_{p_{data}} \left[\frac{1}{2} \|s_{\theta}(x)\|^2 + \nabla_x \cdot s_{\theta}(x) \right].$$

+,

#Calculus/Divergence $\int u(x) \cdot \nabla_x v(x) dx$, $\int u(x) \nabla_x \cdot v(x) dx$ (Advance) *Hyvarinen score matching* 的 *integration by parts* 推导。

从理想 *Fisher divergence* 开始:

$$J(\theta) = \frac{1}{2} \int p(x) \|s_{\theta}(x) - \nabla_x \log p(x)\|^2 dx.$$

这里先把 p_{data} 简写成 p , 展开平方:

$$J(\theta) = \frac{1}{2} \int p(x) \|s_{\theta}(x)\|^2 dx - \int p(x) s_{\theta}(x) \cdot \nabla_x \log p(x) dx + \frac{1}{2} \int p(x)$$

最后一项和 θ 无关, 所以可以记成 *constant*。中间项利用:

$$p(x) \nabla_x \log p(x) = \nabla_x p(x).$$

因此:

$$-\int p(x) s_{\theta}(x) \cdot \nabla_x \log p(x) dx = -\int s_{\theta}(x) \cdot \nabla_x p(x) dx.$$

对每个维度做 *integration by parts*。若 $s_{\theta}(x) p(x)$ 在边界处衰减足够快, *boundary term* 为 0:

[!Notes]- (Advance) 多维 *integration by parts* 的通用公式

一维 *integration by parts* 是:

$$\int_a^b u(x) v'(x) dx = u(x) v(x) \Big|_a^b - \int_a^b u'(x) v(x) dx.$$

在 *score matching* 里, 把 $u(x) = s_{\theta}(x)$, $v(x) = p(x)$, 则:

$$\int_a^b s_{\theta}(x) \frac{dp(x)}{dx} dx = s_{\theta}(x) p(x) \Big|_a^b - \int_a^b p(x) \frac{ds_{\theta}(x)}{dx} dx.$$

如果边界项 $s_{\theta}(x) p(x) \Big|_a^b = 0$, 就得到:

$$\int_a^b s_{\theta}(x) \frac{dp(x)}{dx} dx = - \int_a^b p(x) \frac{ds_{\theta}(x)}{dx} dx.$$

多维时, 令 $x = (x_1, \dots, x_d)$, $s_{\theta}(x) = (s_1(x), \dots, s_d(x))$ 。cross term 是:

$$\int s_{\theta}(x) \cdot \nabla p(x) dx = \sum_{i=1}^d \int s_i(x) \frac{\partial p(x)}{\partial x_i} dx.$$

对每一项, 把除了 x_i 以外的变量暂时看成常数, 对 x_i 做一维 *integration by parts*:

$$\int s_i(x) \frac{\partial p(x)}{\partial x_i} dx = \int [s_i(x) p(x)] \partial_{\Omega_i} dx - \int p(x) \frac{\partial s_i(x)}{\partial x_i} dx.$$

若所有边界项都为 0, 则:

$$\int s_i(x) \frac{\partial p(x)}{\partial x_i} dx = - \int p(x) \frac{\partial s_i(x)}{\partial x_i} dx.$$

对 $i = 1, \dots, d$ 求和:

$$\int s_{\theta}(x) \cdot \nabla p(x) dx = - \int p(x) \sum_{i=1}^d \frac{\partial s_i(x)}{\partial x_i} dx.$$

因为 *divergence* 定义为:

$$\nabla \cdot s_{\theta}(x) = \sum_{i=1}^d \frac{\partial s_i(x)}{\partial x_i},$$

所以得到多维通用公式:

$$\int s_{\theta}(x) \cdot \nabla p(x) dx = - \int p(x) \nabla \cdot s_{\theta}(x) dx.$$

更紧凑地看, 这是 *divergence theorem* 的直接结果。因为:

$$\nabla \cdot (p(x) s_{\theta}(x)) = s_{\theta}(x) \cdot \nabla p(x) + p(x) \nabla \cdot s_{\theta}(x).$$

两边对区域 Ω 积分:

$$J(\theta) = \frac{1}{2} \int p(x) \|s_{\theta}(x)\|^2 dx - \int p(x) s_{\theta}(x) \cdot \nabla_x \log p(x) dx + \frac{1}{2} \int p(x) \| \nabla_x \log p(x) \|^2 dx.$$

$$\int_{\Omega} s_{\theta}(x) \cdot \nabla p(x) dx + \int_{\Omega} p(x) \nabla \cdot s_{\theta}(x) dx = \int_{\Omega} \nabla \cdot (p(x) s_{\theta}(x)) dx.$$

由 *divergence theorem*:

$$\int_{\Omega} \nabla \cdot (p(x) s_{\theta}(x)) dx = \int_{\partial \Omega} p(x) s_{\theta}(x) \cdot n(x) dS.$$

若 *boundary flux* 为 0, 即 $p(x) s_{\theta}(x)$ 在无穷远或边界上足够快衰减, 就得到同一个公式。

$$\int s_{\theta}(x) \cdot \nabla_x p(x) dx = - \int p(x) \nabla_x \cdot s_{\theta}(x) dx.$$

所以中间项变成:

$$-\int s_{\theta}(x) \cdot \nabla_x p(x) dx = \int p(x) \nabla_x \cdot s_{\theta}(x) dx.$$

代回原目标, 去掉和 θ 无关的 *constant*:

$J(\theta) = \mathbb{E}_{p_{\text{data}}} \left[\frac{1}{2} \ s_{\theta}(x)\ ^2 + \nabla_x \cdot s_{\theta}(x) \right] + \text{const.}$
这就是 <i>score matching objective</i>。它不再需要 <i>true score</i> $\nabla_x \log p_{\text{data}}(x)$，只需要从 p_{data} sample，并能计算 <i>network score field</i> 的 <i>divergence</i>。 这里： $\frac{1}{2} \ s_{\theta}(x)\ ^2$ regularizes score magnitude。 $\nabla_x \cdot s_{\theta}(x)$ 是 divergence term，用来匹配 data density 的 curvature。 这个目标的意义是：不用知道 <i>normalized density</i>，也不用知道 <i>partition function</i>，只要能从 data distribution sample，就能学习 score。 (Advance) Score 为什么比 <i>density</i> 更容易。 对 <i>EBM</i>：
$\log p(x) = -E(x) - \log Z.$
直接算 $\log p(x)$ 需要 Z，但是求梯度时：
$\nabla_x \log p(x) = -\nabla_x E(x).$
因此 <i>score</i> 丢掉了 <i>normalization constant</i>。Diffusion / <i>score-based generation</i> 的核心优势之一，就是把“学 <i>normalized probability</i>”改成“学往高概率区域走的方向”。 2.1 Denoising Score Matching (DSM):把 score 学习变成 denoising p.9 Vincent (2011) 的 Denoising Score Matching (DSM) 解决了 Hyvarinen objective 中 divergence computation 很贵的问题。 给 clean data x_0 加 Gaussian noise:
$q_{\sigma}(x \mid x_0) = \mathcal{N}(x; x_0, \sigma^2 I).$
对应的 marginal noised distribution 是：
$p_{\sigma}(x) = \int q_{\sigma}(x \mid x_0) p_{\text{data}}(x_0) dx_0.$
2.1.1 Gaussian Kernel Gaussian perturbation kernel 的 conditional score 有 closed form:
$\nabla_x \log q_{\sigma}(x \mid x_0) = -\frac{x - x_0}{\sigma^2}.$
若写 $x = x_0 + \sigma \epsilon$, $\epsilon \sim \mathcal{N}(0, I)$，则：
$\nabla_x \log q_{\sigma}(x \mid x_0) = -\frac{\epsilon}{\sigma}.$
2.1.2 Conditional Score to Marginal Score DSM 使用如下 MSE:
$\mathbb{E}_{x_0, \epsilon} \left[\ s_{\theta}(x) - \nabla_x \log q_{\sigma}(x \mid x_0)\ ^2 \right].$
虽然 target 是 conditional score,但训练出的 optimal predictor 是 marginal score:
$s_{\theta}^*(x) \approx \nabla_x \log p_{\sigma}(x).$
+ <i>MSE Properties #Loss/MSE when minimum</i> (Advance) 为什么 <i>conditional score target</i> 会学到 <i>marginal score</i>。 这里的关键词是 <i>marginalization</i>。Noisy sample x 的 <i>marginal density</i> 是把 <i>clean image</i> x_0 积分掉：
$p_{\sigma}(x) = \int q_{\sigma}(x \mid x_0) p_{\text{data}}(x_0) dx_0.$
对 x 求梯度，并把梯度移进积分：
$\nabla_x p_{\sigma}(x) = \int \nabla_x q_{\sigma}(x \mid x_0) p_{\text{data}}(x_0) dx_0.$
用 <i>identity</i> $\nabla_x q = q \nabla_x \log q$:
$\nabla_x p_{\sigma}(x) = \int q_{\sigma}(x \mid x_0) \nabla_x \log q_{\sigma}(x \mid x_0) p_{\text{data}}(x_0) dx_0.$
再用 <i>Bayes rule</i>:

$q_{\sigma}(x \mid x_0) p_{\text{data}}(x_0) = p(x_0 \mid x) p_{\sigma}(x).$
所以：
$\nabla_x p_{\sigma}(x) = p_{\sigma}(x) \int p(x_0 \mid x) \nabla_x \log q_{\sigma}(x \mid x_0) dx_0.$
两边除以 $p_{\sigma}(x)$:
$\nabla_x \log p_{\sigma}(x) = \mathbb{E}_{x_0 \mid x} [\nabla_x \log q_{\sigma}(x \mid x_0)].$
因此 <i>marginal score</i> 是 <i>conditional score</i> 在 <i>posterior</i> $p(x_0 \mid x)$ 下的平均。
也可以从 <i>MSE objective</i> 正向推出这一点。固定某个 <i>noisy input</i> x，令 <i>network</i> 在这个点的输出是一个向量 $a = s_{\theta}(x)$，而随机 <i>target</i> 是：
$Y = \nabla_x \log q_{\sigma}(x \mid x_0), \quad x_0 \sim p(x_0 \mid x).$
对这个固定的 x，局部 <i>MSE</i> 是：
$J(a) = \mathbb{E}_{x_0 \mid x} [\ a - Y\ ^2].$
对 a 求梯度：
$\nabla_a J(a) = 2\mathbb{E}_{x_0 \mid x} [a - Y] = 2 \left(a - \mathbb{E}_{x_0 \mid x} [Y] \right).$
令梯度为 0:
$a^* = \mathbb{E}_{x_0 \mid x} [Y] = \mathbb{E}_{x_0 \mid x} [\nabla_x \log q_{\sigma}(x \mid x_0)] = \nabla_x \log p_{\sigma}(x).$
所以 <i>DSM</i> 的 <i>MSE target</i> 虽然每次给的是某个 <i>sample pair</i> (x_0, x) 的 <i>conditional score</i>，但 <i>squared loss</i> 的 <i>pointwise minimizer</i> 就是 <i>marginal score</i>。换句话说，模型被迫学习“所有可能 <i>clean image</i> 对当前 <i>noisy image</i> 的平均去噪方向”。 2.2 Noise Conditional Score Network (NCSN):多噪声尺度的 score network p.10 单一 noise level σ 只能描述某个 scale 的 data structure。大 σ 看到 global structure，小 σ 看到 local details。Noise Conditional Score Network (NCSN) 训练一个带 noise condition 的 score network:
$x_{\sigma} = x_0 + \sigma \epsilon.$
训练目标:
$\frac{1}{2} \mathbb{E}_{x_0, \epsilon} \left[\left\ s_{\theta}(x_0 + \sigma \epsilon, \sigma) + \frac{\epsilon}{\sigma} \right\ ^2 \right].$
对多个 noise levels $\{\sigma_i\}_{i=1}^L$，NCSN 使用：
$\mathcal{L}(\theta) = \frac{1}{L} \sum_{i=1}^L \sigma_i^2 \ell(\theta, \sigma_i).$
这里乘 σ_i^2 是为了 balance magnitude，因为 score 大小通常随 $\frac{1}{\sigma}$ 缩放。 2.3 Annealed Langevin Dynamics: 从粗到细采样 p.11 NCSN 的 sampling 使用 Annealed Langevin Dynamics。核心过程是： - 从 prior noise 初始化 x。 - 对 noise levels $\sigma_1 > \sigma_2 > \dots > \sigma_L$ 从大到小循环。 - 在每个 σ_i 上做若干步 Langevin update:
$x \leftarrow x + \alpha_i s_{\theta}(x, \sigma_i) + \sqrt{2\alpha_i} z, \quad z \sim \mathcal{N}(0, I).$
step size 常写成：
$\alpha_i = \varepsilon \cdot \frac{\sigma_i^2}{\sigma_L^2}.$
直觉： - large σ: 先决定 layout / semantics / global shape。 - small σ: 逐渐补 texture / edge / high-frequency details。 <i>Score Matching</i> 到 <i>DDPM</i> 的缺口。

<i>NCSN</i> 已经能生成图像，但 <i>sampling</i> 仍然依赖 <i>Langevin MCMC</i>。每个 <i>noise level</i> 要跑很多步，高维 <i>mizing</i> 也慢。
<i>DDPM</i> (<i>Denoising Diffusion Probabilistic Model</i>) 把这个思路改写成一个 <i>likelihood-based latent-variable model</i>: <i>forward process</i> 固定加噪，<i>reverse process</i> 学习逐步去噪，并且可以用 <i>ELBO</i> 推出简单稳定的 <i>noise prediction objective</i>。 3 Denoising Diffusion Probabilistic Model (DDPM): 从 ELBO 到 noise prediction p.14 Diffusion 的基本想法：
$x_0 \rightarrow x_t \rightarrow x_T \approx \mathcal{N}(0, I)$
是 fixed forward noising process: 生成时学习 reverse chain:
$x_T \rightarrow x_{T-1} \rightarrow \dots \rightarrow x_0.$
forward process 是固定 Gaussian Markov chain:
$q(x_{1:T} \mid x_0) = \prod_{t=1}^T q(x_t \mid x_{t-1}).$
reverse process 是 learned generative chain:
$p_{\theta}(x_{0:T}) = p(x_T) \prod_{t=1}^T p_{\theta}(x_{t-1} \mid x_t).$
训练目标来自 Variational Lower Bound / Evidence Lower Bound (ELBO)。 3.1 Forward Transition: 逐步加 Gaussian noise p.15 DDPM 的 forward transition:
$q(x_t \mid x_{t-1}) = \mathcal{N} \left(\sqrt{1 - \beta_t} x_{t-1}, \beta_t I \right).$
定义：
$\alpha_t = 1 - \beta_t, \quad \bar{\alpha}_t = \prod_{s=1}^t \alpha_s.$
注意这里课件文本本里都显示为 α_t，但实际 DDPM notation 通常区分 single-step α_t 和 cumulative $\bar{\alpha}_t$，后面笔记用 $\bar{\alpha}_t$ 表示累计 signal retention。 随着 t 增大，$\bar{\alpha}_t \rightarrow 0$, image information 被逐渐 erase。 3.2 Closed-Form Marginal and Reparameterization p.16 线性 Gaussian transform 的 composition 仍然是 Gaussian，因此可以直接从 x_0 sample 任意 timestep 的 x_t：
$q(x_t \mid x_0) = \mathcal{N} \left(\sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t) I \right).$
对应 reparameterization:
$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, \quad \epsilon \sim \mathcal{N}(0, I).$
这一步非常重要：训练时不用真的从 x_0 迭代加噪到 x_t，只要随机采 t 和 ϵ，就能构造 network input。 3.3 Reverse Process: 学习去噪 Gaussian transition p.18 learned reverse transition 写成：
$p_{\theta}(x_{t-1} \mid x_t) = \mathcal{N} \left(\mu_{\theta}(x_t, t), \Sigma_{\theta}(x_t, t) \right).$
DDPM 试图近似 true posterior:
$q(x_{t-1} \mid x_t, x_0).$
+ <i>#Probability</i> $q(a b, c)$ and <i>Bayes Notes</i>.
(consider $q(a b c)!$) use <i>Bayes!</i>
$q(a b, c)q(b c) = q(b a, c)q(a c)$
(Advance) 为什么 reverse transition 可以写成 Gaussian。这里要区分两个 <i>distribution</i>。
第一，给定 x_0 的 <i>posterior</i>:

$q(x_{t-1} \mid x_t, x_0)$
是严格 <i>Gaussian</i>。因为 <i>forward step</i> 是：
$q(x_t \mid x_{t-1}) = \mathcal{N}(\sqrt{\alpha_t} x_{t-1}, \beta_t I),$
且从 x_0 到 x_{t-1} 的 <i>marginal</i> 也是 <i>Gaussian</i>:
$q(x_{t-1} \mid x_0) = \mathcal{N} \left(\sqrt{\bar{\alpha}_{t-1}} x_0, (1 - \bar{\alpha}_{t-1}) I \right).$
<i>Bayes rule</i> 给出：
$q(x_{t-1} \mid x_t, x_0) \propto q(x_t \mid x_{t-1}) q(x_{t-1} \mid x_0).$
把右边看作 x_{t-1} 的函数。第一项是：
$q(x_t \mid x_{t-1}) \propto \exp \left(-\frac{1}{2\beta_t} \ x_t - \sqrt{\alpha_t} x_{t-1}\ ^2 \right).$
第二项是：
$q(x_{t-1} \mid x_0) \propto \exp \left(-\frac{1}{2(1 - \bar{\alpha}_{t-1})} \ x_{t-1} - \sqrt{\bar{\alpha}_{t-1}} x_0\ ^2 \right).$
两个关于 x_{t-1} 的 <i>Gaussian</i> 相乘，指数里仍然是 x_{t-1} 的二次函数；complete the square 后归一化，仍然得到 <i>Gaussian</i>:
$q(x_{t-1} \mid x_t, x_0) = \mathcal{N}(\bar{\mu}_t(x_t, x_0), \bar{\beta}_t I).$
第二，真正 <i>sampling</i> 时理想上想要的是不知道 x_0 的 <i>reverse transition</i>:
$q(x_{t-1} \mid x_t) = \int q(x_{t-1} \mid x_t, x_0) p(x_0 \mid x_t) dx_0.$
这是许多 <i>Gaussian posterior</i> 的 <i>mixture</i>。因为 $p(x_0 \mid x_t)$ 通常很复杂，所以 $q(x_{t-1} \mid x_t)$ 不一定严格是 <i>Gaussian</i>。
<i>DDPM</i> 的建模选择是用 <i>Gaussian</i>:
$p_{\theta}(x_{t-1} \mid x_t) = \mathcal{N}(\mu_{\theta}(x_t, t), \Sigma_{\theta}(x_t, t))$
去近似真实 <i>reverse transition</i>。这个近似合理的原因是每一步 β_t 很小，<i>reverse dynamics</i> 是局部的小步去噪；在连续时间极限中，<i>reverse process</i> 是 <i>reverse-time SDE</i>，每个 <i>infinitesimal step</i> 都是 <i>Gaussian increment</i>。 实际模型通常不直接预测 x_0，而是预测加入到 x_t 中的 noise:
$\epsilon_{\theta}(x_t, t) \approx \epsilon.$
sampling 从 pure Gaussian noise 开始:
$x_T \sim \mathcal{N}(0, I), \quad x_{t-1} \sim p_{\theta}(x_{t-1} \mid x_t).$
3.4 ELBO Decomposition: 三个 loss 组件 p.19 negative ELBO:
$-\log p_{\theta}(x_0) \leq L_{\text{VLB}} = L_T + \sum_{t=2}^T L_{t-1} + L_0.$
(Advance) <i>ELBO decomposition</i> 的详细推导。 <i>DDPM</i> 把 $x_{1:T}$ 当作 <i>latent variables</i>。模型 <i>joint distribution</i> 是：
$p_{\theta}(x_{0:T}) = p(x_T) \prod_{t=1}^T p_{\theta}(x_{t-1} \mid x_t).$
<i>forward posterior</i> / <i>variational distribution</i> 是：

$q(x_{1:T} x_0) = \prod_{t=1}^T q(x_t x_{t-1}).$	
目标是 <i>maximize likelihood</i> $p_\theta(x_0)$, 或者 <i>minimize negative log-likelihood</i> :	
$-\log p_\theta(x_0).$	
先把 <i>latent variables</i> 积分出来:	
$p_\theta(x_0) = \int p_\theta(x_{0:T}) dx_{1:T}.$	
乘除同一个 $q(x_{1:T} x_0)$:	
$p_\theta(x_0) = \int q(x_{1:T} x_0) \frac{p_\theta(x_{0:T})}{q(x_{1:T} x_0)} dx_{1:T} = \mathbb{E}_{q(x_{1:T} x_0)} \left[\frac{p_\theta(x_{0:T})}{q(x_{1:T} x_0)} \right].$	
对 $-\log$ 用 <i>Jensen inequality</i> . 因为 $-\log$ 是 <i>convex</i> :	
$-\log \mathbb{E}_q[R] \leq \mathbb{E}_q[-\log R], \quad R = \frac{p_\theta(x_{0:T})}{q(x_{1:T} x_0)}.$	
因此:	
$-\log p_\theta(x_0) \leq \mathbb{E}_q \left[-\log \frac{p_\theta(x_{0:T})}{q(x_{1:T} x_0)} \right] = \mathbb{E}_q [\log q(x_{1:T} x_0) - \log p_\theta(x_{1:T} x_0)].$	
这就是 <i>variational upper bound on NLL</i> , 也就是 <i>negative ELBO</i> :	
$L_{\text{VLB}} = \mathbb{E}_q [\log q(x_{1:T} x_0) - \log p_\theta(x_{0:T})].$	
接下来把它拆成 <i>DDPM</i> 的三个部分。关键是把 <i>forward chain</i> 反向分解。由 <i>chain rule</i> :	
$q(x_{1:T} x_0) = q(x_T x_0) \prod_{t=2}^T q(x_{t-1} x_t, x_0).$	
这是同一个 <i>conditional joint distribution</i> 的反向 <i>factorization</i> 。于是:	
$\log q(x_{1:T} x_0) = \log q(x_T x_0) + \sum_{t=2}^T \log q(x_{t-1} x_t, x_0).$	
模型 <i>joint</i> 的 <i>log</i> 是:	
$\log p_\theta(x_{0:T}) = \log p(x_T) + \sum_{t=1}^T \log p_\theta(x_{t-1} x_t).$	
把 $t = 1$ 的项单独拿出来:	
$\sum_{t=1}^T \log p_\theta(x_{t-1} x_t) = \log p_\theta(x_0 x_1) + \sum_{t=2}^T \log p_\theta(x_{t-1} x_t).$	
代入 L_{VLB} :	
$L_{\text{VLB}} = \mathbb{E}_q \left[\log \frac{q(x_T x_0)}{p(x_T)} + \sum_{t=2}^T \log \frac{q(x_{t-1} x_t, x_0)}{p_\theta(x_{t-1} x_t)} - \log p_\theta(x_0 x_1) \right].$	
现在逐项识别 <i>KL / reconstruction term</i> .	
第一项只关于 x_T , 在 $q(x_T x_0)$ 下取期望:	

$L_T = D_{\text{KL}}(q(x_T x_0) \ p(x_T)).$	
中间每个 t , 在 $q(x_t x_0)$ 下先固定 x_t, x_0 , 再对 x_{t-1} 取期望:	
$L_{t-1} = D_{\text{KL}}(q(x_{t-1} x_t, x_0) \ p_\theta(x_{t-1} x_t)).$	
最后一项是 <i>final reconstruction likelihood</i> :	
$L_0 = -\log p_\theta(x_0 x_1).$	
所以:	
$L_{\text{VLB}} = L_T + \sum_{t=2}^T L_{t-1} + L_0.$	
这个 <i>decomposition</i> 的意义是: L_T 让 <i>terminal noisy distribution match prior</i> , L_{t-1} 训练每一步 <i>reverse transition</i> , L_0 负责最后一步从 x_1 <i>recover pixel-level</i> x_0 。三个部分分别是: Term; Meaning $L_T = D_{\text{KL}}(q(x_T x_0) \ p(x_T))$; forward endpoint 和 standard Gaussian prior 的 KL; 如果 β_t fixed, 通常是 constant $L_{t-1} = D_{\text{KL}}(q(x_{t-1} x_t, x_0) \ p_\theta(x_{t-1} x_t))$; true posterior 和 model reverse transition 的 KL $L_0 = -\log p_\theta(x_0 x_1)$; final reconstruction / discretized likelihood 因为 $q(x_{t-1} x_t, x_0)$ 和 $p_\theta(x_{t-1} x_t)$ 都是 Gaussian, 中间的 KL 可以用 <i>closed-form</i> 计算, 不需要 high-variance Monte Carlo。 <i>Discretized Gaussian</i> 的小细节。 原始 <i>DDPM</i> 里 <i>pixel</i> 是 integer $\{0, 1, \dots, 255\}$ 再 <i>scale</i> 到 $[-1, 1]$ 。连续 <i>Gaussian density</i> 不能直接给 <i>discrete pixel value</i> 分配 <i>probability mass</i> , 所以 $p_\theta(x_0 x_1)$ 会把 <i>Gaussian</i> 在 <i>pixel bin</i> 周围的小区间上积分。	
这个 <i>discretized Gaussian</i> 只影响 L_0 。对 L_{t-1} 来说, x_{t-1} 是 <i>continuous latent variable</i> , 不需要这个 <i>trick</i> 。	
3.5 From ELBO to Regression: Gaussian KL 变成 mean squared error p.21 中间项是两个 Gaussian 的 KL:	
$L_{t-1} = D_{\text{KL}}(q(x_{t-1} x_t, x_0) \ p_\theta(x_{t-1} x_t)).$	
写成:	
$q(x_{t-1} x_t, x_0) = \mathcal{N}(\bar{\mu}_t, \tilde{\beta}_t I),$	
$p_\theta(x_{t-1} x_t) = \mathcal{N}(\mu_\theta(x_t, t), \sigma_t^2 I).$	
如果 <i>variance matched / fixed</i> , KL 的关键部分就是 <i>mean squared error</i> :	
$D_{\text{KL}} = \frac{1}{2\sigma_t^2} \ \bar{\mu}_t - \mu_\theta(x_t, t)\ ^2 + \text{const.}$	
(Advance) 为什么 <i>matched variance</i> 的 <i>Gaussian KL</i> 只剩 <i>mean squared error</i> . 先写一般的 <i>multivariate Gaussian KL</i> 。令:	
$D_{\text{KL}}(q \ p) = \frac{1}{2} \left[\log \frac{ \Sigma_p }{ \Sigma_q } - d + \text{tr}(\Sigma_p^{-1} \Sigma_q) + (\mu_p - \mu_q)^T \Sigma_p^{-1} (\mu_p - \mu_q) \right].$	
在 <i>DDPM</i> 这个项里:	
$q(x_{t-1} x_t, x_0) = \mathcal{N}(\bar{\mu}_t, \tilde{\beta}_t I),$	
$p_\theta(x_{t-1} x_t) = \mathcal{N}(\mu_\theta, \sigma_t^2 I).$	

如果 <i>variance fixed</i> , $\tilde{\beta}_t$ 和 σ_t^2 都不由 <i>network mean parameter</i> 决定。把它们代入 <i>KL</i> 公式:	
$\log \frac{ \sigma_t^2 I }{ \tilde{\beta}_t I } = d \log \frac{\sigma_t^2}{\tilde{\beta}_t},$	
$\text{tr}((\sigma_t^2 I)^{-1} (\tilde{\beta}_t I)) = \frac{d \tilde{\beta}_t}{\sigma_t^2}.$	
这些项都只依赖 <i>variance schedule</i> , 不依赖 μ_θ 。因此对训练 <i>mean network</i> 来说, 他们都是 <i>constant</i> . 唯一和 μ_θ 有关的是 <i>quadratic mean term</i> :	
$(\mu_\theta - \bar{\mu}_t)^T (\sigma_t^2 I)^{-1} (\mu_\theta - \bar{\mu}_t) = \frac{1}{\sigma_t^2} \ \mu_\theta - \bar{\mu}_t\ ^2.$	
所以:	
$D_{\text{KL}} = \frac{1}{2\sigma_t^2} \ \mu_\theta - \bar{\mu}_t\ ^2 + \text{const.}$	
如果进一步做 <i>matched variance</i> , 令 $\sigma_t^2 = \tilde{\beta}_t$, 则 <i>determinant</i> 和 <i>trace</i> 部分甚至变成完全固定的数; 训练目标对 μ_θ 来说就是 <i>weighted mean squared error</i> 。 3.6 Posterior Mean Derivation: $\bar{\mu}_t$ 的闭式表达 p.22 forward marginals:	
$q(x_{t-1} x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_{t-1}) I),$	
$q(x_t x_{t-1}) = \mathcal{N}(\sqrt{\alpha_t} x_{t-1}, \beta_t I).$	
由 Bayes rule:	
$q(x_{t-1} x_t, x_0) \propto q(x_t x_{t-1}) q(x_{t-1} x_0).$	
取 $\log$ 并对 x_{t-1} <i>complete the square</i> , 可以得到 <i>posterior mean</i> :	
$\bar{\mu}_t = \frac{\sqrt{\bar{\alpha}_{t-1}} \beta_t x_0 + \sqrt{\alpha_t} (1 - \bar{\alpha}_{t-1}) x_t}{1 - \bar{\alpha}_t}.$	
注意只有 x_{t-1} 是变量! 其余的可以视为常数	
报导里的 <i>quadratic coefficient</i> .	
<i>log posterior</i> 中关于 x_{t-1} 的二次项来自两部分:	
$-\frac{1}{2\beta_t} \ x_t - \sqrt{\alpha_t} x_{t-1}\ ^2 - \frac{1}{2(1 - \bar{\alpha}_{t-1})} \ x_{t-1} - \sqrt{\bar{\alpha}_{t-1}} x_0\ ^2.$	
<i>quadratic coefficient</i> 是:	
$\frac{\alpha_t}{\beta_t} + \frac{1}{1 - \bar{\alpha}_{t-1}} = \frac{1 - \bar{\alpha}_t}{\beta_t (1 - \bar{\alpha}_{t-1})}.$	
<i>linear term</i> 除以 <i>quadratic coefficient</i> 后得到上面的 $\bar{\mu}_t$ 。	
3.7 Noise Prediction Parameterization p.23 把	
$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$	
代入 $\bar{\mu}_t$, 可以改写成:	
$\bar{\mu}_t = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon \right).$	
因此, 预测 <i>reverse mean</i> 等价于预测 <i>noise</i> ϵ . 这就是从 <i>likelihood-based ELBO</i> 到 <i>simple noise regression</i> 的桥。	
3.8 Simplified DDPM Objective p.24 实践中常用 <i>simplified loss</i> :	
$L_{\text{simple}} = \mathbb{E}_{t, x_0, \epsilon} [\ \epsilon - \epsilon_\theta(x_t, t)\ _2^2].$	
给定 ϵ_θ 后, 可以恢复 <i>clean image estimate</i> :	
$\hat{x}_0 = \frac{x_t - \sqrt{1 - \bar{\alpha}_t} \epsilon_\theta(x_t, t)}{\sqrt{\bar{\alpha}_t}}.$	

<i>DDPM</i> 影响力很大的原因之一是: 这个 <i>objective</i> 稳定、简单、容易 <i>scale</i> 。	
4 SDE and Probability-Flow ODE: 统一 DDPM, Score SDE 和 Flow View p.27 Stochastic Differential Equation (SDE) 描述 <i>drift</i> + <i>random noise</i> 的连续时间过程:	
$dx = f(x, t) dt + g(t) dW_t.$	
其中:	
$f(x, t)$ 是 <i>drift</i>, 控制 <i>deterministic trend</i>. $g(t)$ 是 <i>diffusion coefficient</i>, 控制 <i>Brownian noise</i> 强度。 <i>forward SDE</i> 是从 <i>data</i> 到 <i>noise</i> 的 <i>continuous-time corruption process</i>.	
<i>reverse-time SDE</i> :	
$dx = [f(x, t) - g(t)^2 \nabla_x \log p_t(x)] dt + g(t) dW_t.$	
<i>reverse</i> 时必须加上 <i>score term</i> :	
$-g(t)^2 \nabla_x \log p_t(x),$	
它像 <i>correction force</i> , 把 <i>probability flow</i> 从 <i>noise distribution</i> 拉回 <i>data distribution</i> 。	
4.1 Probability-Flow ODE (PF-ODE) p.28 Probability-Flow ODE (PF-ODE) 是一个 <i>deterministic process</i> , 和 <i>reverse SDE</i> 有相同的 <i>marginal distributions</i> :	
$dx = \left[f(x, t) - \frac{1}{2} g(t)^2 \nabla_x \log p_t(x) \right] dt.$	
它给出三种等价视角: View; Core object DDPM / ELBO; latent-variable likelihood objective Score SDE; learn $\nabla_x \log p_t(x)$ Probability-Flow ODE; deterministic vector field sampling	
4.2 Historical Takeaway: modern diffusion 的三条线汇合 p.29 现代 <i>diffusion</i> 可以看成三种传统的 <i>convergence</i> :	
- VAE / ELBO view: <i>diffusion</i> 是 <i>deep latent-variable model</i>. - Score matching view: 学习 $\nabla_x \log p_t(x)$。 - ODE / flow view: <i>generation</i> 是沿 <i>learned vector field</i> 从 <i>noise</i> 走到 <i>data</i>。	
(Advance) 为什么 <i>PF-ODE</i> 很重要。 <i>Reverse SDE</i> 有 <i>stochastic noise</i> , 因此 <i>sampling path</i> 不唯一。 <i>PF-ODE</i> 是 <i>deterministic</i> 的, 给了我们使用 <i>numerical ODE solver</i> 的入口, 例如 <i>Euler</i> 、 <i>Heun</i> 、 <i>DPM-Solver</i> 。	
后面把 <i>Flow Matching</i> 和 <i>PF-ODE</i> 对齐, 就是在说: 很多看起来不同的新生成模型, 其实是在学习同一个概率流的 <i>velocity field</i> 。	
5 Gaussian Path and Normalization: 用一个路径写下 diffusion / flow p.32 大部分 <i>modern diffusion</i> 和 <i>flow models</i> 都可以写成 <i>Gaussian path</i> :	
$x_t = \alpha_t x_0 + \sigma_t \epsilon, \quad x_0 \sim p_{\text{data}}, \quad \epsilon \sim \mathcal{N}(0, I), \quad t \in [0, 1].$	
条件分布:	
$q(x_t x_0) = \mathcal{N}(\alpha_t x_0, \sigma_t^2 I).$	
$\alpha_t x_0$ 是保留的 <i>signal</i> , σ_t^2 是 <i>injected noise variance</i> . Signal-to-Noise Ratio (SNR):	
$\text{SNR}(t) = \frac{\alpha_t^2}{\sigma_t^2}, \quad \lambda_t = \log \text{SNR}(t).$	
λ_t 在 <i>clean side</i> 大, 在 <i>noise side</i> 小. 现代 <i>diffusion</i> 很多 <i>scheduler / weighting</i> 都喜欢用 $\log\text{-SNR}$ λ 作为坐标。	
5.1 Three Famous Forward Processes p.33 Process; Path; Typical use Variance Exploding (VE); $x_t = x_0 + \sigma_t \epsilon$; NCSN / EDM Variance Preserving (VP); $x_t = \alpha_t x_0 + \sigma_t \epsilon$, $\alpha_t^2 + \sigma_t^2 = 1$; DDPM / IDDPM Linear Interpolation; $x_t = (1 - t)x_0 + t\epsilon$; Flow Matching / Rectified Flow Classical DDPM 通常是 VP-style; VE 常和 NCSN 关联。	

5.2 Normalization Removes the Difference p.34

定义 overall scale:

c_t = \sqrt{\alpha_t^2 + \sigma_t^2}.

把路径除以 c_t:

\tilde{x}_t = \frac{x_t}{c_t}, \quad \tilde{\alpha}_t = \frac{\alpha_t}{c_t}, \quad \tilde{\sigma}_t = \frac{\sigma_t}{c_t}.

此时:

\tilde{\alpha}_t^2 + \tilde{\sigma}_t^2 = 1.

所以不同 forward process 的差别, 很大程度上只是 scale / parameterization 不同. 归一化后, 它们落到同一个 unit circle 上.

5.3 Worked Normalization Examples p.35

VE path:

x_t = x_0 + \sigma_t \epsilon.

归一化后:

\tilde{\alpha}_t = \frac{1}{\sqrt{1 + \sigma_t^2}}, \quad \tilde{\sigma}_t = \frac{\sigma_t}{\sqrt{1 + \sigma_t^2}}.

VP path 已经满足:

\alpha_t^2 + \sigma_t^2 = 1.

Linear interpolation:

x_t = (1 - t)x_0 + t\epsilon.

归一化后:

\tilde{\alpha}_t = \frac{1 - t}{\sqrt{(1 - t)^2 + t^2}}, \quad \tilde{\sigma}_t = \frac{t}{\sqrt{(1 - t)^2 + t^2}}.

5.4 Forward Processes in One View p.36

Name; Path; Normalized?; Typical use

DDPM / VP; x_t = \alpha_t x_0 + \sigma_t \epsilon, \alpha_t^2 + \sigma_t^2 = 1; yes; DDPM, IDDPM VE / EDM; x_t = x_0 + \sigma_t \epsilon; no; NCSN, EDM

Flow Matching; x_t = (1 - t)x_0 + t\epsilon; no; Rectified Flow / Flow Matching

只有 VP 在所有时刻保持 variance normalized.

6 Five Equivalent Quantities: prediction target 的代数等价 p.37

沿 Gaussian path:

x_t = \alpha_t x_0 + \sigma_t \epsilon.

clean data、noise、score、velocity 和 PF-ODE velocity 是五个 algebraically equivalent quantities. 模型预测哪个 head, 更多是 parameterization 和 optimization 的选择.

6.1 Per-Sample Targets and Conditional Expectations p.39

给定 x_t, \alpha_t, \sigma_t, 可以写:

\epsilon = \frac{x_t - \alpha_t x_0}{\sigma_t}.

per-sample targets 包括:

- clean data: x_0.
- noise: \epsilon.
- score: conditional score.
- velocity:

u_t = \alpha'_t x_0 + \sigma'_t \epsilon.

- PF-velocity:

\tilde{f}(x, t) = f - \frac{1}{2} g(t)^2 s_t^*.

但 neural network 只能看到 x_t 和 t, 不能看到 hidden pair (x_0, \epsilon). 在 squared loss 下, optimal predictor 是 conditional expectation:

\operatorname{argmin}_f \mathbb{E} \left[\|f(x_t) - y\|^2 \right] = \mathbb{E}[y \mid x_t].

因此:

\epsilon^*(x_t, t) = \mathbb{E}[\epsilon \mid x_t], \quad D^*(x_t, t) = \mathbb{E}[x_0 \mid x_t], \quad v^*(x_t, t) = \mathbb{E}[u_t \mid x_t].

6.2 Noise and Clean Data Conversion p.41

从 forward path:

x_t = \alpha_t x_0 + \sigma_t \epsilon.

直接解出 per-sample noise:

\epsilon = \frac{x_t - \alpha_t x_0}{\sigma_t}.

对两边取 conditional expectation:

\mathbb{E}[\epsilon \mid x_t] = \frac{x_t - \alpha_t \mathbb{E}[x_0 \mid x_t]}{\sigma_t}.

用 optimal denoiser:

D^*(x_t, t) = \mathbb{E}[x_0 \mid x_t],

则:

\epsilon^*(x_t, t) = \frac{x_t - \alpha_t D^*(x_t, t)}{\sigma_t}.

6.3 Score as Conditional Score Expectation p.42

由于:

q(x_t \mid x_0) = \mathcal{N}(\alpha_t x_0, \sigma_t^2 I),

conditional score 是:

\nabla x_t \log q(x_t \mid x_0) = -\frac{x_t - \alpha_t x_0}{\sigma_t^2} = -\frac{\epsilon}{\sigma_t}.

marginal score 等于 conditional score 的 posterior expectation:

s_t^*(x_t) := \nabla x_t \log p_t(x_t) = \mathbb{E}_{x_0} \left[-\frac{x_t - \alpha_t x_0}{\sigma_t^2} \mid x_t \right] = -\frac{x_t - \alpha_t D^*(x_t, t)}{\sigma_t^2}.

这说明 score 和 denoiser 是等价的: 知道 D^* 就知道 score.

6.3.1 Why Marginal Score Is a Posterior Expectation p.43

从 marginal density 开始:

p_t(x_t) = \int q(x_t \mid x_0) p_0(x_0) dx_0.

对 x_t 求梯度:

\nabla x_t p_t(x_t) = \int q(x_t \mid x_0) \nabla x_t \log q(x_t \mid x_0) p_0(x_0) dx_0.

用 Bayes rule:

q(x_t \mid x_0) p_0(x_0) = p(x_0 \mid x_t) p_t(x_t).

于是:

\nabla x_t p_t(x_t) = p_t(x_t) \int p(x_0 \mid x_t) \nabla x_t \log q(x_t \mid x_0) dx_0.

两边除以 p_t(x_t):

\nabla x_t \log p_t(x_t) = \int p(x_0 \mid x_t) \nabla x_t \log q(x_t \mid x_0) dx_0 = \mathbb{E}_{x_0} \left[\nabla x_t \log q(x_t \mid x_0) \mid \tilde{f}(x, t) \right].

6.4 Velocity: Path Derivative p.44

对 Gaussian path 求时间导数:

u_t = \frac{dx_t}{dt} = \alpha'_t x_0 + \sigma'_t \epsilon.

用

\epsilon = \frac{x_t - \alpha_t x_0}{\sigma_t}

代入:

u_t = \frac{\sigma'_t}{\sigma_t} x_t + \left(\alpha'_t - \alpha_t \frac{\sigma'_t}{\sigma_t} \right) x_0.

取 conditional expectation:

v^*(x_t, t) = \mathbb{E}[u_t \mid x_t] = \frac{\sigma'_t}{\sigma_t} x_t + \left(\alpha'_t - \alpha_t \frac{\sigma'_t}{\sigma_t} \right) D^*(x_t, t).

6.5 Denoiser as the Bridge p.45

optimal denoiser:

D^*(x_t, t) = \mathbb{E}[x_0 \mid x_t].

所有 target 都可以通过 D^* 表示:

Noise:

\epsilon^*(x_t, t) = \frac{x_t - \alpha_t D^*(x_t, t)}{\sigma_t}.

Score:

s_t^*(x_t) = -\frac{x_t - \alpha_t D^*(x_t, t)}{\sigma_t^2}.

Velocity:

v^*(x_t, t) = \frac{\sigma'_t}{\sigma_t} x_t + \left(\alpha'_t - \alpha_t \frac{\sigma'_t}{\sigma_t} \right) D^*(x_t, t).

因此 noise prediction、x_0 prediction、score prediction、velocity prediction 在数学上可以相互转换.

6.6 PF-Velocity from Fokker-Planck Equation p.46

考虑 Itô diffusion SDE:

dx = f(x, t)dt + g(t)dW_t.

marginal density 满足 Fokker-Planck equation:

\frac{\partial p_t}{\partial t} = -\nabla \cdot (f p_t) + \frac{1}{2} g(t)^2 \nabla^2 p_t.

结合 Fokker-Planck equation:

\frac{\partial p_t}{\partial t} = -\nabla \cdot \left[\left(f - \frac{1}{2} g(t)^2 \nabla x \log p_t \right) p_t \right].

因此 probability-flow velocity 是:

\tilde{f}(x, t) := f(x, t) - \frac{1}{2} g(t)^2 \nabla x \log p_t(x).

6.7 PF-ODE Velocity Equals Flow Matching p.47

对 Gaussian path:

x_t = \alpha_t x_0 + \sigma_t \epsilon,

associated linear SDE 的 drift 和 diffusion coefficient 可以写成:

f(x, t) = \frac{\alpha'_t}{\alpha_t} x,

g(t)^2 = 2\sigma_t^2 \left(\frac{\sigma'_t}{\sigma_t} - \frac{\alpha'_t}{\alpha_t} \right).

代入 PF-ODE velocity:

\tilde{f}(x, t) = \frac{\alpha'_t}{\alpha_t} x + \sigma_t^2 \left(\frac{\alpha'_t}{\alpha_t} - \frac{\sigma'_t}{\sigma_t} \right) \nabla x \log p_t(x).

再用 score-denoiser relation:

\nabla x \log p_t(x) = s_t^*(x) = -\frac{x - \alpha_t D^*(x, t)}{\sigma_t^2}.

化简得到:

\tilde{f}(x, t) = \frac{\sigma'_t}{\sigma_t} x + \left(\alpha'_t - \alpha_t \frac{\sigma'_t}{\sigma_t} \right) D^*(x, t).

这和上一节的 Flow Matching velocity:

v^*(x_t, t) = \frac{\sigma'_t}{\sigma_t} x_t + \left(\alpha'_t - \alpha_t \frac{\sigma'_t}{\sigma_t} \right) D^*(x_t, t)

逐项相同. 所以:

Flow Matching learns the diffusion probability-flow ODE.

6.8 The Big Equivalence p.50

这一讲最核心的 unified view:

Diffusion $\approx$ Score learning $\approx$ Denoising $\approx$ Flow Matching $\approx$ Probability-flow

方法之间的主要差异来自:

- forward path: (α_t, σ_t) 怎么选.
- time parameterization: 用 t 还是用 $\log\text{-SNR}$ λ .
- sampling solver: Euler、Heun、DPM-Solver、stochastic sampler.
- target head: 预测 ϵ, x_0, s, v .
- loss weighting: $w(\lambda)$.

6.9 Sampling Loop p.51

ODE sampling 从 noise side 开始:

x\lambda_{\min} \sim \mathcal{N}(0, I).

一般 ODE form:

\frac{dx}{dt} = v_\theta(x, t).

Diffusion PF-ODE form:

\frac{dx}{dt} = f(t)x - \frac{1}{2} g(t)^2 s_\theta(x, t).

常见 solver:

- Euler.
- Heun / second-order solver.
- DPM-Solver family.
- stochastic sampler with churn / noise injection.

few-step generation 的核心就是: solver 更强, 或者训练目标更适合少步采样.

7 Loss Objectives, Weighting, and Schedules: 从 ELBO 到 log-SNR 坐标 p.54

最后一部分讨论: 不同 noise levels 的 loss 应该怎么加权.

对任意 Gaussian-forward diffusion, ELBO 可以写成:

L_{\text{VLB}} = L_T + \sum_{t=1}^T L_{t-1}.

每个中间项:

L_{t-1} = D_{\text{KL}} \left(q(x_{t-1} \mid x_t, x_0) \parallel p_\theta(x_{t-1} \mid x_t) \right).

若 matched variance $\sigma_t^2 = \tilde{\beta}_t$, 则:

L_{t-1} = \frac{1}{2\tilde{\beta}_t} \left\| \mu_\theta(x_t, t) - \tilde{\mu}_t(x_t, x_0) \right\|^2 + \text{const}.

7.1 x-Prediction Parameterization p.55

posterior mean:

\hat{\mu}_t = \frac{\sqrt{\alpha_t-1}\beta_t x_0 + \sqrt{\alpha_t}(1 - \tilde{\alpha}_{t-1})x_t}{1 - \tilde{\alpha}_t}.

如果 model mean 用 x_0-predictor $D_\theta(x_t, t)$ 参数化, 就是把 x_0 替换成 D_θ :

\mu_\theta = \frac{\sqrt{\alpha_t-1}\beta_t D_\theta + \sqrt{\alpha_t}(1 - \tilde{\alpha}_{t-1})x_t}{1 - \tilde{\alpha}_t}.

因此:

\left\| \mu_\theta - \hat{\mu}_t \right\|^2 = \frac{\tilde{\alpha}_{t-1}\beta_t^2}{(1 - \tilde{\alpha}_t)^2} \left\| D_\theta(x_t, t) - x_0 \right\|^2.

7.2 The Weight Is the SNR Drop p.56

DDPM posterior variance:

\tilde{\beta}_t = (1 - \tilde{\alpha}_{t-1}) \frac{\beta_t}{1 - \tilde{\alpha}_t}.

代入 KL coefficient:

$$L_{t-1} = \frac{1}{2} \frac{\bar{\alpha}_{t-1} \beta_t}{(1 - \bar{\alpha}_{t-1})(1 - \bar{\alpha}_t)} \|D_{\theta}(x_t, t) - x_0\|^2 + \text{const.}$$

定义 SNR:

$$r_t = \frac{\bar{\alpha}_t}{1 - \bar{\alpha}_t}.$$

由于:

$$\bar{\alpha}_t = (1 - \beta_t)\bar{\alpha}_{t-1}, \quad \bar{\alpha}_{t-1} - \bar{\alpha}_t = \bar{\alpha}_{t-1}\beta_t,$$

有:

$$r_{t-1} - r_t = \frac{\bar{\alpha}_{t-1} - \bar{\alpha}_t}{(1 - \bar{\alpha}_{t-1})(1 - \bar{\alpha}_t)} = \frac{\bar{\alpha}_{t-1}\beta_t}{(1 - \bar{\alpha}_{t-1})(1 - \bar{\alpha}_t)}.$$

所以:

$$L_{t-1} = \frac{1}{2} (r_{t-1} - r_t) \|D_{\theta}(x_t, t) - x_0\|^2 + \text{const.}$$

解释: ELBO 不会均匀惩罚所有 timesteps; 它在 SNR 下降更多的 interval 给更大权重。

7.3 Continuous-Time SNR Loss p.57

连续时间写作:

$$x_{\tau} = \alpha_{\tau} x_0 + \sigma_{\tau} \epsilon.$$

SNR:

$$r(\tau) = \frac{\alpha_{\tau}^2}{\sigma_{\tau}^2}.$$

DDPM VLB 的 x -prediction form 是:

$$\mathcal{L}_{\text{VLB}} = \frac{1}{2} \sum_{t=1}^T (r_{t-1} - r_t) \mathbb{E} \left[\|D_{\theta}(x_t, t) - x_0\|^2 \right] + \text{const.}$$

连续极限中, SNR 随 forward noising 下降:

$$\frac{dr(\tau)}{d\tau} < 0.$$

因此:

$$\mathcal{L} = \frac{1}{2} \int_0^1 -\frac{dr(\tau)}{d\tau} \mathbb{E} \left[\|D_{\theta}(x_{\tau}, \tau) - x_0\|^2 \right] d\tau + \text{const.}$$

这里 $-\frac{dr(\tau)}{d\tau} > 0$, 表示 SNR drop faster 的地方权重更高。

7.4 Change Variables from Time to SNR p.59

令:

$$\rho = r(\tau).$$

因为:

$$d\rho = \frac{dr(\tau)}{d\tau} d\tau, \quad -\frac{dr(\tau)}{d\tau} d\tau = -d\rho,$$

且 $r(\tau)$ 从 $r_{\max}$ 下降到 $r_{\min}$, 所以:

$$\mathcal{L} = \frac{1}{2} \int_{r_{\min}}^{r_{\max}} \mathbb{E} \left[\|D_{\theta}(x_{\rho}(\rho), \tau(\rho)) - x_0\|^2 \right] d\rho + \text{const.}$$

结论: 用 SNR 作为坐标后, time-dependent VLB weight 被 Jacobian 抵消, loss 在 SNR ρ 上是 uniform 的。

7.5 epsilon-Prediction Is Not Uniform in SNR p.60

从 SNR 坐标下的 x_0 -prediction loss:

$$\mathcal{L}_{\text{diff}} = \frac{1}{2} \int \mathbb{E} \left[\|D_{\theta}(x_r, r) - x_0\|^2 \right] dr.$$

设:

$$x_r = \alpha x_0 + \sigma \epsilon, \quad r = \frac{\alpha^2}{\sigma^2}.$$

若使用 ϵ -prediction, 则:

$$D_{\theta}(x_r, r) - x_0 = \frac{\sigma}{\alpha} (\epsilon - \epsilon_{\theta}).$$

而:

$$\frac{\sigma^2}{\alpha^2} = \frac{1}{r}.$$

所以:

$$\mathcal{L}_{\text{diff}} = \frac{1}{2} \int \frac{1}{r} \mathbb{E} \left[\|\epsilon - \epsilon_{\theta}(x_r, r)\|^2 \right] dr.$$

结论: 在 SNR coordinate 中, ϵ -prediction 带有 $\frac{1}{r}$ 权重, 不是 uniform over SNR。

7.6 Log-SNR Makes epsilon-Prediction Uniform p.61

为去掉 $\frac{1}{r}$, 使用 log-SNR:

$$\lambda = \log r, \quad r = e^{\lambda}, \quad dr = e^{\lambda} d\lambda.$$

代入:

$$\frac{1}{r} dr = e^{-\lambda} e^{\lambda} d\lambda = d\lambda.$$

因此:

$$\mathcal{L}_{\text{diff}} = \frac{1}{2} \int_{\lambda_{\min}}^{\lambda_{\max}} \mathbb{E} \left[\|\epsilon - \epsilon_{\theta}(x_{\lambda}, \lambda)\|^2 \right] d\lambda.$$

结论: log-SNR 是让 ϵ -prediction 在 noise levels 上 uniform 的自然坐标。

7.7 General Weighted Diffusion Objective p.62

定义 per-log-SNR loss:

$$\ell_{\theta}(\lambda) = \mathbb{E}_{x_0, \epsilon} \left[\|\epsilon - \epsilon_{\theta}(x_{\lambda}, \lambda)\|^2 \right].$$

general weighted objective:

$$\mathcal{L}_w(\theta) = \frac{1}{2} \int_{\lambda_{\min}}^{\lambda_{\max}} w(\lambda) \ell_{\theta}(\lambda) d\lambda.$$

特殊情况:

- ELBO-like: $w(\lambda) = 1$.
- perceptual / heuristic weighting: $w(\lambda) \neq 1$.
- 不同 diffusion method 很多时候就是选了不同的 $w(\lambda)$.

7.8 Target Integral and Monte Carlo p.63

目标积分:

$$\mathcal{L}_w = \frac{1}{2} \int w(\lambda) \ell_{\theta}(\lambda) d\lambda.$$

训练时用 Monte Carlo: sample noise level $\lambda \sim q(\lambda)$.

如果 naive average:

$$\mathbb{E}_{\lambda \sim q} [\ell_{\theta}(\lambda)] = \int q(\lambda) \ell_{\theta}(\lambda) d\lambda,$$

它优化的是 $q(\lambda)$ weighted objective, 不是目标的 $w(\lambda)$. unbiased importance sampling estimator:

$$\hat{\mathcal{L}}_w = \frac{1}{2} \frac{w(\lambda)}{q(\lambda)} \ell_{\theta}(\lambda), \quad \lambda \sim q.$$

7.9 Four Design Choices p.64

diffusion training 可以拆成四个独立设计:

- Path: $x_t = \alpha_t x_0 + \sigma_t \epsilon$.
- Prediction target: ϵ, x_0, s, v .
- Objective weighting: $w(\lambda)$, 表示我们真正想重视哪些 noise levels.
- Noise proposal / schedule: $q(\lambda)$, 表示训练时实际怎样 sample noise levels.

不要混淆 w 和 q :

- $w(\lambda)$ is desired weighting, 是 objective.
- $q(\lambda)$ 是 sampling distribution, 是 Monte Carlo estimator 的 proposal.

7.10 Weighting vs Schedule and Optimal Proposal p.65

课件给出的 practical shortcut:

$$\hat{\mathcal{L}} = \frac{1}{2} a(\lambda) \ell_{\theta}(\lambda),$$

则 effective weight 是:

$$w_{\text{eff}}(\lambda) = q(\lambda) a(\lambda).$$

variance-optimal proposal 满足:

$$q^*(\lambda) \propto w(\lambda) \sqrt{\text{Var}[\ell(\lambda)]}.$$

实践经验:

- 选 w : 决定想让模型学什么。
- 选 q : 决定训练梯度是否稳定、estimator variance 是否小。
- q 太小而 w 很大时, importance correction 会 explode.
- q 太大而 w 很小时, sample 会被浪费。

7.11 Training Loop p.67

diffusion training loop 可以看成 integral 的 Monte Carlo estimator:

- Sample $x_0 \sim p_{\text{data}}$, $\epsilon \sim \mathcal{N}(0, I)$.
- Sample noise level $\lambda \sim q(\lambda)$.
- Convert to $\alpha(\lambda), \sigma(\lambda)$.
- Form noisy input:

$$x_{\lambda} = \alpha x_0 + \sigma \epsilon.$$

- Predict target: $\epsilon_{\theta}, x_{\theta}, v_{\theta}$ or s_{θ} .
- Apply weight: $\frac{w(\lambda)}{q(\lambda)}$.
- Backpropagate.

7.12 Are Weighting Schemes Just Heuristics? p.68

常见 weighting schemes:

Scheme; Intuition

Uniform; $w(\lambda) = 1$

EDM / Sigmoid; focus on mid-noise levels

Min-SNR; clip high-SNR weights

这些常被当成 engineering heuristic. 但 Kingma & Gao (2023) 给出解释:

$$\mathcal{L}_w = \int w(\lambda) \ell(\lambda) d\lambda$$

在 monotone weighting 下, 可以改写成 Gaussian-noise-augmented data 上的 ELBO mixture.

结论: weighting 不是纯 hack: 它可以被理解为对不同程度 augmented data 做 likelihood training.

7.13 Truncated ELBO as Augmented-Data ELBO p.69

在某个 λ 截断 chain, 把 x_{λ} 当作 observed augmented datum:

$$x_0 \sim q_{\text{data}}(x_0), \quad x_{\lambda} = \alpha_{\lambda} x_0 + \sigma_{\lambda} \epsilon, \quad \epsilon \sim \mathcal{N}(0, I).$$

对 x_{λ} 的 ELBO 中, latent variables 是更 noisy 的 states:

$$x_{\lambda_{\min}} \rightarrow \cdots \rightarrow x_{\lambda'} \rightarrow x_{\lambda}, \quad \lambda' < \lambda.$$

关键点: reverse process 是 $x_{\lambda_{\min}} \rightarrow x_{\lambda}$, 不是直接到 x_0 。

7.14 Monotone Weighting as Mixture of Augmented-Data ELBOs p.70

truncated ELBO at λ_0 包含所有 $\lambda \leq \lambda_0$ 的 terms:

$$K(\lambda_0) = \int_{\lambda_{\min}}^{\lambda_0} \ell(\lambda) d\lambda.$$

若 $\lambda_0 \sim \rho$, 则:

$$\mathbb{E}_{\lambda_0 \sim \rho} K(\lambda_0) = \int_{\lambda_{\min}}^{\lambda_{\max}} \ell(\lambda) \Pr(\lambda_0 \geq \lambda) d\lambda.$$

effective weight:

$$w(\lambda) = \Pr(\lambda_0 \geq \lambda).$$

这要求 $w(\lambda)$ 对 λ non-increasing.

解释:

- 小 λ , 更 noisy, 被很多 truncation 包含, 所以 weight 高。
- 大 λ , 更 clean, 只被较大 λ_0 包含, 所以 weight 低。
- $w(\lambda) = 1$ 是普通 full ELBO on clean data x_0 。

7.15 Practical Interpretation p.71

实用理解:

- Uniform weighting $w = 1$: 更接近 exact likelihood, 所有 noise levels 同等重要。
- Sigmoid / EDM style weighting: 更关注 mid-noise levels, 能有更好的 perceptual quality.
- high-resolution images 通常需要更多 fine details, 因此可能需要把 sigmoid center 往 high λ 移动。

这解释了为什么 diffusion practitioners 可以自由设计 weighting function: 在 monotone 条件下, 它仍然可以被解释为 likelihood training, 只是 data 被 adaptive Gaussian noise augmentation 了。

8 Lecture Map: 这讲的最短记忆版 p.2

这讲的主线可以压缩为:

- Image generation 是 density modeling, 但 explicit density 很受限。
- EBM 绕开 normalized density, Langevin dynamics 只需要 score。
- Score Matching $\nabla_x \log p(x)$, Denoising Score Matching 把它变成 denoising。
- NCSN 用多个 noise levels + annealed Langevin sampling。
- DDPM 用 fixed forward noising + learned reverse denoising, 并从 ELBO 推出 noise prediction。
- SDE / PF-ODE 说明 score-based diffusion 和 deterministic flow 是同一组 marginals 的不同路径。
- Gaussian path $x_t = \alpha_t x_0 + \sigma_t \epsilon$ 统一 VE、VP、Flow Matching。
- x_0 、 ϵ 、score、velocity、PF-velocity 都可以通过 denoiser D^* 互相转换。
- loss weighting 本质是在 log-SNR coordinate 上选择 $w(\lambda)$, 训练 loop 是 Monte Carlo estimator。
- monotone weighting 可以解释为 augmented-data ELBO mixture, 不只是 heuristic。

这讲最容易混的三组概念。

score vs. noise prediction: score 是 $\nabla_x \log p_t(x)$; noise prediction 是预测和 λ 的 ϵ 。在 Gaussian path 下它们可通过 σ_t 和 α_t 互换。

****time t vs. log-SNR λ **:** t 是人为 schedule 坐标, $\lambda = \log(\alpha_t^2/\sigma_t^2)$ 更接近 signal/noise 的物理量。很多 loss weighting 在 λ 坐标下更自然。

****objective weight w vs. proposal q **:** w 决定目标函数重视哪里; q 决定训练时从哪 sample. importance weighting $\frac{w}{q}$ 负责让 estimator 对目标无偏。

created: 2026-05-31

updated: 2026-05-31

1 Non-Diffusion Image Generation Paradigms: 四种 diffusion 之外的生成路线 p.3

这一讲讨论的核心不是继续推导 diffusion, 而是看 image generation 里另外几类重要路线:

Paradigm; Exact Likelihood; Sampling Speed; Training Stability; Sample Quality; Typical Bottleneck

Normalizing Flow (NF); Yes; Fast, often 1-step; Stable, Maximum Likelihood Estimation (MLE); Good in modern variants; Jacobian constraints

Generative Adversarial Network (GAN); No; Fast, often 1-step; Unstable; Excellent; mode collapse, training art

Autoregressive Model (AR); Yes; Slow, $O(N)$; Stable, MLE; Good; serial generation, fixed order

Flexible-Order / Masked Generation; No / Approx; Medium, 8–64 steps; Stable; Good; mask schedule, editing

这四条路线的分歧可以理解成: 是否显式写出 $p(x)$, 是否需要可计算 likelihood, 采样是不是 serial, 以及模型如何处理 image 的二维空间结构。

(Advance) Terms and Table Justification.

- **Exact likelihood:** 模型能对给定样本 x 直接计算 normalized density 或 probability mass $p_{\theta}(x)$, 而不是只会 sample。
- **Sampling speed:** 从 latent/noise/token state 生成一张 image 需要多少 sequential steps. Flow 和 GAN 常是一到少数几次 forward pass; standard AR 要接 N 个元素逐个生成; masked/flexible-order 方法用少量 refinement steps 折中。
- **Training stability:** objective 是否是半模型的 maximum likelihood / cross-entropy, 还是两个模型的 game dynamics. MLE 通常更稳定; GAN 的 generator 和 discriminator 同时变化, 因此更容易 oscillate。
- **Sample quality:** 主要指视觉 fidelity. GAN 的 discriminator 直接惩罚 perceptual artifacts, 所以历史上 high-frequency detail 很强; Flow/AR 的 likelihood objective 更稳定, 但不一定直接优化人类视觉偏好。
- **Maximum Likelihood Estimation (MLE):** 最大似然估计。给定训练数据, 选择让真实样本 likelihood 最大的参数:

$$\theta^* = \arg \max_{\theta} \sum_i \log p_{\theta}(x_i).$$

实际训练常写成最小化 negative log-likelihood:

$$\mathcal{L}_{\text{NLL}} = - \sum_i \log p_{\theta}(x_i).$$

Flow / Autoregressive Model (AR) 通常说训练稳定, 就是因为它们优化的是这个明确的 likelihood loss, 而不是 Generative Adversarial Network (GAN) 的 two-player game。

表中的结论来自各模型 *objective* 的结构，而不是孤立经验：Flow/AR 显式分解 $p_{\theta}(x)$ ，因此有 *likelihood*；GAN 只定义 *sample generator* $G(z)$ 和 *discriminator score*，通常没有 *tractable density*；Flexible-order masked model 的 *probability* 往往依赖 *iterative mask schedule*，所以 *likelihood* 不是标准 *exact AR chain*。（*Advance*）*Intuition*。
Diffusion 把 *sampling* 写成一个逐步 *denoising process*；这一讲的四类方法则分别选择了不同的代价：

- *Normalizing Flow (NF)*：用可逆变换换取 *exact likelihood*。
- *Generative Adversarial Network (GAN)*：放弃显式 *density*，用 *discriminator* 提供 *perceptual training signal*。
- *Autoregressive Model (AR)*：用 *chain rule* 得到稳定 *likelihood*，但生成是顺序的。
- *Flexible-Order / Masked Generation*：打乱固定 *raster order*，让多个位置可以并行预测或迭代 *refinement*。

2 Normalizing Flow (NF)：用可逆变换连接 image space 和 latent space p.5

Normalizing Flow (NF) 的核心问题是：

How to turn a complex image distribution into a simple latent distribution?

它学习一个 *invertible map*，将复杂数据分布映射到简单 *latent distribution*，通常是 *standard Gaussian*。生成时从 *latent distribution* 采样，再通过 *inverse map* 回到 *image space*。（*Advance*）*New Terms*: *Conditioner* 和 *MLE Context*。

- *conditioner*：在 *coupling / autoregressive flow* 中，用来根据已有变量预测 *transform parameters* 的 *neural network*。它通常输出 *shift*、*scale*、*mean*、*variance* 等参数。
- *conditioner* 本身不需要可逆；只要它输出的整体 *transform* 可逆即可。

以 *Real-valued Non-Volume Preserving (RealNVP)* 为例：

$$y_b = x_b \odot \exp(s(x_a)) + t(x_a).$$

这里 $s(\cdot)$ 和 $t(\cdot)$ 就是 *conditioners*：它们读取 x_a ，输出对 x_b 做 *affine transform* 所需的 *scale* 和 *shift*。*inverse* 时可以先由 $y_a = x_a$ 得到 x_a ，再重新计算 $s(y_a)$ 、 $t(y_a)$ ，所以不需要反解 *conditioner network*。

在 *Transformer-based Masked Autoregressive Flow (TarFlow)* 中，*Transformer* 就是更强的 *conditioner*：它根据前面的 *patches* $x_{<i}$ 预测当前 *patch* 的 $\mu_i(x_{<i})$ 和 $\sigma_i(x_{<i})$ 。

2.1 Explicit Density Model 的动机：不只要“看起来像” p.6

Flow 的价值在于它能回答一些 *implicit model* 难以直接回答的问题：

- 这张 *image* 在 *model* 下的 *likelihood* 有多大？
- 给定 *sample*，它映射到 *latent space* 的哪个位置？
- 能否 *encode* 和 *decode*，而且没有 *information loss*？

因此 Flow 特别适合 *density estimation*、*anomaly detection*、*compression* 和 *invertible editing*。

（*Advance*）*Comparison with GAN and Diffusion*。
GAN 通常不能直接计算 *density*，因为它只定义了从 *noise* 到 *image* 的 *generator*。

Diffusion 有概率解释，但 *likelihood* 往往来自 *variational bound* 或 *probability flow ODE* 等间接方式；相比之下 Flow 的 $p_X(x)$ 来自 *exact change of variables*。

所以 Flow 的强项不是“最像真实照片”，而是 *generation*、*likelihood evaluation* 和 *latent inference* 三件事在同一个 *invertible transform* 里统一。

2.2 Data Space, Latent Space 和 Invertible Mapping p.7

Flow 的基本形式是：

$$z = f_{\theta}(x), \quad x = f_{\theta}^{-1}(z).$$

（*Advance*）*Why the Two Equations Are Equivalent*。
这里假设 f_{θ} 是 *bijection*。*bijection* 的定义包含两件事：

- *injective*：如果 $f_{\theta}(x_1) = f_{\theta}(x_2)$ ，则 $x_1 = x_2$ ，所以不同 *images* 不会 *collapse* 到同一个 *latent*。
- *surjective*：latent space 中每个合法 z 都有某个 x 满足 $z = f_{\theta}(x)$ 。

因此 *inverse function* f_{θ}^{-1} 存在，并满足：

$$f_{\theta}^{-1}(f_{\theta}(x)) = x, \quad f_{\theta}(f_{\theta}^{-1}(z)) = z.$$

所以 *forward encoding* $z = f_{\theta}(x)$ 和 *reverse generation* $x = f_{\theta}^{-1}(z)$ 是同一个可逆 *coordinate change* 的两个方向。

forward encoding 把真实 *image* x 映射到 *latent* z ，*reverse generation* 从 *latent* z 生成 *image* x 。*base distribution* 通常选：

$$z \sim p_Z(z) = \mathcal{N}(0, I)$$

或其他 *factorized distribution*。训练目标是让真实数据通过 f_{θ} 后接近这个简单分布。

（*Advance*）*Invertibility* 的硬约束。

Invertibility 意味着 *probability mass* 必须 *one-to-one transport*。普通 *encoder* 可以丢弃信息，比如把 *image* 压成 *embedding*；Flow 不可以这样做，因为每个 x 都必须能从 z 精确反推回来。

这带来一个很直接的后果：Flow 的 *input* 和 *output dimension* 通常要匹配，*architecture* 也不能像普通 *Convolutional Neural Network (CNN)* 或 *Transformer* 那样自由。

2.3 Change of Variables: Flow 的核心 likelihood 公式 p.8
change-of-variables formula 给出 *exact data density*：

$$\log p_X(x) = \log p_Z(f_{\theta}(x)) + \log \left| \det J_{f_{\theta}}(x) \right|.$$

（*Advance*）*Derivation: Change of Variables*。

先从 *probability mass conservation* 出发。对一个很小的 *data-space volume element* dx ，经过 $z = f_{\theta}(x)$ 后变成 *latent-space volume element* dz 。多元微积分给出：

$$dz = \left| \det J_{f_{\theta}}(x) \right| dx.$$

同一块 *probability mass* 在两个坐标系中必须相等：

$$p_X(x)dx = p_Z(z)dz.$$

代入 $z = f_{\theta}(x)$ 和 *volume scaling*：

$$p_X(x)dx = p_Z(f_{\theta}(x)) \left| \det J_{f_{\theta}}(x) \right| dx.$$

两边消去 dx ：

$$p_X(x) = p_Z(f_{\theta}(x)) \left| \det J_{f_{\theta}}(x) \right|.$$

对两边取 $\log$ ，得到正式公式。这里使用绝对值是因为 *determinant* 可能为负，但 *volume scaling* 必须非负。

其中：

- 第一项 $\log p_Z(f_{\theta}(x))$ ：latent $z = f_{\theta}(x)$ 在 *base distribution* 下有多 likely。
- 第二项 $\log \left| \det J_{f_{\theta}}(x) \right|$ ：local volume correction，描述 *mapping* 在 x 附近压缩或拉伸空间。

如果 *mapping* 局部压缩 *volume*，同样的 *probability mass* 被挤进更小区域，*density* 上升；如果 *mapping* 局部拉伸 *volume*，*density* 下降。

2.4 Maximum Likelihood Objective 和 Layer-wise Log-Det p.9

实际 Flow 通常是多层 *invertible layer* 的组合：

$$f = f_K \circ \cdots \circ f_1.$$

composition 仍然 *invertible*。*determinant* 满足：

$$\det J_f = \prod_{k=1}^K \det J_{f_k}, \quad \log \left| \det J_f \right| = \sum_{k=1}^K \log \left| \det J_{f_k} \right|.$$

（*Advance*）*Derivation: Layer-wise Log-Det*。

对 *composition* $f = f_K \circ \cdots \circ f_1$ ，*chain rule* 给出 *Jacobian matrix*：

$$J_f(x) = J_{f_K}(h_{K-1}) \cdots J_{f_2}(h_1) J_{f_1}(x),$$

其中 $h_k = f_k \circ \cdots \circ f_1(x)$ 。利用 *determinant* 的乘法性质：

$$\det(AB) = \det A \det B,$$

可得：

$$\det J_f(x) = \prod_{k=1}^K \det J_{f_k}(h_{k-1}).$$

再利用 $\log \prod_k a_k = \sum_k \log a_k$ ，得到 *total log-det* 是 *per-layer log-det* 的和。*negative log-likelihood loss* 是：

$$\mathcal{L}(\theta) = -\mathbb{E}_{x \sim p_{\text{data}}} \left[\log p_Z(f_{\theta}(x)) + \log \left| \det J_{f_{\theta}}(x) \right| \right].$$

（*Advance*）*Derivation: Negative Log-Likelihood Objective*。

Maximum Likelihood Estimation (MLE) 从下面的目标出发：

$$\theta^* = \arg \max_{\theta} \mathbb{E}_{x \sim p_{\text{data}}} \log p_{\theta}(x).$$

用 *change-of-variables* 展开 *model likelihood*：

$$\log p_{\theta}(x) = \log p_Z(f_{\theta}(x)) + \log \left| \det J_{f_{\theta}}(x) \right|.$$

优化器通常最小化 *loss*，所以把最大化目标乘以 -1 ，就得到正文的 $\mathcal{L}(\theta)$ 。这证明该 *loss* 不是额外假设，而是 *exact likelihood* 的直接符号形式。

（*Advance*）为什么 *layer-wise log-det* 重要。

如果每层的 *log determinant* 都容易算，总 *log determinant* 就是逐层求和，这就是 *Flow architecture* 设计的核心：每一层可能很受限，但整体通过 *stacking* 得到复杂变换，同时保持 *likelihood tractable*。

2.5 Jacobian Bottleneck: 为什么 Flow architecture 被强约束 p.10
对 $256 \times 256 \times 3$ 的 *image*，维度约为 200,000。generic network 的 *Jacobian* 会是：

$$200,000 \times 200,000$$

级别的 *dense matrix*。直接计算 *determinant* 在时间和内存上都不可行，而且训练时每个 *batch* 都要重复计算。

（*Advance*）*Why Dense Jacobian Determinant Is Infeasible*。

对 d 维输入，generic dense *Jacobian* 是 $d \times d$ *matrix*。直接 *determinant* 或 *LU decomposition* 的 *typical cost* 是 $O(d^3)$ ，*memory* 至少是 $O(d^2)$ 。

对 $256 \times 256 \times 3$ *image*：

$$d = 256 \cdot 256 \cdot 3 = 196,608.$$

仅存一个 *dense Jacobian* 就需要约 $d^2 \approx 3.9 \times 10^{10}$ 个 *entries*。即使用 *float32*，也超过 150 GB；训练时还要 *batch*、*activations* 和 *gradients*。因此 Flow 必须设计出特殊 *Jacobian structure*，不能依赖 *generic determinant*。

因此 Flow architecture 不能完全自由。常见 *practical ideas*：

- 使用 *triangular structure*，让 *determinant* 变成 *diagonal product*。
- 堆叠很多 simple *invertible layers*，每层快速计算 *log-det*，最后求和。

2.6 Triangular Jacobian: 把 determinant 变成 diagonal sum p.11

如果 J 是 *triangular matrix*，则：

$$\det J = \prod_i J_{ii}, \quad \log \left| \det J \right| = \sum_i \log \left| J_{ii} \right|.$$

（*Advance*）*Proof: Triangular Matrix Determinant*。

对 *triangular matrix*，所有非对角“错误方向”的 *entries* 都为 0。*determinant* 的 *Leibniz formula* 是：

$$\det J = \sum_{\pi \in S_d} \text{sgn}(\pi) \prod_{i=1}^d J_{i,\pi(i)}.$$

如果 J 是 *lower triangular*，那么当某个 $\pi(i) > i$ 时， $J_{i,\pi(i)} = 0$ ，该 *permutation* 的乘积为 0。唯一不会碰到上三角零项的 *permutation* 是 *identity* $\pi(i) = i$ 。因此：

$$\det J = \prod_i J_{ii}.$$

对绝对值取 $\log$ ：

$$\log \left| \det J \right| = \log \prod_i \left| J_{ii} \right| = \sum_i \log \left| J_{ii} \right|.$$

这把高维 *matrix decomposition* 变成 *element-wise summation*。（*Advance*）*Intuition*。

Flow 的很多经典结构都在回答同一个问题：怎么让一个看起来像 *neural network* 的 *mapping* 仍然有 *triangular* 或 *block-triangular Jacobian*？

Coupling layer 的答案是：把变量分成两半，一半不变，另一半由前一半 *condition* 后更新。这样 *Jacobian* 就出现 *block triangular structure*。

2.7 Non-linear Independent Components Estimation (NICE): additive coupling p.12

Non-linear Independent Components Estimation (NICE) 的核心是把变量分成两部分，只 *transform* 其中一半：

$$y_a = x_a, \quad y_b = x_b + m(x_a).$$

inverse 非常简单：

$$x_a = y_a, \quad x_b = y_b - m(y_a).$$

$m(\cdot)$ 可以是复杂 *neural network*，因为它不需要被求逆；真正需要 *invert* 的只是 *additive map*。

（*Advance*）*Term Explanation and Inverse Proof*。

coupling layer 指把 *variables* 分成两组：一组保持不变，用它 *condition* 一个 *network*；另一组被这个 *network* 的输出修改。它的技巧是：*conditioner* $m(\cdot)$ 可以很复杂，但 *inverse* 不需要反解 m 。

对 *NICE*：

$$y_a = x_a, \quad y_b = x_b + m(x_a).$$

第一式直接给出 $x_a = y_a$ 。代入第二式：

$$y_b = x_b + m(y_a).$$

移项得到：

$$x_b = y_b - m(y_a).$$

因此无论 m 是多复杂的 *neural network*，只要 *forward pass* 可计算，整个 *coupling transform* 就可逆。

2.7.1 NICE 的 triangular Jacobian 和 volume-preserving 性质 p.13

NICE 的 *Jacobian* 是 *block triangular*：

$$J = \begin{bmatrix} I & 0 \\ \frac{\partial m(x_a)}{\partial x_a} & I \end{bmatrix}.$$

（*Advance*）*Derivation: NICE Jacobian and Volume Preservation*。

对 $y_a = x_a$ ，有：

$$\frac{\partial y_a}{\partial x_a} = I, \quad \frac{\partial y_a}{\partial x_b} = 0.$$

对 $y_b = x_b + m(x_a)$ ，有：

$$\frac{\partial y_b}{\partial x_b} = I, \quad \frac{\partial y_b}{\partial x_a} = \frac{\partial m(x_a)}{\partial x_a}.$$

把四个 *block* 合起来就得到正文的 J 。这是 *block lower triangular matrix*，*determinant* 等于 *diagonal block determinants* 的乘积：

$$\det J = \det(I) \det(I) = 1.$$

因此 $\log \left| \det J \right| = 0$ 。*volume-preserving* 的结论由 $\left| \det J \right| = 1$ 得到：局部 *volume element* dx 映射后大小不变。

因此：

$$\det J = 1, \quad \log \left| \det J \right| = 0.$$

这意味着 NICE 是 **volume-preserving**: 它可以重新排列 probability mass, 但不能局部压缩或扩张 volume.

(Advance) Pros and Cons.

NICE 的优点是极其简单, training 稳定; 缺点是每层只能 translation, 不能 scaling, 因此 per-layer expressiveness 有限。

实际上需要很多 coupling layers、permutations 和 final scaling transform 来弥补 expressiveness.

2.8 Real-valued Non-Volume Preserving (RealNVP): affine coupling p.15

Real-valued Non-Volume Preserving (RealNVP) 把 additive coupling 扩展成 affine coupling:

$$y_a = x_a, \quad y_b = x_b \odot \exp(s(x_a)) + t(x_a).$$

inverse 是:

$$x_a = y_a, \quad x_b = (y_b - t(y_a)) \odot \exp(-s(y_a)).$$

log-determinant 为:

$$\log |\det J| = \sum_i s(x_a)_i.$$

这里 $\exp(s(x_a))$ 保证 scaling factor 非零, 从而 inverse 存在。

(Advance) Symbol Explanation: $\odot$ and Affine Coupling.

$\odot$ 表示 element-wise multiplication, 也就是逐元素相乘。如果

$$x_b = (x_{b,1}, \dots, x_{b,d}), \quad s(x_a) = (s_1, \dots, s_d),$$

那么:

$$x_b \odot \exp(s(x_a)) = (x_{b,1}e^{s_1}, \dots, x_{b,d}e^{s_d}).$$

RealNVP 的 affine coupling 可以逐元素理解成:

$$y_{b,i} = x_{b,i} \cdot e^{s_i(x_a)} + t_i(x_a).$$

这就是 affine transform: 先 scale, 再 shift. 这里的 scale $e^{s_i(x_a)}$ 和 shift $t_i(x_a)$ 都由另一半变量 x_a 预测出来。因为 x_a 自己保持不变, 所以 inverse 时可以先从 y_a 得到 x_a , 再用同一个 scale/shift 把 y_b 反解回 x_b :

$$x_{b,i} = (y_{b,i} - t_i(y_a))e^{-s_i(y_a)}.$$

直觉上, NICE 只做 $x_b + m(x_a)$ 这种 translation; RealNVP 多了 $\odot \exp(s(x_a))$ 这一步, 所以能对不同维度做 local volume expansion / compression, 因而比 additive coupling 更 expressive.

(Advance) Derivation: RealNVP Inverse and Log-Det.

RealNVP 的 forward transform 是:

$$y_a = x_a, \quad y_b = x_b \odot \exp(s(x_a)) + t(x_a).$$

第一式给出 $x_a = y_a$. 代入第二式:

$$y_b = x_b \odot \exp(s(y_a)) + t(y_a).$$

先减去 shift, 再除以 scale:

$$x_b = (y_b - t(y_a)) \odot \exp(-s(y_a)).$$

Jacobian 仍然是 block triangular, diagonal block 分别是 I 和 $\text{diag}(\exp(s(x_a)))$, 所以:

$$\det J = \det(I) \prod_i \exp(s(x_a)_i) = \exp\left(\sum_i s(x_a)_i\right).$$

取 log 得:

$$\log |\det J| = \sum_i s(x_a)_i.$$

$\exp(\cdot)$ 的作用是确保 scale 始终正且非零, 因此 inverse 不会除以 0。

(Advance) 为什么 RealNVP 比 NICE 更 expressive.

NICE 只能做 translation, 不改变 local volume; RealNVP 可以通过 $s(x_a)$ 局部压缩或扩张 volume, 因此 density 可以被显式调节。

代价是 mask design、scale stabilization 和 layer depth 都变得更重要。

2.8.1 RealNVP Image Architecture: mask、squeeze 和 multi-scale p.16

RealNVP 在 image 上使用几种 structure:

- checkerboard mask: 空间上交替选择 conditioning pixels 和 updated pixels, 让邻域信息互相 condition.

- channel split: 按 channel 分组, 一组预测另一组。

- squeeze: 把 spatial resolution 减半、channel 数变为四倍, 让局部邻域信息更容易在 channel dimension 混合。

- multi-scale: 逐步 factor out 一些 latents, 使信息在不同 scale 进入 latent space.

2.8.2 RealNVP 的优点和 coupling limitation p.17

RealNVP 是 coupling-based Flow 的经典形式。它的优点:

- forward 和 inverse 都快。

- log-det 直接可得。

- 非 volume-preserving, 比 additive coupling 强很多。

限制也很明确:

- 每层只直接更新一部分 variables。

- 很依赖 mask、scale choices 和 depth。

- information mixing 不足会限制 expressiveness.

核心 trade-off 是:

$$\text{restricted single layer} \iff \text{globally tractable likelihood}.$$

(Advance) Why This Trade-off Holds.

“restricted single layer” 指单个 coupling layer 只更新一部分 coordinates, 所以 expressiveness 局部受限; 但正因为另一部分 coordinates 保持 identity, Jacobian 才会是 block triangular.

block triangular 直接带来 tractable log-det:

$$\det \begin{bmatrix} A & 0 \\ C & D \end{bmatrix} = \det(A) \det(D).$$

因此结论是: 单层结构受限换来了全局 likelihood 可计算; 多层 stacking、mask permutation、squeeze 和 multi-scale 用来补回 expressiveness.

2.9 Flow 的关键性质和主要限制 p.19

Flow 作为 generative model 有三个核心优势:

- exact likelihood: training 和 evaluation 使用同一个 objective.

- exact latent inference: 给定 x , forward pass 得到 $z = f_\theta(x)$, 不需要 approximate posterior.

- composable: 很多简单 invertible layers 可以堆成复杂 distribution transform.

它也有几个主要限制: p.20

- structural constraints: 为了 tractable log-det, 不能自由使用 arbitrary architecture.

- dimension pressure: invertibility 要求 matched dimensions, 不能像普通 encoder 那样丢信息。

- visual quality: continuous high-dimensional density 可能给出 high-likelihood but bad-looking samples.

- sampling speed:取决于 structure.coupling flow 很快,但 autoregressive flow 或 iterative components 可能慢。

2.10 Transformer-based Masked Autoregressive Flow (TarFlow): 用 Transformer conditioner 复兴 Flow p.21

早期 Flow 图像质量不够强, 一个重要原因是 conditioner expressiveness 不足。

Transformer-based Masked Autoregressive Flow (TarFlow) 的想法是:

- 把 image 切成 patch sequence.

- 在 patches 上堆叠 autoregressive Transformer blocks.

- 每层仍然满足 triangular Jacobian. 因此 likelihood 仍然 tractable.

- conditioner 从简单网络升级成 Transformer, 能构建更复杂的 long-range image structure.

TarFlow 可以理解为:

$$\text{TarFlow} = \text{Transformer-based Masked Autoregressive Flow}.$$

(Advance) Term Explanation: Masked Autoregressive Flow.

autoregressive 表示第 i 个变量的 transform 只依赖 earlier variables $x_{<i}$.

masked 表示通过 attention mask 或 ordering constraint 强制模型看不到 future positions. 这样 Transformer conditioner 虽然很强, 但不会破坏 triangular Jacobian.

Flow 表示整个 transformation 仍然可逆, 且 likelihood 仍然由 base density 加 log-det 得到。

2.10.1 TarFlow Formulation: patch autoregressive flow p.22

把 image 切成 N 个 patches 后, autoregressive Flow 写作:

$$z_i = \frac{x_i - \mu_i(x_{<i})}{\sigma_i(x_{<i})}, \quad x_i = \mu_i(x_{<i}) + \sigma_i(x_{<i})z_i.$$

(Advance) Derivation: Why TarFlow Has Triangular Jacobian.

对每个 patch i :

$$z_i = \frac{x_i - \mu_i(x_{<i})}{\sigma_i(x_{<i})}.$$

z_i 只依赖 $x_{<i}$, 不依赖任何 x_j with $j > i$. 因此 Jacobian entry $\partial z_i / \partial x_j$ 在 $j > i$ 时为 0, 整个 Jacobian 是 triangular.

对 diagonal entry:

$$\frac{\partial z_i}{\partial x_i} = \frac{1}{\sigma_i(x_{<i})},$$

因为 μ_i 和 σ_i 只依赖 $x_{<i}$, 对 x_i 是 constants. 于是 forward encoding $x \mapsto z$ 的 log-det 是:

$$\log \left| \det \frac{\partial z}{\partial x} \right| = \sum_i \log \frac{1}{\sigma_i} = - \sum_i \log \sigma_i.$$

如果看 reverse generation $z \mapsto x$, diagonal entry 是 σ_i , 所以 reverse log-det 是 $\sum_i \log \sigma_i$. slides 中的 scale-term sum 取决于讨论 forward 还是 inverse direction; 两者只差一个符号。

==** 其中 μ_i 和 σ_i 由 Transformer 根据 context $x_{<i}$ 预测.**== 因为 position i 只依赖 earlier positions, Jacobian 仍然 triangular, log-det 是 scale terms 的和:

$$\log |\det J| = \sum_i \log \sigma_i.$$

TarFlow 还会 across layers 交替 autoregressive directions. 如果所有层都用同一个 order, information flow 会受到 directional bottleneck; 交替方向可以让不同位置经过若干层后互相影响。

2.10.2 TarFlow Trick 1 & 2: noise augmentation 和 post-training denoising p.23

TarFlow 在训练输入上加入 Gaussian noise:

$$\epsilon \sim \mathcal{N}(0, \sigma^2 I).$$

ImageNet 64 × 64 使用 $\sigma = 0.05$, 比 uniform dequantization noise 大约一个数量级。动机是: 如果没有 noise, $f^{-1}(z)$ 只在离散训练点附近见过数据; 但 sampling 时 latent Gaussian 会映射到连续空间, 容易形成 out-of-distribution (OOD) generalization 问题。

(Advance) New Terms: Dequantization, OOD, Score.

- uniform dequantization: 把离散 pixel value 加上很小的 uniform noise, 使离散图像可以被 continuous density model 处理。它通常只是解决 discrete-to-continuous mismatch, 不会显著扩展 data support.

- out-of-distribution (OOD): 测试/采样时遇到的输入落在训练 distribution 支持之外或很少见的区域。TarFlow 这里的 OOD 指 inverse flow 在连续 Gaussian samples 上生成时, 可能访问训练中没有覆盖的 continuous input neighborhoods.

- score: density 的 log-gradient, $s(y) = \nabla_y \log p(y)$. 它指向 density 上升最快的方向, 所以可用于 denoising.

因为模型训练在 noisy distribution 上, raw samples 也会带 noise. TarFlow 使用 Tweedie's formula 做 score-based denoising:

$$x = y + \sigma^2 \nabla_y \log p_{\text{model}}(y).$$

(Advance) Derivation: Tweedie's Formula.

假设 clean data 是 x , noisy observation 是:

$$y = x + \epsilon, \quad \epsilon \sim \mathcal{N}(0, \sigma^2 I).$$

noisy density 是:

$$p(y) = \int p(x)p(y \mid x)dx.$$

对 y 求梯度:

$$\nabla_y p(y) = \int p(x)\nabla_y p(y \mid x)dx.$$

Gaussian likelihood 满足:

$$\nabla_y \log p(y \mid x) = -\frac{y-x}{\sigma^2} = \frac{x-y}{\sigma^2}.$$

因此:

$$\nabla_y p(y) = \int p(x)p(y \mid x)\frac{x-y}{\sigma^2}dx.$$

两边除以 $p(y)$:

$$\nabla_y \log p(y) = \frac{1}{\sigma^2} (\mathbb{E}[x \mid y] - y).$$

移项得到 posterior mean:

$$\mathbb{E}[x \mid y] = y + \sigma^2 \nabla_y \log p(y).$$

TarFlow 用 model score $\nabla_y \log p_{\text{model}}(y)$ 近似 true noisy score, 于是得到正文中的 denoising update.

(Advance) How the Post-Process Is Computed in Practice.

“不需要额外训练 network”的意思是: 训练完 TarFlow 后, 不再训练一个单独的 denoiser. denoising direction 直接来自 Flow model 的 own likelihood gradient.

具体步骤:

- 先从 base distribution 采样:

$$z \sim \mathcal{N}(0, I).$$

- 用 Flow 的 inverse generation 得到 raw sample:

$$y = f_\theta^{-1}(z).$$

因为模型训练时加入了 Gaussian noise, 所以这个 y 往往是 noisy sample.

- 把 y 当作需要求梯度的 input, 再走一次 forward encoding:

$$z_y = f_\theta(y).$$

同时计算 Flow likelihood:

$$\log p_{\text{model}}(y) = \log p_Z(f_\theta(y)) + \log |\det J_{f_\theta}(y)|.$$

- 用 backprop 对 input y 求 gradient:

$$s_\theta(y) = \nabla_y \log p_{\text{model}}(y).$$

- 用 Tweedie's formula 做一次 correction:

$$\hat{x} = y + \sigma^2 s_\theta(y).$$

所谓“大约两次 forward pass”的意思是：一次 reverse pass 生成 raw sample y ，再一次 forward likelihood pass 加一次 backward 求 input gradient. backward 的计算量通常和一次 forward 同量级，所以整体成本大约是少数几次 model evaluation，而不是像 diffusion 那样需要几十到上百步 denoising network calls. 这个 post-process 只需要大约两次 forward pass，不需要额外训练 network. (Advance) Tweedie’s Formula 的直觉.

对 noisy observation $y = x + \epsilon$ ，posterior mean 可以写成当前 noisy point 加上 score direction 的修正。score $\nabla_y \log p(y)$ 指向 noisy distribution 的高密度区域，因此 $y + \sigma^2 \nabla_y \log p(y)$ 会把样本往更干净的数据 manifold 推。

这里的关键是：noise augmentation 和 denoising 没有改变 TarFlow 作为 Flow 的身份：likelihood 仍然通过 change-of-variables 计算。

2.10.3 TarFlow Trick 3: conditional 和 unconditional guidance p.24

TarFlow 可以使用 Classifier-Free Guidance (CFG)。训练时随机丢掉 class labels，例如以 10% 概率使用 null condition；推理时对每个 flow block 的 prediction 做 extrapolation：

$$\tilde{\mu}_t^i = (1 + w) \mu_t^i(c) - w \mu_t^i(\text{null}),$$

$$\tilde{\alpha}_t^i = (1 + w) \alpha_t^i(c) - w \alpha_t^i(\text{null}).$$

$w > 0$ 会把 conditional prediction 推离 unconditional prediction，提高 class alignment 和 sample sharpness. (Advance) Conditional Input and Class Embedding. conditional TarFlow 生成时有两类输入：sampled latent z 和 condition c 。训练时大多数样本给 class label c ，少数样本把 c 替换成 null，让同一个模型同时学 conditional prediction 和 unconditional prediction。

在 ImageNet class-conditional generation 中， c 通常不是自然语言词 embedding，而是 class id 经过 learnable embedding table 得到的 class embedding：

$$e_c = \text{Embed}(c).$$

如果是 text-to-image，condition 才通常来自 text encoder 的 token / sentence embeddings。这里 slide 讨论 class labels，所以可以把 c 理解为 class label embedding；null 也对应一个特殊 null embedding。

TarFlow 还提出 unconditional guidance：没有 label 可 drop 时，用 attention temperature τ 构造一个 inferior-quality baseline。将 attention logits 除以 τ 再 softmax， $\tau \neq 1$ 会让 attention 过 smooth 或过 sharp，然后使用：

$$\tilde{\mu}_t^i = (1 + w_i) \mu_t^i(1) - w_i \mu_t^i(\tau), \quad w_i = \frac{i}{T-1} \cdot w.$$

(Advance) Term Explanation and Why Guidance Is Extrapolation. Classifier-Free Guidance (CFG) 的训练阶段让同一个 model 同时学 conditional prediction 和 unconditional prediction：有时输入 class condition c ，有时把 condition 替换成 null。推理时使用：

$$\tilde{\mu} = (1 + w) \mu(c) - w \mu(\text{null}) = \mu(c) + w(\mu(c) - \mu(\text{null})).$$

第二种写法说明它是在 conditional direction 上做 extrapolation：从 $\mu(\text{null})$ 指向 $\mu(c)$ 的方向继续走 w 倍。若 conditional prediction 代表“符合类别”的方向，extrapolation 会增强 class alignment。

attention temperature τ 是 softmax 前 logits 的缩放：

$$\text{softmax}(a/\tau).$$

$\tau > 1$ 让 distribution 更 smooth， $\tau < 1$ 让 distribution 更 sharp. TarFlow 的 unconditional guidance 把 $\tau \neq 1$ 的 prediction 当作 inferior-quality baseline，再做同样的 extrapolation。

2.11 Flow Design Checklist 和 Part I 总结 p.26

理解一个 Flow model 时，可以问五个问题：

- mapping 是否 strictly invertible?
 - log-det 是否能 efficient compute?
 - conditioner 是否足够强，能捕捉 long-range image structure?
 - reverse sampling 是否真的高效，还是隐藏了 autoregressive / iterative cost?
 - MLE training 得到的 samples 是否符合 human preference?
- NICE $\rightarrow$ RealNVP $\rightarrow$ TarFlow 的主线是：

invertibility feasible $\rightarrow$ tractable and expressive $\rightarrow$ modern high-quality generation.

(Advance) Part I Conclusion.

Flow 的价值没有过时，只是被重新组合了。早期瓶颈来自 structural constraints；现代方向则通过 Transformer conditioner、noise augmentation、denoising 和 guidance 把 Flow theory 接回 contemporary image generation practice. p.27

3 Generative Adversarial Network (GAN): 从 density modeling 到 adversarial game p.29

Generative Adversarial Network (GAN) 从一个与 Flow 很不同的前提出发：

- 不显式写下 data density $p(x)$ 。
 - generator 学习从 noise 到 image 的 mapping。
 - discriminator 持续学习区分 real 和 fake。
- GAN 的强项是 one-step sampling 和 strong visual quality：代价是 training 变成 non-convex, non-concave two-player game，stability、mode coverage 和 evaluation 都更复杂。

(Advance) New Terms.

- implicit generative model: 只定义 sampler $x = G(z)$ ，不直接给出可计算的 $P\theta(x)$ 。
- discriminator: 二分类器，估计输入 image 是 real 还是 fake。
- adversarial game: generator 和 discriminator 的目标相反：一个试图骗过对方，一个试图区分真假。
- mode coverage: generated distribution 是否覆盖真实数据的所有主要 modes，而不是只生成少数看起来很好的类型。
- non-convex, non-concave: generator 参数上不是 convex minimization, discriminator 参数上也不是 concave maximization，因此没有简单的全局优化保证。

3.1 Implicit Generative Model 的动机 p.30

“high probability”和“looking realistic”不总是同一件事。Maximum likelihood model 往往倾向 cover all modes，因此可能产生 over-averaged 或不够 sharp 的结果；pixel-level likelihood 也不完全等于 perceptual quality。GAN 的 solution 是让 discriminator 学会 real 和 generated images 之间的 gap。generator 不直接最小化某个固定 distance，而是接受一个动态变化的 adversarial signal：

better discriminator $\Rightarrow$ generator forced to fix more artifacts.

(Advance) Why the Conclusion Holds.

discriminator 的训练目标是把真实样本判为 real，把生成样本判为 fake。若 discriminator 学会某类 artifact，例如 checkerboard pattern、blur 或不自然 texture，则 fake sample 在该方向上的 score 会降低。

generator 的梯度来自 discriminator score 对 image 的梯度：

$$\nabla_{\theta_G} D(G\theta(z)) = \nabla_x D(x)|_{x=G\theta(z)} \nabla_{\theta_G} G\theta(z).$$

因此 discriminator 越能指出 artifact，generator 越能收到“沿哪个 image-space direction 改”的信号。这就是 adversarial signal 改善 perceptual artifacts 的原因。

3.2 Generator 和 Discriminator 的基本框架 p.31

Generator $G(z; \theta_G)$ ：

- input: random noise $z \sim p(z)$ 。
- output: synthetic image $x = G(z)$ 。
- goal: 产生 discriminator 无法区分的 samples。

Discriminator $D(x; \theta_D)$ ：

- input: real 或 generated image。
- output: $D(x) \in [0, 1]$ ，表示 being real 的概率。
- goal: 最大化 real/fake classification accuracy。

GAN 没有 explicit likelihoods：它通过 comparing samples 学习。

3.3 Original GAN Objective: minimax game p.32

original GAN minimax objective 是：

$$\min_G \max_D \mathbb{E}_{x \sim p_{\text{data}}} [\log D(x)] + \mathbb{E}_{z \sim p(z)} [\log(1 - D(G(z)))].$$

(Advance) Derivation: Original GAN Objective from Binary Classification. 对 discriminator 来说，real images 的 label 是 1，generated images 的 label 是 0。binary cross-entropy log-likelihood 是：

$$\ell(D; x, y) = y \log D(x) + (1 - y) \log(1 - D(x)).$$

real sample $x \sim p_{\text{data}}$ 时 $y = 1$ ，得到 $\log D(x)$ ；fake sample $G(z)$ 时 $y = 0$ ，得到 $\log(1 - D(G(z)))$ 。

对两类样本分别取 expectation，discriminator 要 maximize：

$$\mathbb{E}_{x \sim p_{\text{data}}} \log D(x) + \mathbb{E}_{z \sim p(z)} \log(1 - D(G(z))).$$

generator 的目标相反：让 fake 更像 real，使第二项变小。因此得到 $\min_G \max_D$ 的 minimax game。

discriminator 想同时提高 real term 和 fake term：generator 想降低 fake 被识别为 fake 的概率。实践中常用 non-saturating generator loss：

$$\max_G \mathbb{E}_z [\log D(G(z))]$$

或等价地最小化：

$$\mathcal{L}_{G,\text{ns}} = -\mathbb{E}_z [\log D(G(z))].$$

这样可以避免 early training 中 discriminator 过强导致的 gradient saturation。

(Advance) Why Non-Saturating Loss Has the Same Fixed Point. original generator minimization 会降低 $\log(1 - D(G(z)))$ ，non-saturating version 会提高 $\log D(G(z))$ 。二者的 pointwise optimum 都要求 generated samples 被 discriminator 判为 real。

当 $P_G = P_{\text{data}}$ 时，optimal discriminator 是 $D^*(x) = 1/2$ ，两个 generator objectives 都没有继续改变 distribution 的方向。因此 fixed point 相同。

差别在 gradient magnitude：original loss 在 $D(G(z)) \approx 0$ 时梯度接近 0；non-saturating loss 在同一位置仍有强梯度。这是 optimization trick，不是改变目标 equilibrium。

3.3.1 Gradient Saturation: 为什么 $\log(1 - D)$ 早期会消失 p.33

original generator loss：

$$L = \log(1 - D(G(z))).$$

令：

$$y = D(G(z)) = \sigma(f),$$

其中 f 是 discriminator logit。chain rule 给出：

$$\frac{\partial L}{\partial f} = \frac{-1}{1 - y} \cdot y(1 - y) = -y.$$

early training 时 strong discriminator 使 $D(G(z)) \approx 0$ ，也就是 $y \approx 0$ ，因此 gradient ≈ 0 。non-saturating loss：

$$L_{\text{ns}} = -\log D(G(z))$$

有：

$$\frac{\partial L_{\text{ns}}}{\partial f} = y - 1.$$

当 $y \approx 0$ 时，gradient ≈ -1 ，仍然提供强信号。

(Advance) Term Explanation: Saturation.

saturation 指 sigmoid output 已经非常接近 0 或 1，再改变 logit f 对 output 的影响很小。数学上 sigmoid derivative 是：

$$\frac{d\sigma(f)}{df} = \sigma(f)(1 - \sigma(f)) = y(1 - y).$$

当 $y \approx 0$ 或 $y \approx 1$ 时，这个 derivative 接近 0。original GAN 的 generator loss 在 strong discriminator 下正好落入 $y \approx 0$ 的区域，所以 early gradient 很弱。

3.4 Optimal Discriminator 和 Jensen-Shannon Divergence (JSD) p.34

给定 fixed G ，optimal discriminator 是：

$$D^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_G(x)}.$$

(Advance) Derivation: Optimal Discriminator.

固定 generator 后， p_G 是常量分布。discriminator objective 可以写成对每个 x 独立最大化：

$$V(D) = \int p_{\text{data}}(x) \log D(x) + p_G(x) \log(1 - D(x)) dx.$$

对某个固定 x ，令 $a = p_{\text{data}}(x)$ ， $b = p_G(x)$ ，最大化：

$$a \log D + b \log(1 - D).$$

求导：

$$\frac{a}{D} - \frac{b}{1 - D} = 0.$$

移项：

$$a(1 - D) = bD \implies a = (a + b)D.$$

因此：

$$D^*(x) = \frac{a}{a + b} = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_G(x)}.$$

它本质上是在 equal priors 下估计 real 和 generated distributions 的 density ratio。把 $D^*(x)$ 代回 generator objective，可得：

$$C(G) = -\log 4 + 2 \cdot JSD(p_{\text{data}} \| p_G),$$

其中 Jensen-Shannon Divergence (JSD) 定义为：

$$JSD(P \| Q) = \frac{1}{2} D_{KL}(P \| M) + \frac{1}{2} D_{KL}(Q \| M), \quad M = \frac{P + Q}{2}.$$

(Advance) Derivation: Substituting D^* Gives JSD.

把 D^* 代回 objective：

$$C(G) = \int p_{\text{data}} \log \frac{p_{\text{data}}}{p_{\text{data}} + p_G} dx + \int p_G \log \frac{p_G}{p_{\text{data}} + p_G} dx.$$

令：

$$M = \frac{p_{\text{data}} + p_G}{2}.$$

则 $p_{\text{data}} + p_G = 2M$ 。第一项变为：

$$\int p_{\text{data}} \log \frac{p_{\text{data}}}{2M} dx = D_{KL}(p_{\text{data}} \| M) - \log 2.$$

第二项同理：

$$\int p_G \log \frac{p_G}{2M} dx = D_{KL}(p_G \| M) - \log 2.$$

相加：

$$C(G) = D_{KL}(p_{\text{data}} \| M) + D_{KL}(p_G \| M) - 2 \log 2.$$

由 $JSD(P \| Q) = \frac{1}{2} D_{KL}(P \| M) + \frac{1}{2} D_{KL}(Q \| M)$ ，可得：

$$C(G) = -\log 4 + 2JSD(p_{\text{data}} \| p_G).$$

理想情况下：

$$p_G = p_{\text{data}} \implies D(x) = \frac{1}{2}$$

everywhere, discriminator 无法区分。

(Advance) Why the Ideal Discriminator Is 1/2.

若 $p_G = p_{data}$, 则:

$$D^*(x) = \frac{p_{data}(x)}{p_{data}(x) + p_G(x)} = \frac{p_{data}(x)}{2p_{data}(x)} = \frac{1}{2}.$$

也就是说, 给定 *image* 后 *posterior probability of real and fake* 相等, *discriminator* 最优也只能随机猜。

3.5 Non-Overlapping Support 和 unstable gradients p.35

高维 image distributions 往往集中在复杂 low-dimensional manifolds 上。训练早期, real images 和 generated images 可能处于几乎不重叠的区域, 因此 discriminator 很容易把它们分开。

generator 需要的不是“你是假的”这个 label. 而是“往哪个方向移动可以更像 real data”的 gradient. 当 discriminator 过强时, generator signal 可能弱化, 表现为:

- oscillation.
- collapse.
- sample diversity 不足.
- discriminator-generator dynamic balance 难以维持.

(Advance) Why Non-Overlapping Support Breaks JSD Gradients.

假设 p_{data} 和 p_G 的 supports 不重叠, 则在 real support 上 $p_G = 0$, 在 fake support 上 $p_{data} = 0$.

对 *optimal discriminator*:

$$D^*(x) = \begin{cases} 1, & x \in \text{supp}(p_{data}), \\ 0, & x \in \text{supp}(p_G). \end{cases}$$

对 fake samples, $D^*(G(z)) = 0$, generator loss 中的 $\log(1 - D^*(G(z))) = \log 1 = 0$ 成为 flat region. discriminator 已经完美区分真假, 但这个二分类结果没有告诉 generator 应该往哪个方向移动 support 才能接近 real manifold。

3.6 Wasserstein Generative Adversarial Network (WGAN): Earth-Mover Intuition p.36

Wasserstein Generative Adversarial Network (WGAN) 用 Wasserstein-1 distance, 也叫 Earth Mover’s distance, 替代 JSD:

$$W(p_{data}, p_G) = \inf_{\gamma \in \Pi} \mathbb{E}_{(x,y) \sim \gamma} [\|x - y\|].$$

(Advance) Term Explanation: Coupling and Earth Mover Distance.

Π 表示所有 couplings 的集合。一个 coupling $\gamma(x, y)$ 是定义在 pair (x, y) 上的 joint distribution. 要求它的 marginals 分别是 p_{data} 和 p_G 。

直觉上, $\gamma(x, y)$ 表示“把 generated mass at y 搬到 real location x 的多少”。transport cost 是 $\|x - y\|$, 所以 expectation 是平均搬运距离。对所有合法搬运方案取 infimum, 就得到最小搬运成本。

直觉是: 把 generated distribution 看成一堆土, 把 real distribution 看成目标形状, W_1 测量把土搬成目标形状的最小 transport cost. p.37 即使两个 distributions 完全不重叠, Wasserstein distance 仍然会随位置平滑变化。discriminator 在 WGAN 中被称为 critic, 输出 real-valued score, 而不是 probability。

3.6.1 为什么 Wasserstein 在 non-overlap 时有连续梯度 p.38

ordinary GAN discriminator 如果能 perfect separate real and fake, 在 gap 中会 saturate, generator 收到 vanishing gradients. WGAN critic 输出 real-valued score, 可以在 non-overlap 时仍然给出方向性变化:

$$\text{non-overlap support} \not\Rightarrow \text{zero useful gradient.}$$

(Advance) Simple Proof: Wasserstein Changes Under Translation.

用一维 toy case 看 non-overlap. 令真实分布是 point mass δ_0 , 生成分布是 δ_θ . 两者 support 只要 $\theta \neq 0$ 就不重叠。

Jensen-Shannon Divergence (JSD) 在所有 $\theta \neq 0$ 时都是常数 $\log 2$, 所以对 θ 的梯度为 0。

Wasserstein-1 distance 是:

$$W_1(\delta_0, \delta_\theta) = |\theta|.$$

当 $\theta > 0$ 时:

$$\frac{d}{d\theta} W_1(\delta_0, \delta_\theta) = 1,$$

当 $\theta < 0$ 时梯度为 -1 . 因此即使 support 不重叠, Wasserstein distance 仍然告诉 generator 应该往 0 的方向移动。

3.6.2 Kantorovich-Rubinstein (KR) Duality 和 1-Lipschitz Critic p.39

Kantorovich-Rubinstein (KR) duality 给出:

$$W_1(p_{data}, p_G) = \sup_{\|f\|_L \leq 1} \mathbb{E}_{x \sim p_{data}} [f(x)] - \mathbb{E}_{x \sim p_G} [f(x)].$$

critic f 必须满足 1-Lipschitz constraint. 它试图增大 real 和 generated samples 的 average score gap: generator 则试图缩小这个 gap. (Advance) Term Explanation: KR Duality and 1-Lipschitz.

Kantorovich-Rubinstein (KR) duality 是 optimal transport 的一个 theorem: primal form 是“找最便宜的搬运计划 γ^* ”, dual form 是“找一个满足 Lipschitz 约束的 potential function f 来最大化 real/fake score gap”。

1-Lipschitz 表示:

$$|f(x) - f(y)| \leq \|x - y\|$$

对所有 x, y 都成立。这个约束防止 critic 用任意陡峭的 score 差把 objective 放大到无意义。

WGAN 的 critic 不是 probability classifier, 因为它不输出 $[0, 1]$ 的 real probability: 它输出的是 transport potential 的 score. generator 最小化:

$$-\mathbb{E}_{x \sim p_G} [f(x)]$$

等价于提高 fake samples 的 critic score, 从而缩小 dual objective 中 real/fake 的 score gap。

(Advance) WGAN 的代价。

WGAN 的 objective 更有意义, 但 1-Lipschitz constraint 必须小心 enforce. 若 constraint 失败, 优化的就不再是 Wasserstein distance. 早期 WGAN 使用 weight clipping, 后来更常见的是 gradient penalty 或 spectral normalization 等方法。

3.7 Mode Collapse: 高质量不等于高覆盖 p.40

mode collapse 指 generator 只覆盖 real distribution 的一部分 modes:

- generator 发现某些 modes 暂时最容易得到 high score.
 - 大量 noise vectors 被映射到少数 modes.
 - discriminator-generator chase 可能造成 mode hopping.
- 危险之处在于 sample grids 可能看起来很漂亮, 但高度重复: Fréchet Inception Distance (FID) 也可能掩盖 coverage 不足. 因此要同时看 fidelity 和 diversity. (Advance) New Terms: Fidelity, Diversity, FID.

- fidelity: 单张或局部 samples 是否真实、清晰、没有 artifact.
 - diversity: 生成分布是否覆盖多种类别、姿态、背景和风格.
 - Fréchet Inception Distance (FID): 把真实图像和生成图像都送入 Inception feature space, 用两个 Gaussian feature distributions 的 Fréchet distance 估计分布差异. FID 同时受 quality 和 diversity 影响, 但它是 aggregate metric, 可能掩盖某些 mode missing.
- (Advance) Why Nice Samples Do Not Prove Good Coverage.

mode 可以理解对 data distribution 中一个高概率区域, 例如某类姿态、背景、类别或风格。mode collapse 是 many-to-one mapping:

$$z_1, z_2, \dots, z_m \mapsto x_{\text{same mode.}}$$

sample grid 只展示有限数量图片; 如果 collapsed mode 本身视觉质量很高, grid 会显得漂亮。coverage 需要检查 generated distribution 是否覆盖真实分布的不同 modes, 因此必须结合 diversity metrics, nearest-neighbor inspection 或 mode counting task.

3.8 Relativistic Pairing Generative Adversarial Network (RpGAN): pairing real and fake p.41

Relativistic Pairing Generative Adversarial Network (RpGAN) 把 real 和 fake samples 在 loss 中配对:

$$\mathbb{E}_{z,x} [f(D(G(z)) - D(x))].$$

classic GAN, 也就是 separated GAN (SepGAN), 把 real 和 fake 独立判断: 一个 global decision boundary 就可能让 generator 把 fake collapse 到刚好越过 boundary 的少数 modes。

RpGAN 则让 fake 相对 random real sample 被评价。直觉是: 每个 fake 都要和真实样本比较, collapse 到单一 mode 时仍然会输给其他 real modes.

(Advance) Why Pairing Changes the Landscape.

在 SepGAN 中, discriminator 可以学习一个 global boundary, 若 generator 找到少数 fake locations 可以越过这个 boundary, 就可能把大量 z 映射到这些位置。

RpGAN 的 loss 依赖 difference:

$$D(G(z)) - D(x).$$

这意味着 fake sample 的质量不是绝对判断, 而是相对当前 random real sample 判断。若 generator collapse 到一个 mode, 当 paired real sample 来自其他 mode 时, fake 仍然需要在 score difference 上胜过它。于是 collapse 到单一 mode 无法同时应对所有 real modes.

(Advance) Landscape 观点。

slide 中提到 Sun et al. 证明 SepGAN 有 $(n^n - n!)$ 个 sub-optimal strict local minima, 每个对应 mode collapse; 而 RpGAN 没有 bad strict local minima.

这说明 mode collapse 不只是“训练技巧不够好”, 也和 loss landscape 的几何结构有关。

3.9 R3GAN: 用 gradient penalty 稳定 RpGAN p.42

RpGAN 改善了 landscape, 但训练 dynamics 仍然可能 diverge. R3GAN 在 RpGAN 上加入 gradient penalties:

$$\mathcal{L}(\theta, \psi) = \mathbb{E}_{z,x} \left[f(D_\psi(G_\theta(z)) - D_\psi(x)) \right] + R_1(\psi) + R_2(\theta, \psi),$$

其中:

$$R_1 = \frac{\gamma}{2} \mathbb{E}_{x \sim p_D} \left[\|\nabla_x D_\psi\|^2 \right], \quad R_2 = \frac{\gamma}{2} \mathbb{E}_{x \sim p_\theta} \left[\|\nabla_x D_\psi\|^2 \right].$$

R_1 penalizes discriminator gradient on real data, R_2 penalizes discriminator gradient on fake data. 两者都需要:

- RpGAN alone: training diverges.
- RpGAN + R_1 alone: fake side gradient 仍可能 explode.
- RpGAN + $R_1 + R_2$: stable training.

R3GAN 在 StackedMNIST 上达到 1000/1000 mode coverage, 而 classic GAN + $R_1 + R_2$ 只有 693/1000. p.43

(Advance) Derivation: What R_1 and R_2 Penalize.

R_1 和 R_2 都是 discriminator score 对 input image 的 gradient norm penalty:

$$\|\nabla_x D_\psi(x)\|^2.$$

R_1 在 real samples $x \sim p_D$ 上取 expectation, R_2 在 fake samples $x \sim p_\theta$ 上取 expectation. 它们的效果是让 discriminator 在 data manifold 和 generator manifold 附近都不要过于陡峭。

为什么这有助于 stability? generator update 包含:

$$\nabla_\theta D_\psi(G_\theta(z)) = \nabla_x D_\psi(x)|_{x=G_\theta(z)} \nabla_\theta G_\theta(z).$$

如果 fake side 的 $\nabla_x D_\psi$ 爆炸, generator 会收到巨大且不稳定的 update. 因此只惩罚 real side 的 R_1 不够; R_2 直接控制 fake samples 附近的 gradient.

在 equilibrium, real 和 generated distributions match, optimal discriminator 不需要在局部改变 score, 因此 $\nabla_x D_\psi = 0$ 是稳定目标。gradient penalty 把训练 dynamics 拉向这个平滑 equilibrium.

(Advance) Modernization.

R3GAN 的方向是把 GAN 从“调参艺术”推进到 principled loss + regularization + modern architecture. slide 中强调: 当 loss 更 principled 后, 可以去掉 minibatch size, path length regularization, style injection、noise 等 ad-hoc tricks, 并用现代 backbone 替换旧结构。

3.10 Adversarial Diffusion Distillation (ADD): GAN idea 迁移到 diffusion distillation p.45

Adversarial Diffusion Distillation (ADD) 的核心是: 用 adversarial loss 帮助一个 few-step student model 从 slow diffusion teacher 中蒸馏。

$$\mathcal{L}_{\text{student}} = \mathcal{L}_{\text{distill}} + \lambda \mathcal{L}_{\text{adv}}.$$

其中:

- $\mathcal{L}_{\text{distill}}$ 来自 diffusion teacher, 保持 semantic consistency 和 distribution direction.

- $\mathcal{L}_{\text{adv}}$ 来自 discriminator, 把 samples 推向 real-image manifold, 加强 sharpness 和 perceptual realism.

这说明 GAN 的思想不一定只作为 standalone generative model 存在: adversarial loss 可以作为 perceptual regularizer 嵌入其他 framework.

(Advance) Why the Student Loss Has Two Terms.

$\mathcal{L}_{\text{distill}}$ 约束 student 不偏离 teacher. 它提供 semantic / distribution direction: 同一个 prompt 或 condition 下, student 应该接近 diffusion teacher 的输出或 denoising trajectory.

$\mathcal{L}_{\text{adv}}$ 约束 student 的 output 落在 real-image manifold 附近。它补的是 distillation loss 容易弱化的 high-frequency realism.

加权和:

$$\mathcal{L}_{\text{student}} = \mathcal{L}_{\text{distill}} + \lambda \mathcal{L}_{\text{adv}}$$

是 multi-objective optimization 的 scalarization. λ 控制 fidelity-to-teacher 和 adversarial realism 的 trade-off: λ 太小, student 可能 blurry; λ 太大, 可能牺牲 semantic consistency.

3.11 GAN-Based Methods 的优点、限制和 Part II 总结 p.47

GAN-based methods 的优点:

- sampling 快, 通常 one forward pass.
- visual quality 和 high-frequency detail 强.
- adversarial loss 可以作为其他 generative frameworks 的 perceptual constraint.
- 适合 low-latency 和 real-time scenarios.

限制:

- training game unstable.
- mode collapse.
- evaluation 复杂, 需要同时看 quality 和 coverage.
- 强依赖 loss, regularization 和 architecture design.

(Advance) Part II Conclusion.

GAN 没有消失, 而是变成了 universal adversarial signal. p.48

原始 GAN 建立 adversarial learning paradigm; WGAN 改进 distribution distance; R3GAN 说明现代 loss 和 architecture 仍能推进 GAN; ADD 则说明 adversarial loss 可以服务 few-step generation 和 distillation.

4 Autoregressive Model (AR): 把图像生成写成 conditional prediction p.50

Autoregressive Model (AR) 使用 chain rule 把 joint image distribution 分解成一串 conditional distributions:

$$p(x) = \prod_i p(x_i \mid x_{<i}).$$

(Advance) Derivation: Chain Rule Factorization.

对 N 个 random variables $x_1, \dots, x_N$, 概率链式法则给出:

$$p(x_1, \dots, x_N) = p(x_1)p(x_2, \dots, x_N \mid x_1).$$

对第二项继续展开:

$$p(x_2, \dots, x_N \mid x_1) = p(x_2 \mid x_1)p(x_3, \dots, x_N \mid x_1, x_2).$$

递归下去得到:

$$p(x_1, \dots, x_N) = \prod_{i=1}^N p(x_i \mid x_1, \dots, x_{i-1}) = \prod_i p(x_i \mid x_{<i}).$$

这不是模型假设, 而是任意 joint distribution 都满足的概率恒等式。AR model 的建模选择在于如何 parameterize 每个 conditional, 以及选择什么 ordering.

x_i 可以是 pixels, discrete tokens, 也可以是 continuous soft tokens. AR 的优点是 objective 清楚、training 稳定、天然适配 next-token Transformer paradigm: 核心 tension 是 image 并不天然是一维序列。

4.1 AR Factorization: chain decomposition 和 ordering p.51

AR decomposition 可以写成:

$$p(x) = p(x_1)p(x_2 \mid x_1) \cdots p(x_N \mid x_{<N}).$$

ordering 是 modeling choice:

- 可以用 raster scan: left-to-right, top-to-bottom.
- 可以用其他 permutations.
- fixed order let causal structure 简单。

但 image 的二维 semantic structure 不一定遵守一维顺序, bottom-right pixel 并不比 top-left pixel 在语义上更“晚”。这也是后面 flexible-order methods 想打破的瓶颈。

(Advance) Term Explanation: Ordering as a Modeling Choice.

ordering 是把二维 image elements 排成一维 sequence 的规则。chain rule 对任何 ordering 都成立: 例如 raster order 和 random permutation 都可以写成:

$$p(x) = \prod_i p(x_i \mid x_{\pi< i}).$$

但 neural network 的 inductive bias 和 inference cost 会受 ordering 强烈影响, raster scan 简单, 但它把 spatially symmetric 的图像强行变成“左上先、右下后”的时间过程。

4.2 Recurrent Image Density Estimator (RIDE):spatial LSTM for pixel generation p.52

Recurrent Image Density Estimator (RIDE) 是一个基于 spatial Long Short-Term Memory (LSTM) 的 pure recurrent image density estimator.

设计:

- 把 image 按 raster order 展开成二维 sequence.
- 使用 multi-dimensional LSTM units 在空间上传播 context.
- hidden states 从 already-generated region 向 current pixel 传递信息。
- 每个 pixel 输出 conditional Gaussian scale mixtures.

抽象形式:

$$h_{ij} = SLSTM(x_{<(ij)}), \quad p(x_{ij} \mid x_{<(ij)}) = p_{\theta}(x_{ij} \mid h_{ij}).$$

RIDE 的意义是证明 pixel-level maximum-likelihood modeling 对 natural images 是可行的, 并且 spatial recurrence 可以捕捉 long-range correlations. (Advance) New Terms and Why the Formula Is AR.

- spatial Long Short-Term Memory (LSTM): 把 LSTM hidden state 从一维 time extension 到二维 grid, 让当前 pixel 接收来自上方、左方或已生成区域的 context.
- conditional Gaussian scale mixture: 用多个 Gaussian-like components 表示一个 pixel 的 conditional density, 比单 Gaussian 能表达更复杂的 brightness/color uncertainty.

公式

$$h_{ij} = SLSTM(x_{<(ij)}), \quad p(x_{ij} \mid x_{<(ij)}) = p_{\theta}(x_{ij} \mid h_{ij})$$

说明 h_{ij} 是 already-generated pixels 的 deterministic summary. 只要 h_{ij} 不依赖 future pixels, 就满足 AR factorization 中的 conditional form.

4.3 Teacher Forcing 的优势被 recurrence 阻塞 p.53

AR 有一个重要训练优势: teacher forcing. 训练时完整 ground-truth image 已知, 因此原则上所有 positions 的 loss 都可以并行计算。

但 RIDE 的 recurrent structure 使 forward pass 仍然 serial:

- inference: 未来 pixels 未知, 必须逐 pixel sample.
- training: 虽然 ground-truth prefix 已知, 但 hidden state h_i 依赖 h_{i-1} .
- 因此 forward computation 仍然被 recurrent chain 限制。

问题变成:

Can we keep AR likelihood, but redesign architecture so training is parallel?

(Advance) Why Teacher Forcing Enables Parallel Loss but Not Recurrent Computation.

teacher forcing 指训练时 conditional distribution 使用 ground-truth prefix, 而不是模型自己 sample 出来的 prefix. 对 AR loss:

$$\mathcal{L} = - \sum_i \log p_{\theta}(x_i \mid x_{<i}),$$

所有 target x_i 和 prefix $x_{<i}$ 在训练时都已知, 所以原则上每个 position 的 loss 可以同时计算。

但 RIDE 的 hidden state 递推是:

$$h_i = F_{\theta}(h_{i-1}, x_{i-1}).$$

即使 x_{i-1} 已知, h_i 也必须等 h_{i-1} 算完。因此 teacher forcing 去掉的是 sampling uncertainty, 不会自动去掉 recurrent dependency.

4.4 Pixel Convolutional Neural Network (PixelCNN): masked convolution p.54

Pixel Convolutional Neural Network (PixelCNN) 用 masked convolution 替代 recurrence, 使训练完全并行:

- ordinary convolution 会看到 future pixels, 造成 answer leakage.
- binary mask 把 convolution kernel 中未来位置的 weights 置零。
- 每个 output 只依赖 already-known pixels.

效果是: 训练时 full image 可以像普通 convnet 一样 parallel compute; 生成时仍然 pixel-by-pixel.

(Advance) Why Masked Convolution Preserves AR Causality.

普通 convolution 在位置 i 的输出会使用 kernel 覆盖范围内的 neighboring pixels, 其中可能包含 future pixels x_j with $j \geq i$. 这会泄露答案。

PixelCNN 使用 binary mask M 修改 kernel K :

$$K_{\text{masked}} = M \odot K.$$

对 future positions, $M = 0$; 对 allowed past positions, $M = 1$. 因此输出满足:

$$\hat{x}_i = f_{\theta}(x_{<i}),$$

不依赖 $x_{\geq i}$. 训练时所有 true pixels 都在 tensor 中, 但 mask 保证 computation graph 不从 future pixels 取信息, 所以可以 parallel compute 而不破坏 AR likelihood.

4.4.1 Pixel Recurrent Neural Network (PixelRNN) 和 PixelCNN 的关系 p.55

PixelCNN 来自论文 Pixel Recurrent Neural Networks. 该 work 中 PixelRNN 本身是 hybrid architecture:

- PixelRNN: 结合 recurrent 和 convolutional components, 使用 Row LSTM 和 Diagonal Bidirectional LSTM (BiLSTM)。
- PixelCNN: 同一篇论文中的 simpler, more scalable variant, 去掉 recurrent state, 只依赖 masked convolution.

PixelCNN training 可以 across spatial positions 并行, 但 long-range dependency modeling 比完整 recurrence 脆弱。

4.4.2 Pixel-level Bottleneck p.57

RIDE 和 PixelCNN 共享根本瓶颈: 它们都 model raw pixels.

对 1024×1024 RGB image, sequence length 是:

$$1024 \times 1024 \times 3 = 3,145,728.$$

即使每一步很快, serial sampling 也不可行。

(Advance) Why Pixel-Level Serial Sampling Is the Bottleneck.

AR generation 必须先 sample x_1 , 再 sample $x_2 \mid x_1$, 一直到 $x_N \mid x_{<N}$. 这产生 N 个 sequential dependencies.

对 $1024 \times 1024 \times 3$, $N = 3,145,728$. 即使每一步只需要 0.1 ms, 单张图也要超过 300 秒。实际 neural network step 通常更重, 所以 high-resolution raw-pixel AR 不现实。

4.5 Vector Quantized Variational Autoencoder (VQ-VAE): 用 discrete latents 压缩 image sequence p.58

pixel-level AR 概念上干净, 但 sequence length 是 killer. 一个自然想法是先压缩:

学习 compact image representation.

- 在 compressed sequence 上跑 AR.
- 生成后 decode 回 image.

Vector Quantized Variational Autoencoder (VQ-VAE) 的 pipeline 是: p.59

- encoder 输出 $z_e(x)$.
- 每个位置选择最近的 codebook vector e_k .
- nearest-neighbor index 成为 discrete token.
- decoder 从 quantized latents 重建 image.

(Advance) Term Explanation: Vector Quantization.

vector quantization 是把连续 encoder output $z_e(x)$ 映射到有限 codebook 中最近的 vector:

$$k = \arg \min_j \|z_e(x) - e_j\|_2, \quad z_q(x) = e_k.$$

因为 k 是 discrete index, image 就被转换成 token grid. 这个过程缩短了 sequence, 但也引入 quantization error: 所有落在同一个 Voronoi cell 内的 continuous features 都会被同一个 code vector 表示。

4.5.1 VQ-VAE Training Losses: three terms 和 straight-through estimator p.60

VQ-VAE 的 loss 包含三部分:

$$L = \log p(x \mid z_q(x)) + \|\text{sg}[z_e(x)] - e\|_2^2 + \beta \|z_e(x) - \text{sg}[e]\|_2^2.$$

(Advance) Sign Convention in the Reconstruction Term.

slide 写作 $\log p(x \mid z_q(x))$, 表示 reconstruction log-likelihood. 作为 minimized loss 时, 通常写成 negative reconstruction log-likelihood:

$$- \log p(x \mid z_q(x)).$$

如果 decoder likelihood 是 fixed-variance Gaussian, 那么 minimizing $-\log p(x \mid z_q(x))$ 等价于 minimizing pixel-space mean squared error up to constants. 这里正文保留 slide 记号, 但理解 loss direction 时要注意符号约定。

三项分别是:

- reconstruction loss: 训练 decoder 从 $z_q(x)$ 重建 x , 也通过 straight-through estimator 训练 encoder.
- codebook loss: 把 embedding vectors e_k 拉向 encoder outputs $z_e(x)$, 类似 dictionary learning / k-means.
- commitment loss: 鼓励 encoder output commit 到某个 codebook entry, 避免无界漂移.

其中 sg[] 是 stop-gradient. nearest-neighbor lookup:

$$z_q(x) = e_k$$

不可微, 所以 backprop 时把 $\nabla_{z_q} L$ 直接复制给 $z_e(x)$, 绕过 argmin 操作, 这就是 straight-through estimator.

(Advance) Derivation: Why the Three Loss Terms Update Different Parts.

记 sg[] 为 stop-gradient: forward value 不变, 但 backward gradient 为 0.

Codebook loss:

$$\|\text{sg}[z_e(x)] - e\|_2^2$$

中, sg[$z_e(x)$] 不接收梯度, 所以梯度只更新 codebook vector e , 把 e 拉向 encoder output.

Commitment loss:

$$\beta \|z_e(x) - \text{sg}[e]\|_2^2$$

中, sg[e] 不接收梯度, 所以梯度只更新 encoder, 让 $z_e(x)$ 靠近被选中的 codebook vector, 避免 encoder output 到处漂移。

reconstruction loss 需要通过 $z_q(x) = e_k$ 影响 encoder, 但 nearest-neighbor arg min 不可微。straight-through estimator 近似设定:

$$\frac{\partial z_q}{\partial z_e} \approx I,$$

所以把 decoder 端传来的 $\nabla_{z_q} L$ 直接复制给 z_e . 这是 biased estimator, 但实践中能让 encoder 学到可重建的 discrete representation.

4.6 VQ-VAE + AR Prior: image-as-language modeling p.61

VQ-VAE + AR prior 通常是 two-stage training:

Stage 1 tokenizer:

- 学习把 images 转成 discrete code indices.

- objective 通常是 reconstruction.

Stage 2 AR prior:

- 学习 code sequence 的 joint distribution:

$$p(k_1, \dots, k_N) = \prod_i p(k_i \mid k_{<i}).$$

(Advance) Derivation: AR Prior on Code Indices.

tokenizer 把 image 变成 discrete code indices $k_1, \dots, k_N$. 这些 indices 仍然是 random variables, 所以 chain rule 直接给出:

$$p(k_1, \dots, k_N) = p(k_1)p(k_2 \mid k_1) \cdots p(k_N \mid k_{<N}).$$

Transformer prior parameterizes 每一项 $p(k_i \mid k_{<i})$, 通常用 softmax over codebook vocabulary. 训练 loss 是 token-level cross-entropy:

$$\mathcal{L}_{\text{AR}} = - \sum_i \log p_{\theta}(k_i \mid k_{<i}).$$

- 生成时先 sample token grid, 再交给 decoder 重建 image.

好处是:复杂连续 image problem 变成 discrete token modeling,更适合 Transformer. 坏处是: tokenizer 是 lossy 的: AR prior 无法恢复 tokenizer 压缩掉的细节。

4.7 JetFormer: flow soft tokens + decoder-only AR Transformer p.62

VQ-VAE 缩短了 sequence, 但 discrete quantization 也引入 bottleneck: finite codebook 会把 local features hard-assign 到某个 code.

- true feature 如果位于两个 codes 之间, 会产生 quantization error.
- tokenizer 常常先训练好并冻结, 后续 AR prior 只能适配固定 representation. JetFormer 的答案是: 用 normalizing flow 作为 invertible image representation module, 再用 decoder-only Transformer 对 soft tokens 做 AR modeling.

4.7.1 JetFormer Overview p.63

JetFormer 的目标是直接 maximize raw images 和 text 的 likelihood. 不依赖 separately pretrained and frozen image tokenizer.

设计:

- normalizing flow 产生 soft image tokens.
- flow invertible, 因此可以 encode 和 decode 回 image.
- decoder-only Transformer 同时处理 text tokens 和 image tokens.

整体路线是:

Raw image $\rightarrow$ Flow $\rightarrow$ Soft tokens $\rightarrow$ AR Transformer $\rightarrow$ Generated so

(Advance) Why JetFormer Can Connect Raw Likelihood and AR Likelihood.

如果 flow 把 raw image x 映射成 soft token sequence $y = f_{\phi}(x)$, 那么 exact likelihood 仍然需要 change-of-variables:

$$\log p_X(x) = \log p_Y(f_{\phi}(x)) + \log \left| \det J_{f_{\phi}}(x) \right|.$$

其中 $p_Y(y)$ 由 AR Transformer factorize:

$$\log p_Y(y) = \sum_i \log p_{\theta}(y_i \mid y_{<i}).$$

所以 JetFormer 的 raw-image likelihood 同时包含 AR conditional likelihood 和 flow log-det correction. 这就是“flow tokenizer 不是普通预处理模块”的数字原因。

4.7.2 JetFormer Flow Soft Tokens: 连续且可逆的 representation p.64

VQ tokenizer 和 Jet flow 的对比:

VQ Tokenizer; Jet Flow

hard mapping to finite codebook; continuous soft tokens
quantization error unavoidable; invertible, no information loss
representation space fixed; representation trained jointly with generation objective

(Advance) 为什么这是 Flow 和 AR 的交叉点.

Flow 负责把 raw image 变成可逆、连续、可 likelihood-evaluable 的 representation: AR Transformer 负责 sequence modeling. 这样既保留了 AR 的 next-token objective, 也避免了 discrete tokenizer 的 hard quantization.

4.7.3 JetFormer AR Head: continuous distribution prediction p.65

discrete AR 输出 vocabulary logits, 而 JetFormer 的 y_i 是 flow 产生的 continuous soft token:

$$p(x) = \prod_i p_{\theta}(y_i \mid y_{<i}).$$

(Advance) Correction and Derivation: Density Is on Soft Tokens.

严格说，若 $y = f_{\phi}(x)$, AR head 直接建模的是 $p_Y(y)$:

$$p_Y(y) = \prod_i p_{\theta}(y_i \mid y_{<i}).$$

raw image density 应写成:

$$p_X(x) = p_Y(f_{\phi}(x)) \left| \det J_{f_{\phi}}(x) \right|.$$

slides 用 $p(x)$ 强调 end-to-end image likelihood, 但数学上中间需要 flow 的 change-of-variables correction. 这个 correction 正是 JetFormer 区别于 frozen tokenizer AR 的地方。

因此 AR head 要预测 continuous distribution. 可选方式包括:

- Gaussian: 简单, 但可能 average away multiple modes.
- Gaussian Mixture Model (GMM): 能表达 multiple candidate modes.
- 更复杂的 continuous densities.

关键困难是: 给定 context, 下一个 image token 可能有多 个 plausible continuations: 简单 mean 会导致 blur.

4.7.4 JetFormer 的优点和限制 p.66

JetFormer 的优点:

- 减少对 pretrained tokenizer 的依赖.
- invertible soft tokens 保留更多信息.
- AR likelihood 更直接连接 raw data.
- 可同时服务 image generation, text modeling 和 multimodal tasks.
- 限制:
 - continuous token output distributions 比 discrete softmax 难训练.
 - flow 和 Transformer joint training 增加复杂度.
 - high-resolution images 仍然带来 long-sequence problem.

4.8 AR Representative Works 的演化 p.67

Model; Core Improvement; Representation; Training Efficiency
RIDE; spatial LSTM 证明 pixel-level AR 可行; raw pixels; low, serial recurrence

PixelCNN; masked convolution 提升 parallel training; raw pixels; medium, convolution parallel
VQ-VAE; 把 image 压缩成 discrete tokens; discrete codebook; high, short sequence

JetFormer; 用 invertible continuous soft tokens 替代 fixed tokenizer; continuous vectors; high, end-to-end

主线是: 如何保持 stable MLE training, 同时缩短 sequence、提升 fidelity、降低 fixed-representation loss.

(Advance) Why the Progression Table Is Ordered This Way.

这个 progression 的隐含 objective 是同时优化三件事:

- likelihood training 是否稳定;
- sequence length 是否短;
- representation 是否丢失太多信息.

RIDE 和 PixelCNN 保持 exact pixel likelihood, 但 sequence 太长; VQ-VAE 缩短 sequence, 但 codebook quantization 有 loss; JetFormer 试图用 invertible flow soft tokens 同时缩短/重排 representation 并保留 information. 每一步都在交换 representation cost, training cost 和 sampling cost.

(Advance) Part III Conclusion.

AR 的演化路径是: pixel-level RIDE / PixelCNN, 到 token-based VQ-VAE, 再到 continuous JetFormer. p.68

它的核心优势是 objective 简单稳定: 不需要 discriminator, 也不需要复杂 sampling chain 来定义 training objective, 只要按顺序预测 next element 即可。

5 Flexible-Order Autoregressive Generation: 打破 fixed raster order p.70

fixed-order AR 的问题是: image 是二维空间结构, 不天然是一维时间序列. raster scan 有几个缺点:

- top-left before bottom-right 没有 semantic necessity.
- semantic layout 往往先于 local texture, 但 raster scan 不反映这个层级.
- token-by-token sampling 需要 $O(N)$ serial steps.
- 对 inpainting, outpainting 和 local editing 不友好.

Flexible-Order Autoregressive Generation 的动机是把 generation order 从 hard-coded rule 变成 designable, learnable 或 queryable object. p.71

representative works:

- Masked Generative Image Transformer (MaskGIT): 通过 bidirectional masking 做 masked parallel decoding.
- Random-Order Autoregressive Image Generation (RandAR): 保留 flexible order, 但切换到 causal attention 以支持 key-value cache (KV-cache), 代价是需要 position instruction tokens.
- (Advance) New Terms and Why Flexible Order Helps.
- raster scan: 按行从左到右、从上到下把 image grid 展平成 sequence.
- inpainting: 给定已知上下文, 填补 image 中间缺失区域.
- outpainting: 在已有 image 外部继续生成, 扩展画面边界.
- key-value cache (KV-cache): causal Transformer 推理时缓存过去 tokens 的 key/value states, 避免每一步重复计算整个 prefix.

fixed-order AR 的 $O(N)$ serial cost 来自每一步只能生成一个 next token. Flexible-order 方法允许多个位置在同一轮被预测或 refined, 因此 sequential depth 从 N 降到少量 rounds. 它没有违反 conditional modeling, 而是改变了“哪些变量已知、哪些变量要预测”的组织方式。

5.1 Masked Generative Image Transformer (MaskGIT): architecture p.72

Masked Generative Image Transformer (MaskGIT) 是 two-stage design, 把 masked language modeling 变成 image generation.

Stage 1 visual tokenizer:

- image $H \times W$ 被压缩成 discrete token grid $h \times w$.
- 使用 Vector Quantized Generative Adversarial Network (VQGAN) encoder / decoder / codebook.
- reconstruction quality 限制 generation ceiling.

Stage 2 bidirectional Transformer:

- 在 token grid 上操作, 而不是 raw pixels.
- 每个 token 可以 attend to all others, 即 bidirectional self-attention.
- 支持 parallel decoding.

(Advance) Why Two-Stage Design Is Needed.

MaskGIT 不直接在 raw pixels 上做 masked prediction, 因为 raw pixel grid 的 sequence 太长且每个 pixel 的 local uncertainty 很低. visual tokenizer 先把 image 压缩成 semantic-ish discrete tokens:

$$x \rightarrow y_1:N.$$

Transformer 再建模 token-level dependency. 这样每个 token 对应一块 image content, masked prediction 更接近 language modeling 中“预测缺失词”的问题。

5.2 MaskGIT Training: Masked Visual Token Modeling (MVTM) p.73

MaskGIT 的 training objective 是从 partial context 预测 masked visual tokens, 类似 Bidirectional Encoder Representations from Transformers (BERT)。

(Advance) New Terms: BERT, VQGAN, MVTM.

- Bidirectional Encoder Representations from Transformers (BERT): 语言模型中的 masked token prediction framework. 训练时随机 mask 一些 words, 让 Transformer 根据左右上下文预测缺失词。
 - Vector Quantized Generative Adversarial Network (VQGAN): 用 vector quantization 学 discrete visual tokens, 并通常结合 adversarial/perceptual loss 提升 reconstruction quality 的 tokenizer.
 - Masked Visual Token Modeling (MVTM): MaskGIT 的 image 版本 masked modeling: 不是预测 missing words, 而是预测 missing visual tokens.
- Masked Visual Token Modeling (MVTM) procedure:
- 把 image 转成 token grid:

$$\mathcal{Y} = [y_i]_{i=1}^N.$$

- 从 scheduling function $\gamma(r)$ 采样 mask ratio $r \in [0, 1]$.
- 用 [MASK] 替换:

$$[\gamma(r) \cdot N]$$

个 tokens.

- bidirectional Transformer 对所有 masked positions 预测:

$$p(y_i \mid Y_{\bar{M}}).$$

- loss 只在 masked tokens 上计算 cross-entropy.

Mask schedule 的设计要求是: $\gamma(r)$ continuous, decreasing, 并且:

$$\gamma(0) \rightarrow 1, \quad \gamma(1) \rightarrow 0.$$

slide 中强调 concave functions, 例如 cosine, 通常最好, 因为它们实现了 less-to-more information flow.

(Advance) Derivation: Mask Count and Masked Cross-Entropy.

token grid 有 N 个 positions. mask schedule $\gamma(r) \in [0, 1]$ 表示当前要 mask 的比例, 因此 mask count 是:

$$m = \lceil \gamma(r)N \rceil.$$

取 ceiling 是因为 token 数必须是 integer, 并且当比例很小但非零时仍至少能 mask 一个 token.

若 masked set 是 M , training loss 只对 masked positions 求和:

$$\mathcal{L}_{\text{MVTM}} = - \sum_{i \in M} \log p_{\theta}(y_i \mid Y_{\bar{M}}, M),$$

其中 $Y_{\bar{M}}$ 是 visible tokens. visible positions 不计算 loss, 因为它们已经作为 context 给出; 让模型预测 visible tokens 会变成 copy task, 不能训练 fill-in-the-blank 能力。

$\gamma(0) \rightarrow 1$ 和 $\gamma(1) \rightarrow 0$ 的含义是: generation 初期几乎全 mask, 后期几乎全 visible. concave schedule 让早期保留较少 high-confidence tokens, 后期逐渐细化, 形成 less-to-more 的 refinement.

(Advance) MaskGIT 和 standard AR 的差别.

standard AR 学的是“按固定顺序写下下一个 token”; MaskGIT 学的是“根据上下文填空”。每个 masked position 可以看到 both sides of context, 因此没有 raster-order constraint.

5.3 MaskGIT Iterative Parallel Decoding p.74

MaskGIT generation 从 fully masked token grid 开始, 迭代并行解码。每次 iteration:

- Predict: 对所有 masked positions 并行输出:

$$p_t \in \mathbb{R}^{N \times K}.$$

- Sample: 抽样 y_t^i , 并用 max probability 作为 confidence c_i .
- Schedule: 根据当前 step 保留:

$$n = \left\lceil \gamma\left(\frac{t}{T}\right) \cdot N \right\rceil$$

个 tokens.

- Mask: 保留 top-n confident tokens, 重新 mask 其余 tokens.

这样 early iterations 先解决 easy tokens 和 global structure, later iterations 再在 resolved context 上填 details. sampling steps 从 $O(N)$ 降到 $T \approx 8-12$.

(Advance) Derivation: Why Iterative Decoding Reduces Serial Steps.

standard AR 每次只能 sample 一个 token:

$$y_1 \rightarrow y_2 \rightarrow \cdots \rightarrow y_N,$$

因此 sequential depth 是 N 。

MaskGIT 在第 t 轮对所有 masked positions 同时预测 logits:

$$p_t \in \mathbb{R}^{N \times K},$$

其中 K 是 codebook size. 然后用 confidence $c_i = \max_k p_t(i, k)$ 选择最可信的 tokens 保留。下一轮只重新预测低置信度 tokens。

sequential dependency 不再是 token-level, 而是 round-level:

$$\text{all masks} \rightarrow \text{round } 1 \rightarrow \cdots \rightarrow \text{round } T.$$

所以 serial steps 从 N 降为 T . 代价是每轮需要并行预测整张 token grid, 并且最终质量依赖 confidence ranking 和 schedule.

5.4 MaskGIT Applications: editing, inpainting 和 outpainting p.75

bidirectional attention 让很多 image manipulation tasks 不需要 retraining:

- class-conditional editing: mark bounding box, 保持 context fixed, 在 box 内按 target class 重新生成内容。
- inpainting: missing region 作为 initial mask。

- outpainting: mask outer regions, 从已有 context 向外扩展。

因为模型本来就会根据 arbitrary visible context 填 masked tokens, 所以这些任务只是 inference-time mask pattern 的改变。

(Advance) Why No Retraining Is Needed for Editing.

MaskGIT training 已经随机 mask 任意 subset M , 并学习:

$$p_{\theta}(Y_M \mid Y_{\bar{M}}).$$

inpainting 时, M 是缺失区域; outpainting 时, M 是外扩区域; class-conditional editing 时, M 是 bounding box 内 tokens. 数学形式仍然是同一个 conditional prediction problem, 只是 mask geometry 改变。因此不需要重新训练 objective.

5.5 MaskGIT 的限制: bidirectional attention 不能用 KV-cache p.76
MaskGIT 的 flexible order 来自 bidirectional attention, 但代价是不能自然使用 KV-cache.

decoder-only causal Transformer 中, previous keys and values 可以 cache; bidirectional attention 每一步都需要重新计算所有 positions, 因为 [MASK] tokens 作为 placeholders, 可以被任意位置 attend.

design dilemma:

- MaskGIT 需要 bidirectional attention 来处理 arbitrary known / unknown tokens.
- causal attention 没有 placeholders, 只看 past.
- 如果想同时支持 flexible order 和 KV-cache, 就需要新机制。

RandAR 的 insight 是: 切换到 causal attention, 但显式告诉模型下一步要预测哪里, 也就是插入 position instruction token.

(Advance) Why Bidirectional Attention Blocks KV-Cache.

causal Transformer 的第 t 步只 attend to positions $\leq t$. 旧 prefix 的 keys/values 不会因为新 token 出现而改变, 所以可以 cache.

bidirectional Transformer 中, 每个 position 可以 attend to every other position. MaskGIT 每轮会改变哪些 tokens 是 [MASK]、哪些 tokens 被 fixed; 这会改变所有 positions 的 attention context. 因此旧的 key/value states 不再对应当前 input state, 必须整体 recompute.

结论: MaskGIT 的 flexibility 来自 bidirectional placeholders, 但 placeholders 的全局交互破坏了 causal prefix cache 的可复用性。

5.6 Random-Order Autoregressive Image Generation (RandAR): position instruction token p.77

Random-Order Autoregressive Image Generation (RandAR) 在 decoder-only Transformer 中实现 arbitrary generation order.

core design: p.78

- 随机 shuffle image tokens.
- 在每个 image token 前插入 position instruction token P_i .
- P_i 编码坐标 (h_i, w_i) , 告诉 model 当前位置要生成哪个 spatial token.
- standard causal decoder-only Transformer 不需要改结构。

训练时使用 random permutations, 只需要一个额外 learnable embedding.

(Advance) Derivation: Random-Order AR with Position Instructions.

选择一个 permutation $\pi = (\pi_1, \dots, \pi_N)$ 。standard AR 可以写成:

$$p(y) = \prod_{t=1}^N p(y\pi_t \mid y_{\pi<t}).$$

但如果只把 tokens 随机排列, 模型不知道当前要预测的是 image grid 上哪个位置。因此 RandAR 在 y_{π_t} 前插入 position instruction P_{π_t} , 让 conditional 变成:

$$p(y\pi_t \mid y_{\pi<_t}, P_{\pi\leq_t}).$$

causal mask 仍然成立: 模型只能看过去 generated tokens 和当前/过去 position instructions. position instruction 提供 query coordinate, random permutation 提供 flexible order.

(Advance) 为什么 position instruction token 关键.

causal Transformer 只能根据 past token next token. 如果 token order 是随机的, 模型必须知道“next token 对应 image grid 的哪个位置”。position instruction token 就是这个 query: 它把 spatial coordinate 作为输入, 让 model 在当前 context 下生成指定位置的 token.

5.7 RandAR Parallel Decoding with KV-Cache p.79

random-order training 带来 inference 优势: parallel decoding.

做法:

- 在 sequence 末尾 append 多个 position instruction tokens.
- model 在一次 forward pass 中预测所有对应 image tokens.
- previously generated context 只 cache 一次, 并复用 KV-cache.

• 不需要 training modification 或 fine-tuning。
RandAR 可以把 inference steps 从 N 降到一个小 fraction, 例如 88 或 48, 并兼容 KV-cache acceleration. slide 中报告可实现 $2.5\times$ latency reduction without quality loss.
(Advance) Why Multiple Position Instructions Enable Parallel Decoding.
在 causal Transformer 中, 如果 sequence 末尾追加多个 position instructions:

... , $P_a, P_b, P_c,$

它们都可以 attend to the same cached generated context. 模型可以在同一次 forward pass 中输出这些 positions 对应的 next-token distributions.

严格来说, 这些并行预测的 tokens 之间没有互相 condition; 因此并行 batch size 越大, approximation 越强. RandAR 的 random-order training 让模型习惯在不同 context subsets 下预测指定位置, 所以这种 parallel decoding 在质量和 latency 之间形成可控 trade-off.

5.8 RandAR Zero-Shot Capabilities p.80

random-order training 给 model 带来 flexible spatial reasoning:

- parallel decoding: 一次 query 多个 position instruction tokens.
- inpainting / outpainting: 给定 context tokens, 再 query missing / outside positions.
- zero-shot resolution extrapolation: 通过新坐标位置扩展 token grid.
- bi-directional feature extraction: 通过 random permutation and repeated rounds 得到更灵活的 context usage.

(Advance) Why Zero-Shot Spatial Tasks Emerge.
RandAR 的 model input 显式包含 coordinate query P_i , 所以 inference 时可以用 query training raster order 之外的位置集合:

- inpainting: query missing coordinates, context 是已知区域 tokens.
- outpainting: query outside coordinates, context 是原图 tokens.
- resolution extrapolation: query 更密或更大的 coordinate grid.

这些任务共享同一个 conditional form:

$$p(y_Q \mid y_C, P_Q, P_C),$$

其中 C 是 context positions, Q 是 queried positions. 只要 coordinate embedding 能合理泛化, 模型就能把 random-order AR 训练得到的 spatial reasoning 迁移到这些 zero-shot layouts.
(Advance) Part IV Conclusion.
Flexible-order generation 的主线是从“固定顺序写图像”转向“按需要查询位置并行填图像”。MaskGIT 用 bidirectional masking 获得灵活性, 但牺牲 KV-cache; RandAR 用 position instruction token 把 flexible order 放回 causal decoder-only Transformer, 从而保留 AR 训练方式和 KV-cache inference 优势。

6 Lecture Takeaways: Beyond Diffusion 的统一视角 p.3

这一讲可以用四组 trade-off 总结:

- Normalizing Flow (NF): exact likelihood 和 invertibility 很漂亮, 但 architecture 必须满足 tractable Jacobian. 现代 TarFlow 用 Transformer conditioner、noise augmentation、denoising 和 guidance 改善 sample quality.
 - Generative Adversarial Network (GAN): 不写 explicit density, 直接通过 discriminator 学 perceptual gap: sampling 快、质量强, 但 training dynamics 和 coverage 是主要难题. 现代 GAN 思想也可作为 adversarial loss 服务 diffusion distillation.
 - Autoregressive Model (AR): objective 稳定清楚, 适合 Transformer; 瓶颈是 sequence length 和 fixed order. VQ-VAE 用 discrete tokenizer 压缩, JetFormer 用 flow soft tokens 尝试 continuous invertible representation.
 - Flexible-Order Autoregressive Generation: 把 order 从 hard-coded raster scan 变成 mask schedule 或 position query; MaskGIT 强在并行 masked decoding, RandAR 强在 causal attention + KV-cache.
- (Advance) Why These Four Takeaways Follow from the Objectives.
- Flow 的 objective 是 exact likelihood:

$$\log p_X(x) = \log p_Z(f(x)) + \log |\det J_f(x)|.$$

所以它天然得到 density 和 invertible inference, 但必须让 J_f tractable.

- GAN 的 objective 是 adversarial classification game, 所以它直接优化 sample realism signal, 但没有 normalized density, 也带来 game stability 问题。

- AR 的 objective 是 chain-rule likelihood:

$$\log p(x) = \sum_i \log p(x_i \mid x_{<i}).$$

所以 training 稳定, 但 generation 顺序依赖强。

• Flexible-order methods 改写“已知 context 和待预测 tokens”的组织方式, 例如 MaskGIT 的 $p(Y_M \mid Y_{\bar{M}})$ 或 RandAR 的 coordinate query. 它们降低 serial depth, 但需要 mask schedule、confidence selection 或 position instruction 来指定生成过程。
(Advance) 一句话地图。
Flow 问“如何 exact density + invertible sample”; GAN 问“如何让 sample perceptually real”; AR 问“如何把 image 写成 stable next-token prediction”; Flexible-Order AR 问“如果图像不是一维句子, 为什么还要按固定顺序写?”
created: 2026-05-22
updated: 2026-05-22
1 Simple YOLO 代码理解笔记
这份笔记按代码文件整理, 目标是帮助理解这个简化 YOLO 项目的主流程。
整体流程是:

```
“‘text
图片和标签
-> Dataset 生成 image_tensor 和 target
-> Model 输出 pred
-> Loss 比较 pred 和 target
-> Utils 解码预测框、计算 IoU、NMS、mAP
“
最终训练配置里:
“‘text
image_size = 448
grid_size = 14
num_classes = 1
“
所以一张图片会被看成 14 x 14 个 grid cell, 每个 cell 输出 6 个值:
“‘text
[objectness, x, y, w, h, class]
“
```

```
1.1 dataset.py
dataset.py 负责把图片和标签读成模型能训练的数据。
每次 Dataset 返回:
“‘text
image_tensor: [3, image_size, image_size]
target: [grid_size, grid_size, 5 + num_classes]
“
最终就是:
“‘text
image_tensor: [3, 448, 448]
target: [14, 14, 6]
“
```

```
1.1.1 objectness 和 class 为什么都是维度
每个 cell 里存:
“‘text
[objectness, x, y, w, h, class...]
“
objectness 表示:
“‘text
这个 cell 里有没有目标
“
类别部分表示:
“‘text
如果这里有目标, 它是什么类别
“
当前只有一个类别 face, 所以类别部分只有 1 维:
“‘text
[objectness, x, y, w, h, face]
“
如果有两个类别, 比如 face 和 person, 每个 cell 会变成:
“‘text
[objectness, x, y, w, h, face, person]
“
```

```
例如:
“‘text
[1, 0.28, 0.72, 0.20, 0.30, 1, 0]
“
表示这个 cell 有目标, 目标是 face.
1.1.2 _load_target 的核心
YOLO 标签格式是:
“‘text
class_id cx cy w h
“
例如:
“‘text
0 0.52 0.48 0.20 0.30
“
含义是:
“‘text
类别 0
中心点在全图 x=0.52, y=0.48
宽度是全图的 0.20
高度是全图的 0.30
“
_load_target() 会先创建一个全 0 的 target:
“‘python
target = torch.zeros((s, s, 5 + c))
“
如果 s=14, c=1, 就是:
“‘text
[14, 14, 6]
“
```

然后逐行读取标签, 把每个目标放进对应的 grid cell.
1.1.3 怎么找到目标属于哪个 grid

```
核心代码:
“‘python
grid_x = min(int(cx * s), s - 1)
grid_y = min(int(cy * s), s - 1)
“
例如:
“‘text
cx = 0.52
cy = 0.48
s = 14
“
那么:
“‘text
grid_x = int(0.52 * 14) = 7
grid_y = int(0.48 * 14) = 6
“
```

```
所以这个目标写入:
“‘python
target[6, 7]
“
```

```
注意顺序是:
“‘text
target[y, x]
“
```

不是 target[x, y].
1.1.4 中心是局部的, 宽高是全局的代码:

```
“‘python
x_in_cell = cx * s - grid_x
y_in_cell = cy * s - grid_y
“
继续上面的例子:
“‘text
cx * s = 7.28
grid_x = 7
x_in_cell = 0.28
cy * s = 6.72
grid_y = 6
y_in_cell = 0.72
“
```

```
“
所以中心点存的是:
“‘text
目标在当前 cell 内部的位置
“
但是 w, h 仍然是全图比例:
“‘text
w = 0.20
h = 0.30
“
原因是:
“‘text
中心点用于决定哪个 cell 负责目标, 所以用 cell 内坐标:
宽高描述整个目标框, 所以用全图坐标。
“
```

```
一个框可能跨多个 grid cell, 但中心点只会落在一个 cell 里。
1.1.5 同一个 cell 多个目标怎么办
这个简化版每个 cell 只能预测一个框。
所以如果多个目标中心落在同一个 cell, 代码保留面积更大的目标:
“‘python
area = w * h
old_area = target[grid_y, grid_x, 3] * target[grid_y, grid_x, 4]
“
```

```
如果当前 cell 为空, 或者新目标面积更大, 就写入新目标。
1.1.6 写入 target
最终写入:
“‘python
target[grid_y, grid_x, 0] = 1.0
target[grid_y, grid_x, 1:5] = [x_in_cell, y_in_cell, w, h]
target[grid_y, grid_x, 5 + class_id] = 1.0
“
```

```
例如:
“‘text
target[6, 7] = [1, 0.28, 0.72, 0.20, 0.30, 1]
“
```

```
1.2 model.py
model.py 负责把图片变成每个 grid cell 的预测。
输入:
“‘text
[batch, 3, 448, 448]
“
输出:
“‘text
[batch, 14, 14, 6]
“
```

```
1.2.1 ConvBlock
ConvBlock 是:
“‘text
Conv2d -> BatchNorm2d -> LeakyReLU
“
作用:
“‘text
提取局部特征
稳定训练
增加非线性表达能力
“
```

```
1.2.2 Backbone
backbone 用多层卷积和池化提取特征。
最终配置中, shape 大致变化是:
“‘text
[batch, 3, 448, 448]
-> [batch, 16, 224, 224]
-> [batch, 32, 112, 112]
-> [batch, 64, 56, 56]
-> [batch, 128, 28, 28]
-> [batch, 256, 14, 14]
-> [batch, 512, 14, 14]
“
空间尺寸变小, 通道数变多。可以理解为:
“‘text
```


细节减少，语义信息增加。

“

1.2.3 Head

head 把 backbone 特征变成预测。

最后一层：

```
“python
nn.Conv2d(256, output_channels, kernel_size=1)
“
```

其中，

```
“python
output_channels = 5 + num_classes
“
```

当前为 6。

卷积输出默认是：

```
“text
[batch, 6, 14, 14]
“
```

但 target 是：

```
“text
[batch, 14, 14, 6]
“
```

所以 forward() 最后做：

```
“python
x.permute(0, 2, 3, 1).contiguous()
“
```

把维度换成和 target 一致。

1.2.4 输出值为什么不直接满足概率约束

模型最后输出的是 raw logits，不是概率。

例如类别输出可能是任意实数：

```
“text
[-1.2, 0.5, 3.0]
“
```

在 loss 或 decode 里才会用 sigmoid 把它变成 0 到 1。

当前代码对类别使用 sigmoid，所以更像 multi-label 处理：

```
“text
每个类别独立判断是不是
“
```

如果是严格互斥多类别，更标准的方式是 softmax + cross entropy。

但本作业只有一个类别 face，所以 sigmoid 简洁且可用。

1.3 loss.py

loss.py 比较模型输出 pred 和 dataset 给出的 target。

输入 shape：

```
“text
pred: [batch, grid, grid, 6]
target: [batch, grid, grid, 6]
“
```

1.3.1 有目标和没目标的 cell 分开

代码：

```
“python
obj_mask = target[..., 0] == 1
noobj_mask = target[..., 0] == 0
“
```

含义：

```
“text
obj_mask: 有目标的 cell
noobj_mask: 没目标的 cell
“
```

有目标的 cell 需要学习：

```
“text
objectness, box, class
“
```

没目标的 cell 只需要学习：

```
“text
objectness = 0
“
```

1.3.2 objectness loss

obj_loss 让有目标的 cell 输出 objectness=1。

noobj_loss 让没目标的 cell 输出 objectness=0。

没目标的 cell 很多，所以 lambda_noobj 通常小一些，最终配置为：

```
“text
```

```
lambda_noobj = 0.5
```

“

1.3.3 MSE 是什么

MSE 全名是：

```
“text
Mean Squared Error
“
```

中文是均方误差。

公式：

```
“text
MSE = (1 / N) * Σ(y_pred - y_true)^2
“
```

代码里使用：

```
“python
reduction="sum"
“
```

所以实际是求和：

```
“text
Σ(y_pred - y_true)^2
“
```

box_loss 用 MSE 比较：

```
“text
[x_in_cell, y_in_cell, w, h]
“
```

并且只有在有目标的 cell 上计算。

1.3.4 BCEWithLogits 数学含义

binary_cross_entropy_with_logits 等价于：

```
“text
p = sigmoid(x)
BCE(x, y) = - [ y log(p) + (1 - y) log(1 - p) ]
“
```

其中：

```
“text
x 是模型 raw logit
y 是 target，通常是 0 或 1
p 是 sigmoid 后的概率
“
```

with_logits 的意思是：

```
“text
输入还没有经过 sigmoid，函数内部会做 sigmoid + BCE
“
```

这样数值更稳定。

1.3.5 多类别 BCE 是怎么加的

当前代码如果有多个类别，会对每个类别分别算 BCE，然后相加。

例如 3 类：

```
“text
target_cls = [1, 0, 0]
pred_cls = [2.0, -1.0, 0.5]
“
```

loss 是：

```
“text
BCE(2.0, 1) + BCE(-1.0, 0) + BCE(0.5, 0)
“
```

这适合 multi-label 思路，不要求类别概率和为 1。

1.3.6 总 loss

最终：

```
“text
total =
lambda_box * box_loss
• lambda_obj * obj_loss
• lambda_noobj * noobj_loss
• lambda_cls * cls_loss
“
```

最终配置：

```
“text
lambda_box = 20
lambda_obj = 1
lambda_noobj = 0.5
lambda_cls = 1
“
```

“

第一版模型能找到大致位置，但 IoU 经常低于 0.5，所以提高 lambda_box 强化定位。

1.4 utils.py

utils.py 用于后处理和评估。

1.4.1 xywh_to_xyxy

YOLO 标签常用：

```
“text
cx, cy, w, h
“
```

但 IoU、NMS、画框更适合：

```
“text
x1, y1, x2, y2
“
```

转换公式：

```
“text
x1 = cx - w / 2
y1 = cy - h / 2
x2 = cx + w / 2
y2 = cy + h / 2
“
```

1.4.2 IoU

IoU 全名：

```
“text
Intersection over Union
“
```

公式：

```
“text
IoU = 交集面积 / 并集面积
“
```

box_iou() 会计算两组框两两之间的 IoU。

如果两个框完全重合：

```
“text
IoU = 1
“
```

如果完全不重合：

```
“text
IoU = 0
“
```

1.4.3 NMS

NMS 全名：

```
“text
Non-Maximum Suppression
“
```

作用：

```
“text
去掉重复预测框，保留高 confidence 的框。
“
```

规则：

```
“text
• 按 score 从高到低排序
• 保留最高分框
• 删除和它 IoU 过高的低分框
• 继续处理剩下的框
“
```

1.4.4 decode_predictions

decode_predictions() 把模型输出变成真实检测结果。

主要步骤：

```
“text
• 对 objectness 做 sigmoid
• 对 box 做 sigmoid
• 对 class score 做 sigmoid
• 把 cell 内坐标还原成全图坐标
• 根据 confidence threshold 过滤
• 对每个类别做 NMS
“
```

最终输出：

```
“text
boxes, scores, labels
“
```

1.5 mAP 和 PR 曲线

1.5.1 mAP 是什么

mAP 全名：

```
“text
mean Average Precision
“
```

如果有多个类别：

```
“text
mAP = 所有类别 AP 的平均
“
```

1.5.2 Precision 和 Recall

Precision：

```
“text
Precision = TP / (TP + FP)
“
```

含义：

```
“text
预测出来的框里，有多少是真的。
“
```

Recall：

```
“text
Recall = TP / (TP + FN)
“
```

含义：

```
“text
真实目标里，有多少被找到了。
“
```

1.5.3 所有框是混在一起排序的

计算 AP 时，不是每张图单独算。

标准做法是：

```
“text
对某一个类别，把整个验证集的所有预测框放在一起，
按 confidence 从高分到低分排序。
“
```

但是判断一个预测框是不是 TP 时，只能和它自己图片里的 ground truth 比 IoU。

也就是说：

```
“text
排序：全验证集同类别预测框混在一起排序
匹配：只和同一张图里的真实框匹配
“
```

1.5.4 给定 confidence threshold 后怎么理解 Precision/Recall

如果给定：

```
“text
confidence >= 0.5
IoU >= 0.7
“
```

那么：

```
“text
Recall = 在 IoU 限制下找到了多少真实框 / 所有真实框
Precision = 保留下来的预测框里有多少是正确的
“
```

一个真实框最多只能被匹配一次。

如果同一张脸有多个预测框都 IoU 达标：

```
“text
最高分那个算 TP
其余重复框算 FP
“
```

1.5.5 mAP@0.5、mAP@0.9、mAP@0.5:0.95

mAP@0.5：

```
“text
IoU >= 0.5 算检测正确
“
```

mAP@0.9：

```
“text
IoU >= 0.9 才算检测正确
“
```

更严格，要求框非常准。

mAP@0.5:0.95：

```
“text
分别计算 IoU=0.50, 0.55, ..., 0.95 下的 AP，
再取平均。
“
本次最终结果：
“text
mAP@0.5 = 0.8038
mAP@0.9 = 0.0312
mAP@0.5:0.95 = 0.4740
“
```

1.6 训练与报告相关脚本

1.6.1 part2_train_eval.py

负责完整 Part 2:

```
“text
训练模型
保存权重
保存 loss_curve.png
计算 mAP
保存 pr_curve_iou_0.7.png
保存 metrics.json
“
```

最终训练命令见 README.md.

1.6.2 scripts/prepare_fddb.py

负责下载 Fddb 并转换为 YOLO 格式。

Fddb 原始标注是椭圆，脚本会先转成外接矩形，再写成：

```
“text
class_id cx cy w h
“
```

数据划分：

```
“text
train: folds 1-8
```

```
val: folds 9-10
```

```
“
```

1.6.3 scripts/visualize_predictions.py

负责画验证集预测图。

图中：

```
“text
绿色框 = ground truth
红色框 = prediction
“
```

1.6.4 report/report.md

最终英文报告，包含：

```
“text
dataset
method
training config
loss curve
PR curve
mAP metrics
prediction examples
discussion
conclusion
“
```

created: 2026-06-08

updated: 2026-06-08

1 Computer Vision: 2025 Final

- One A4 cheat sheet is allowed. Calculators are forbidden.

1.1 Multiple Choice Questions (21 pts)

Please choose all **correct choices** for each problem to get 3 points. **Incorrect choice or no choice** will receive 0 points; **missing choices** will receive 1 point.

- Which filter(s) is (are) not able to shift the figure (a) to figure (b)?

(Note: Figure (b) is obtained by shifting figure (a) to the right)

(a) $\begin{pmatrix} 0 & 0 & 0 \\ 2 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}$;

(b) $\begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 2 \\ 0 & 0 & 0 \end{pmatrix}$;

(c) $\begin{pmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix}$;

(d) $\begin{pmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix}$.

- Which of the following statements about convolution in image processing is true?

(a) Convolution involves flipping the filter in both dimensions and then applying cross-correlation.

(b) Convolution is not shift-invariant, meaning the output depends on the position of the neighborhood in the image.

(c) The convolution operation does not distribute over addition.

(d) Using a larger kernel in a Gaussian filter results in more smoothing.

- Suppose we are using a Hough transform to do line fitting, but we notice that our system is detecting two lines where there is actually one in some example image. Which of the following is the most likely to alleviate this problem?

(a) Increase the size of the cells in the Hough transform.

(b) Decrease the size of the cells in the Hough transform.

(c) Sharpen the image.

(d) Make the image larger.

- Which of the following statements about image classification and recognition tasks is true?

(a) Pose estimation is concerned with estimating the attributes of objects, such as color and size.

(b) Action recognition is the task of determining the pose of each object in an image.

(c) Scene classification involves recognizing what type of scene is being shown in a picture.

(d) Detection is the task of identifying the presence of objects in an image.

- What is the biggest benefit of image rectification for stereo matching?

(a) Epipoles are moved to the center of the image.

(b) Image contents are uniformly scaled to a desirable size.

(c) All epipolar lines intersect at the vanishing point.

(d) All epipolar lines are perfectly vertical.

(e) All epipolar lines are perfectly horizontal.

- Which of the following is(are) a part(s) of the SIFT calculation pipeline?

(a) Difference of Gaussians.

(b) Histogram of Gradients.

(c) Laplacian of Gaussians.

(d) Orientation Histogram.

(e) Image Pyramid.

(f) None of the above.

- You start training your neural network, but the loss is almost completely flat **from the beginning**. What could be the cause?

(a) The learning rate is too low.

(b) The weight initialization scale is incorrectly set.

(c) The regularization strength is too high.

(d) The class distribution is very uneven in the dataset.

(e) None of the above.

1.2 True or False (21 pts)

For each problem, a correct answer is worth 1 point, and the explanation is worth 2 points.

- Even if we use global average pooling after the last convolution layer in ResNet, we cannot train the model with input images of variable sizes.

- The mean filter is a better tool to remove the impulse (salt and pepper) noise, comparing to the medium filter.

- If the input to a CNN is a zero image (i.e., all pixel values are 0), the class probabilities will come out uniform.

- The regularization mechanism and choosing a larger model capacity are common approaches to alleviate underfitting.

- During backpropagation, when the gradient flows backwards

through sigmoid / tanh / ReLU activations, it cannot change sign.

- Given 2 cameras and their intrinsics and extrinsics, we can calculate the depth by $Z = \frac{fT}{d}$, where f is the focal length, T is the distance between two cameras, and d is the disparity.

- The Canny edge detector is a linear filter, because it uses the Gaussian filter to blur the image and then uses the linear filter to compute the gradient.

1.3 Short Answer Problems (12 pts)

- List 3 methods to prevent overfitting. (3)

- Suppose we would like to use RANSAC to find circles in $\mathbb{R}^2$. Let $D = \{(x_i, y_i)\}_{1 \leq i \leq n}$ be our data, and I is the random seed group of points used in RANSAC.

(a) The next step of RANSAC is to fit a circle (a curve, not a filled circle!) to the points in I . Please formulate this as an optimization problem. That is, represent fitting a circle to the points as a problem of the form

$$\min \sum_{i \in I} L(x_i, y_i, c_x, c_y, r),$$

where L is a function for you to determine which gives the distance from (x_i, y_i) to the circle with center (c_x, c_y) and radius r . (2)

(b) What might go wrong in solving the problem you came up with in (a) when $|I|$ is too small? (2)

(c) The next step in RANSAC is to determine what the inliers are, given the circle (c_x, c_y, r) . Using these inliers, we refit the circle and determine new inliers in an iterative fashion. Define mathematically what an inlier is for this problem. Mention any free variables. (2)

- You are using K-means clustering in color space to segment an image. However, you notice that although pixels of similar color are indeed clustered together, the pixels in the same cluster are not spatially contiguous in the image. Describe a method to overcome this problem in the K-means framework. (3)

1.4 Long Answer Problems (46 pts)

In this section, the detailed process of calculations, proof, and explanations are required. No points will be awarded if you only write a short answer. Please write the answers in the space provided or attach an extra sheet with a clear structure of your answers. We give partial credit for good explanations of what you are trying to do.

1.4.1 Convolutional Architectures (9 pts)

Consider a convolutional neural network block whose input size is $64 \times 64 \times 8$. The block consists of the following layers:

- A convolutional layer 32 filters with height and width 3, stride 1 and 0 padding, which has both a weight and a bias (i.e. CONV3-32)
- A 2×2 max pooling layer with stride 2 and 0 padding (i.e. POOL-2)
- A batch normalization layer (i.e. BATCHNORM)

Compute the output volume dimensions and number of parameters of the layers

(You can write the output shapes in the format (H, W, C) where H, W, C are the height, width, and channel dimensions, respectively. You only need to give the formula for counting parameters, no need for the final values.)

(a) What are the output volume dimensions and number of parameters for CONV3-32? (3)

(b) What are the output volume dimensions and number of parameters for POOL-2? (3)

(c) What are the output volume dimensions and number of parameters for BATCHNORM? (3)

1.4.2 Laplacian of Gaussians (16 pts)

Hint: subquestions (a), (b), (c) are somewhat independent. You can complete them in any order.

(a) You want to use a convolution with a Laplacian of Gaussian to find edges in an image. *(Note: This image is 2D, left and right are white, middle is black.)*

Suppose that you have the following black and white image. In the image, white is represented by the value 1 and black the value 0.

Question: Draw a one-dimensional representation of the intensity. Then convolve the image with a Laplacian of Gaussian filter, showing the result of the convolution. (4)

Hint: Your sketches do not need to be scaled correctly because we have not specified the width of the Gaussian. Please ignore border effects, i.e., ignore the effect of convolving the kernel with the

border of the image. Draw your answer on the following figure (in answer sheet).

(b) Recall the formula of 2D Gaussian and Laplacian of Gaussian:

$$G(x, y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right),$$
$$\text{LoG}(x, y, \sigma) = \nabla^2 G(x, y, \sigma) = \frac{\partial^2 G}{\partial x^2} + \frac{\partial^2 G}{\partial y^2} = \frac{1}{\pi\sigma^4} \left(\frac{x^2 + y^2}{2\sigma^2} - 1\right) e^{-\frac{x^2 + y^2}{2\sigma^2}}$$

Suppose that you wish to compute the 2D Laplacian of Gaussian of an image. Explain how to accelerate this convolution using separable filters. (4)

Hint: consider separable filters

(c) In our lecture, we introduce using Difference of Gaussians $\text{DoG}(x, y, \sigma, k) = G(x, y, k\sigma) - G(x, y, \sigma)$ to approximate Laplacian of Gaussian. Now we want to understand why.

Question: You need to first compute the partial derivatives of the 2D Gaussian with respect to σ , i.e. $\frac{\partial G}{\partial \sigma}$. Then prove the following relationship between Difference of Gaussians and Laplacian of Gaussian:

$$\lim_{k \rightarrow 1} \frac{\partial \text{DoG}(x, y, \sigma, k)}{k - 1} = ? \text{LoG}(x, y, \sigma).$$

Specify what $?$ is. It should be an expression that only uses σ and constants. (8)

1.4.3 Optical Flow (10 pts)

- Derive the optical flow equation from the brightness constancy equation. Clearly state any assumption you make during derivation. (3)

- Can the optical flow equation be solved given two consecutive frames without further assumption? Which values can be computed given two consecutive frames? Which values cannot be computed without additional information? Why? (4)

- What is the aperture problem in optical flow? Please point out which method solves the problem, and explain how. (3)

1.4.4 Projective Geometry (11 pts)

Define a camera coordinate system which is only rotated and translated from a world coordinate system.

Hint: For any invertible matrix A and vectors x, y , we have $(Ax) \times (Ay) = \det(A)(A^{-1})^T(x \times y)$.

- Prove that parallel lines in the world reference system are still parallel in the camera reference system. (2)

- Consider a unit square $pqrs$ in the world reference system, where p, q, r, s are four points. Will the same square in camera reference system always have unit area? Prove or provide a counter example. (3)

- Now consider affine transformations. Given some vector p , an affine transformation is defined as $A(p) = Mp + b$, where M is an invertible matrix. Prove that under any affine transformation, the ratio of parallel line segments is invariant, but the ratio of non-parallel line segments is not invariant. (3)

- Do the properties in the first three questions still hold under perspective projection? Justify your answers briefly. (3)